

Maestría en Ingeniería de Software y Sistemas
Informáticos

Administración de Sistemas, Virtualización, Cloud Computing y DevOps

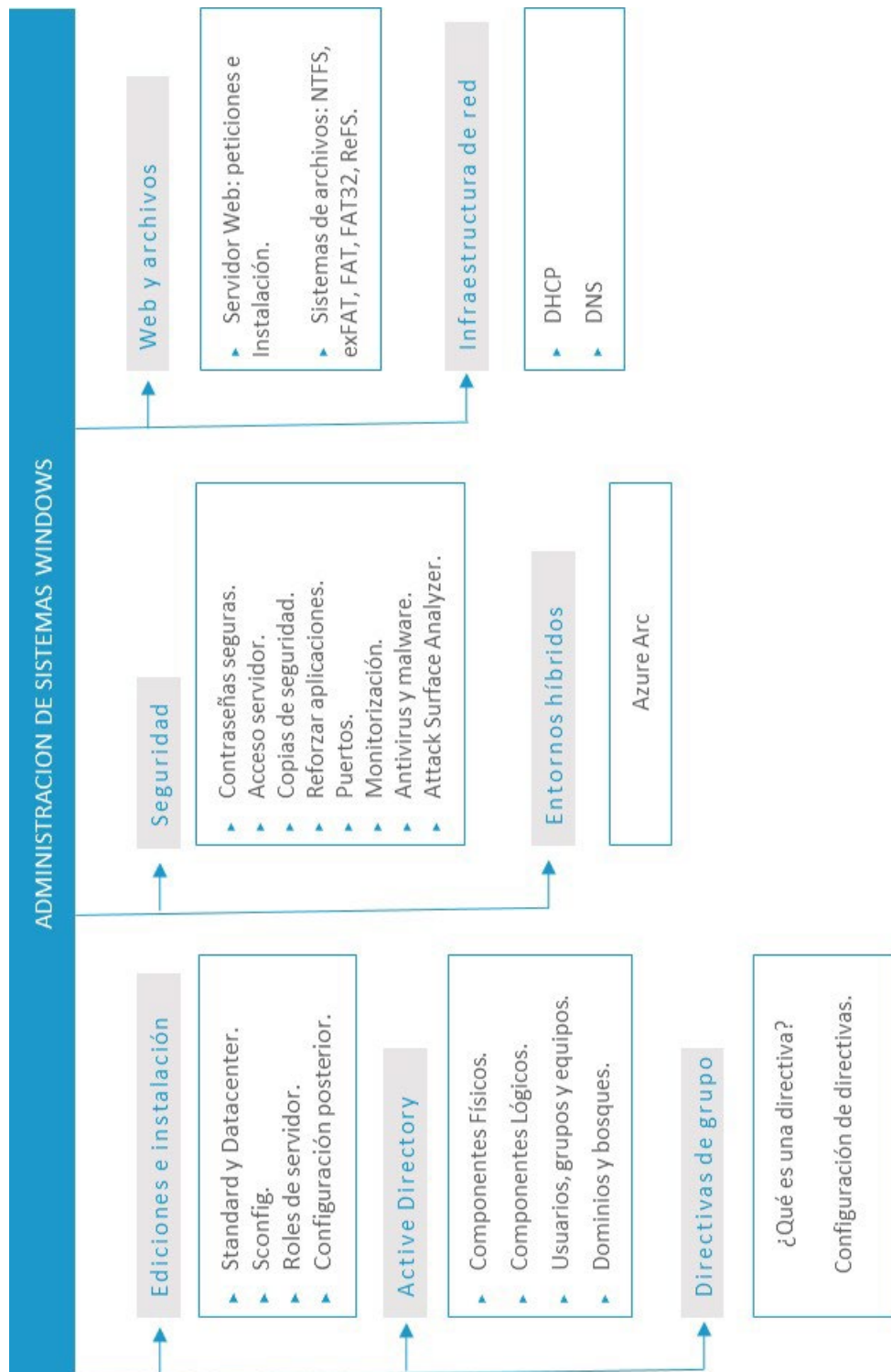
Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Administración de sistemas Windows

Índice

Esquema	3
Ideas clave	4
1.1. Introducción y objetivos	4
1.2. Ediciones e instalación de Windows	5
1.3. Servicios de dominio de Active Directory	10
1.4. Implementación objetos de directiva de grupo	19
1.5. Servidor web, servidor de archivos y monitorización	24
1.6. Infraestructura de red de Windows server	32
1.7. Seguridad en redes Windows	42
1.8. Entornos híbridos	49
1.9. Referencias bibliográficas	52
A fondo	53
Test	55

Esquema



1.1. Introducción y objetivos

En este tema vamos a estudiar los **conceptos** asociados a **Windows Server** que es un sistema operativo de tipo servidor, preparado para gestionar una red de ordenadores mediante un sistema de dominios y un directorio activo que permite una administración centralizada.

Existen varios **roles principales** dentro de Windows Server que cualquier empresa necesita instalar y configurar. Entre los más importantes tenemos los roles de ficheros, impresión, DNS, DHCP y servidor web y comentaremos su **configuración** en un entorno empresarial.

Finalizaremos el tema con una aproximación a la **integración** de Windows Server con las nubes públicas. Hoy en día, ya casi no existen instalaciones únicas de Windows Server, sino que se poseen las denominadas redes híbridas con conexiones a las nubes públicas.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Diferenciar las versiones existentes del producto, así como las diferentes instalaciones.

Aprender a instalar un directorio activo y manejaremos las diferentes consolas de gestión.

Aplicar políticas de grupo para configurar entornos empresariales y como afectan a los clientes basados en Windows conectados a la red.

Instalar servidores de ficheros y servidores web.

Conocer los servicios de red principales en un entorno Windows Server.

Aplicar las configuraciones de seguridad más habituales en entornos corporativos con infraestructura Windows

Introducirnos en el mundo híbrido, empresa-nube pública.

En los **vídeos** de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Instalación Windows Server. Después de la instalación limpia de un Windows Server crearemos un directorio activo.

Vídeo 2. Gestionaremos usuarios y máquinas en el directorio activo creado en el vídeo anterior. Así mismo instalaremos nuevos roles como el role DHCP.

1.2. Ediciones e instalación de Windows

Tal y como pide Microsoft (2022), al instalar Windows Server, debe elegir entre instalar el servidor con o sin la Experiencia de escritorio.

Importante

Esta es una decisión fundamental porque no puede agregar ni quitar la Experiencia de escritorio después de instalar el servidor.

Esta es una decisión fundamental porque no puede agregar ni quitar la Experiencia de escritorio después de instalar el servidor.

Server Core es una instalación de Windows Server sin la Experiencia de escritorio. Server Core está disponible para las ediciones **Standard y Datacenter**, pero no está disponible para Windows Server Essentials, y la versión gratuita de Hyper-V Server solo está disponible como una instalación de Server Core.

Puede administrar y configurar Server Core en el propio servidor a través de PowerShell, la línea de comandos o a través de la herramienta basada en texto denominada Sconfig.

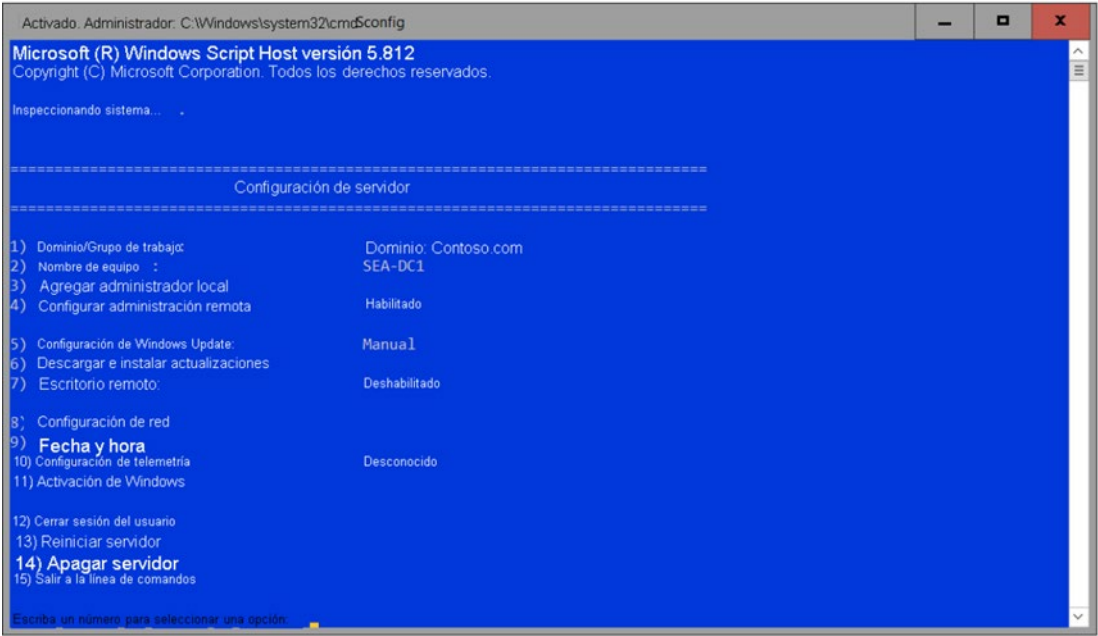


Figura 1. Configuración Windows Server Core con Sconfig. Fuente: elaboración propia.

La administración remota es el método normal de administración del servidor mediante el uso de varias herramientas, como la comunicación remota de PowerShell, la herramienta de **administración remota del servidor** (RSAT) y **Windows Admin Center**. En la siguiente tabla se describen las herramientas disponibles localmente.

Herramienta	Función
Cmd.exe	Permite ejecutar herramientas de línea de comandos tradicionales, como ping.exe, ipconfig.exe y netsh.exe.
PowerShell.exe	Inicia una sesión de Windows PowerShell en la implementación de Server Core. Puede ejecutar los cmdlets de PowerShell en una sesión de Windows PowerShell.
Regedt32.exe	Proporciona acceso a la herramienta de edición gráfica para el acceso al registro en el entorno de Server Core.
Msinfo32.exe	La herramienta de información del sistema le permite observar información del sistema acerca de la implementación de Server Core.
Sconfig.cmd	Actúa como una herramienta línea de comandos controlada por menús para realizar tareas comunes de administración del servidor.
Taskmgr.exe	Inicia el Administrador de tareas.
Notepad.exe	Abre un editor de texto gráfico.
Timedate.cpl	Abre el elemento de fecha y hora en el panel de control.
Cmd.exe	Permite ejecutar herramientas de línea de comandos tradicionales, como ping.exe, ipconfig.exe y netsh.exe.

Tabla 1. Herramientas Windows server. Fuente: elaboración propia.

Server Core tiene ventajas sobre Windows Server con Experiencia de escritorio y es la instalación recomendada para la mayoría de los escenarios, pero puede que no sea adecuado en todos los casos. En la tabla siguiente se enumeran las principales ventajas y desventajas.

Ventajas	Inconvenientes
Superficie pequeña que usa menos recursos del servidor y menos espacio en disco, tan solo 5 GB para una instalación básica.	No se pueden instalar varias aplicaciones en Server Core. Las aplicaciones incluyen System Center VM Manager 2019, System Center Data Protection Manager 2019, SharePoint Server 2019, Project Server 2019 y versiones de Exchange anteriores a Exchange 2019.
Dado que Server Core instala menos componentes, hay menos actualizaciones de software. Esto reduce el número de reinicios mensuales necesarios y el tiempo necesario para atender Server Core.	Varios roles y servicios de rol no están disponibles, como el host de sesión de Servicios de Escritorio remoto, Web Access y el servicio de puerta de enlace, servidor de fax, servidor SMTP y Windows PowerShell ISE.
La superficie de ataque pequeña hace que Server Core sea mucho menos vulnerable a los ataques.	No se pueden instalar muchas líneas de aplicaciones empresariales de proveedor en Server Core. Sin embargo, la característica de compatibilidad de aplicaciones a petición puede ayudar a mitigar esto en algunos casos.

Tabla 2. Ventajas e inconvenientes del Server Core. Fuente: elaboración propia.

Roles de servidor disponibles

Aunque algunos roles de servidor no están disponibles, en las implementaciones de Server Core están disponibles los siguientes:

Servicios de certificados de Active Directory.

AD DS.

Servidor DHCP.

Servidor DNS.

Servicios de archivo (incluido Administrador de recursos del servidor de archivos).

Active Directory Lightweight Directory Services (AD LDS).

Hyper-V.

Servicios de impresión y documentos.

Servicios de multimedia de transmisión por secuencias.

Servidor web (incluido un subconjunto de ASP.NET).

Servidor Windows Server Update.

Servidor de Active Directory Rights Management.

Servidor de acceso remoto.

Configuración posterior a la instalación

Después de instalar Windows Server, normalmente debe completar las siguientes acciones:

Configurar la dirección IP.

Establecer el nombre de equipo.

Unirse a un dominio de Active Directory.

Configure la zona horaria.

Habilitar las actualizaciones automáticas.

Agregar roles y características.

Habilitar la característica Escritorio remoto.

Configurar Firewall de Windows Defender.

Puede realizar esta configuración posterior a la instalación mediante Sconfig. También puede usar herramientas como Windows Admin Center y la comunicación remota de PowerShell para completar estas tareas en Server Core.

Nota

Aunque puede habilitar Escritorio remoto en Server Core, solo habilita la conectividad RDP con la interfaz de la línea de comandos.

1.3. Servicios de dominio de Active Directory

Tal y como afirma Microsoft (2022), un AD DS y sus servicios relacionados forman la base de las **redes empresariales** que ejecutan los sistemas operativos Windows. La base de datos de AD DS es el almacén central de todos los **objetos del dominio**, como cuentas de usuario, cuentas de equipo y grupos. AD DS proporciona un **directorio jerárquico** en el que se pueden realizar búsquedas y un **método** para aplicar ajustes de configuración y seguridad a los objetos de una empresa.

¿Qué son los componentes lógicos?

AD DS incluye componentes lógicos y físicos. Debe comprender cómo funcionan conjuntamente los componentes de AD DS a fin de administrar la infraestructura de forma eficaz. Además, puede usar las opciones de AD DS para realizar acciones como las siguientes:

- Instalación, configuración y actualización de aplicaciones.

- Administración de la infraestructura de seguridad.

- Habilitación del servicio de acceso remoto y DirectAccess.

- Emisión y administración de certificados digitales.

Los componentes lógicos de AD DS son estructuras que se usan para implementar un diseño de AD DS adecuado para una organización.

En la tabla siguiente se describen los tipos de componentes lógicos que contiene una base de datos de AD DS.

Componente lógico	Descripción
Partición	Una partición, o contexto de nomenclatura, es una parte de la base de datos de AD DS. Aunque la base de datos consta de un archivo denominado Ntds.dit, las diferentes particiones contienen datos diferentes. Por ejemplo, la partición de esquema contiene una copia del esquema de Active Directory. La partición de configuración contiene los objetos de configuración del bosque y la partición de dominio contiene los usuarios, equipos, grupos y otros objetos específicos del dominio. Active Directory almacena copias de las particiones en varios controladores de dominio y las actualiza a través de la replicación de directorios.
Schema	Un esquema es el conjunto de definiciones de los tipos de objeto y los atributos que se usan para definir los objetos creados en AD DS.
Domain	Un dominio es un contenedor administrativo lógico para objetos como usuarios y equipos. Un dominio se asigna a una partición específica y puede organizarlo con relaciones de principal-secundario respecto de otros dominios.
Árbol de dominios	Un árbol de dominios es una colección jerárquica de dominios que comparten un dominio raíz común y un espacio de nombres del sistema de nombres de dominio (DNS) contiguo.
Bosque	Un bosque es una colección de uno o varios dominios que tienen una raíz AD DS común, un esquema común y un catálogo global común.
OU	Una unidad organizativa es un objeto contenedor de usuarios, grupos y equipos que proporciona un marco para la delegación de derechos administrativos y la administración mediante la vinculación de objetos de directiva de grupo (GPO).
Contenedor	Un contenedor es un objeto que proporciona un marco de organización para su uso en AD DS. Puede usar los contenedores predeterminados o bien crear contenedores personalizados. No se pueden vincular GPO a contenedores.

Tabla 3. Componentes lógicos. Fuente: elaboración propia.

¿Qué son los componentes físicos?

AD DS incluye componentes lógicos y físicos. Debe comprender cómo funcionan conjuntamente.

Componente físico	Descripción
Controlador de dominio	Un controlador de dominio contiene una copia de la base de datos de AD DS. Para la mayoría de las operaciones, cada controlador de dominio puede procesar los cambios y replicarlos en todos los demás controladores de dominio del dominio.
Almacén de datos	Existe una copia del almacén de datos en cada controlador de dominio. La base de datos de AD DS usa la tecnología de base de datos Microsoft Jet y almacena la información de directorio en el archivo Ntds.dit y en los archivos de registro asociados. La carpeta C:\Windows\NTDS almacena estos archivos de forma predeterminada.
Servidor del catálogo global	Un servidor del catálogo global es un controlador de dominio que hospeda el catálogo global, que es una copia parcial de solo lectura de todos los objetos de un bosque de varios dominios. Un catálogo global acelera las búsquedas de objetos que pueden estar almacenados en controladores de un dominio diferente del bosque.
Controlador de dominio de solo lectura (RODC)	Un RODC es una instalación especial de solo lectura de AD DS. Los RODC son comunes en las sucursales donde la seguridad física no es óptima, el soporte de TI es menos avanzado que en los centros corporativos principales, o las aplicaciones de línea de negocio necesitan ejecutarse en un controlador de dominio.
Bosque	Un bosque es una colección de uno o varios dominios que tienen una raíz AD DS común, un esquema común y un catálogo global común.
OU	Una unidad organizativa es un objeto contenedor de usuarios, grupos y equipos que proporciona un marco para la delegación de derechos administrativos y la administración mediante la vinculación de objetos de directiva de grupo (GPO).
Contenedor	Un contenedor es un objeto que proporciona un marco de organización para su uso en AD DS. Puede usar los contenedores predeterminados o bien crear contenedores personalizados. No se pueden vincular GPO a contenedores.

Tabla 4. Componentes físicos. Fuente: elaboración propia.

Definición de usuarios, grupos y equipos

Además de los componentes y objetos de alto nivel, **AD DS** contiene **otros objetos** como usuarios, grupos y equipos.

En AD DS, debe proporcionar una cuenta de usuario a todos los usuarios que requieran **acceso a los recursos de red**. Con esta cuenta de usuario, los usuarios pueden autenticarse en el dominio de AD DS y acceder a los recursos de red.

En Windows Server, una cuenta de usuario es un objeto que contiene toda la información que define a un usuario. Una cuenta de usuario incluye:

- El nombre de usuario.

- Una contraseña de usuario.

- Pertenencias a grupos.

Una cuenta de usuario también contiene parámetros que se pueden configurar en función de los requisitos de la organización.

El nombre de usuario y la contraseña de una cuenta de usuario sirven como **credenciales de inicio de sesión** del usuario. Un objeto de usuario también incluye otros atributos que describen y administran el usuario. Puede usar lo siguiente para crear y administrar objetos de usuario en AD DS:

- Centro de administración de Active Directory.

- Usuarios y equipos de Active Directory.

- Windows Admin Center.

- Windows PowerShell.

- La herramienta de línea de comandos dsadd.

Aunque podría ser práctico asignar permisos y derechos a cuentas de usuario individuales en redes pequeñas, esto se vuelve poco práctico e ineficiente en redes empresariales grandes.

Por ejemplo, si varios usuarios necesitan el mismo nivel de acceso a una carpeta, resulta más eficaz crear un grupo que contenga las cuentas de usuario necesarias y, a continuación, asignar los permisos necesarios al grupo.

Sugerencia

Como ventaja adicional, puede cambiar los permisos de archivo de los usuarios si los agrega o los quita de los grupos en lugar de editar los permisos de archivo directamente.

Tal y como afirma Panek W. (2019), antes de implementar grupos en su organización, debe comprender el **ámbito** de varios tipos de grupos de AD DS. Además, debe entender cómo usar los tipos de grupo para administrar el acceso a los recursos o para asignar **derechos y responsabilidades** de administración.

Los **equipos**, al igual que los usuarios, son entidades de **seguridad**, en este sentido: Tienen una cuenta con un nombre de inicio de sesión y una contraseña que Windows cambia automáticamente de forma periódica.

Se autentican con el dominio.

Pueden pertenecer a grupos y tener acceso a los recursos, y puede configurarlos mediante una directiva de grupo.

Una **cuenta de equipo** comienza su ciclo de vida cuando crea el objeto de equipo y lo une a su dominio. Después de unir la cuenta de equipo a su dominio, las **tareas administrativas** cotidianas incluyen:

Configuración de las propiedades del equipo.

Traslado del equipo entre unidades organizativas.

Administración del propio equipo.

Cambio de nombre del objeto de equipo, así como su restablecimiento, deshabilitación, habilitación y finalmente eliminación.

Definición de usuarios, grupos y equipos

Un **bosque de AD DS** es una colección de uno o varios árboles de AD DS que contienen uno o varios dominios de AD DS. Dominios en un recurso compartido de bosque:

Una raíz común.

Un esquema común.

Un catálogo global.

Un **dominio de AD DS** es un contenedor administrativo lógico para objetos como:

Usuarios.

Grupos.

Computers.

Un **bosque** es un contenedor de nivel superior en AD DS. Cada bosque es una colección de uno o varios **árboles de dominios** que comparten un esquema de directorio común y un catálogo global. Un árbol de dominios es una colección de **uno o varios dominios** que comparten un espacio de nombres contiguo. El **dominio raíz** del bosque es el primer dominio que se crea en el bosque.

El dominio raíz del bosque contiene objetos que no existen en otros dominios del bosque. Dado que siempre se crean estos objetos en el **primer controlador de dominio**, un bosque puede estar formado por un solo dominio con un único controlador de dominio, o puede estar formado por varios dominios en varios árboles de dominio.

En el gráfico siguiente se muestra Contoso.com como dominio raíz del bosque. Debajo hay dos dominios, Adatum.com en un árbol independiente y Seattle.Contoso.com como un elemento secundario de Contoso.com.

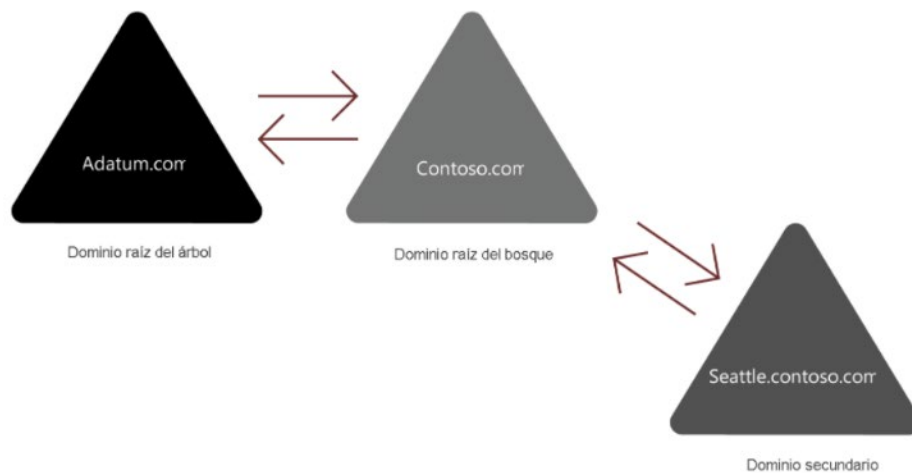


Figura 2. Definición de bosques y dominios. Fuente: Microsoft, 2022.

Los siguientes objetos existen en el dominio raíz del bosque:

- El rol de maestro de esquema.
- El rol de maestro de nombres de dominio.
- El grupo de administradores de empresa.
- El grupo de administradores de esquema.

¿Qué es un dominio de AD DS?

Un dominio de AD DS es un **contenedor lógico** para administrar usuarios, equipos, grupos y otros objetos. La base de datos de AD DS almacena todos los objetos de dominio y cada controlador de dominio almacena una copia de la base de datos.

En el gráfico siguiente se muestra un dominio de AD DS. Contiene usuarios, equipos y grupos.

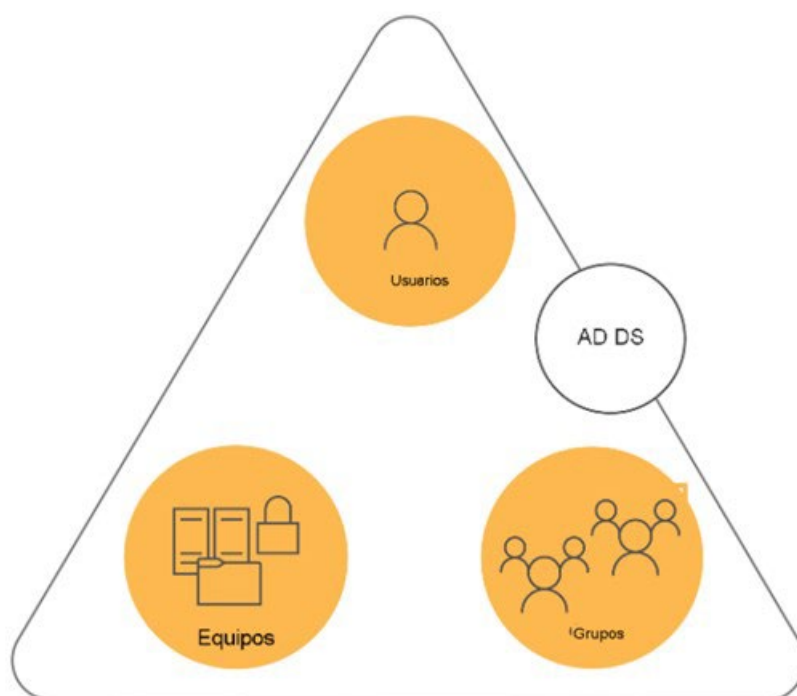


Figura 3. Dominios. Fuente: Microsoft, 2022.

Los objetos que se usan con más frecuencia se describen en la tabla siguiente:

Objeto	Descripción
Cuentas de usuario	Las cuentas de usuario contienen información sobre los usuarios, incluida la información necesaria para autenticar a un usuario durante el proceso de inicio de sesión y compilar el <i>token</i> de acceso del usuario.
Cuentas de equipo	Cada equipo unido a un dominio tiene una cuenta en AD DS. Puede usar cuentas de equipo para equipos unidos a un dominio de la misma manera que usa las cuentas de usuario para los usuarios.
Grupos	Los grupos organizan usuarios o equipos para simplificar la administración de permisos y los objetos de la directiva de grupo en el dominio.
Cuentas de usuario	Las cuentas de usuario contienen información sobre los usuarios, incluida la información necesaria para autenticar a un usuario durante el proceso de inicio de sesión y compilar el <i>token</i> de acceso del usuario.

Tabla 5. Objetos directorio activo. Fuente: elaboración propia.

Nota

AD DS permite que un único dominio contenga casi 2000 millones de objetos. Esto significa que la mayoría de las organizaciones solo necesitan implementar un único dominio.

Un dominio de AD DS suele describirse como:

Un **límite de replicación**. Al realizar cambios en cualquier objeto del dominio, el controlador de dominio en el que se produjo el cambio lo replica en todos los demás controladores de dominio del dominio. Si existen varios dominios en el bosque, solo se replican los subconjuntos de los cambios en otros dominios. AD DS usa un modelo de replicación de arquitectura multimaestro que permite a todos los controladores de dominio realizar cambios en los objetos del dominio.

Una **unidad administrativa**. El dominio de AD DS contiene una cuenta de administrador y una cuenta de administradores de grupo. De forma predeterminada, la cuenta de administrador forma parte del grupo de administradores de dominio y el grupo de administradores de dominio forma parte de todos los grupos de administradores locales de los equipos unidos a un dominio. Además, de forma predeterminada, los miembros del grupo de administradores de dominio tienen control total sobre todos los objetos del dominio.

Nota

La cuenta de administrador del dominio raíz del bosque tiene derechos adicionales.

Un dominio de AD DS proporciona:

Autenticación. Cada vez que se inicia un equipo unido a un dominio o un usuario inicia sesión en un equipo unido a un dominio AD DS lo autentica. La autenticación comprueba que el equipo o el usuario tiene la identidad adecuada en AD DS comprobando sus credenciales.

Autorización. Windows usa tecnologías de autorización y control de acceso para determinar si los usuarios autenticados pueden tener acceso a los recursos.

Sugerencia

Las organizaciones con estructuras administrativas descentralizadas o con múltiples ubicaciones podrían considerar la implementación de múltiples dominios en el mismo bosque para acomodar sus necesidades administrativas.

A continuación, en la lección magistral *Instalación de un directorio activo* se profundiza y se muestra el proceso de configuración de un entorno de red corporativo con un directorio activo configurado y sus herramientas de trabajo habituales.

En el siguiente vídeo *Instalación y configuración de un Windows Server* vamos a realizar una instalación de un sistema operativo Windows Server. También realizaremos la instalación y configuración de un directorio activo y veremos los pasos iniciales de gestión.



Accede al vídeo

1.4. Implementación objetos de directiva de grupo

Tal y como afirma Microsoft (2022), las primeras versiones de Windows Server, la característica directiva de grupo de los sistemas operativos Windows ha proporcionado una infraestructura con la que los administradores pueden definir la configuración de forma centralizada y, después, implementarla en los equipos de las organizaciones.

Por lo tanto, el personal de TI de las empresas puede definir, aplicar y actualizar toda su configuración mediante la configuración de los GPO. Al usar la configuración de los GPO, puede modificarse todo el sitio o un dominio de la organización, o bien limitar su enfoque a una sola unidad organizativa.

Sugerencia

El filtrado basado en los atributos de los equipos físicos y en la pertenencia a grupos de seguridad también permite a la empresa definir con más detalle el destino de su configuración de GPO.

¿Qué es una directiva de grupo?

Directiva de grupo es un **marco** de los sistemas operativos Windows con componentes que residen en AD DS, en controladores de dominio y en cada una de las instancias de servidor y cliente de Windows. Con estos componentes, puede **administrar** la configuración en un dominio de AD DS. **Define** la configuración de la directiva de grupo en un objeto de directiva de grupo (GPO).

Un **GPO** es un objeto que contiene una o más configuraciones de directiva que se aplican a una o varias opciones de configuración para un usuario o un equipo.

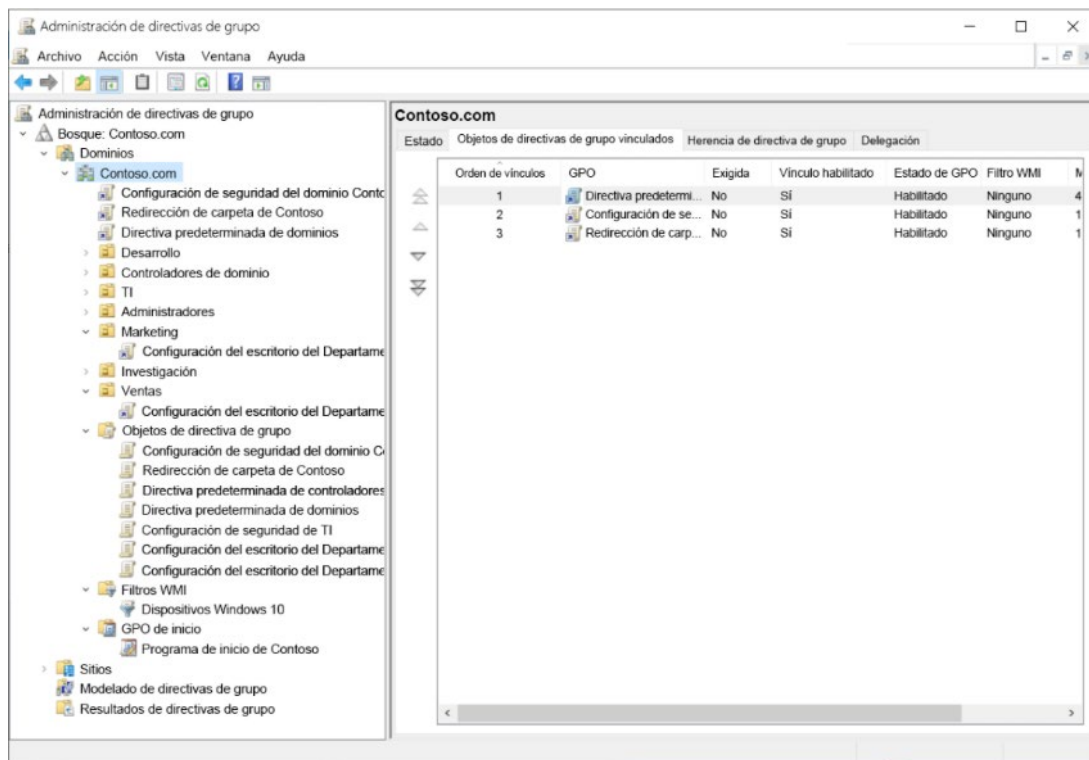


Figura 4. Directivas de grupo. Fuente: Microsoft, 2022.

Una directiva de grupo es una **herramienta administrativa** de gran eficacia. Los GPO se pueden usar para insertar diversas configuraciones en un gran número de usuarios y equipos. Dado que pueden aplicarse a distintos niveles, desde el equipo local a un dominio, las configuraciones también pueden enfocarse de forma precisa.

Una directiva de grupo se usa principalmente para **configurar los valores** que no quiere que configuren los usuarios. Además, puede usarla para **estandarizar** los entornos de escritorio en todos los equipos de una unidad organizativa (UO) o de toda una organización. También puede usar las directivas de grupo para proporcionar **seguridad adicional**, establecer algunas configuraciones avanzadas del sistema y para otros fines que se describen en una unidad de demostración posterior.

El componente más pormenorizado de una directiva de grupo es una configuración de directiva individual. Una configuración de directiva individual define una configuración específica, por ejemplo, una **configuración de directiva** que impide que un usuario acceda a las herramientas de edición del registro. Si define esa

configuración de directiva y, a continuación, la aplica a un usuario, este no podrá ejecutar herramientas como Regedit.exe.

Algunas configuraciones afectan a un usuario, lo que se conoce como parámetros de configuración de usuario o **directivas de usuario** y otras afectan al equipo, lo que se conoce como parámetros de configuración del equipo o **directivas de equipo**.

Importante

La configuración no afecta a los grupos directamente y solo se aplica a los objetos de usuario y de equipo.

Una directiva de grupo administra varias configuraciones de directiva y su marco es **extensible**. Con las directivas de grupo puede administrar prácticamente todos los valores configurables.

Para **definir** una configuración de directiva:

En el Editor de administración de directivas de grupo, busque la configuración de la directiva y, a continuación, seleccione Entrar. Se abre el cuadro de diálogo Propiedades de la configuración de directiva.

Cambie el estado de la directiva a Habilitada o Deshabilitada. La mayoría de las configuraciones de directiva pueden tener tres estados: No configurada, Habilitada y Deshabilitada.

Si es necesario, configure los valores adicionales y, cuando haya finalizado, seleccione Aceptar.

Los GPO almacenan la configuración de la directiva de grupo. En un GPO nuevo, el valor predeterminado de una configuración de directiva es No configurada. Al habilitar o deshabilitar una configuración de directiva, Windows Server modifica la configuración de los usuarios y los equipos a los que se aplica el GPO.

Nota

Al devolver una configuración al valor No configurada, esta se devuelve a su valor predeterminado.

Para crear un GPO en un dominio:

En Administración de directivas de grupo, haga clic con el botón derecho o acceda al menú contextual del contenedor Objetos de directiva de grupo y, a continuación, seleccione Nuevo.

Para modificar los valores de configuración de un GPO, haga clic con el botón derecho o acceda al menú contextual del GPO y, a continuación, seleccione Editar. Al hacerlo, se abre el complemento Editor de administración de directivas de grupo.

El Editor de administración de directivas de grupo muestra todas las configuraciones de directiva **disponibles** en un GPO en una jerarquía organizada, que comienza con la división entre la configuración del equipo y la configuración de usuario: el nodo **Configuración del equipo** y el nodo **Configuración de usuario**.

Los GPO se muestran en un contenedor denominado Objetos de directiva de grupo. Los dos niveles siguientes de la jerarquía son los nodos denominados **Directivas** y **Preferencias**. Al avanzar por la jerarquía, vemos que el Editor de administración de directivas de grupo muestra distintas carpetas, llamadas nodos o grupos de configuración de directivas. Las configuraciones de directiva se encuentran en las carpetas.

1.5. Servidor web, servidor de archivos y monitorización

¿Qué es un servidor web IIS?

Afirma Microsoft (2022) que las iniciales de **Internet Information Services**, aunque es más conocido como servidor web, en realidad son conjuntos de servicios que transforman un sistema Microsoft Windows en un servidor capaz de ofrecer servicios Web, FTP.

Por muchos años IIS fue un CD-ROM separado del Sistema Operativo denominado Option Pack, recién con Windows 2000 se incorporó al instalador del sistema como un paquete más instalable variando las limitaciones dependiendo de la versión de Windows.

Desde 2016 es un **servidor** que ha venido en crecimiento. Las estadísticas de Netcraft lo ubican como el servidor más utilizado como servidor web con cerca de 688 millones de instalaciones seguidos por Nginx con 358 millones y Apache con 313 millones.

Sin embargo, a la hora de ver la cuota total de sitios web activos en internet los números cambian y tanto Apache como Nginx superan a IIS como servidor en sitios web más activos donde cae al 10% de los sitios.

Es **apto** tanto como servidor web en **Internet** y en **Intranet**, siendo en Intranet o en servidores privados donde tiene mayor cuota debido al mercado empresarial que lo utilizan para aplicaciones internas en empresas de mediana y gran escala, ya que va de la mano de Visual Studio.NET muy arraigado en el sector privado empresarial.

¿Cómo maneja las peticiones el servidor IIS?

A diferencia de Apache y Nginx que dividen la carga de trabajo en diversos subprocesos, dependiendo de la configuración incluso un subproceso por cada petición, IIS opta por el **modelo de proceso único**, es decir que un solo proceso maneja todas las peticiones.

También, cabe decir que para determinadas operaciones especiales se delegan a **subprocesos**, es decir que el proceso principal hace uso de subproceso sin embargo solo delega una parte de la responsabilidad porque es siempre el proceso principal quien recibe y responde la petición.

Una de las **ventajas** más importantes de IIS es el **manejo de las peticiones** y como estas pueden ser atendidas de manera concurrente, sobre todo por la alta necesidad que tienen las aplicaciones de realizar peticiones al servidor para enviar y recibir datos, así mismo como importantes mejoras en la entrada y salida asíncrona de datos que permiten aumentar la performance de la aplicación.

Requisitos mínimos de IIS

Los requisitos de hardware mínimos son relativamente **bajos**, sin embargo, hay que entender que estos requisitos son para **instalar e iniciar los servicios** únicamente. A la hora de utilizarlo en un servidor en producción la potencia del procesador y la cantidad de memoria depende de la cantidad de tráfico, de la cantidad de usuarios y la carga de trabajo que se espera.

Para la versión de Windows Server en cuestión del procesador el requisito mínimo es un CPU de 1,4 Ghz, algo que cumple la mayoría de los CPU modernos, luego el rendimiento aumenta al aumentar la cantidad de MHz y núcleos de CPU.

Mínimo:

Procesador de 64 bits a 1,4 GHz.

Compatible con el conjunto de instrucciones x64.

Admite DEP y NX.

Admite CMPXCHG16b, LAHF/SAHF y PrefetchW.

Admite la traducción de direcciones de segundo nivel (EPT o NPT).

En lo referido a memoria si el servidor es instalado sin escritorio, es decir solo consola, la cantidad de memoria mínima requerida son 512MB mientras que con el escritorio el mínimo sube a 2GB mínimo. Obviamente, para un servidor que soporte clientes la cantidad de memoria debe ser incrementada de acuerdo con el tráfico, peticiones, usuarios y carga de trabajo esperadas.

Mínimo:

512 MB (2 GB para la opción de instalación Servidor con Experiencia de escritorio)

Tipo ECC (código de corrección de errores) o tecnología similar.

Características del Servidor Internet Information Services

La arquitectura de IIS es **modular** para permitir una gestión ordenada y la posibilidad de agregar funciones adicionales, brinda soporte para los varios protocolos: HTTP/HTTPS, FTP/FTPS, SMTP y NNTP, provee soporte SSL para Socket Layer y, adicionalmente, provee múltiples alternativas de autenticación como Kerberos entre otros.

Nativamente, el **lenguaje** dinámico soportado es ASP.NET, sin embargo, también es posible configurar para utilizar PHP, VBScript, Perl e incluso Java (Java Server Pages y Servlets) y prácticamente cualquier lenguaje como módulo adicional o a través de la interface CGI.

Además de los módulos para soportar lenguajes de programación y scripting IIS, está formado por otros módulos como pueden ser:

Módulos de seguridad: se utilizan para agregar capas de seguridad a las peticiones y proteger los datos. Un ejemplo claro son los métodos de autenticación que luego pueden ser implementados en las aplicaciones para la validación de usuarios y sesiones.

Módulos de contenido: se utilizan para la generación y envío de contenido al usuario, como pueden ser archivos estáticos o dinámico generados por aplicaciones .NET, PHP, PERL o CGI.

Módulos de compresión: son módulos que permiten la compresión de contenido. Existen dos módulos, uno para contenido dinámico y otro para contenido estático.

Módulos de almacenamiento caché: brinda soporte para caché de datos en memoria con el fin de mejorar el rendimiento de las aplicaciones reduciendo el tiempo y la latencia en tiempo de respuesta sobre todo en aplicaciones de alto tráfico.

Módulos de registro y diagnóstico: es un módulo que provee diferentes herramientas como por ejemplo de registro, que permite registro de actividad web como las peticiones. A su vez, también provee un monitor de solicitudes para supervisar el funcionamiento del servidor web y de las aplicaciones.

Instalación de Servidor IIS

Para instalar Internet Information Server en Windows Server se debe primero abrir el panel de **Administración del Servidor** y hacer clic en **Agregar roles y características**, tal como aparece en la imagen:



Figura 5. Instalación Internet Information Services. Fuente: elaboración propia.

¿Qué es un sistema de archivos?

Antes de que pueda almacenar datos en un volumen, primero debe **formatearlo**. Para ello, debe seleccionar el sistema de archivos que el volumen deberá usar. Hay varios sistemas de archivos disponibles, cada uno con ventajas y desventajas.

Un sistema de archivos ofrece una **variedad de características** que implementan el almacenamiento y recuperación de archivos en los dispositivos de almacenamiento. Permite organizar los archivos en una estructura jerárquica y controla tanto el formato como la convención de nomenclatura de estos. Los sistemas de archivos admiten una **amplia gama** de dispositivos de almacenamiento, incluidos los discos duros y los medios extraíbles.

Todos los sistemas de archivos disponibles en el sistema operativo Windows constan de los siguientes **componentes de almacenamiento**:

Archivos. Son una agrupación lógica de datos relacionados.

Directorios. Son una colección jerárquica de directorios y archivos.

Volúmenes. Son una colección de directorios y archivos.

¿Cuáles son las características distintivas de los tipos de sistemas de archivos de Windows Server?

Algunos de los **tipos de sistema de archivos** de Windows Server son:

Tabla de asignación de archivos (FAT), FAT32 y tabla de asignación de archivos extendida (exFAT).

Sistema de archivos NT (NTFS).

Sistema de archivos resistente (ReFS).

El sistema de archivos **FAT** es el **más sencillo** disponible en la compatibilidad con sistemas operativos Windows. Realiza un seguimiento de los objetos del sistema de archivos mediante una tabla de nivel del volumen. FAT conserva dos copias de la tabla para ofrecer resistencia. Tanto las dos tablas como el directorio raíz deben residir en una ubicación fija del disco con formato.

Debido a la **limitación de tamaño** de la tabla de asignación de archivos, no puede usar FAT para crear volúmenes mayores de 4 gigabytes (GB). Para alojar discos de mayor tamaño, Microsoft desarrolló **FAT32**, que admite particiones de hasta 64 GB.

exFAT es el sistema de archivos diseñado para las unidades flash USB, que incluye compatibilidad para volúmenes de **tamaños mayores** que los disponibles con FAT32. Funciona con los dispositivos multimedia, como TV de pantalla plana modernos, centros y reproductores multimedia portátiles.

Ni FAT, ni FAT32 ofrecen seguridad en el nivel de sistema de archivos. No debe crear volúmenes FAT o FAT32 en los discos conectados a los servidores que ejecuten cualquiera de los sistemas operativos Windows Server. Sin embargo, puede plantearse el uso de FAT, FAT32 o exFAT para dar formato a los **medios externos**, como las unidades **flash USB**.

Sugerencia

A partir de Windows Server 2016, FAT y FAT32 admiten el cifrado a través del Sistema de cifrado de archivos (EFS).

NTFS

Tradicionalmente, NTFS ha sido la opción de sistema de archivos **más común** para los sistemas operativos Windows Server. NTFS ofrece muchas **mejoras** con respecto a FAT, que aprovechan las estructuras de datos avanzadas para mejorar el rendimiento, la confiabilidad y el uso del espacio en disco. NTFS también ofrece **seguridad integrada**, con funciones de control de acceso como las listas de control de acceso (ACL), la auditoría, el registro en diario del sistema de archivos y el cifrado. NTFS también admite la compresión y el cifrado del sistema de archivos, aunque son mutuamente excluyentes. Por ello, no se pueden aplicar en el mismo archivo o carpeta.

Nota

NTFS es necesario cuando se implementan volúmenes en servidores que hospedan varios roles y características de Windows Server, como Active Directory Domain Services (AD DS), VSS y el Sistema de archivos distribuido (DFS).

ReFS

Microsoft lanzó ReFS en Windows Server 2012 para mejorar las capacidades de NTFS. Uno de los principales puntos fuertes de ReFS, como se indica en su nombre, es su **resistencia mejorada** a los datos dañados gracias a un mecanismo de detección más preciso y a una funcionalidad para corregir los problemas de integridad en línea. ReFS también ofrece **compatibilidad** con tamaños mayores de archivos y volúmenes individuales, incluida la de duplicación.

En la mayoría de los casos, ReFS es la opción de sistema de archivos óptima para los volúmenes de datos en Windows Server 2022. Sin embargo, debe tener en cuenta que ReFS no ofrece una paridad de características completa con NTFS. Por ejemplo, ReFS no admite la compresión y el cifrado en el nivel de archivo. Tampoco es la opción adecuada para los volúmenes de arranque y los medios extraíbles.

La instalación del servidor de ficheros la realizaremos como el resto de los roles desde el administrador de servidor, agregando el rol de archivos que podemos ver en la figura siguiente:

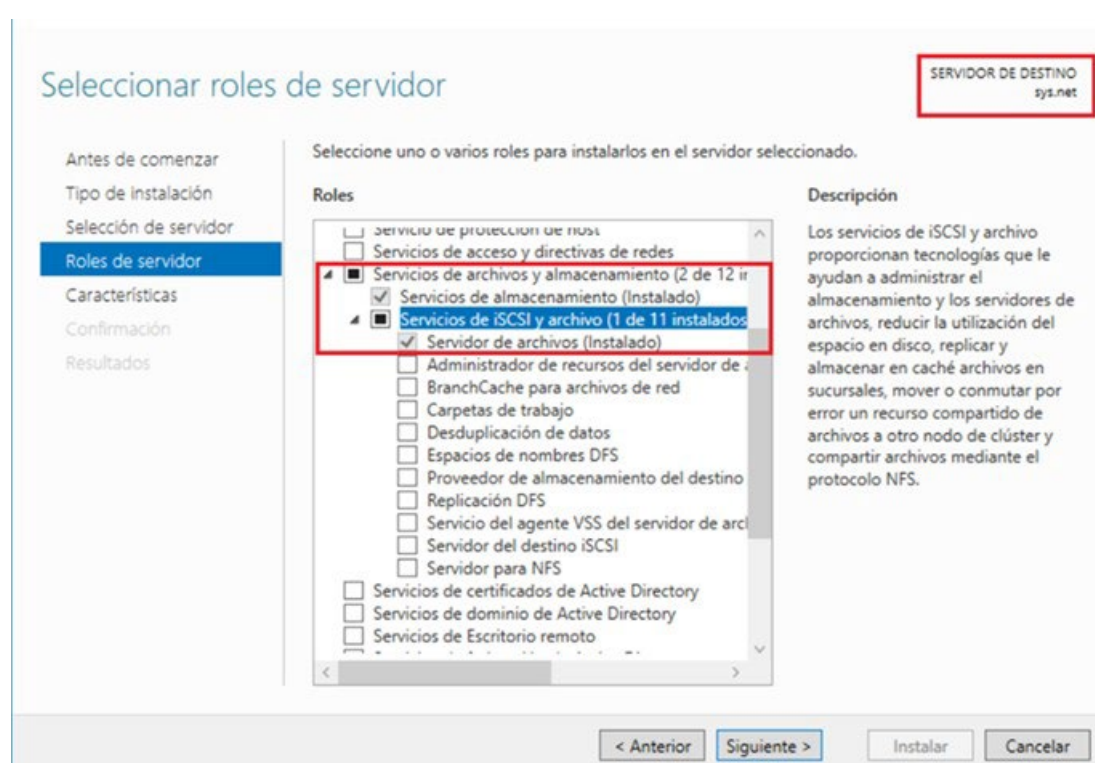


Figura 6. Instalación del rol servidor de archivos. Fuente: elaboración propia.

Monitorización

En el pasado, la administración del entorno de TI suponía la utilización de distintas herramientas en varias consolas. Windows Admin Center combina esas herramientas en una **única consola** que se puede implementar fácilmente y con acceso sencillo a través de una interfaz web.

Desde esta consola de administración es ahora mucho más **fácil la monitorización** de toda la plataforma Windows de los entornos corporativos.

Windows Admin Center es una **aplicación web modular** formada por los cuatro módulos siguientes:

Administrador del servidor.

Clústeres de conmutación por error.

Clústeres hiperconvergid.

Cientes de Windows 10.

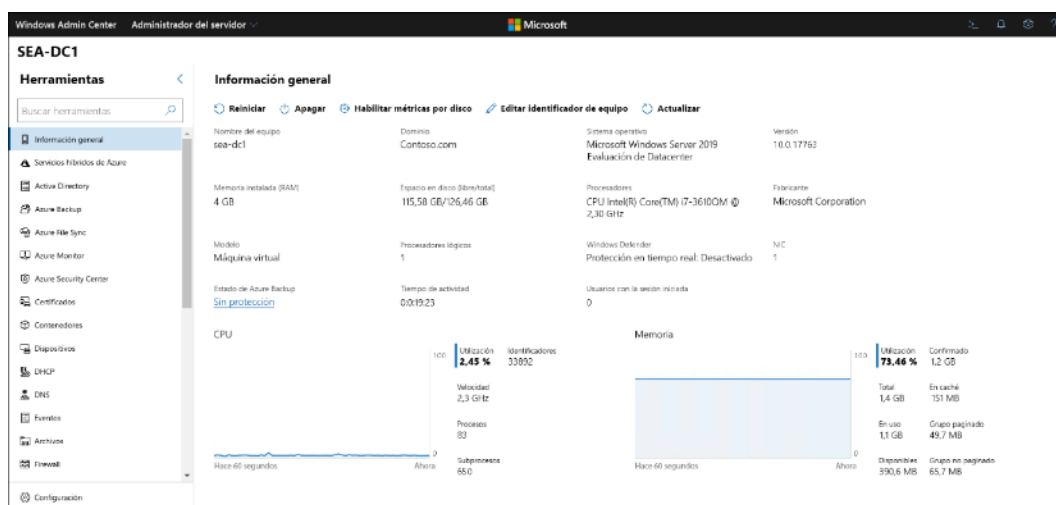


Figura 7. Consola de administración y monitorización: Windows Admin Center. Fuente: elaboración propia.

1.6. Infraestructura de red de Windows server

DHCP

Microsoft (2022) nos enseña que este servicio **configura automáticamente** los dispositivos de red con la información de configuración de dirección IP. Esto puede ayudar al personal de soporte técnico de TI de la empresa al simplificar y centralizar la asignación de las configuraciones de dirección IP. Si la empresa decide no usar DHCP, cada vez que agregue un cliente a su red deberá configurar su interfaz de red con información sobre la red a la que se va a conectar.

Sugerencia

La información que debe configurar incluye la dirección IP, la máscara de subred de la red, la configuración de DNS de cliente para la resolución de nombres y la puerta de enlace predeterminada para el acceso a otras redes.

Ventajas de DHCP

La principal ventaja de utilizar DHCP es **reducir el mantenimiento** necesario para configurar la información de dirección IP en los dispositivos de red. Muchas organizaciones administran miles de dispositivos informáticos, como impresoras, escáneres, *smartphones*, equipos de escritorio y portátiles. Por este motivo, no es práctico administrar manualmente las configuraciones de dirección IP de red para organizaciones de este tamaño.

Como DHCP es un **proceso automatizado**, es más preciso que configurar manualmente la información de dirección IP. Esto resulta especialmente importante para los usuarios que no conocen ni entienden el proceso de configuración.

DHCP **facilita la actualización** de la información de configuración de la dirección IP. Como administrador, cuando hace un cambio en el servicio de red, como proporcionar un servidor nuevo de sistema de nombres de dominio (DNS), solo realiza una actualización única en los servidores DHCP y ese cambio lo reciben todos los clientes de DHCP. Por ejemplo, un usuario móvil con un equipo portátil que usa DHCP obtiene automáticamente la información nueva de configuración de la dirección IP cuando se conecta a una red nueva.

Nota

De manera predeterminada, todos los sistemas operativos Windows recuperan automáticamente una dirección IP después de la instalación inicial del sistema operativo (SO).

Funcionamiento de DHCP

El servicio cliente DHCP se ejecuta en todos los equipos con Windows que tienen **propiedades TCP/IP** configuradas para obtener una dirección IP automáticamente. El cliente DHCP se comunica con un **servidor DHCP** para obtener la información de configuración de dirección IP. Los clientes pueden usar la dirección DHCP asignada durante un período determinado, lo que se conoce como **tiempo de la concesión**. El servidor DHCP está configurado con un grupo de direcciones y opciones de configuración. Esta información determina qué información de configuración de dirección IP se entrega a los clientes. En el gráfico siguiente se presenta el **proceso de comunicación** de cuatro pasos.

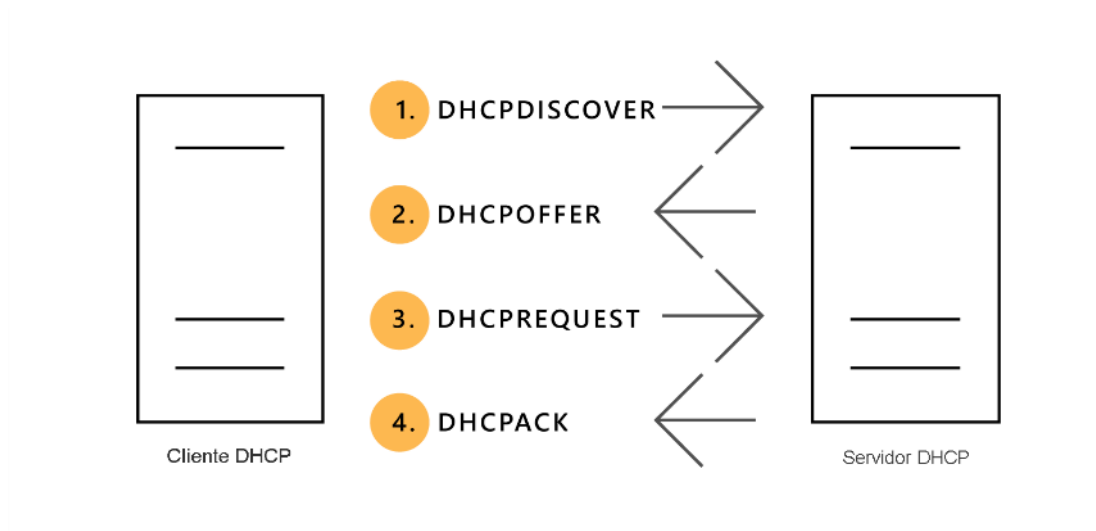


Figura 8. Proceso de comunicación DHCP. Fuente: Microsoft, 2022.

La comunicación para la generación de concesiones DHCP usa **difusiones de dirección IP**. Como las difusiones de dirección IP no se enrutan, debe configurar un **servidor DHCP** en cada subred o configurar una **retransmisión DHCP**. Muchos enrutadores incluyen la funcionalidad de retransmisión DHCP.

Los cuatro **pasos** de la **generación de concesiones** son los siguientes:

1. El cliente DHCP difunde un paquete DHCPDISCOVER. Los únicos equipos que responden son los que tienen el rol del servidor DHCP o los equipos o enrutadores que ejecutan un agente de retransmisión DHCP. En el último caso, el agente de retransmisión DHCP reenvía el mensaje al servidor DHCP que configuró para retransmitir las solicitudes.

Un servidor DHCP responde con un paquete DHCPOFFER, que contiene una dirección potencial para el cliente. Si varios servidores DHCP reciben el paquete DHCPDISCOVER, pueden responder varios servidores DHCP.

El cliente recibe el paquete DHCPOFFER. Si el cliente recibe varios paquetes DHCPOFFER, selecciona la primera respuesta. Después, el cliente envía un paquete DHCPREQUEST que contiene un identificador de servidor. Esto informa a los servidores DHCP que reciben la difusión del paquete DHCPOFFER del servidor que el cliente eligió aceptar.

Los servidores DHCP reciben el paquete DHCPREQUEST. Los servidores que el cliente no aceptó usan este mensaje como notificación de que el cliente rechazó la oferta del servidor. El servidor elegido almacena la información del cliente de la dirección IP en la base de datos DHCP y responde con un mensaje DHCPACK. Si el servidor DHCP no puede proporcionar la dirección ofrecida en el DHCPOFFER inicial, el servidor DHCP envía un mensaje DHCPNAK.

Renovación de la concesión de DHCP

Cuando la concesión de DHCP alcanza el cincuenta por ciento del tiempo de la concesión, el cliente intenta renovar la concesión automáticamente. Este proceso se realiza en segundo plano. Es posible que un equipo tenga la misma dirección IP asignada por DHCP durante mucho tiempo. Esto se debe a que el equipo renueva la concesión varias veces.

Para intentar renovar la concesión de la dirección IP, el cliente envía un mensaje DHCPREQUEST de unidifusión. El servidor que concedió originalmente la dirección IP

envía un mensaje DHCPACK de vuelta al cliente. Este mensaje contiene los parámetros nuevos que cambiaron desde que se creó la concesión original. Tenga en cuenta que estos paquetes no se difundirán porque, en este momento, el cliente tiene una dirección IP que puede usar para las comunicaciones de unidifusión.

Nota

Al actualizar las opciones de configuración de DHCP, es posible que los clientes no obtengan las opciones actualizadas hasta que se complete el cincuenta por ciento del tiempo de la concesión. Por ejemplo, si configura un tiempo de la concesión de seis días, es posible que los clientes no obtengan las opciones actualizadas durante tres días.

Si el cliente DHCP no puede ponerse en contacto con el servidor DHCP, el cliente espera hasta que expire el 87,5 por ciento del tiempo de la concesión. En este momento, el cliente envía una difusión DHCPREQUEST (en lugar de una unidifusión) para obtener una renovación y la solicitud va a todos los servidores DHCP, y no solo al servidor que proporcionó la concesión original. Sin embargo, esta solicitud de difusión es para una renovación, no para una concesión nueva.

Como los equipos cliente pueden moverse mientras están apagados (por ejemplo, un equipo portátil conectado a una subred nueva), también intentan realizar la renovación durante el proceso de inicio o cuando el equipo detecta un cambio de la red. Si la renovación se realiza correctamente, se restablece el período de la concesión.

Versión 6 de DHCP

La versión 6 de DHCP (DHCPv6) admite **configuraciones con estado y sin estado** para configurar clientes en un entorno IPv6. La configuración con estado ocurre cuando el servidor DHCPv6 asigna la dirección IPv6 al cliente, junto con datos DHCP adicionales. La configuración sin estado ocurre cuando el enrutador asigna la dirección IPv6

automáticamente y el servidor DHCPv6 solo asigna otras opciones de configuración de IPv6.

¿Qué es DNS?

DNS es un servicio de **resolución de nombres estándar** del sector. Los servidores que ejecutan el rol de servidor DNS responden a las solicitudes de los hosts y otros dispositivos de red para resolver nombres de equipo y nombres de host en direcciones IP. Los servidores DNS también pueden resolver direcciones IP en nombres. Los servidores DNS usan **registros de recursos DNS** almacenados en zonas DNS. Si un servidor DNS no puede resolver una consulta para un dispositivo de red, podría hacer referencia a otros servidores DNS mediante sugerencias de raíz o reenvíos.

Nota

El formato exacto de la zona depende del tipo de la zona.

Puede crear registros de recursos DNS manualmente, o bien los hosts IP y los dispositivos de red pueden realizar registros DNS automáticamente con un servidor DNS.

El uso más común de DNS consiste en resolver nombres de **dominio completos** (FQDN), como sea-dc1.contoso.com, en direcciones IP. Los usuarios requieren esta funcionalidad para acceder a recursos de red y sitios web.

Los administradores usan la resolución de nombres en DNS cuando configuran y administran aplicaciones, en parte porque los usuarios pueden recordar los nombres con mayor facilidad que las direcciones IP. Los servidores y clientes Windows unidos a un dominio también usan DNS para buscar controladores de dominio en un dominio de AD DS.

Nombres de dominio DNS

La estructura de nomenclatura que usa DNS es el espacio de nombres DNS. Es **jerárquico**; es decir que se inicia con un dominio raíz. Ese **dominio raíz** puede tener cualquier número de **subdominios**. A su vez, cada subdominio puede tener cualquier número de subdominios propios.

Los nombres de dominio pueden ser **públicos** (accesibles desde Internet) o **privados**. Si son privados, decida cómo definirá el espacio de nombres de su organización. Si son públicos, debe trabajar con la Corporación de Internet para la Asignación de Nombres y Números (ICANN) u otras autoridades de registro de nombres de Internet que puedan delegar (vender) nombres únicos. A partir de estos nombres, podrá crear los nombres de subdominio correspondientes.

Sugerencia

Para facilitar la obtención de certificados de confianza para las aplicaciones y la autenticación, se suele usar un nombre de dominio público que esté registrado en Internet.

Servidores DNS

Un servidor DNS responde a las **solicitudes de registros DNS** que realizan los solucionadores de DNS. Por ejemplo, un cliente de Windows 10 puede enviar una solicitud DNS para resolver sea-dc1.contoso.com a un servidor DNS, y la respuesta del servidor DNS incluye la dirección IP de sea-dc1.contoso.com.

Un servidor DNS recupera esta información desde una base de datos local que contiene **registros de recursos**. Si el servidor DNS no tiene la información solicitada, también puede reenviar las solicitudes DNS a otro servidor DNS. Un servidor DNS también **almacena en caché** la información solicitada previamente de otros servidores DNS.

Aunque puede configurar cualquier servidor de Windows como un servidor DNS, también es habitual instalar el rol del servidor DNS en cualquier equipo de Windows Server que se haya promovido a un controlador de dominio de AD DS.

Nota

Windows Server está configurado para ser un servidor DNS cuando se instala el rol del servidor DNS.

Registros de recursos y zonas DNS

Cuando un servidor DNS es responsable de resolver las solicitudes de un espacio de nombres específico, se crea una **zona en el servidor** DNS que corresponde a dicho espacio de nombres. Por ejemplo, si un servidor DNS es responsable de contoso.com, se crearía una zona contoso.com. Dentro de la zona, se crean registros de recursos que contienen la información que usará para responder a las consultas.

Solucionadores de DNS

Un solucionador de DNS es un **cliente**, como un cliente de Windows, que necesita resolver registros DNS. En Windows, el servicio de cliente DNS envía las solicitudes DNS al servidor DNS que está configurado en las propiedades de IP. Después de recibir respuesta a una solicitud de DNS, Windows almacena en caché la respuesta para uso futuro. Esta es la **memoria caché** del solucionador de DNS.

Nota

El equipo con Windows 10 comprueba la memoria caché de DNS antes de enviar solicitudes al servidor DNS en caso de que haya resuelto recientemente el registro necesario.

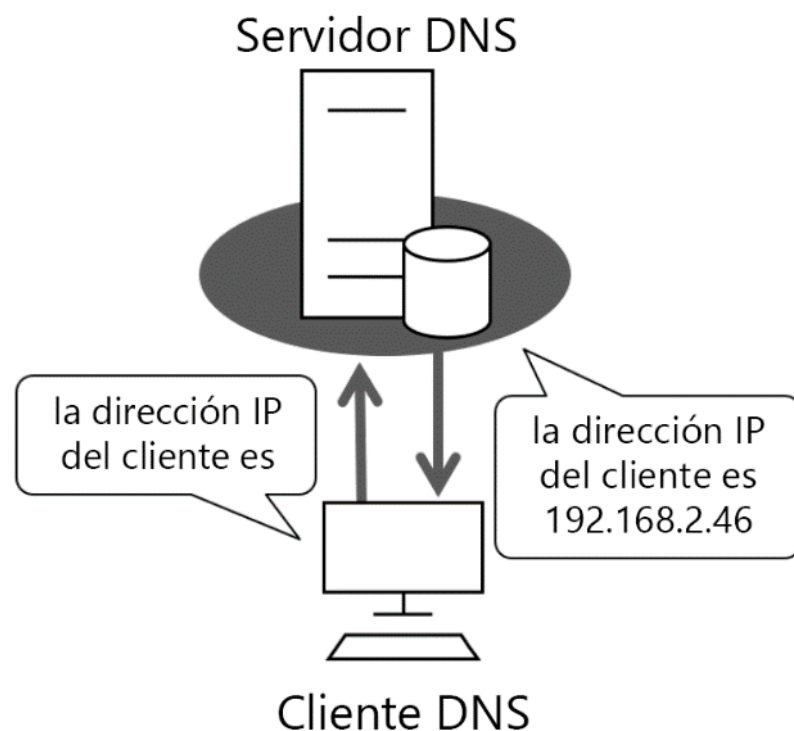


Figura 9. Cliente DNS. Fuente: Microsoft, 2022.

Sugerencia

Puede acceder al contenido de la memoria caché del solucionador de DNS mediante el cmdlet `Get-DnsClientCache` o la ejecución del comando `ipconfig /displaydns`. Puede borrar el contenido de la memoria caché del solucionador de DNS mediante el cmdlet `Clear-DnsClientCache` o el comando `ipconfig /flushdns`.

Es importante mencionar que el DNS no es la única manera en que los equipos host que ejecutan Windows pueden resolver nombres. Un equipo con **Windows 10** puede usar los **métodos** siguientes según la configuración:

Comprobar la memoria caché del solucionador de DNS.

Comprobar el archivo Hosts que se encuentra en `C:\Windows\System32\Drivers\etc`.

Enviar una solicitud al servidor DNS configurado.

Transmitir una consulta de nombre NETBIOS.

Comprobar el archivo LMHOSTS que se encuentra en C:\Windows\System32\Drivers\etc.

Usar un servicio de resolución de nombres NETBIOS configurado, como el Servicio de nombres de Internet de Windows (WINS).

Sugerencia

Puede instalar la característica de servidor WINS en Windows Server si su organización requiere la resolución de nombres NETBIOS para admitir una aplicación específica.

Nombres de equipo y nombres de host

Un nombre de host es un **nombre descriptivo** que está asociado a la **dirección IP** de un host y lo identifica como un **host TCP/IP**. Un nombre de host no puede superar los 255 caracteres y solo debe contener caracteres alfanuméricos, puntos y guiones. Un nombre de host es un alias o un nombre de dominio completo. Los elementos del nombre usan puntos como separadores. Las aplicaciones usan el nombre de dominio completo estructurado en Internet. payroll.contoso.com es un ejemplo de nombre de dominio completo.

Un nombre **NETBIOS** es un nombre **no jerárquico** que usan algunas **aplicaciones antiguas**. Un nombre NETBIOS de dieciséis caracteres identifica al recurso NETBIOS de una red. Dicho nombre representa un único equipo o un grupo de equipos. NETBIOS usa los quince primeros caracteres para el nombre de un equipo específico y el decimosexto y último carácter para identificar un recurso o servicio de ese equipo. Un ejemplo de nombre NETBIOS es SEA-SVR2.

1.7. Seguridad en redes Windows

Tal y como afirma Panek W. (2019), hay que instalar **parches de seguridad** y realizar **actualizaciones** cada cierto tiempo.

Normalmente, las vulnerabilidades o agujeros en la seguridad conocidos se vuelven a cerrar en un corto período de tiempo gracias a las actualizaciones. Sin embargo, para garantizar la efectividad de estas, es importante que esté al corriente de los parches de seguridad y las actualizaciones necesarias y las instale lo antes posible. Asegúrese de que se instalan también parches de seguridad y actualizaciones para sus complementos o plug-in. Puede establecer que las actualizaciones se instalen automáticamente en la configuración de Windows Update.

Nota

Si quiere, puede probar (previamente a la instalación en el servidor) los parches de seguridad y las actualizaciones de seguridad para sus aplicaciones y complementos y, así, ver los posibles efectos en su entorno. Tenga en cuenta que para dicha comprobación necesita un segundo servidor.

Utilice siempre contraseñas seguras

Las contraseñas débiles facilitan a los posibles atacantes el acceso a su servidor. En caso de ataque, el atacante podría usar su servidor para actividades maliciosas, usar los recursos de su servidor o, posiblemente, tomar el control del servidor y bloquearlo. Por eso le recomendamos utilizar siempre contraseñas seguras y complejas y cambiarlas regularmente.

Tenga en cuenta los siguientes **criterios** para crear una contraseña **sólida y segura**:

Utilice una contraseña distinta para cada servidor y para cada base de datos que se ejecute en el servidor.

Utilice una contraseña que no esté en los diccionarios.

Utilice siempre una contraseña que difiera significativamente de las contraseñas anteriores.

No utilice ningún dato de su entorno personal, como cumpleaños, nombres, etc.

No utilice una contraseña que contenga el nombre de usuario o el nombre de la empresa.

No comparta su contraseña con terceros.

Combine diferentes tipos de caracteres, como letras, números y caracteres especiales.

Una contraseña segura contiene:

Al menos ocho caracteres.

Letras mayúsculas y minúsculas: a-z, A-Z.

Números: 0-9.

Caracteres especiales.

Restrinja el acceso al servidor

El acceso al sistema solo debería concederse a los usuarios que trabajen activamente en él.

Establezca una rutina para las copias de seguridad

La **pérdida de datos** puede provocar daños graves y costosos. Por esta razón, le recomendamos establecer lo antes posible una rutina adecuada para realizar copias de seguridad.

Algunos de los **factores** que debe tener en cuenta son, por ejemplo:

Determinar la situación de riesgo: depende de la finalidad del servidor y de su dependencia con respecto a la base de datos.

Clasificar los datos: ¿qué tipo de datos se quieren almacenar? ¿Quiere realizar una copia de seguridad de los datos relevantes para el sistema o de los datos personales?

Disponer de datos: ¿qué aplicaciones dependen de los datos y en qué forma? ¿Pueden funcionar las aplicaciones también sin los datos que quiere almacenar?

Además, debe considerar las siguientes **cuestiones antes de establecer una rutina**:

¿Qué pérdida de datos es aceptable?

¿Cuánto tiempo se tardaría en restaurar los datos?

¿Qué importancia tiene el volumen de datos y cómo evolucionará el volumen de datos?

¿Cómo se debe hacer una copia de seguridad de los datos?

¿Existen plazos para la eliminación y el almacenamiento de datos?

¿Son confidenciales los datos? ¿Es necesaria una protección especial para acceder a estos? ¿Existen requisitos legales?

¿Cuándo se puede crear la copia de seguridad sin perjudicar a otros procesos?

¿Cuánto tiempo debe almacenar las copias de seguridad?

Otro punto importante que debe considerar al establecer su rutina es el **tipo de copia de seguridad** que quiere realizar. Se hace una distinción básica entre los siguientes tipos de copias de seguridad:

Copia de seguridad **completa**: una copia de seguridad completa contiene todos los datos seleccionados para la copia de seguridad.

Copia de seguridad **diferencial**: una copia de seguridad diferencial realiza una copia de seguridad de todos los archivos que han sido modificados o añadidos desde la última copia de seguridad completa. Los cambios se realizan siempre en relación con la copia de seguridad completa. Las copias de seguridad diferenciales aumentan día a día hasta que se vuelve a realizar una copia de seguridad

completa. Sin embargo, requieren menos espacio en disco que una copia de seguridad completa y pueden realizarse más rápido.

Para recuperar datos de una copia de seguridad diferencial, también debe tener acceso a la última copia de seguridad completa. Cada copia de seguridad diferencial se puede modificar independientemente del resto.

Copia de seguridad **incremental**: las copias de seguridad incrementales ahorran mucho espacio y se pueden realizar rápidamente.

Con una copia de seguridad incremental, solo se realiza una copia de seguridad de los datos que se han creado o modificado desde la última copia de seguridad. No importa si se trata de una copia de seguridad completa o incremental.

Por lo tanto, para restaurar una copia de seguridad incremental, usted también debe tener acceso a otras copias de seguridad realizadas previamente, ya que las copias de seguridad son interdependientes. Si elimina una de las copias de seguridad incrementales anteriores o una copia de seguridad completa, no podrá restaurar todo el paquete.

Además de desarrollar una rutina adecuada para las copias de seguridad y de crear copias de seguridad con regularidad, también debe asegurarse de comprobar con regularidad si las **copias de seguridad** se pueden **recuperar**.

Refuerce sus aplicaciones

Dependiendo de la aplicación instalada, existen diferentes riesgos y amenazas. Para protegerse contra estos riesgos, debe reforzar sus aplicaciones. A continuación, hemos recopilado algunos **consejos**:

Consulte la documentación pertinente o visite el sitio web del fabricante si necesita obtener más información sobre cómo instalar e implementar una aplicación de forma segura.

Lea y siga las mejores prácticas para instalar cada aplicación.

Realice una búsqueda de vulnerabilidades conocidas en su aplicación. Puede consultar la lista de información registrada sobre vulnerabilidades de seguridad conocidas, también conocida como Vulnerabilidades y exposiciones comunes (CVE®), para obtener más información. Aquí le dejamos el enlace a la página principal de CVE®.

Compruebe las vulnerabilidades de su servidor utilizando un programa como Nmap. Realice una prueba de penetración para identificar otras vulnerabilidades.

Instale solo aplicaciones necesarias.

Instale solo las aplicaciones que realmente necesite. Cuantas más aplicaciones instale en el servidor, mayor será el riesgo de vulnerabilidades.

Nota

Intente instalar aplicaciones solo de fuentes oficiales. Las aplicaciones de fuentes no oficiales pueden contener malware o virus.

Deshabilite o desinstale los servicios que no necesite

Dependiendo del sistema operativo utilizado y del tipo de instalación, también se descargan e instalan diferentes programas y servicios adicionales que, a menudo, no son necesarios. Por eso es importante que los identifique y los deshabilite desactivando las directrices del sistema correspondientes.

Abra solo los puertos que realmente necesite

Los puertos abiertos rara vez suponen un riesgo para la seguridad. Si reduce el número de aplicaciones de terceros, el número de puertos necesarios disminuirá y será más manejable. Estos puertos abiertos solo se convierten en un riesgo si los programas conectados o de terceros tienen vulnerabilidades de seguridad y los atacantes se aprovechan de ello. Este riesgo aumenta cuando el número de aplicaciones que instala en el servidor es mayor.

Monitoree su servidor

La monitorización es una **herramienta importante** para aumentar la **seguridad** de su servidor. Solo así se puede detectar a tiempo un fallo en el servidor o un fallo en los componentes o aplicaciones individuales. Esto también se aplica a ciertos tipos de ataques informáticos. Cuando le atacan, una respuesta rápida es esencial para detener el ataque y minimizar el daño causado.

Además, le recomendamos que compruebe regularmente las entradas del registro de seguridad de su servidor.

Compruebe regularmente si hay malware en su servidor

El malware, el ransomware o programa de software malicioso y los virus pueden causar daños considerables en su equipo. Asegúrese de tener instalados y activos un software antivirus y antispyware. Actualice regularmente sus firmas de antivirus y antimalware. Asegúrese también de que el software antivirus esté siempre activo y supervisando el tráfico de datos. Igualmente, le recomendamos realizar regularmente un análisis completo de los discos duros o unidades SSD.

Configure la política para bloquear las cuentas en uso durante los ataques

Esta configuración le permite defenderse contra ataques de fuerza bruta. En los ataques de fuerza bruta, los ciberdelincuentes intentan adivinar las contraseñas de una o más cuentas de usuario probando automáticamente diferentes combinaciones de letras y números. Cuantas más combinaciones se prueben, mayores serán las posibilidades de éxito del ataque.

Después del bloqueo, no se podrá acceder a las cuentas afectadas durante un cierto período de tiempo.

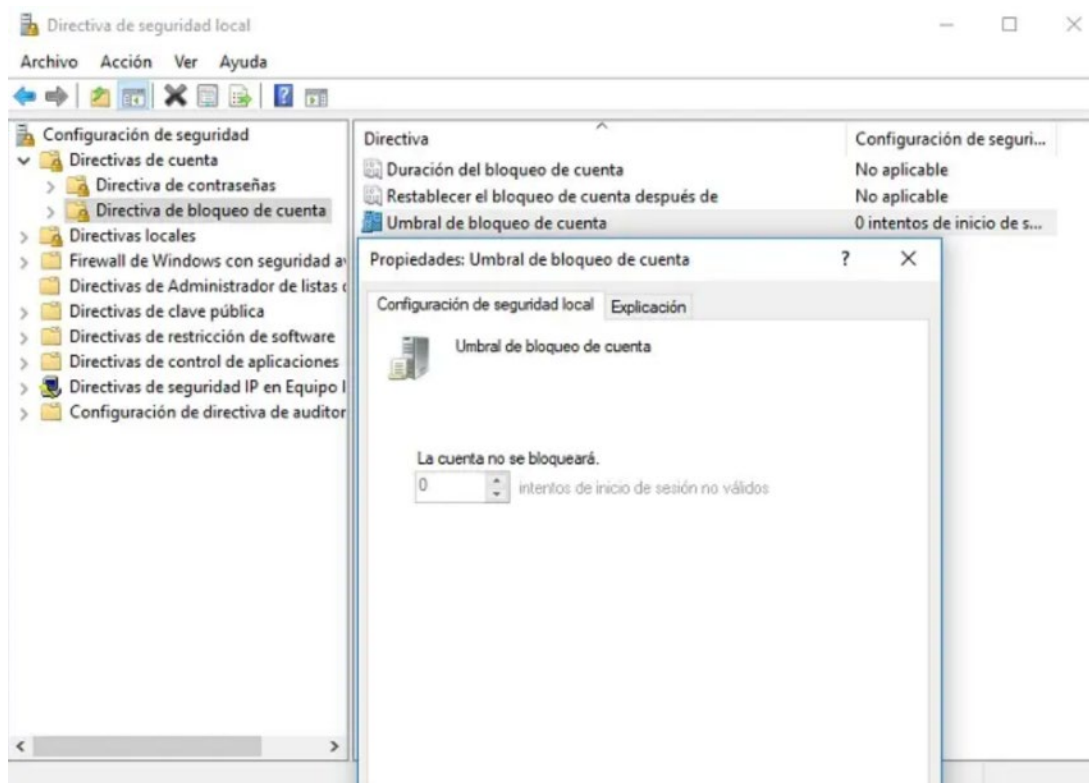


Figura 10. Opciones de seguridad en directiva de seguridad. Fuente: elaboración propia.

Use la herramienta Attack Surface Analyzer

Microsoft ha desarrollado una herramienta de software gratuita (Attack Surface Analyzer) que le permite analizar la superficie de ataque en busca de vulnerabilidades de seguridad. Este software muestra los cambios que se han producido en el sistema operativo después de instalar un nuevo software.

Puedes utilizar esta o cualquier otra herramienta de seguridad disponible para Windows. Revisar la sección «A fondo» para más información.

En este segundo vídeo, *Gestión de un directorio activo*, vamos a gestionar usuarios y máquinas de una red Windows Server y aprenderemos también a instalar roles y características nuevas dentro de nuestra red como pueda ser el servicio DHCP.



Accede al vídeo

1.8. Entornos híbridos

Hoy en día la instalación de Windows server en los centros de datos no corresponden a una instalación aislada. La demanda de integración con las nubes públicas provoca que la mayoría de las redes actuales se denominen redes híbridas, con una parte en los centros de datos locales y una conexión con la nube pública.

Por ello Windows server se integra de una forma nativa con la nube pública de Microsoft denominada Azure.

Azure Arc es un servicio que proporciona un conjunto de tecnologías para organizaciones que quieren simplificar sus entornos complejos y distribuidos. Proporciona un enfoque centralizado, unificado y de autoservicio para administrar lo siguiente:

Windows Server.

Servidores Linux.

Clústeres de Kubernetes.

Servicios de datos de Azure.

Azure Arc también amplía la adopción de procedimientos en la nube como DevOps y Azure Security en entornos locales, de varias nubes y perimetrales. Además de extender el plano de control para la administración, Azure Arc permite a las empresas ejecutar servicios de datos de Azure en cualquier lugar.

Se han realizado mejoras continuas en el controlador de tejido de Azure (el plano de control de Azure). Este plano de control es responsable de administrar el ciclo de vida de los recursos, como las VM, las instancias de bases de datos, los clústeres de Apache Hadoop y los clústeres de Kubernetes.

Por ejemplo, cada vez que la empresa aprovisiona, escala, detiene o finaliza un recurso, como una VM de Azure, el controlador de tejido de Azure procesa esta operación. Entre el controlador de tejido y los recursos hay otra capa denominada Azure Resource Manager, que automatiza el ciclo de vida de los recursos. Azure tiene proveedores de recursos para cada uno de estos tipos de recursos hospedados en Azure.

Nota

Azure Resource Manager ofrece una capa de administración que le permite crear, actualizar y eliminar recursos de su suscripción a Azure.

Funcionalidades de Azure Arc

Aunque Azure Arc tiene la posibilidad de modificar drásticamente la estrategia de administración en escenarios híbridos, actualmente está limitado a dos características disponibles para los sistemas registrados, como se describe en la tabla siguiente.

Características	Descripción
Configuración de invitado de Azure Policy.	Azure Policy admite la auditoría de recursos de Azure Arc para validar estos valores como configuraciones del sistema operativo (SO), aplicaciones y configuración del entorno. La única excepción a esto es la capacidad de configurar la zona horaria en el sistema operativo Windows. Al auditar los equipos de destino, tiene la opción de revisar los registros de forma local o remota mediante el comando Azure VM Run, disponible en Azure Portal.
Compatibilidad con el contexto de recursos: acceso a datos de Log Analytics.	El contexto de recursos le permite restringir el ámbito de acceso a los datos de Log Analytics en función de los permisos para el recurso de Azure correspondiente. Dado que Azure Arc presenta los equipos Windows locales individuales como recursos de Azure, puede controlar el acceso a sus respectivos registros si aplica permisos de RBAC a estos recursos.

Tabla 6. Funcionalidades Azure Arc. Fuente: elaboración propia.

1.9. Referencias bibliográficas

Microsoft (2022). *Uso de DHCP para simplificar la configuración de dirección IP.*

<https://docs.microsoft.com/es-es/learn/modules/deploy-manage-dynamic-host-configuration-protocol/2-use-dynamic-host-configuration-protocol-simplify-ip-configuration>

Microsoft. (2022). *Definición de bosques y dominios de AD DS.*

<https://docs.microsoft.com/es-es/learn/modules/introduction-to-ad-ds/4-define-forests-domains>

Microsoft. (2022). *Definición de bosques y dominios de AD DS.*

<https://docs.microsoft.com/es-es/learn/modules/introduction-to-ad-ds/4-define-forests-domains>

Microsoft. (2022). *Definición de objetos de directiva de grupo (GPO).*

<https://docs.microsoft.com/es-es/learn/modules/implement-group-policy-objects/2-define-gpos>

Microsoft. (2022). *Examen de la arquitectura de DNS.* <https://docs.microsoft.com/es-es/learn/modules/implement-windows-server-dns/2-explore-dns-architecture>

Microsoft. (2022). *Uso de DHCP para simplificar la configuración de dirección IP.*

<https://docs.microsoft.com/es-es/learn/modules/deploy-manage-dynamic-host-configuration-protocol/2-use-dynamic-host-configuration-protocol-simplify-ip-configuration>

Panek, W. (2019). *MCSA Windows Server 2019 Complete Study Guide.* Sybex.

Microsoft Learn

Microsoft. (2022). *Descubra su ruta de aprendizaje*. <http://docs.microsoft.com/es-es/learn/>

Centro de recursos multimedia de Microsoft donde puedes ampliar información sobre todos los temas de esta unidad.

Documentación Windows Server

Microsoft. (2022). *Documentación de Windows Server*. <http://docs.microsoft.com/es-es/windows-server/>

Centro de recursos propios sobre Windows Server. Entre los apartados principales tenemos: actualizar Windows Server, identidad y acceso, clústeres de conmutación por error, administración, redes, control remoto, seguridad y control, *storage* y virtualización.

Azure Arc

Microsoft. (2022). *Azure Arc*. <https://azure.microsoft.com/es-es/services/azure-arc/>

Azure Arc ofrece una administración simplificada, un desarrollo de aplicaciones más rápido y servicios de Azure coherentes. Organice, controle y proteja fácilmente los servidores Windows y Linux, SQL Server y los clústeres de Kubernetes de los centros de datos, el perímetro y los entornos multinube en Azure. Diseñe y compile aplicaciones nativas de nube en cualquier lugar sin sacrificar la visibilidad y el control centralizados. Obtenga las ventajas de innovación y la nube de Azure mediante la implementación de servicios coherentes de datos, aplicaciones y aprendizaje automático de Azure en cualquier infraestructura.

¿Qué versión de Windows no tiene interfaz gráfico?

- A. Standard y Datacenter GUI.
- B. Essentials Core.
- ☐ C. Standard y Datacenter Core.
- D. Ninguna de las anteriores.

Server Core es una instalación de Windows Server sin la Experiencia de escritorio. Server Core está disponible para las ediciones Standard y Datacenter, pero no está disponible para Windows Server Essentials y la versión gratuita de Hyper-V Server solo está disponible como una instalación de Server Core.

¿Qué herramienta actúa como una línea de comandos controlada por menús?

- A. Msinfo32.exe.
- ☐ B. Sconfig.
- C. Taskbar.
- D. CMD.

Actúa como una herramienta línea de comandos controlada por menús para realizar tareas comunes de administración del servidor.

¿Qué tipo de componentes tiene un directorio activo?

- ☐ A. Componentes lógicos y físicos.
- B. Componentes virtuales.
- C. Componentes lógicos.
- D. Componentes híbridos.

AD DS incluye componentes lógicos y físicos. Debe comprender cómo funcionan conjuntamente los componentes de AD DS a fin de administrar la infraestructura de forma eficaz.

¿Qué es un bosque en un sistema Windows?

- ☐ A. Un bosque es una colección de varios dominios.
- ☐ B. Un bosque no existe en sistemas Windows.
- ☐ C. Un bosque es lo mismo que un dominio.
- ☐ D. Un bosque es una relación de confianza entre dominios.

Cada bosque es una colección de uno o varios árboles de dominios que comparten un esquema de directorio común y un catálogo global.

¿Para qué sirve una directiva de grupo?

- ☐ A. Para crear usuarios en un dominio.
- ☐ B. Para monitorizar recursos de un dominio.
- ☐ C. Para crear un dominio y un bosque.
- ☐ D. Para administrar la configuración de un dominio.

Directiva de grupo es un marco de los sistemas operativos Windows con componentes que residen en AD DS, en controladores de dominio y en cada una de las instancias de servidor y cliente de Windows. Con estos componentes puede administrar la configuración en un dominio de AD DS.

¿Qué protocolos puedo usar en un IIS?

- ☐ A. HTTP/HTTPS, FTP.
- ☐ B. HTTP/HTTPS, FTP/FTPS, SMTP y NNTP.
- ☐ C. Solo HTTP y HTTPS.
- ☐ D. Solo FTP.

La arquitectura de IIS es modular para permitir una gestión ordenada y la posibilidad de agregar funciones adicionales, brinda soporte para los varios protocolos: HTTP/HTTPS, FTP/FTPS, SMTP y NNTP, provee soporte SSL para Socket Layer, y, adicionalmente, provee múltiples alternativas de autenticación como Kerberos, entre otros.

Windows Admin Center es:

- A. Un sistema operativo Windows para empresas pequeñas.
- B. Un sistema de nube privada.
- ☐ C. Una consola de administración y monitorización.
- D. Una consola de infraestructura de red.

En el pasado, la administración del entorno de TI suponía la utilización de distintas herramientas en varias consolas. Windows Admin Center combina esas herramientas en una única consola que se puede implementar fácilmente y con acceso sencillo a través de una interfaz web.

¿Qué hace un servidor DNS?

- A. Nos sirve direcciones IPs a las redes empresariales.
- B. Nos mantiene usuarios en directorio activo.
- C. Nos saca a Internet.
- ☐ D. Responde a las solicitudes del host.

DNS es un servicio de resolución de nombres estándar del sector. Los servidores que ejecutan el rol de servidor DNS responden a las solicitudes de los hosts y otros dispositivos de red para resolver nombres de equipo y nombres de host en direcciones IP.

¿Qué herramienta puedes usar para buscar vulnerabilidades en tu red Windows?

- A. Foca.
- ☐ B. Attack Surface Analyzer.
- C. Solo las herramientas open-source certificadas por Microsoft.
- D. Solo las certificadas por Microsoft.

Microsoft ha desarrollado una herramienta de software gratuita (Attack Surface Analyzer) que le permite analizar la superficie de ataque en busca de vulnerabilidades de seguridad.

¿Qué es Azure Arc?

- A. Una nueva versión de Windows.
- B. No existe este producto.
- C. Sirve para integrar Windows y Linux en una empresa.
- _ D. Una herramienta para entornos híbridos.

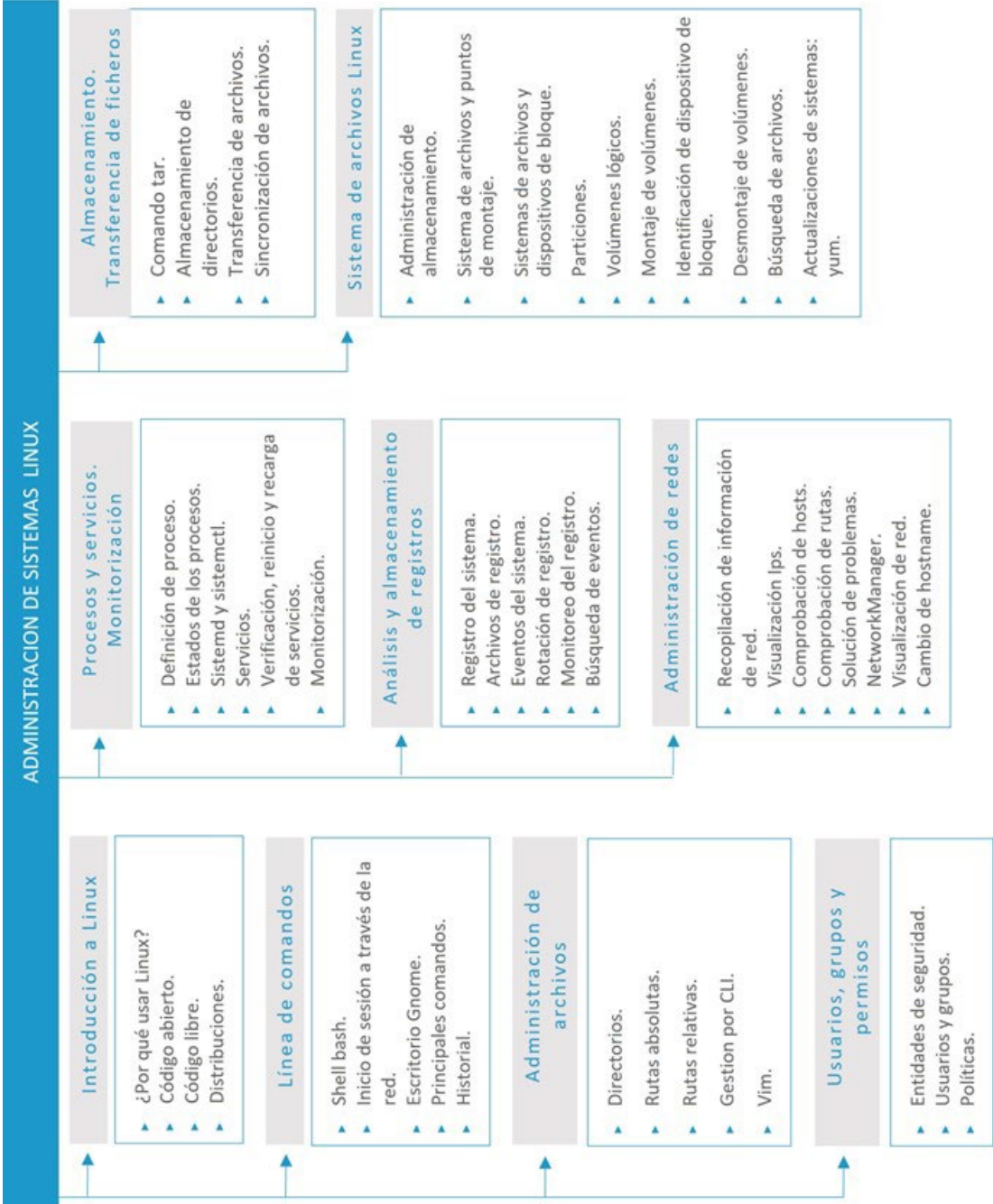
Azure Arc es un servicio que proporciona un conjunto de tecnologías para organizaciones que quieren simplificar sus entornos complejos y distribuidos.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Administración de sistemas Linux

Índice

Esquema	3
Ideas clave	4
2.1. Introducción y objetivos	4
2.2. Primeros pasos en Linux	5
2.3. Introducción a línea de comandos	12
2.4. Administración de archivos desde línea de comandos	23
2.5. Usuarios, grupos y permisos	31
2.6. Procesos, servicios y daemons. Monitorización	41
2.7. Análisis y almacenamiento de registros	50
2.8. Administración de redes	56
2.9. Almacenamiento y transferencia de archivos	65
2.10. Acceso a los sistemas de archivos de Linux.	
Actualizaciones	72
2.11. Referencias bibliográficas	91
A fondo	92
Test	94



2.1. Introducción y objetivos

En este tema vamos a estudiar los **conceptos** asociados al sistema operativo **Linux**. Si bien hay muchas distribuciones y versiones se va a ver un repaso general del sistema para adquirir los conocimientos base de este sistema operativo que vamos a usar a lo largo del resto de temas de esta asignatura.

Empezaremos con un **repaso teórico** sobre Linux y su **nacimiento** e iremos viendo todas las **áreas importantes** para un administrador de sistemas o un operado de sistemas como son gestión de usuarios, seguridad, redes, almacenamiento, Shell, comandos principales, procesos, daemons y sistema.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Obtener la habilidad suficiente para realizar las tareas principales de administración de sistemas en Linux.

Desarrollar las habilidades necesarias para un administrador de sistemas Linux.

En los **vídeos** de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Instalación en un entorno virtualizado de una distribución de Linux Red Hat. Veremos todas las opciones de configuración y realizaremos el proceso completo para tener un sistema operativo Linux empresarial instalado.

Vídeo 2. Comandos Linux. Aprenderemos más de 30 comandos importantes para administradores de sistemas.

2.2. Primeros pasos en Linux

¿Por qué debería aprender sobre Linux?

Según RedHat (2022), es muy importante que los profesionales de TI entiendan la tecnología de Linux.

El **uso** de Linux está **generalizado**; si usa Internet, probablemente ya esté interactuando con sistemas Linux en su vida diaria. Quizás la forma más obvia en que interactúa con sistemas Linux es al navegar por la World Wide Web o al usar sitios de comercio electrónico para comprar y vender productos.

Sin embargo, Linux se usa para mucho más que eso. Linux **administra los sistemas** de punto de venta y los mercados de valores del mundo y, también, se usa en los televisores inteligentes y en los sistemas de entretenimiento durante el vuelo. Se usa en la mayoría de las quinientas supercomputadoras más importantes del mundo.

Linux proporciona las **tecnologías fundamentales** que impulsan la revolución de la nube y las herramientas utilizadas para desarrollar la próxima generación de aplicaciones de microservicios basadas en contenedores, tecnologías de almacenamiento basado en software y soluciones de grandes volúmenes de datos.

En el centro de **datos modernos**, Linux y Microsoft Windows son los principales actores y, además, Linux es un segmento en crecimiento en ese espacio. Algunas de las muchas razones por las que debe aprender a usar Linux incluyen:

Un usuario de Windows necesita interoperar con Linux.

En el desarrollo de aplicaciones, Linux aloja la aplicación o su tiempo de ejecución.

En la computación en nube, las instancias de la nube en el entorno de nube privada o pública usan Linux como sistema operativo.

Con las aplicaciones móviles o Internet de las cosas (IoT), hay muchas posibilidades de que el sistema operativo de su dispositivo use Linux.

Si está buscando nuevas oportunidades en TI, hay mucha demanda de habilidades de Linux.

¿Qué hace que Linux sea excelente?

Hay muchas respuestas diferentes a la pregunta «¿Qué hace que Linux sea excelente?». Tres de ellas son:

Linux es software de **código abierto**.

Al ser de código abierto, no solo se puede ver cómo funciona el sistema, sino también se pueden probar cambios y compartirlos libremente para que otros los utilicen. El modelo de código abierto facilita el desarrollo de mejoras, lo que acelera la innovación.

Linux **proporciona fácil acceso** a una interfaz de línea de comandos (CLI) potente y programable.

Linux se construyó alrededor de la filosofía de diseño básica de que los usuarios pueden realizar todas las tareas de administración desde la CLI. Esto facilita la automatización, la implementación y el aprovisionamiento, así como simplifica la administración de sistemas locales y remotos. A diferencia de otros sistemas operativos, estas capacidades se han incorporado desde el principio y la idea ha sido siempre permitir estas importantes capacidades.

Linux es un **sistema operativo modular** que hace posible reemplazar o eliminar componentes fácilmente.

Los componentes del sistema se pueden actualizar y mejorar según sea necesario.

Un sistema Linux puede ser una estación de trabajo de desarrollo de uso general o un dispositivo de software en su mínima expresión.

¿Qué es el software de código abierto?

El software de código abierto es software con **código fuente** que cualquiera puede usar, estudiar, modificar y compartir.

El código fuente es el conjunto de instrucciones en lenguaje humano que se usan para hacer un programa.

Puede interpretarse como un **script** o compilarse en un archivo ejecutable binario que la computadora ejecuta directamente. Al crear el código fuente, se le aplican los derechos de autor y su titular controla los términos bajo los cuales se puede copiar, adaptar y distribuir el software. Los usuarios pueden usar este software bajo una licencia de software.

Otros tipos de software tienen un código fuente que solo la persona, el equipo o la organización que lo crearon pueden verlo, cambiarlo o distribuirlo. Este software a veces se denomina **software «patentado»** o de **«código cerrado»**. Por lo general, la licencia solo permite que el usuario final ejecute el programa y no proporciona acceso al código fuente o brinda acceso muy limitado a este.

El software de código abierto es diferente. Cuando el titular de los derechos de autor proporciona software con una licencia de código abierto, otorga a los usuarios el **derecho de ejecutar el programa** y también de ver, modificar, compilar y redistribuir el código fuente a otros sin pagar regalías.

El código abierto promueve la colaboración, el intercambio, la transparencia y la innovación rápida, ya que alienta a otras personas, más allá de los desarrolladores originales, a realizar modificaciones y mejoras en el software, así como a compartirlo con los demás.

El hecho de que el software sea de código abierto no significa que de alguna manera no se pueda usar o proporcionar para fines comerciales. El código abierto es una parte crítica de las **operaciones comerciales** de muchas organizaciones. Algunas

licencias de código abierto permiten que el código se reutilice en productos de código cerrado.

Se puede vender el código fuente abierto, pero los términos de las verdaderas **licencias de código abierto**, generalmente, permiten que el cliente redistribuya el código fuente. Más comúnmente, los proveedores como Red Hat proporcionan ayuda comercial con la implementación, el soporte y la ampliación de soluciones basadas en productos de código abierto.

El código abierto tiene muchos **beneficios** para el usuario:

Control: vea lo que hace el código y cámbielo para mejorarlo.

Formación: aprenda del código real y desarrolle aplicaciones más útiles.

Seguridad: inspeccione el código confidencial y corríjalo ya sea con la ayuda de los desarrolladores originales o sin ella.

Estabilidad: el código puede sobrevivir a la pérdida del desarrollador o distribuidor original.

La conclusión es que el código abierto permite la creación de mejor software con un mayor retorno de la inversión mediante la colaboración.

Tipos de licencias de código abierto

Hay más de una forma de proporcionar software de código abierto. Los términos de la licencia de software controlan cómo puede reutilizarse el código fuente o cómo puede combinarse con otro código y existen cientos de licencias de código abierto diferentes. Sin embargo, para ser de código abierto, las licencias deben permitir que los usuarios usen, vean, cambien, compilen y distribuyan libremente el código.

Hay **dos clases** amplias de licencias de código abierto que son particularmente importantes:

Las licencias **copyleft**, que están diseñadas para fomentar que el código se mantenga abierto.

Las licencias **permisivas**, que están diseñadas para maximizar la reutilización del código.



Figura 1. Símbolo licencias con copyleft. Fuente: Wikipedia, 2022.

Las licencias **copyleft** o de «uso compartido para todos» requieren que cualquier persona que distribuya el código fuente, con o sin cambios, también deba extender esta libertad para que los demás también puedan copiar, cambiar y distribuir el código. La **ventaja** básica de estas licencias es que ayudan a que el código existente y las mejoras a ese código sigan siendo de código abierto, para **aumentar** la cantidad de **código fuente abierto disponible**. Las licencias copyleft más comunes son la Licencia Pública General de GNU (GNU General Public License, GPL) y la Licencia Pública General Reducida de GNU (GNU Lesser General Public License, LGPL).

Las licencias **permisivas** están diseñadas para maximizar la reutilización del código fuente. Los usuarios pueden usar el código fuente para cualquier propósito siempre que se conserven las declaraciones de derechos de autor y de licencia, incluso al reutilizar ese código bajo licencias más restrictivas o incluso de propiedad exclusiva. Esto simplifica mucho la reutilización de este código, pero con el riesgo de **incentivar mejoras** de propiedad exclusiva. Algunas licencias permisivas de código abierto de uso común son la licencia **MIT/X11**, la licencia **BSD simplificada** y la licencia de software **Apache 2.0**.

¿Quién desarrolla el software de código abierto?

Es un error común pensar que el código abierto lo desarrolla únicamente un «ejército de voluntarios». En la actualidad, el desarrollo de código abierto es una actividad mayoritariamente **profesional**. Muchas organizaciones pagan a los desarrolladores para que trabajen en proyectos de código abierto para desarrollar y contribuir con las mejoras que ellos y sus clientes necesitan.

Los **voluntarios** y la **comunidad académica** desempeñan un papel importante y pueden hacer contribuciones vitales, en especial en áreas de tecnologías nuevas. La combinación de desarrollo formal e informal proporciona un entorno altamente dinámico y productivo.

¿Qué es una distribución de Linux?

Una distribución de Linux es un sistema operativo instalable construido a partir de un **kernel de Linux**, que admite librerías y programas de usuario. Un sistema operativo Linux completo no está desarrollado por una sola organización, sino por un conjunto de **comunidades independientes** de desarrollo de código abierto que trabajan con componentes individuales de software. Una distribución proporciona una manera fácil para que los usuarios instalen y administren un sistema Linux que funcione.

En 1991, un joven estudiante de informática llamado **Linus Torvalds** desarrolló un **kernel** tipo Unix y lo denominó **Linux**, con licencia de software de código abierto bajo la GPL. El kernel es el **componente central** del sistema operativo, que administra el hardware, la memoria y la programación de los programas en ejecución.

A continuación, este kernel de Linux podría complementarse con otro software de código abierto, como utilidades y programas del Proyecto GNU, la interfaz gráfica de X Window System de MIT y muchos otros componentes de código abierto, como el

servidor de correo Sendmail o el servidor web Apache HTTP, para desarrollar un sistema operativo completo de código abierto tipo Unix.

Sin embargo, uno de los desafíos para los usuarios de Linux era reunir todas estas piezas de muchas fuentes diferentes. Muy temprano en su historia, los desarrolladores de Linux comenzaron a trabajar para proporcionar una distribución de herramientas precompiladas y probadas que los usuarios pudieran descargar y usar para configurar los sistemas Linux rápidamente.

Existen muchas **distribuciones** de Linux diferentes, con diferentes objetivos y criterios tanto para seleccionar como para apoyar el software proporcionado por su distribución. Sin embargo, las distribuciones generalmente tienen muchas **características comunes**:

Las distribuciones constan de un kernel de Linux y programas de espacio de usuario compatibles.

Las distribuciones pueden ser pequeñas y de propósito único o incluir miles de programas de código abierto.

Las distribuciones deben proporcionar un medio para instalar y actualizar la distribución y sus componentes.

El proveedor de la distribución debe admitir ese software, e idealmente, debe participar directamente en la comunidad que lo desarrolla.



Figura 2. Existen múltiples distribuciones Linux. Fuente: Redes zone, 2015.

2.3. Introducción a línea de comandos

Introducción a la Shell Bash

RedHat (2022) afirma que una línea de comandos es una interfaz basada en texto que puede usarse para introducir instrucciones en un sistema de cómputo. Un programa denominado shell proporciona la **línea de comandos** de Linux. Se han desarrollado diversas opciones para el programa de shell a lo largo de los años y pueden configurarse diferentes usuarios para que usen diferentes shells. Sin embargo, la mayoría de los usuarios usan la shell predeterminada actual.

Uno de los Shell de Linux es GNU Bourne-Again Shell (**bash**). Bash es una versión mejorada de una de las shells más exitosas que se usa en los sistemas tipo UNIX: la shell Bourne (**sh**).

Si una shell se usa de manera interactiva, muestra una cadena cuando espera un comando del usuario. Esto recibe el nombre de **prompt de shell**. Cuando un usuario normal inicia una shell, el prompt predeterminado finaliza con un carácter \$, como se muestra a continuación.

```
[user@host ~]$
```

El carácter \$ se reemplaza con el carácter # si la shell se está ejecutando como el superusuario root. Con esto, resulta más evidente que se trata de una shell de superusuario, lo que permite evitar accidentes y errores que pueden afectar a todo el sistema. El prompt de shell de superusuario se muestra a continuación.

```
[root@host ~]#
```

El uso de **bash** para ejecutar comandos puede ser eficaz. La shell bash proporciona un lenguaje de secuencia de comandos capaz de admitir la automatización de tareas. La shell tiene capacidades adicionales que pueden simplificar operaciones o posibilitar aquellas que son difíciles de realizar con herramientas gráficas.

Conceptos básicos de la shell

Los comandos ingresados en el prompt de shell están compuestos por tres **partes básicas**:

Comando que se ejecuta.

Opciones que permiten ajustar el comportamiento del comando.

Argumentos que generalmente son destinos del comando.

El **comando** es el nombre del programa que se ejecuta. Puede estar seguido de **una o más opciones** que ajustan el comportamiento del comando o lo que hará. Las opciones generalmente comienzan con uno o dos guiones (-a o --all, por ejemplo) para que se distingan de los argumentos. Los comandos pueden estar seguidos por

uno o más argumentos, que generalmente indican un destino sobre el cual el comando debe operar.

Por ejemplo, el comando **usermod -L user01** tiene un comando (**usermod**), una opción (-L) y un argumento (user01). El efecto de este comando es bloquear la contraseña de la cuenta del usuario user01.

Muchos administradores de sistemas eligen no ejecutar un entorno gráfico en sus servidores. Esto permite que los servicios del servidor usen los recursos que de otra forma usaría el entorno gráfico.

Si inicia sesión con la pantalla de inicio de sesión gráfica, su entorno gráfico se iniciará en la primera consola virtual que no se encuentre en uso en una sesión de inicio de sesión. Normalmente, la sesión gráfica reemplazará al prompt de inicio de sesión en la segunda consola virtual (tty2). Sin embargo, si esa consola está en uso en una sesión de inicio de sesión de texto activa (no solo en un prompt de inicio de sesión), en su lugar, se usa la siguiente consola virtual libre.

La pantalla de inicio de sesión gráfico se sigue ejecutando en la primera consola virtual (tty1). Si ya ha iniciado sesión en una sesión gráfica e inicia sesión con otro usuario en la pantalla de inicio de sesión gráfica, o si usa el ítem de menú **Switch User** (Cambiar usuario) para cambiar de usuario en el entorno gráfico sin cerrar sesión, se iniciará otro entorno gráfico para ese usuario en la siguiente consola virtual libre.

Cuando cierra sesión en un entorno gráfico, la sesión se cerrará y la consola física volverá automáticamente a la pantalla de inicio de sesión gráfica de la primera consola virtual.

Inicio de sesión a través de la red

Los usuarios y administradores de Linux a menudo necesitan obtener acceso de shell a un sistema remoto conectándose a él a través de la red. En un entorno de cómputo moderno, muchos servidores sin periféricos son en realidad **máquinas virtuales** o se ejecutan como instancias de nube pública o privada. Estos sistemas no son físicos y no tienen consolas de hardware reales. Es posible que ni siquiera puedan proporcionar acceso a su consola física o consola serial (simulada).

En Linux, la forma más común de obtener un prompt de shell en un sistema remoto es usar Secure Shell (SSH). La mayoría de los sistemas Linux y macOS proporcionan el programa de línea de comandos OpenSSH `ssh` para este propósito.

En este ejemplo, un usuario con un prompt de shell en la máquina `host` usa `ssh` para iniciar sesión en el sistema Linux remoto `remotehost` como el usuario `remoteuser`:

```
[user@host ~]$ ssh remoteuser@remotehost
remoteuser@remotehost's password: password
[remoteuser@remotehost ~]$
```

El **comando ssh** cifra la conexión para **proteger** la comunicación contra el espionaje o el robo de contraseñas y contenido.

Algunos sistemas (como las nuevas instancias en la nube) no permiten que los usuarios usen una contraseña para iniciar sesión con `ssh` para mejorar la seguridad. Una forma alternativa de autenticarse en una máquina remota sin ingresar una contraseña es a través de la autenticación de **clave pública**.

Con este método de autenticación, los usuarios tienen un archivo de identidad especial que contiene una **clave privada**, equivalente a una contraseña, que mantienen en secreto. Su cuenta en el servidor está configurada con una **clave pública**, que no tiene que ser secreta. Al iniciar sesión, los usuarios pueden configurar

ssh para proporcionar la clave privada y, si su clave pública coincidente está instalada en esa cuenta en ese servidor remoto, iniciarán sesión sin necesitar una contraseña.

En el siguiente ejemplo, un usuario con un prompt de shell en la máquina host inicia sesión en remotehost como remoteuser con ssh, mediante autenticación de clave pública. La opción **-i** se usa para especificar el archivo de clave privada del usuario, que es **mylab.pem**. La clave pública coincidente ya está configurada como una clave autorizada en la cuenta remoteuser.

```
[user@host ~]$ ssh -i mylab.pem remoteuser@remotehost  
[remoteuser@remotehost ~]$
```

Para que esto funcione, el archivo de clave privada debe ser legible solo para el propietario del archivo. En el ejemplo anterior, donde la clave privada está en el archivo **mylab.pem**, el comando **chmod 600 mylab.pem** podría usarse para garantizar esto. La forma de configurar los permisos de archivo se describe en más detalle en un capítulo posterior.

Los usuarios también pueden tener configuradas claves privadas que se prueban automáticamente, pero ese asunto está fuera del alcance de esta sección. En las referencias que aparecen al final de esta sección, hay más información acerca de este tema.

Introducción al entorno de escritorio de GNOME

El entorno de escritorio es la **interfaz gráfica de usuario** en un sistema **Linux**. Proporciona un escritorio integrado para usuarios y una plataforma de desarrollo unificada sobre un marco (framework) gráfico proporcionado por Wayland (opción predeterminada) o la herramienta heredada X Window System.

GNOME Shell proporciona las funciones principales de la interfaz de usuario para el entorno de escritorio GNOME. La aplicación GNOME Shell es muy personalizable.

Cualquier tema se puede seleccionar persistentemente en el inicio de sesión al hacer clic en el ícono de engranaje junto al botón **Sign In** (iniciar sesión) que está disponible después de seleccionar su cuenta, pero antes de ingresar la contraseña.

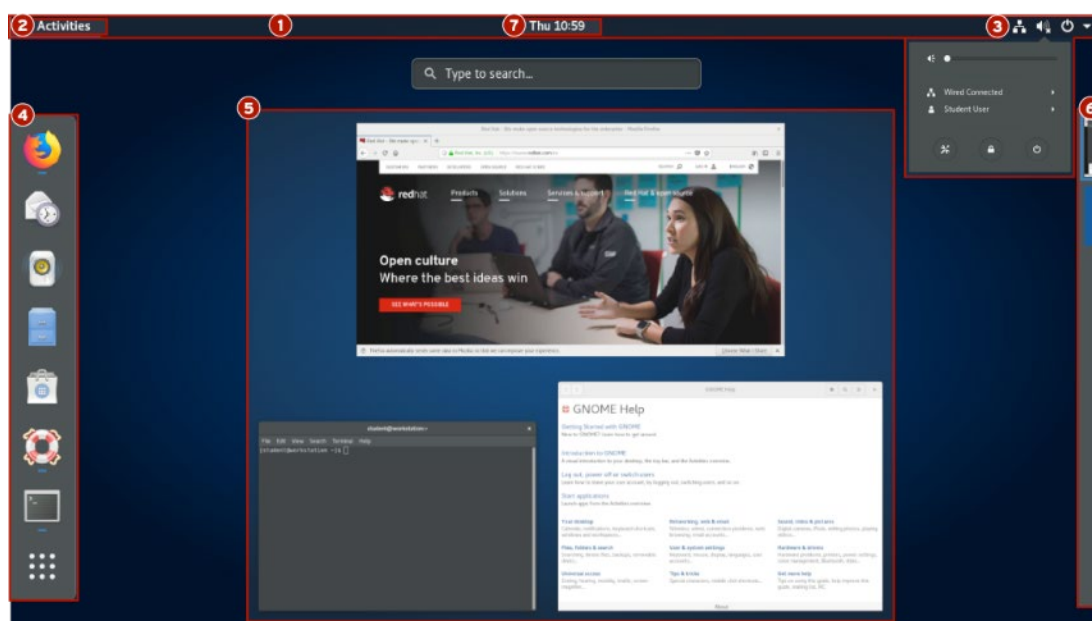


Figura 3. Partes Shell de Gnome. Fuente: Red Hat, 2021.

La primera vez que inicia sesión como usuario nuevo se ejecuta un programa de configuración inicial para ayudar a configurar parámetros de cuenta básicos. Una vez completa, se inicia la aplicación GNOME Help en la pantalla **Getting Started with GNOME** (primeros pasos con GNOME). Esta pantalla incluye vídeos y documentación para ayudar a orientar a los nuevos usuarios hacia el entorno de GNOME3. Para iniciar rápidamente GNOME Help, haga clic en el botón **Activities** (Actividades) a la izquierda de la barra superior y en el guion que aparece a la izquierda de la pantalla; haga clic en el icono de la boya salvavidas para iniciarla.

GNU Bourne-Again Shell (**bash**) es un programa que interpreta comandos escritos por el usuario. Cada secuencia escrita en la shell puede tener un máximo de tres partes: el comando, las opciones (que suelen comenzar con - o --) y los argumentos. Cada palabra escrita en la shell está separada de las otras por espacios. Los comandos son nombres de programas que están instalados en el sistema. Cada comando tiene sus propias opciones y argumentos.

Cuando esté listo para ejecutar un comando, presione la tecla **Intro**. Escriba cada comando en una línea separada. La salida del comando se muestra antes de que aparezca el siguiente prompt de shell.

```
[user@host]$ whoami
user
[user@host]$
```

Si quiere escribir más de un comando en una sola línea, use un **punto y coma (;)** como **separador de comando**. El punto y coma pertenece a la clase de caracteres denominada metacaracteres que tienen un significado especial para bash. En este caso, la salida de ambos comandos se mostrará antes de que aparezca el siguiente prompt de shell.

En el siguiente ejemplo, se muestra cómo combinar dos comandos (**command1** y **command2**) en la línea de comandos.

```
[user@host]$ command1;command2
```

Ejemplos de comandos simples

El **comando date** muestra la fecha y la hora actuales. Además, puede usarlo el superusuario para configurar el **reloj del sistema**. Un argumento que comienza con el signo más (+) especifica una secuencia de formato para el comando de fecha.

```
[user@host ~]$ date
Sat Jan 26 08:13:50 IST 2019
[user@host ~]$ date +%R
08:13
[user@host ~]$ date +%x
01/26/2019
```

El comando **passwd** cambia la contraseña propia del usuario. La contraseña original de la cuenta debe especificarse antes de que se permita un cambio.

De manera predeterminada, **passwd** se configura para solicitar una **contraseña más sólida** que esté compuesta por letras minúsculas, letras mayúsculas, números y símbolos y que no se base en una palabra del diccionario. El superusuario puede usar el comando **passwd** para cambiar las contraseñas de otros usuarios.

```
[user@host ~]$ passwd
Changing password for user.
Current password: old_password
New password: new_password
Retype new password: new_password
passwd: all authentication tokens updated successfully.
```

Linux no requiere de extensiones de nombre de archivo para clasificar los archivos por tipo. El **comando file** detecta el comienzo del contenido de un archivo y muestra qué tipo de archivo es. Los archivos que se clasificarán pasan como argumentos para el comando.

```
[user@host ~]$ file /etc/passwd
/etc/passwd: ASCII text
[user@host ~]$ file /bin/passwd
/bin/passwd: setuid ELF 64-bit LSB shared object, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
for GNU/Linux 3.2.0,
BuildID[sha1]=a3637110e27e9a48dced9f38b4ae43388d32d0e4, stripped
[user@host ~]$ file /home
/home: directory
```

Visualización del contenido de los archivos

Uno de los comandos más simples y de uso frecuente en Linux es **cat**. El comando **cat** permite crear archivos individuales o múltiples, ver el contenido de los archivos, concatenar el contenido de varios archivos y redirigirlo a un terminal o a archivos. En el ejemplo, se muestra cómo ver el contenido del archivo `/etc/passwd`.

```
[user@host ~]$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
...output omitted...
```

Use el siguiente comando para mostrar el contenido de varios archivos.

```
[user@host ~]$ cat file1 file2
Hello World!!
Introduction to Linux commands.
```

Algunos archivos son muy largos y pueden ocupar más espacio para mostrar que el proporcionado por el terminal. El comando **cat** no muestra el contenido de un archivo como páginas. El comando **less** muestra una página del archivo a la vez y le permite desplazarse a su gusto.

El comando **less** le permite avanzar y retroceder por las páginas de los archivos que son más largos de lo que puede mostrar una ventana de terminal. Use las teclas **UpArrow** (Flecha arriba) y **DownArrow** (Flecha abajo) para desplazarse hacia arriba y hacia abajo. Presione **q** para salir del comando.

Los comandos **head** y **tail** muestran el comienzo y el final de un archivo, respectivamente. De manera predeterminada, estos comandos muestran diez líneas del archivo, pero ambos tienen la **opción -n** que permite la especificación de una cantidad diferente de líneas. El archivo que se mostrará pasa como un argumento para estos comandos.

```
[user@host ~]$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
[user@host ~]$ tail -n 3 /etc/passwd
gdm:x:42:42:./var/lib/gdm:/sbin/nologin
gnome-initial-setup:x:977:977:./run/gnome-initial-setup:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
```

El comando **wc** cuenta líneas, palabras y caracteres en un archivo. Puede usar la opción **-l**, **-w** o **-c** para mostrar solo la cantidad de líneas, palabras o caracteres, respectivamente.

```
[user@host ~]$ wc /etc/passwd
 45  102 2480 /etc/passwd
[user@host ~]$ wc -l /etc/passwd ; wc -l /etc/group
45 /etc/passwd
70 /etc/group
[user@host ~]$ wc -c /etc/group /etc/hosts
966 /etc/group
516 /etc/hosts
1482 total
```

Continuación de un comando largo en otra línea

Los comandos con muchas opciones y argumentos pueden volverse largos rápidamente y la ventana de comandos los delimita automáticamente cuando el cursor llega al margen derecho. En cambio, para **facilitar la lectura** del comando, puede escribir un comando largo utilizando más de una línea.

Para ello, utilizará un carácter de barra invertida (\), al que se conoce como carácter de escape, para ignorar el significado del carácter que se encuentra inmediatamente después de la barra invertida. Aprendió que, al ingresar un carácter de nueva línea, presionando la tecla **Enter** (Intro) se le indica a la shell que la entrada del comando está completa y que ejecute el comando. Con el escape del carácter de nueva línea, se le indica a la shell que se traslade a una nueva línea de comando sin la ejecución del comando. La shell reconoce la solicitud mostrando un prompt de continuación, denominado prompt secundario, mediante el carácter mayor que (>) de forma predeterminada, en una nueva línea vacía. Los comandos pueden continuarse a lo largo de muchas líneas.

```
[user@host]$ head -n 3 \  
> /usr/share/dict/words \  
> /usr/share/dict/linux.words  
==> /usr/share/dict/words <==  
1080  
10-point  
10th  
  
==> /usr/share/dict/linux.words <==  
1080  
10-point  
10th  
[user@host ~]$
```

Historial de comandos

El comando **history** muestra una lista de los comandos ejecutados anteriormente con un número de comando como prefijo.

El carácter de signo de admiración, (!), es un metacarácter que se usa para expandir los comandos anteriores sin tener que volver a escribirlos. El comando **!number** se expande hasta el comando que coincide con el número especificado. El comando

!string se expande hasta el comando más reciente que comienza con la secuencia especificada.

```
[user@host ~]$ history
```

```
...output omitted...
```

```
23 clear
```

```
24 who
```

```
25 pwd
```

```
26 ls /etc
```

```
27 uptime
```

```
28 ls -l
```

```
29 date
```

```
30 history
```

```
[user@host ~]$ !ls
```

```
ls -l
```

```
total 0
```

```
drwxr-xr-x. 2 user 6 Mar 29 21:16 Desktop
```

```
...output omitted...
```

```
[user@host ~]$ !26
```

```
ls /etc
```

```
abrt                                hosts                                pulse
```

```
adjtime                            hosts.allow                          purple
```

```
aliases                            hosts.deny                           qemu-ga
```

```
...output omitted...
```

2.4. Administración de archivos desde línea de comandos

RedHat (2022) afirma que todos los archivos de un sistema Linux se guardan en sistemas de archivos que están organizados en un árbol de directorios **invertido** individual conocido como **jerarquía de sistema de archivos**. Este árbol está invertido porque se dice que la root del árbol está en la parte superior de la jerarquía y las ramas de los directorios y subdirectorios se extienden debajo de root.

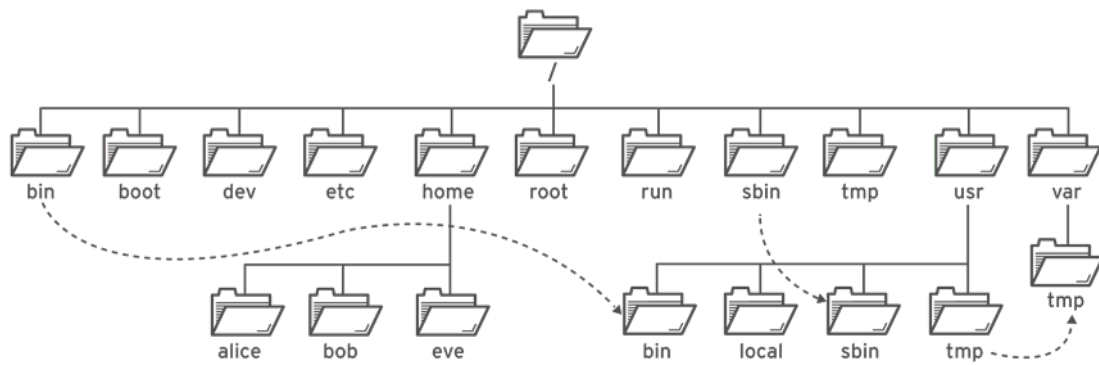


Figura 4. Directorios importantes del sistema de archivos en Red Hat. Fuente: Red Hat, 2021.

El directorio `/` es el **directorío raíz** (root) que está en la parte superior de la jerarquía del sistema de archivos. El carácter `/` también se usa también como un **separador de directorío** en los nombres de archivo. Por ejemplo, si `etc` es un subdirectorio del directorío `/`, podemos llamar a ese directorío `/etc`. De la misma manera, si el directorío `/etc` contiene un archivo con el nombre `issue`, podemos referirnos a ese archivo como `/etc/issue`.

Los subdirectorios de `/` se usan con fines estandarizados para **organizar archivos por tipo y objetivo**. Esto facilita la posibilidad de encontrar archivos. Por ejemplo, en el directorío raíz, el subdirectorio `/boot` se usa para guardar archivos que se necesitan para arrancar el sistema.

Los siguientes términos ayudan a describir el contenido del directorío del sistema de archivos:

- El contenido estático no se modifica hasta que se edita o se reconfigura en forma explícita.
- El contenido dinámico o variable se puede modificar o adjuntar mediante procesos activos.
- El contenido persistente se mantiene después de un arranque nuevo, como los parámetros de configuración.
- El contenido de tiempo de ejecución es contenido específico del proceso o del sistema que se elimina en el reinicio.

Ubicación	Propósito
/usr	Software instalado, bibliotecas compartidas, incluye archivos y datos de programa de solo lectura. Los subdirectorios importantes incluyen: ▶ /usr/bin: Comandos del usuario. ▶ /usr/sbin: Comandos de administración del sistema. ▶ /usr/local: Software personalizado a nivel local.
/etc	Archivos de configuración específicos para este sistema.
/var	Datos variables específicos de este sistema que deberían conservarse entre los arranques. Los archivos que cambian en forma dinámica (por ejemplo, bases de datos, directorios caché, archivos de registro, documentos en cola de impresión y contenido de sitio web) pueden encontrarse en /var.
/run	Datos de tiempo de ejecución para procesos que se iniciaron desde el último arranque. Esto incluye archivos de ID de proceso y archivos de bloqueo, entre otros elementos. El contenido de este directorio se vuelve a crear en el arranque nuevo.
/home	Los directorios de inicio son aquellos donde los usuarios habituales guardan sus datos personales y los archivos de configuración.
/root	Es el directorio de inicio para el superusuario administrativo, root.
/tmp	Es un espacio con capacidad de escritura por parte de cualquier usuario del sistema para archivos temporales. Los archivos a los que no se haya accedido, y que no se hayan cambiado ni modificado durante 10 días se eliminan de este directorio automáticamente. Existe otro directorio temporal, /var/tmp, en el que los archivos que no tuvieron acceso, cambios ni modificaciones durante más de 30 días se eliminan automáticamente.
/boot	Son los archivos necesarios para iniciar el proceso de arranque.
/dev	Contiene archivos de dispositivo especiales que el sistema usa para acceder al hardware.

Tabla 1. Directorios importantes en la distribución de Red Hat Enterprise Linux. Fuente: elaboración propia.

Rutas absolutas y rutas relativas

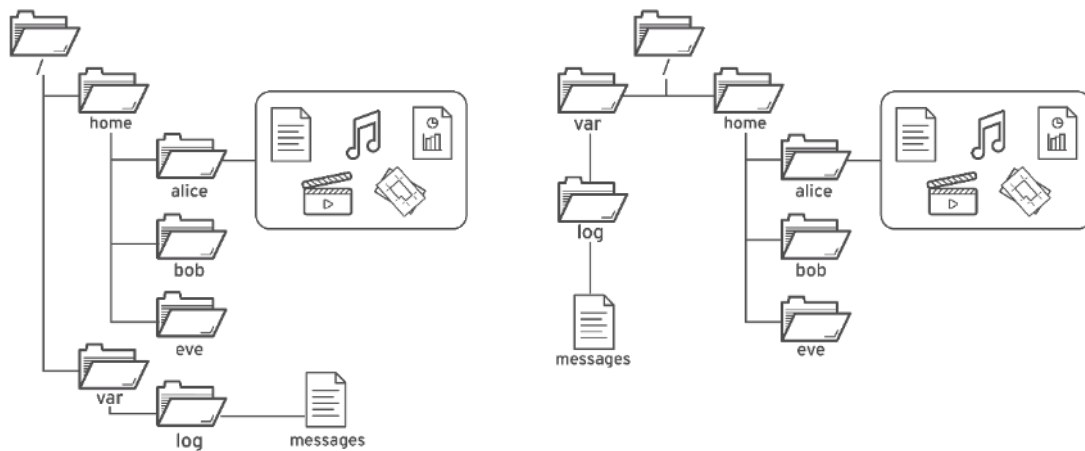


Figura 5. La vista del explorador de archivos habitual (izquierda) equivale a la vista descendente (derecha).
Fuente: Red Hat, 2021.

La **ruta** de un archivo o directorio especifica su ubicación única en el sistema de archivos. Si se sigue la ruta de un archivo, pasa por uno o más subdirectorios con nombre que están delimitados por una barra (/) hasta que se llega al destino. Los directorios, también llamados **carpetas**, contienen otros archivos y otros subdirectorios. Se puede hacer referencia a estos de la misma manera que con los archivos.

Rutas absolutas

Una ruta absoluta es un **nombre** totalmente **calificado** que especifica la **ubicación exacta** de los archivos en la jerarquía del sistema de archivos. Comienza en el directorio raíz (/) y especifica cada subdirectorio que se debe atravesar para llegar al archivo específico. Cada archivo del sistema de archivos tiene un **único nombre** de ruta absoluta, reconocido con una regla simple: un nombre de ruta con una barra (/) como primer carácter es el nombre de la ruta absoluta. Por ejemplo, el nombre de ruta absoluta para el archivo de registro de mensajes del sistema es `/var/log/messages`. Los nombres de rutas absolutas pueden ser **extensos**; en consecuencia, los archivos también pueden ubicarse en forma relativa al directorio de trabajo actual para su prompt de shell.

El directorio de trabajo actual y las rutas relativas

Cuando un usuario inicia sesión y abre una ventana de comandos, la ubicación inicial es, por lo general, el directorio de inicio del usuario. Los procesos del sistema también tienen un directorio inicial. Los usuarios y los procesos navegan hacia otros directorios, según sea necesario; los términos **directorios de trabajo o directorio de trabajo actual** se refieren a su ubicación actual.

Al igual que una ruta absoluta, una ruta relativa identifica un **archivo único** y especifica solo la **ruta necesaria** para llegar al archivo desde el directorio de trabajo. Para reconocer nombres de ruta relativas, se sigue una regla simple: un nombre de ruta que no tenga otro carácter más que una **barra** como primer carácter es un nombre de ruta relativa. Un usuario en el directorio /var podría referirse en forma relativa al archivo de registro del mensaje como log/messages.

Los sistemas de archivos Linux, que incluyen, entre otros, ext4, XFS, GFS2 y GlusterFS, distinguen entre **letras minúsculas** y **mayúsculas**. Si se crea un archivo FileCase.txt y un archivo filecase.txt en el mismo directorio, se obtienen dos archivos exclusivos.

Los sistemas de archivos que no pertenecen a Linux podrían funcionar de manera diferente. Por ejemplo, VFAT, NTFS de Microsoft y HFS+ de Apple tienen un comportamiento de conservación de tipo de letra. A pesar de que estos sistemas de archivos no distinguen entre minúsculas y mayúsculas, muestran los nombres de archivo con el uso de **mayúsculas** original usado cuando se creó el archivo. Por lo tanto, si intentó crear los archivos en el ejemplo anterior en un sistema de archivos VFAT, ambos nombres se tratarían como nombres que apuntan al mismo archivo y no a dos archivos diferentes.

Administración de archivos de la línea de comandos

Para administrar archivos, debe poder crearlos, eliminarlos, copiarlos y moverlos. También debe organizarlos de forma lógica en directorios, los cuales también debe poder crear, eliminar, copiar y mover.

La siguiente tabla resume algunos de los **comandos de administración de archivos** más comunes. El resto de esta sección analizará con más detalle las formas de usar estos comandos.

Actividad	Sintaxis de comandos
Crear un directorio.	<code>mkdir directory</code>
Copiar un archivo.	<code>cp file new-file</code>
Copiar un directorio y su contenido.	<code>cp -r directory new-directory</code>
Mover o cambiar el nombre de un archivo o directorio.	<code>mv file new-file</code>
Eliminar un archivo.	<code>rm file</code>
Eliminar un directorio que contiene archivos.	<code>rm -r directory</code>
Eliminar un directorio vacío.	<code>rmdir directory</code>

Tabla 2. Comandos comunes de administración de archivos. Fuente: elaboración propia.

Editar archivos con Vim

Un principio clave de diseño de Linux es que la **configuración** y la **información** habitualmente se almacenan en archivos basados en **texto**. Estos archivos se pueden estructurar de varias maneras, como listas de configuraciones, en formatos tipo INI, como XML o YAML estructurados, etc. Sin embargo, la ventaja de los archivos de texto es que se pueden **ver y editar** con cualquier editor de texto **simple**.

Vim es una versión mejorada del editor vi que se distribuye con los sistemas Linux y UNIX. Vim es **altamente configurable** y eficaz para usuarios avanzados; incluye funciones como edición en pantalla partida, formateo de color y resaltado para la edición de texto.

Debe saber cómo usar al menos un editor de texto que se puede utilizar desde un prompt de shell de solo texto. Así, puede editar archivos de configuración basados en texto desde una ventana de terminal o desde inicios de sesión remotos a través de ssh o de la consola web. Entonces, no necesitará acceso a un escritorio gráfico para editar archivos en un servidor y, de hecho, es posible que ese servidor no necesite ejecutar un entorno de escritorio gráfico.

Pero entonces, ¿por qué aprender Vim en lugar de otras opciones posibles?

La razón clave es que Vim casi siempre está instalado en un **servidor**, si hay un editor de texto. Esto se debe a que vi se especificó a través del estándar POSIX que Linux y muchos otros sistemas operativos tipo UNIX cumplen en gran parte.

Además, Vim se utiliza a menudo como la **implementación de vi** en otros sistemas operativos o distribuciones comunes. Por ejemplo, macOS actualmente incluye una instalación ligera de Vim de forma predeterminada. Por lo tanto, las habilidades de Vim que aprenda para Linux también podrían ayudarlo a resolver cosas en otro lugar.

Inicio de Vim

Vim se puede instalar en Linux de **dos** maneras diferentes. Esto puede afectar las funciones y los comandos Vim que tiene disponibles.

Es posible que su servidor solo tenga instalado el paquete **vim-minimal**. Esta es una instalación muy ligera que incluye solo el conjunto de características principales y el comando vi básico. En este caso, puede abrir un archivo para editarlo con **vi filename** y tendrá disponibles todas las funciones básicas que se analizan en esta sección.

Alternativamente, su servidor puede tener el instalado el paquete **vim-enhanced**. Esto proporciona un conjunto de características mucho **más completo**, un sistema de ayuda en línea y un programa tutorial. Para iniciar Vim en este modo mejorado, utiliza el comando vim.

```
[user@host ~]$ vim filename
```

Modos de operación de Vim

Una característica inusual de Vim es que tiene varios modos de operación, incluido el modo de comandos, el modo de comandos ampliado, el modo de edición y el modo visual. Según el modo, es posible que emita comandos, edite texto o trabaje con bloques de texto en el modo visual. Como usuario nuevo de Vim, siempre debe estar al tanto del modo actual, ya que las pulsaciones de teclas tienen diferentes efectos en los diferentes modos.

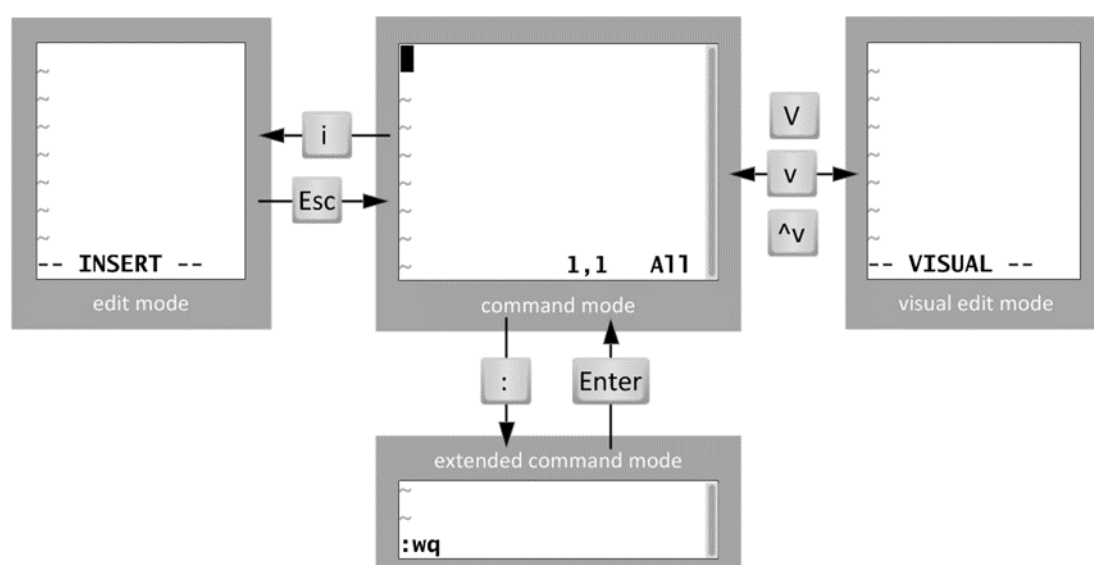


Figura 6. Cambio entre modos de Vim. Fuente: Red Hat Academy, 2021.

Cuando se abre Vim por primera vez, arranca en el **modo de comandos**, usado para navegar, cortar y pegar y otro tipo de manipulación de texto. Ingrese en cada uno de los otros modos con pulsaciones de tecla de caracteres únicos para acceder a funciones de edición específicas:

Al pulsar la tecla **i** se ingresa al modo de inserción, en el cual todo el texto ingresado se convierte en contenido de archivo. Al presionar **Esc** se vuelve al modo de comandos.

Al pulsar la tecla **v** se ingresa al modo visual, en el cual se pueden seleccionar varios caracteres para la manipulación de texto. Utilice **Shift+V** (Mayús V) para líneas múltiples y **Ctrl+V** para seleccionar en bloque. La misma pulsación de tecla utilizada para ingresar al modo visual (**v**, **Shift+V** (Mayús V) o **Ctrl+V**) se utiliza para salir.

Al pulsar la tecla: se inicia el modo de comandos ampliado para tareas como la escritura del archivo (para guardarlo) y la salida del editor Vim.

En el siguiente vídeo, *Instalación de sistemas operativos Linux*, vamos a realizar una instalación de un sistema operativo Linux Red Hat Enterprise. Veremos la configuración de todos los parámetros necesarios para empezar a trabajar profesionalmente con esta distribución de Linux.



Accede al vídeo

2.5. Usuarios, grupos y permisos

RedHat (2022) afirma que una cuenta de usuario se utiliza para proporcionar **límites de seguridad** entre diferentes personas y programas que pueden ejecutar comandos.

Los usuarios tienen nombres de usuario para identificarlos como usuarios humanos y facilitar el trabajo con ellos. Internamente, el sistema distingue las cuentas de usuario por el número de identificación único que se les asigna, el **ID de usuario o UID**. Si las personas utilizan una cuenta de usuario, generalmente se le asignará una contraseña secreta que el usuario utilizará para demostrar que es el usuario autorizado real al iniciar sesión.

Las cuentas de usuario son fundamentales para la **seguridad del sistema**. Cada proceso (programa en ejecución) en el sistema se ejecuta como un usuario particular. Cada **archivo** tiene un usuario particular como su **propietario**. La propiedad del archivo ayuda al sistema a aplicar el **control de acceso** para los usuarios de los archivos. El usuario asociado con un proceso de ejecución determina los archivos y directorios accesibles para ese proceso.

Hay tres **tipos** principales de **cuenta de usuario**: el superusuario, el usuario del sistema y el usuario normal.

La cuenta de **superusuario** es para la administración del sistema. El nombre del superusuario es root y la cuenta tiene UID 0. El superusuario tiene acceso completo al sistema.

El sistema tiene cuentas de **usuario del sistema** que utilizan los procesos que proporcionan servicios de soporte. Estos procesos o daemons, por lo general, no necesitan ejecutarse como superusuario. Son cuentas asignadas sin privilegios que les permiten proteger sus archivos y otros recursos entre sí y de los usuarios habituales del sistema. Los usuarios no inician sesión de forma interactiva mediante una cuenta de usuario del sistema.

La mayoría de los usuarios tienen cuentas de **usuario normal** que utilizan para su trabajo diario. Al igual que los usuarios del sistema, los usuarios normales tienen acceso limitado al sistema.

Puede utilizar el comando **id** para mostrar información acerca del usuario con sesión iniciada actualmente.

```
[user01@host ~]$ id
uid=1000(user01)          gid=1000(user01)          groups=1000(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Para ver el **propietario de un archivo**, use el comando **ls -l**. Para ver el **propietario de un directorio**, use el comando **ls -ld**. En la siguiente salida, la tercera columna muestra el nombre de usuario.

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 0 Feb 5 11:10 file1
[user01@host]$ ls -ld dir1
drwxrwxr-x. 2 user01 6 Feb 5 11:10 dir1
```

Para ver la información del proceso, use el comando **ps**. La opción predeterminada es mostrar solo los procesos que están en la shell actual. Agregue la opción **a** para ver todos los procesos con un terminal. Para ver el usuario relacionado con un proceso, incluya la opción **u**. En la siguiente salida, la primera columna muestra el nombre de usuario.

```
[user01@host]$ ps -au
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	777	0.0	0.0	225752	1496	tty1	Ss+	11:03	0:00	/sbin/agetty
-o -p -- \u --noclear tty1 linux										
root	780	0.0	0.1	225392	2064	ttyS0	Ss+	11:03	0:00	/sbin/agetty
-o -p -- \u --keep-baud 115200,38400,9600										
user01	1207	0.0	0.2	234044	5104	pts/0	Ss	11:09	0:00	-bash
user01	1319	0.0	0.2	266904	3876	pts/0	R+	11:33	0:00	ps au

En la salida de los comandos anteriores, se muestran los usuarios por nombre, pero, internamente, el sistema operativo usa los **UID** para realizar un **seguimiento** de los usuarios. La asignación de nombres de usuario a UID se define en las bases de datos de la información de la cuenta. De forma predeterminada, los sistemas usan el archivo `/etc/passwd` para almacenar información sobre los usuarios locales.

Cada línea del archivo `/etc/passwd` contiene información sobre un usuario. Se divide en siete campos separados por dos puntos. Este es un ejemplo de una línea de `/etc/passwd`:

```
user01: x: 1000: 1000: User One: /home/user01: /bin/bash
```


Nombre de usuario para este usuario (user01).

La contraseña del usuario se almacenaba aquí en formato cifrado. Se trasladó al archivo `/etc/shadow`, que se analizará más adelante. Este campo siempre debe ser X.

El número de UID para esta cuenta de usuario (1000).

El número de GID para el grupo principal de esta cuenta de usuario (1000). Los grupos se analizarán más adelante en esta sección.

El nombre real para este usuario. (User One).

El directorio de inicio de este usuario. (`/home/user01`). Este es el directorio de trabajo inicial cuando se inicia la shell y contiene los datos del usuario y los parámetros de configuración.

El programa de shell predeterminado para este usuario, que se ejecuta al iniciar sesión (**`/bin/bash`**). Para un usuario normal, por lo general este es el programa que proporciona el prompt de línea de comando del usuario. Un usuario del sistema podría usar **`/sbin/nologin`** si no se permiten inicios de sesión interactivos para ese usuario.

¿Qué es un grupo?

Un grupo es una colección de usuarios que necesitan compartir el acceso a archivos y otros recursos del sistema.

Los grupos se pueden utilizar para otorgar **acceso a los archivos** a un conjunto de usuarios en lugar de a un solo usuario.

Como los usuarios, los grupos tienen **nombres de grupo** para facilitar el trabajo con ellos. Internamente, el sistema distingue los grupos por el número de identificación único que se les asigna, el **ID de grupo** o **GID**.

La asignación de nombres de grupo a GID se define en las bases de datos de la información de la cuenta de grupo. De forma predeterminada, los sistemas usan el archivo **`/etc/group`** para almacenar información sobre los grupos locales.

Cada línea del archivo `/etc/group` contiene información sobre un grupo. Cada entrada de grupo se divide en cuatro campos separados con dos puntos. Este es un ejemplo de una línea de `/etc/group`:

```
group01: x: 10000: user01,user02,user03
```

1. Nombre de grupo para este grupo (group01).

Campo de contraseña de grupo obsoleto. Este campo siempre debe ser X.

El número de GID para este grupo (10 000).

Una lista de usuarios que son miembros de este grupo como grupo adicional (user01, user02 y user03).

Grupos principales y grupos adicionales

Cada usuario tiene exactamente un **grupo principal**. Para usuarios locales, este es el grupo enumerado por número de GID en el archivo `/etc/passwd`. De forma predeterminada, este es el grupo que será **propietario** de los nuevos **archivos** creados por el usuario.

Normalmente, cuando crea un nuevo usuario normal, se crea un nuevo grupo con el mismo nombre que ese usuario. Ese grupo se usa como el grupo principal del nuevo usuario y ese usuario es el único miembro de este grupo privado de usuarios. Se deduce que esto ayuda a **simplificar la administración de los permisos** de archivos, que se analizará más adelante en este curso.

Los usuarios también pueden tener **grupos adicionales**. La pertenencia a grupos adicionales se determina a través del **archivo /etc/group**. A los usuarios se les otorga acceso a los archivos en función de si alguno de sus grupos tiene acceso. No importa si el grupo o los grupos que tienen acceso son principales o adicionales para el usuario.

Por ejemplo, si el usuario user01 tiene un grupo principal user01 y grupos adicionales wheel y webadmin, entonces ese usuario puede leer archivos legibles para cualquiera de esos tres grupos.

El **comando id** también se puede utilizar para averiguar sobre la membresía de grupo de un usuario.

```
[user03@host ~]$ id
uid=1003(user03) gid=1003(user03)
groups=1003(user03),10(wheel),10000(group01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

En el ejemplo anterior, user03 tiene el grupo user03 como su grupo principal (gid). El elemento groups enumera todos los grupos de este usuario y, además del grupo principal user03, el usuario tiene los grupos wheel y group01 como grupos adicionales.

El superusuario

La mayoría de los sistemas operativos tienen una especie de superusuario; un usuario que tiene todo el poder sobre el sistema.

Es el **usuario root**. Este usuario tiene el poder de anular los privilegios normales del sistema de archivos y se usa para **manejar y administrar el sistema**. Para realizar tareas como la instalación o eliminación de software y para administrar los directorios y los archivos del sistema, los usuarios deben aumentar sus privilegios al usuario root.

El usuario root solo entre los usuarios normales puede controlar la mayoría de los dispositivos, pero hay algunas excepciones. Por ejemplo, los usuarios normales pueden controlar dispositivos desmontables, como dispositivos USB. Por lo tanto, los usuarios normales pueden agregar y eliminar archivos y administrar de otro modo un

dispositivo desmontable, pero solo el usuario root puede **administrar los discos duros** «fijos» de manera predeterminada.

Sin embargo, este privilegio ilimitado viene acompañado de una **responsabilidad**. El usuario root tiene poder ilimitado para **dañar el sistema**: eliminar archivos y directorios, eliminar cuentas de usuarios, agregar puertas traseras, etc. Si la cuenta del usuario root está comprometida, alguien más tendrá control administrativo del sistema. A lo largo de este curso, se les indicará a los administradores que inicien sesión como usuario normal y que escalen los privilegios a root solo cuando sea necesario.

La cuenta root en Linux es casi equivalente a la cuenta de administrador local en Microsoft Windows. En Linux, la mayoría de los administradores del sistema inician sesión en el sistema como un usuario sin privilegios y usan distintas herramientas para ganar privilegios de usuario root temporalmente.

Los comandos **useradd**, **userdel** y **userdel** nos permiten gestionar a los usuarios. Los correspondientes comandos para grupos son **groupadd**, **groupmod** y **groupdel**.

Permisos del sistema de archivos de Linux

Los permisos de archivos controlan el acceso a los archivos. Los permisos de archivos de Linux son simples pero **flexibles**, lo que hace que sean **fáciles de comprender y de aplicar** y, al mismo tiempo, permiten gestionar fácilmente los casos más normales de permisos.

Los archivos tienen **tres categorías de usuario** a las que se les aplican permisos. El propietario del archivo es un usuario que normalmente es quien lo creó. También es posible que el propietario del archivo sea un solo grupo, generalmente el grupo primario del usuario que creó el archivo, pero esto se puede cambiar. Pueden establecerse **diferentes permisos** para el usuario o grupo propietario y para otros usuarios del sistema que no sean el usuario o un miembro del grupo propietario.

Los permisos más **específicos** tienen **prioridad**. Los permisos de usuario anulan los permisos de grupo, lo cual anula los permisos de otros.

En la Figura 7 joshua es un miembro de los grupos joshua y web y allison es miembro de los grupos allison, wheel y web. Cuando joshua y allison necesitan colaborar, los archivos deben asociarse con el grupo web y los permisos del grupo deben permitir el acceso deseado.

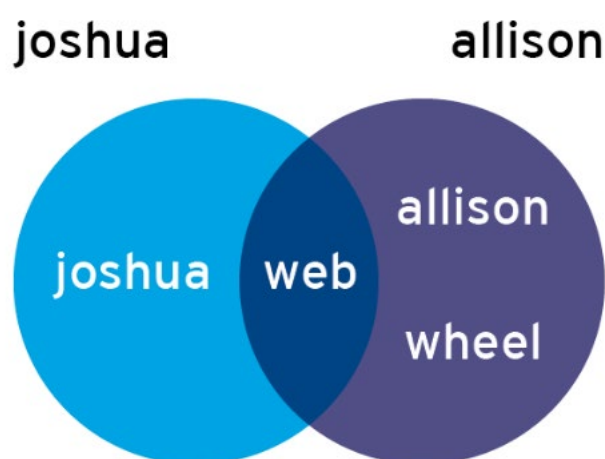


Figura 7. Ejemplo de pertenencia a un grupo para facilitar la colaboración. Fuente: Red Hat, 2021.

Se utilizan tres categorías de permisos: **lectura**, **escritura** y **ejecución**. En la siguiente tabla, se explica cómo estos permisos afectan el acceso a archivos y directorios.

Permiso	Efecto en los archivos	Efecto en los directorios
r (lectura)	Puede leerse el contenido del archivo.	El contenido del directorio (nombres de archivo) puede detallarse.
w (escritura)	Puede cambiarse el contenido del archivo.	Cualquier archivo en el directorio puede crearse o eliminarse.
x (ejecución)	Los archivos pueden ejecutarse como comandos.	El directorio puede convertirse en el directorio de trabajo actual (usted puede cd en él, pero también se requiere permiso de lectura para enumerar los archivos que se encuentran allí).

Tabla 3. Efectos de los permisos en archivos y directorios. Fuente: elaboración propia.

Los **usuarios** normalmente poseen permisos tanto de **lectura** como de **ejecución** en los directorios de solo lectura, por lo que pueden enumerar el directorio y tener acceso completo de solo lectura a su contenido.

Si un usuario **solo** posee acceso **de lectura** en un directorio, los nombres de los archivos dentro de este pueden enumerarse, pero no estará disponible otra información, incluidos permisos o marcas de tiempo y tampoco se podrá acceder a ellos. Si un usuario **solo** tiene acceso **de ejecución** en un directorio, no puede enumerar nombres de archivo en el directorio. Si el usuario conoce el nombre de un archivo para el cual tiene permiso de lectura, puede acceder al contenido de ese archivo desde fuera del directorio al especificar explícitamente el nombre de archivo relativo.

Todo usuario que sea propietario o cuente con permisos de escritura para el directorio donde se encuentra el archivo puede **quitar un archivo**, sin importar la propiedad ni los permisos del archivo en sí. Esto puede anularse con un **permiso especial**, el **sticky bit**, que trataremos más adelante en este capítulo.

Nota

Los permisos de archivos Linux funcionan de manera diferente al sistema de permisos del sistema de archivos NTFS para Microsoft Windows.

En Linux, los permisos se aplican solo al archivo o directorio en el que están configurados. Es decir, los permisos en un directorio no se heredan de forma automática por los subdirectorios o los archivos que se encuentran en él. Sin embargo, los permisos de un directorio pueden bloquear el acceso al contenido del directorio según qué tan restrictivos sean.

El **permiso de lectura** en un directorio de **Linux** es casi equivalente a **Mostrar el contenido de la carpeta** en Windows.

El **permiso de escritura** en un directorio de Linux es equivalente a **Modify** (Modificación) en Windows. Esto implica la posibilidad de eliminar archivos y subdirectorios. En Linux, si escritura y sticky bit están establecidos en un directorio, solo el propietario del archivo o subdirectorio puede eliminarlos, lo que se asemeja al comportamiento del permiso de escritura de Windows.

El **usuario root** en Linux posee los permisos equivalentes a **Full Control** (Control total) de Windows en todos los archivos. Sin embargo, el usuario root puede tener acceso restringido por la política de SELinux del sistema que utiliza contextos de seguridad del proceso y de los archivos. SELinux se analizará en un curso posterior.

Visualización de permisos y propiedades de archivos y directorios

La opción **-l** del comando **ls** muestra información detallada sobre permisos y propiedad:

```
[user@host~]$ ls -l test
-rw-rw-r--. 1 student 0 Feb  8 17:36 test
```

Utilice la opción **-ld** para mostrar información detallada sobre un directorio y no sobre su contenido.

```
[user@host ~]$ ls -ld /home  
drwxr-xr-x. 5 root 4096 Jan 31 22:00 /home
```

2.6. Procesos, servicios y daemons.

Monitorización

Definición de un proceso

RedHat (2022) afirma que un proceso es una **instancia** de un programa ejecutable que se inició y se encuentra **en funcionamiento**. Un proceso consta de lo siguiente:

Un espacio de direcciones que incluye la memoria asignada

Características de seguridad que incluyen credenciales y privilegios de propiedad.

Uno o más subprocesos de ejecución de código de programa.

Estado del proceso.

El **entorno de un proceso** incluye lo siguiente:

Variables locales y globales.

Un contexto de programación actual.

Recursos asignados del sistema, como descriptores de archivos y puertos de red.

Un proceso (principal) existente duplica su propio espacio de direcciones (fork) para crear una nueva estructura de proceso (secundaria). Se asigna una identificación de proceso (**PID**) única a cada proceso nuevo para su rastreo y por motivos de seguridad. La PID y la identificación del proceso principal (PPID) son elementos del entorno del proceso nuevo. Cualquier proceso puede crear un proceso secundario.

A través del **proceso de bifurcación**, un proceso secundario hereda identidades de seguridad, descriptores de archivos actuales y anteriores, privilegios de recursos y puertos, variables de entorno y código de programa. Luego, un proceso secundario puede ejecutar su propio código de programa.

Normalmente, un proceso principal se encuentra **inactivo** mientras se ejecuta el proceso secundario, lo que establece que una solicitud (espera) se señale cuando finalice el proceso secundario. Tras su finalización, el proceso secundario ya ha cerrado o descartado sus recursos y su entorno.

El único recurso restante, llamado **zombi**, es una entrada en la tabla de procesos. El proceso principal, que se señala como activo una vez que finaliza el proceso secundario, limpia la tabla de procesos de la entrada del proceso secundario y libera el último recurso del proceso secundario. El proceso principal continúa con la ejecución de su propio código de programa.

Descripción de los estados del proceso

En un sistema operativo de funciones múltiples, cada CPU (o núcleo de CPU) puede trabajar en un proceso en un momento dado. Mientras se ejecuta un proceso, sus requisitos inmediatos en cuanto a asignación de recursos y tiempo de la CPU **cambian**. Los procesos reciben un estado que cambia en la medida en que las circunstancias así lo exijan.

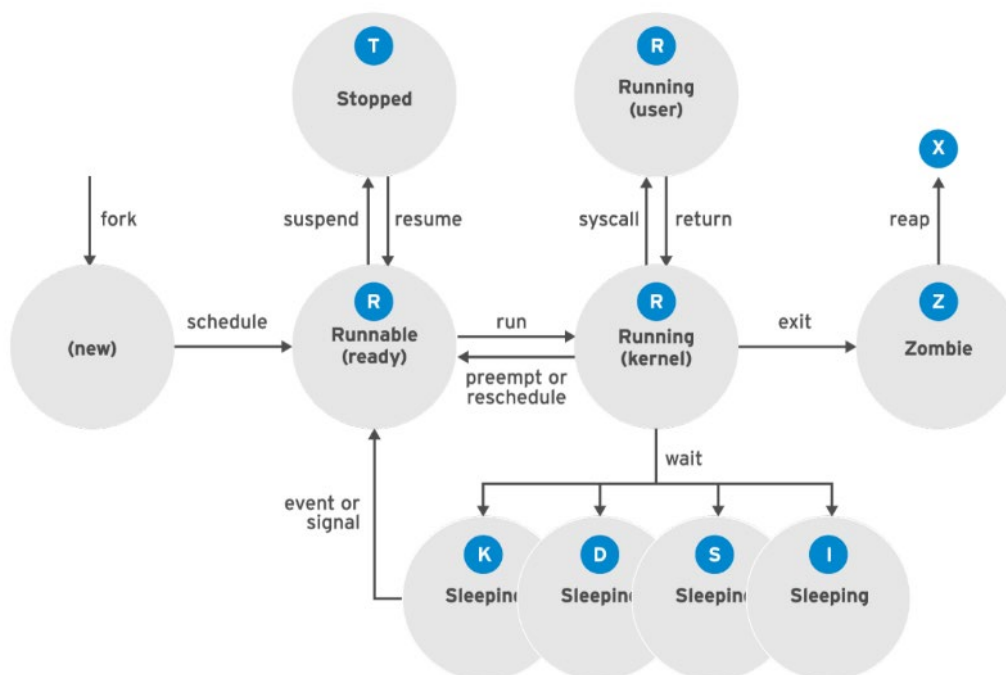


Figura 8. Estados de los procesos de Linux. Fuente: Red Hat Academy. Train the trainers, 2021. Manuscrito no publicado.

Los estados de los procesos de Linux se ilustran en el diagrama anterior y se describen en la siguiente tabla.

Por qué los estados de los procesos son importantes

Al solucionar problemas de un sistema, es importante comprender cómo se comunica el **kernel** con los procesos y cómo los procesos se comunican entre sí.

En la creación del proceso, el sistema asigna un estado al proceso. La columna **S** del comando `top` o la columna **STAT** de `ps` muestra el estado de cada proceso. En un sistema de una sola CPU, solo se puede ejecutar un proceso a la vez. Es posible ver varios procesos con un estado de R. Sin embargo, no todos se ejecutarán consecutivamente, algunos de ellos estarán en estado en espera.

```
[user@host ~]$ top
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	244344	13684	9024	S	0.0	0.7	0:02.46	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd

...output omitted...

```
[user@host ~]$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	2	0.0	0.0	0	0	?	S	11:57	0:00	[kthreadd]
student	3448	0.0	0.2	266904	3836	pts/0	R+	18:07	0:00	ps aux

...output omitted...

El proceso se puede suspender, detener, reanudar, finalizar e interrumpir mediante el uso de **señales**. Las señales se analizarán en más detalle, posteriormente, en este capítulo. Las señales pueden ser usadas por otros procesos, por el propio kernel o por usuarios conectados al sistema.

Lista de procesos

El **comando ps** se usa para elaborar una lista de los procesos actuales. Puede proporcionar **información detallada** de los procesos que incluye:

La identificación del usuario (UID) que determina los privilegios del proceso.

La identificación del proceso (PID) única.

La CPU y el tiempo real empleado.

La cantidad de memoria que el proceso ha asignado en diversas ubicaciones.

La ubicación del proceso stdout, conocido como terminal de control.

El estado del proceso actual.

Realización de trabajos en segundo plano

Cualquier comando o tubería puede iniciarse en segundo plano si se anexa el signo **ampersand** (&) al final de la línea de comandos. La shell Bash muestra un **número de trabajo** (exclusivo de la sesión) y el identificador de proceso del proceso secundario

nuevo. La shell no espera que finalice el proceso secundario, sino que muestra el prompt de shell.

```
[user@host ~]$ sleep 10000 &  
[1] 5947  
[user@host ~]$
```

Monitoreo del proceso en tiempo real

El **programa top** es una vista dinámica de los procesos del sistema que muestra un encabezado del resumen seguido de un proceso o lista de subprocesos similares a la información de ps. A diferencia de la salida de ps estática, top continuamente se actualiza a un intervalo configurable y ofrece capacidades de **reorganización, ordenado y resaltado de columnas**. Las configuraciones del usuario se pueden guardar y hacer persistentes.

Las columnas de salidas predeterminadas se diferencian de otras herramientas de recursos en:

La ID del proceso (PID).

El nombre de usuario (USER) es el propietario del proceso.

La memoria virtual (VIRT) es toda la memoria que está usando el proceso, incluido el conjunto residente, las bibliotecas compartidas y cualquier página de memoria asignada o intercambiada (con la etiqueta VSZ en el comando ps).

La memoria residente (RES) es la memoria física que usa el proceso, incluido cualquier objeto residente compartido (con la etiqueta RSS en el comando ps).

El estado del proceso S se muestra como:

D = Suspensión ininterrumpida.

R = En ejecución o ejecutable.

S = En espera.

T = Detenido o en seguimiento.

Z = Zombi.

El tiempo de CPU (TIME) es el tiempo total de procesamiento desde que comenzó el proceso. Se puede alternar para incluir el tiempo acumulativo de todos los procesos secundarios.

El nombre del comando de proceso (COMMAND).

Introducción a systemd

El daemon de systemd **administra el inicio** para Linux, incluido el inicio del servicio y la gestión de servicios en general. Activa los **recursos del sistema**, los daemons del servidor y otros procesos, tanto en el momento del arranque como en un sistema que está en funcionamiento.

Los **daemons** son procesos que esperan o se ejecutan en **segundo plano** y realizan diversas tareas. Generalmente, los daemons se inician automáticamente en el momento del **arranque** y continúan ejecutándose hasta que se apaga el sistema o son detenidos manualmente. Por convención, los nombres de muchos programas daemon finalizan con la letra **d**.

A menudo, un servicio en el contexto de **systemd** hace referencia a **uno o más daemons**, pero iniciar o detener un servicio puede, en cambio, hacer una modificación por única vez en el estado del sistema, que no implica dejar un proceso daemon en ejecución después de esto (que se denomina oneshot).

Visualización de los estados de servicio

Visualice el estado de una unidad específica con **systemctl status name.type**. Si no se proporciona el tipo de unidad, **systemctl** mostrará el estado de una unidad de servicio, en caso de que exista una.

```
[root@host ~]# systemctl status sshd.service
```

```
● sshd.service - OpenSSH server daemon
```

```
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
```

```
   Active: active (running) since Thu 2019-02-14 12:07:45 IST; 7h ago
```

```
 Main PID: 1073 (sshd)
```

```
   CGroup: /system.slice/ssh.service
```

```
           └─1073 /usr/sbin/sshd -D ...
```

```
Feb 14 11:51:39 host.example.com systemd[1]: Started OpenSSH server daemon.
```

```
Feb 14 11:51:39 host.example.com sshd[1073]: Could not load host key: /et...y
```

```
Feb 14 11:51:39 host.example.com sshd[1073]: Server listening on 0.0.0.0
```

```
....
```

```
Feb 14 11:51:39 host.example.com sshd [1073]: Server listening on :: port 22.
```

```
Feb 14 11:53:21 host.example.com sshd[1270]: error: Could not load host k...y
```

```
Feb 14 11:53:22 host.example.com sshd[1270]: Accepted password for root f...2
```

```
...output omitted...
```

Verificación del estado de un servicio

El comando **systemctl** proporciona métodos para verificar los estados específicos de un servicio. Por ejemplo, use el siguiente comando para verificar que la unidad de servicio se encuentre activa (en ejecución) en ese momento:

```
[root@host ~]# systemctl is-active sshd.service
```

```
active
```

El comando devuelve el estado de la unidad de servicio, el cual generalmente es **active** (activo) o **inactive** (inactivo).

Ejecute el siguiente comando para verificar si una unidad de servicio está habilitada para iniciarse automáticamente durante el arranque del sistema:

```
[root@host ~]# systemctl is-enabled sshd.service
enabled
```

El comando informa si la unidad de servicio está habilitada para iniciarse en el momento del arranque, lo cual, generalmente, se informa como estado **enabled** (habilitado) o **disabled** (deshabilitado).

Para verificar **si la unidad falló** durante el arranque, ejecute el siguiente comando:

```
[root@host ~]# systemctl is-failed sshd.service
active
```

Reinicio y recarga de servicios

Durante el reinicio de un servicio en ejecución, el servicio se detiene y, luego, se inicia. Al reiniciar el servicio, la ID del proceso cambia y una nueva ID del proceso se asocia durante el arranque. Para reiniciar un servicio en ejecución, use el **argumento restart** con el **comando systemctl**. En el siguiente ejemplo, se muestra cómo reiniciar el servicio `sshd.service`:

```
[root@host ~]# systemctl restart sshd.service
```

Algunos servicios tienen la capacidad de volver a cargar sus archivos de configuración sin necesidad de reiniciar. Este proceso se llama **recarga del servicio**. La recarga de un servicio no cambia la ID del proceso asociada con diversos procesos del servicio. Para recargar un servicio en ejecución, use el argumento `reload` con el **comando systemctl**. En el siguiente ejemplo, se muestra cómo volver a cargar el servicio `sshd.service` después de modificar la configuración:

```
[root@host ~]# systemctl reload sshd.service
```

En caso de que no esté seguro de si el servicio cuenta con la funcionalidad para volver a cargar los cambios del archivo de configuración, use el **argumento reload-or-restart**

con el **comando systemctl**. El comando vuelve a cargar los cambios de configuración si la funcionalidad de recarga está disponible. De lo contrario, el comando reinicia el servicio para implementar los nuevos cambios de configuración:

```
[root@host ~]# systemctl reload-or-restart sshd.service
```

¿Qué es OpenSSH?

OpenSSH implementa el **protocolo Secure Shell** o SSH en los sistemas Linux. El protocolo SSH permite que los sistemas se comuniquen de forma **cifrada y segura** a través de una red insegura.

Puede usar el comando ssh para crear una **conexión segura** a un sistema remoto, autenticarse como un usuario específico y obtener una sesión de shell interactiva en el sistema remoto como ese usuario. También puede usar el comando ssh para **ejecutar un comando individual** en el sistema remoto sin acceder a la shell interactiva.

Con el siguiente comando ssh, podría iniciar sesión en el servidor remoto remotehost utilizando el mismo nombre de usuario que el usuario local actual. En este ejemplo, el sistema remoto le solicita que se autentique con la contraseña de ese usuario.

```
[user01@host ~]$ ssh remotehost
user01@remotehost's password: redhat
...output omitted...
[user01@remotehost ~]$
```


2.7. Análisis y almacenamiento de registros

Registro del sistema

RedHat (2022) afirma que los procesos y el kernel del sistema operativo llevan un registro de los eventos que suceden. Estos registros se usan para realizar una **auditoría del sistema y solucionar problemas**.

Muchos sistemas registran registros de eventos en archivos de texto que se guardan en el **directorio /var/log**. Estos registros se pueden inspeccionar utilizando las utilidades de texto normales, como **less** y **tail**.

Varias distribuciones Linux incluye un sistema de registro estándar que se basa en el protocolo Syslog. Muchos programas usan este sistema para registrar eventos y organizarlos en archivos de registro.

El servicio **systemd-journald** está en el corazón de la arquitectura de registro de eventos del sistema operativo. **Recolecta mensajes** de eventos desde muchas fuentes, incluido el kernel, la salida de las primeras etapas del proceso de arranque, la salida y el error estándar de los daemons cuando se inician y se ejecutan y los eventos de syslog. Luego, **los reestructura** en un formato estándar y **los escribe** en un diario de sistema indexado y estructurado. De forma predeterminada, este diario se almacena en un sistema de archivos que no persiste en los reinicios.

Sin embargo, el **servicio rsyslog** lee los mensajes de syslog recibidos por **systemd-journald** desde el diario a medida que llegan. Luego procesa los eventos del registro del sistema, los registra en sus archivos de registro o los reenvía a otros servicios de acuerdo con su propia configuración.

El **servicio rsyslog** ordena y escribe mensajes de syslog en los archivos de registro que no persisten en los reinicios en `/var/log`. El servicio rsyslog **clasifica los mensajes** de registro en archivos de registro específicos según el tipo de programa que envió cada mensaje, o instalación, y la prioridad de cada mensaje de syslog.

Además de los archivos de mensajes de syslog, el **directorio `/var/log`** contiene archivos de registro de **otros servicios** en el sistema. En la siguiente tabla, se enumeran algunos archivos útiles del directorio `/var/log`.

Archivo de registro	Tipo de mensajes almacenados
<code>var/log/messages</code>	La mayoría de los mensajes de syslog se registran aquí. Las excepciones incluyen mensajes relacionados con tareas de autenticación y procesamiento de correos electrónicos, con ejecución de trabajos programados y aquellos relacionados exclusivamente con tareas de depuración.
<code>/var/log/secure</code>	Mensajes de syslog relacionados con eventos de seguridad y autenticación.
<code>/var/log/maillog</code>	Mensajes de syslog relacionados con el servidor de correo.
<code>/var/log/cron</code>	Mensajes de syslog relacionados con la ejecución de trabajos programados.
<code>/var/log/boot.log</code>	Mensajes de la consola que no son de syslog relacionados con el inicio del sistema.

Tabla 4. Archivos de registro del sistema seleccionados. Fuente: elaboración propia.

Registro de eventos en el sistema

Muchos programas usan el protocolo syslog para registrar eventos en el sistema. Cada mensaje se clasifica por **facility** (tipo de mensaje) y **prioridad** (gravedad del mensaje). Los tipos (facilities) disponibles se detallan en la página de manual `rsyslog.conf`.

En la siguiente tabla, se enumeran las ocho **prioridades estándares de syslog**, de mayor a menor.

Código	Prioridad	Gravedad
0	emerg	El sistema no se puede usar.
1	alert	Se debe implementar una acción de inmediato.
2	crit	Condición crítica.
3	err	Condición de error no crítica.
4	warning	Condición de advertencia.
5	notice	Evento normal, pero importante.
6	info	Evento informativo.
7	depurar	Mensaje de nivel de depuración.

Tabla 5. Descripción general de las prioridades de syslog. Fuente: elaboración propia.

El servicio rsyslog usa el tipo (facility) y la prioridad de los mensajes de registro para determinar cómo resolverlos. Esto está configurado por **reglas** en el **archivo /etc/rsyslog.conf** y cualquier archivo en el **directorio /etc/rsyslog.d** que tiene una extensión de nombre de archivo de .conf. Los paquetes de software pueden agregar reglas fácilmente instalando un archivo apropiado en el directorio /etc/rsyslog.d.

Cada regla que controla cómo ordenar los mensajes de syslog es una línea en uno de los archivos de configuración. En el **lado izquierdo** de cada línea, se indican el **tipo** (facility) y la **gravedad** del mensaje de syslog que se corresponde con la regla. En el **lado derecho** de cada línea, se indica en qué archivo se debe **guardar** el mensaje de registro (o a donde más enviar el mensaje). Un asterisco (*) es un comodín que coincide con todos los valores.

Por ejemplo, la siguiente línea registraría los mensajes enviados al facility authpriv en cualquier prioridad para el archivo /var/log/secure:

authpriv.*

/var/log/secure

Los mensajes de registro a veces coinciden con **más de una regla** en `rsyslog.conf`. En tales casos, un mensaje se almacena en más de un archivo de registro. Para limitar los mensajes almacenados, la palabra clave **none** en el campo de prioridad señala que no se deben almacenar mensajes para el tipo (facility) indicado en el archivo dado.

En lugar de registrar mensajes de syslog en un archivo, también pueden imprimirse en las terminales de todos los usuarios que hayan iniciado sesión. El archivo **rsyslog.conf** tiene una configuración para imprimir todos los mensajes de syslog con la **prioridad emerg** en las terminales de todos los usuarios que hayan iniciado sesión.

Rotación del archivo de registro

La herramienta **logrotate** rota los archivos de registro para evitar que ocupen demasiado espacio en el sistema de archivos que contiene el directorio `/var/log`. Cuando se rota un archivo de registro, se le **cambia el nombre** con una extensión que indica la **fecha** en que se rotó. Por ejemplo, el archivo `/var/log/messages` anterior puede pasar a ser `/var/log/messages-20190130` si se rota el 30-1-2019. Una vez que se rotó el archivo de registro anterior, se crea un nuevo archivo de registro y se notifica al servicio que escribe en este.

Después de una determinada cantidad de rotaciones, habitualmente después de cuatro semanas, el archivo de registro más viejo se descarta para **liberar espacio** en disco. Un trabajo programado ejecuta el programa `logrotate` **a diario** para verificar si es necesario rotar algún registro. La mayoría de los archivos de registro se rotan **semanalmente**, pero `logrotate` rota un poco más rápidamente o más lento, o cuando alcanza un **tamaño determinado**.

La configuración de `logrotate` no se aborda en este curso. Si desea obtener más información, consulte la página del **Manual logrotate(8)**.

Análisis de una entrada de syslog

Los mensajes de registro comienzan con el mensaje **más antiguo** en la parte superior y el mensaje **más nuevo** al final del archivo de registro. El servicio rsyslog usa un **formato estándar** al registrar las entradas en los archivos de registro. En el siguiente ejemplo, se explica la anatomía de un mensaje de registro en el archivo de registro `/var/log/secure`.

```
Feb 11 20:11:48 localhost sshd[1433]: Failed password for student from
172.25.0.10 port 59344 ssh2
```

La marca de tiempo cuando se grabó la entrada de registro.

El host desde donde se envió el mensaje de registro.

El nombre del programa o el proceso y el número de PID que envió el mensaje de registro.

El mensaje real enviado.

Monitoreo de registros

Para reproducir problemas e inconvenientes, es útil monitorear uno o más archivos de registro para eventos. El **comando `tail -f /path/to/file`** proporciona las últimas diez líneas del archivo especificado y continúa ofreciendo líneas nuevas en el archivo a medida que se escriben.

Por ejemplo, para monitorear los intentos fallidos de inicio de sesión, ejecute el comando **tail** en un terminal y, luego, en otro terminal, ejecute el comando **ssh** como el usuario root mientras un usuario intenta iniciar sesión en el sistema.

En el primer terminal, ejecute el siguiente comando tail:

```
[root@host ~]# tail -f /var/log/secure
```

En el segundo terminal, ejecute el siguiente comando **ssh**:

```
[root@host ~]# ssh root@localhost
root@localhost's password: redhat
...output omitted...
[root@host ~]#
```

Envío manual de mensajes Syslog

El **comando logger** puede enviar mensajes al servicio rsyslog. De manera predeterminada, envía el mensaje al tipo (**facility**) **user** con la notice prioridad (**user.notice**), a menos que se especifique lo contrario con la opción **-p**. Es útil probar los cambios en la configuración del servicio rsyslog.

Para enviar un mensaje **al servicio rsyslog** que se graba en el archivo de registro `/var/log/boot.log`, ejecute el siguiente **comando logger**:

```
[root@host ~]# logger -p local7.notice "Log entry created on host"
```

Cómo encontrar eventos

El servicio **systemd-journald** almacena datos de registro en un archivo **binario estructurado e indexado**, que se denomina diario (journal). Estos datos incluyen información adicional sobre el evento de registro. Por ejemplo, en el caso de los eventos de syslog, esto incluye el tipo (facility) y la prioridad del mensaje original.

```
[root@host ~]# journalctl
...output omitted...
Feb 21 17:46:25 host.lab.example.com systemd[24263]: Stopped target Sockets.
Feb 21 17:46:25 host.lab.example.com systemd[24263]: Closed D-Bus User
Message Bus Socket.
lines 1464-1487/1487 (END) q
```

El **comando journalctl** destaca los mensajes de registro importantes: los mensajes con prioridad **notice o warning** se muestran en negrita y los mensajes con prioridad **error o una prioridad mayor** se muestran en rojo.

La clave para usar en forma correcta el diario (journal) para la solución de problemas y auditorías es **limitar las búsquedas** en el diario (journal) para mostrar solo la salida relevante.

De manera predeterminada, **journalctl -n** muestra las diez últimas entradas de registro. Puede ajustar esto con un argumento opcional que especifique cuántas entradas de registro se mostrarán.

2.8. Administración de redes

Recopilación de la información de interfaz de red

Identificación de interfaces de red

RedHat (2022) afirma que el comando `ip link` listará todas las interfaces de red disponibles en su sistema:

```
[user@host ~]$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode
DEFAULT group default qlen 1000
    link/ether 52:54:00:00:00:0a brd ff:ff:ff:ff:ff:ff
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode
DEFAULT group default qlen 1000
    link/ether 52:54:00:00:00:1e brd ff:ff:ff:ff:ff:ff
```

En el ejemplo anterior, el servidor tiene tres interfaces de red: el **dispositivo de bucle** invertido que está conectado al propio servidor y **dos interfaces Ethernet**, ens3 y ens4.

Para configurar cada interfaz de red correctamente, necesita saber **cuál** está conectada **a qué red**. En muchos casos, sabrá la dirección MAC de la interfaz conectada a cada red, ya sea porque está impresa físicamente en la tarjeta o el servidor o porque es una máquina virtual y usted sabe cómo está configurada.

La **dirección MAC** del dispositivo aparece después de link/ether para cada interfaz. Así que ya sabes que la tarjeta de red con la dirección MAC 52:54:00:00:00:0a es la interfaz de red ens3.

Visualización de las direcciones IP

Use el **comando ip** para ver la información del dispositivo y de la dirección. Una sola interfaz de red puede tener varias direcciones IPv4 o IPv6.

```
[user@host ~]$ ip addr show ens3
```

```
2: ens3: <BROADCAST,MULTICAST, UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
```

Visualización de estadísticas de rendimiento

El comando **ip** también puede usarse para mostrar las estadísticas sobre el **rendimiento** de la red. Se pueden usar contadores para cada interfaz de red para identificar la presencia de **problemas de red**. Los contadores registran estadísticas para cosas como la cantidad de paquetes recibidos (RX) y transmitidos (TX), errores de paquetes y paquetes que se omitieron.


```
[user@host ~]$ ip -s link show ens3
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UP qlen 1000
link/ether 52:54:00:00:00:0a brd ff:ff:ff:ff:ff:ff
    RX: bytes  packets  errors  dropped overrun mcast
    269850      2931      0        0          0        0
    TX: bytes  packets  errors  dropped carrier collsns
    300556      3250      0        0          0        0
```

Comprobación de la conectividad entre hosts

El **comando ping** se usa para comprobar la **conectividad**. El comando continúa ejecutándose hasta que se presione **Ctrl+c**, a menos que se indiquen otras opciones para limitar la cantidad de paquetes enviados.

```
[user@host ~]$ ping -c3 192.0.2.254
PING 192.0.2.1 (192.0.2.254) 56(84) bytes of data.
64 bytes from 192.0.2.254: icmp_seq=1 ttl=64 time=4.33 ms
64 bytes from 192.0.2.254: icmp_seq=2 ttl=64 time=3.48 ms
64 bytes from 192.0.2.254: icmp_seq=3 ttl=64 time=6.83 ms

--- 192.0.2.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 3.485/4.885/6.837/1.424 ms
```

El comando **ping6** es la versión de IPv6 de **ping en Linux**. Se comunica a través de **IPv6** y toma direcciones IPv6, pero de lo contrario funciona como ping.

```
[user@host ~]$ ping6 2001:db8:0:1::1
PING 2001:db8:0:1::1(2001:db8:0:1::1) 56 data bytes
64 bytes from 2001:db8:0:1::1: icmp_seq=1 ttl=64 time=18.4 ms
64 bytes from 2001:db8:0:1::1: icmp_seq=2 ttl=64 time=0.178 ms
64 bytes from 2001:db8:0:1::1: icmp_seq=3 ttl=64 time=0.180 ms
^C
--- 2001:db8:0:1::1 ping statistic ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.178/6.272/18.458/8.616 ms
[user@host ~]$
```

Cuando hace ping a las direcciones de vínculo local y al grupo multidifusión de todos los nodos de vínculo local (ff02::1), la interfaz de red que se debe usar debe especificarse explícitamente con un identificador de zona de alcance (como ff02::1%ens3). Si esto se deja afuera, se muestra el **error connect: Invalid argument**.

Solución de problemas de ruta

El enrutamiento de la red es complejo y, a veces, el tráfico no se comporta como se espera. Aquí hay algunas herramientas de diagnóstico útiles.

Visualización de la tabla de enrutamiento

Use el **ip** comando con el **route Opción** para mostrar información de enrutamiento.

```
[user@host ~]$ ip route
default via 192.0.2.254 dev ens3 proto static metric 1024
192.0.2.0/24 dev ens3 proto kernel scope link src 192.0.2.2
10.0.0.0/8 dev ens4 proto kernel scope link src 10.0.0.11
```

Esto muestra la tabla de **enrutamiento de IPv4**. Todos los paquetes que estén destinados para la red 10.0.0.0/8 se envían directamente al destino mediante la ens4 del dispositivo. Todos los paquetes que estén destinados para la red 192.0.2.0/24 se envían directamente al destino mediante la ens3 del dispositivo. Todos los demás

paquetes se envían al enrutador predeterminado que está ubicado en 192.0.2.254, y también mediante la ens3 del dispositivo.

Seguimiento de rutas tomadas por el tráfico

Para rastrear la ruta que toma el tráfico de la red para llegar a un host remoto a través de múltiples enrutadores, use **tracert** o **tracpath**. Esto puede identificar si hay un problema con uno de sus enrutadores o con uno intermedio. Ambos comandos usan paquetes de **UDP** para realizar el seguimiento de una ruta de forma predeterminada; sin embargo, muchas redes bloquean el tráfico de UDP e ICMP. El comando **tracert** tiene opciones para realizar el seguimiento de la ruta con paquetes UDP (predeterminado), ICMP (-I) o TCP (-T). Sin embargo, en general, el comando **tracert** no está instalado de forma predeterminada.

```
[user@host ~]$ tracpath access.redhat.com
...output omitted...
 4:  71-32-28-145.rcmt.qwest.net                48.853ms asymm  5
 5:  dcp-brdr-04.inet.qwest.net                 100.732ms asymm  7
 6:  206.111.0.153.ptr.us.xo.net                 96.245ms asymm  7
 7:  207.88.14.162.ptr.us.xo.net                 85.270ms asymm  8
 8:  ae1d0.cir1.atlanta6-ga.us.xo.net            64.160ms asymm  7
 9:  216.156.108.98.ptr.us.xo.net                108.652ms
10:  bu-ether13.atlmgamq46w-bcr00.tbone.rr.com  107.286ms asymm 12
...output omitted...
```

Cada línea de la salida de **tracpath** representa un **router o hop** por donde pasa el paquete entre el origen y el destino final. Proporciona información adicional como disponible que incluye la sincronización en ambos sentidos (RTT) y cualquier cambio en el tamaño de la unidad de transmisión máxima (MTU). La **indicación asymm** significa que el tráfico llegó a ese enrutador y regresó de ese enrutador usando diferentes rutas (asimétricas). Los enrutadores que se muestran son los que se usan para el **tráfico saliente**, no el tráfico de retorno.

Solución de problemas en puertos y servicios

Los servicios TCP usan **sockets** como extremos para la comunicación y se componen de una dirección IP, protocolo y número de puerto. En general, los servicios están atentos a los puertos estándares mientras que los clientes usan un puerto disponible en forma aleatoria. Los nombres más conocidos de puertos estándares están enumerados en el archivo **/etc/services**.

El comando **ss** se usa para mostrar las **estadísticas** del socket. El comando **ss** tiene por objeto **reemplazar** la herramienta anterior **netstat** que es parte del paquete **net-tools**, que algunos administradores de sistemas pueden conocer más, pero que no siempre está instalada.

```
[user@host ~]$ ss -ta
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	128	*:sunrpc	*:*
LISTEN	0	128	*:ssh	*:*
LISTEN	0	100	127.0.0.1:smtp	*:*
LISTEN	0	128	*:36889	*:*

Descripción de conceptos de NetworkManager

NetworkManager es un daemon que monitorea y administra los parámetros de red.

Además del daemon, hay un applet del área de notificaciones de **GNOME** que proporciona información sobre el estado de la red. Las herramientas gráficas y de la línea de comandos se comunican con **NetworkManager** y guardan archivos de configuración en el directorio **/etc/sysconfig/network-scripts**.

Un **dispositivo** es una interfaz de red.

Una **conexión** es un grupo de parámetros que pueden configurarse para un dispositivo.

Solo puede haber una **conexión activa** para un dispositivo por vez. Pueden existir múltiples conexiones para que las usen distintos dispositivos o para permitir que una configuración se modifique para el mismo dispositivo. Si necesita cambiar temporalmente la configuración de red, en lugar de cambiar la configuración de una conexión, puede cambiar la conexión que está activa para un dispositivo. Por ejemplo, un dispositivo para una interfaz de red inalámbrica en una laptop puede usar diferentes conexiones para la red inalámbrica en un sitio de trabajo y para la red inalámbrica en el hogar.

Cada conexión tiene un nombre o **ID** que la identifica.

La utilidad **nmcli** se usa para crear y editar archivos de conexión desde la línea de comandos.

Visualización de información de redes

El comando **nmcli dev status** muestra el estado de todos los dispositivos de red:

```
[user@host ~]$ nmcli dev status
```

DEVICE	TYPE	STATE	CONNECTION
eno1	ethernet	connected	eno1
ens3	ethernet	connected	static-ens3
eno2	ethernet	disconnected	--
lo	loopback	unmanaged	--

El comando **nmcli con show** muestra una lista de todas las conexiones. Para enumerar solo las conexiones activas, agregue la opción **--active**.

```
[user@host ~]$ nmcli con show
```

NAME	UUID	TYPE	DEVICE
eno2	ff9f7d69-db83-4fed-9f32-939f8b5f81cd	802-3-ethernet	--
static-ens3	72ca57a2-f780-40da-b146-99f71c431e2b	802-3-ethernet	ens3
eno1	87b53c56-1f5d-4a29-a869-8a7bdaf56dfa	802-3-ethernet	eno1

```
[user@host ~]$ nmcli con show --active
```

NAME	UUID	TYPE	DEVICE
static-ens3	72ca57a2-f780-40da-b146-99f71c431e2b	802-3-ethernet	ens3
eno1	87b53c56-1f5d-4a29-a869-8a7bdaf56dfa	802-3-ethernet	eno1

Cambio de nombre del host del sistema

El comando **hostname** muestra o modifica provisoriamente el nombre del host totalmente calificado del sistema.

```
[root@host ~]# hostname
```

```
host@example.com
```

Puede especificarse un nombre del host estático en el archivo `/etc/hostname`. Se usa el comando `hostnamectl` para modificar este archivo y puede usarse para ver el estado del nombre del host totalmente calificado del sistema. Si este archivo no existe, el nombre del host se establece mediante una consulta de DNS invertida una vez que la interfaz tiene una dirección IP asignada.

```
[root@host ~]# hostnamectl set-hostname host@example.com
[root@host ~]# hostnamectl status
  Static hostname: host.example.com
        Icon name: computer-vm
        Chassis: vm
        Machine ID: 73ab164e278e48be9bf80e80714a8cd5
        Boot ID: 6b1cbc4177164ef58c0e9ed4adb2904f
  Virtualization: kvm
  Operating System: Red Hat Enterprise Linux 8.0 beta (Ootpa)
        CPE OS Name: cpe:/o:redhat:enterprise_linux:8.0:beta
        Kernel: Linux 4.18.0-60.el8.x86_64
  Architecture: x86-64
[root@host ~]# cat /etc/hostname
host@example.com
[root@host ~]# cat /etc/resolv.conf
# Generated by NetworkManager
domain example.com
search example.com
nameserver 172.25.254.254
```

NetworkManager actualiza el archivo `/etc/resolv.conf` con los parámetros de configuración de DNS en los archivos de configuración de conexión. Use **nmcli** para modificar las conexiones.

Prueba de resolución de nombres DNS

El comando de host **HOSTNAME** puede usarse para probar la **conectividad** del servidor DNS.

```
[root@host ~]# host classroom.example.com
classroom.example.com has address 172.25.254.254
[root@host ~]# host 172.25.254.254
254.254.25.172.in-addr.arpa domain name pointer classroom.example.com.
```

DHCP reescribe automáticamente el archivo `/etc/resolv.conf` a medida que se inician las interfaces, a menos que usted especifique `PEERDNS=no` en los archivos de configuración de interfaz correspondientes. Configure esto con el **comando nmcli**.

```
[root@host ~]# nmcli con mod "static-ens3" ipv4.ignore-auto-dns yes
```

2.9. Almacenamiento y transferencia de archivos

El comando tar

RedHat (2022) afirma que el almacenamiento y la compresión de archivos es útil cuando se realizan **copias de seguridad** y se transfieren datos a través de una **red**. Uno de los comandos más antiguos y usados para crear y trabajar con colecciones de archivos de copias de seguridad es el comando **tar**.

Con el comando tar, los usuarios pueden reunir grandes conjuntos de archivos en un solo archivo (colección de archivos).

Una colección de archivos tar es una secuencia estructurada de datos de archivos mezclados con metadatos sobre cada archivo y un índice para poder extraer archivos individuales.

La colección de archivos puede comprimirse con **gzip**, **bzip2** o **xz**.

Con el comando tar, también se puede **enumerar** el contenido de las colecciones de archivos o **extraer** sus archivos a su sistema actual.

Almacenamiento de archivos y directorios

La primera opción para usar cuando se crea una colección de archivos nueva es la opción **c**, seguida de la opción **f**. Luego, un solo espacio, el nombre de archivo de la colección de archivos que se creará y, por último, la lista de los archivos y directorios que deben agregarse al archivo. La colección de archivos se crea en el **directorio actual**, a menos que se especifique de otro modo.

Antes de crear una colección de archivos tar, **verifique** que no haya otra colección de archivos en el directorio con el mismo nombre de la colección de archivos nueva que se creará. El comando tar **sobrescribe** una colección de archivos existente **sin advertencia**.

Con el siguiente comando, se crea una colección de archivos con el nombre archive.tar con el contenido de file1, file2 y file3 en el directorio de inicio del usuario.

```
[user@host ~]$ tar -cf archive.tar file1 file2 file3
[user@host ~]$ ls archive.tar
archive.tar
```

Para que tar pueda archivar los archivos seleccionados, es obligatorio que el **usuario** que ejecute el comando tar pueda **leer** los archivos. Por ejemplo, la creación de una colección de archivos nueva de la carpeta /etc y todo su contenido requiere privilegios de root porque solo este tipo de usuario root tiene permitido leer todos los archivos que están presentes en el directorio /etc.

Un usuario sin privilegios puede crear una colección de archivos del directorio /etc, pero la colección de archivos **omite los archivos** que no incluyen el permiso de lectura para el usuario y **omite los directorios** que no incluyen permiso de lectura y ejecución para el usuario.

Para crear la colección de archivos tar con el nombre /root/etc.tar, con el directorio /etc como contenido con el usuario root:

```
[root@host ~]# tar -cf /root/etc.tar /etc
tar: Removing leading `/' from member names
[root@host ~]#
```

Transferencia de archivos con copia de segura

OpenSSH es útil para ejecutar los comandos de shell en sistemas remotos en forma segura. El comando de copia segura, **scp**, parte del conjunto OpenSSH, copia archivos desde un sistema remoto al sistema local o desde el sistema local a un sistema remoto. El comando usa el **servidor SSH** para la autenticación y cifra datos cuando se transfieren.

Puede especificar una ubicación remota para el origen o destino de los archivos que está copiando. El formato de la ubicación remota debe ser en la forma `[user@]host:/path`. Los `user@` parte del argumento es opcional. Si falta, se utilizará su nombre de usuario local actual. Cuando ejecute el comando, su **cliente scp** se autenticará en el servidor SSH remoto al igual que `ssh`, utilizando la autenticación basada en claves o le pide su contraseña.

En el siguiente ejemplo, se muestra cómo copiar los archivos locales `/etc/yum.conf` y `/etc/hosts` en `host`, en el directorio de inicio de `remoteuser` en el **sistema remoto** `remotehost`:

```
[user@host ~]$ scp /etc/yum.conf /etc/hosts
remoteuser@remotehost:/home/remoteuser
remoteuser@remotehost's password: password
yum.conf          100% 813   0.8KB/s   00:00
hosts             100% 227   0.2KB/s   00:00
```

También puede copiar un archivo en la otra dirección, desde un sistema remoto al sistema de archivos local. En este ejemplo, el archivo `/etc/hostname` en `remotehost` se copia al directorio local `/home/user`. El comando **scp autentica a remotehost** como usuario `remoteuser`.

```
[user@host ~]$ scp remoteuser@remotehost:/etc/hostname /home/user
remoteuser@remotehost's password: password
hostname                                100%  22    0.0KB/s   00:00
```

Para copiar un **árbol de directorios completo** de manera recursiva, use la opción **-r**. En el siguiente ejemplo, el directorio remoto `/var/log` en `remotehost` se copia de manera recursiva en el directorio local `/tmp/` en `host`. Debe conectarse al sistema remoto como `root` para asegurarse de que puede leer todos los archivos en el directorio remoto `/var/log`.

```
[user@host ~]$ scp -r root@remoteuser:/var/log /tmp
root@remotehost's password: password
...output omitted...
```

Transferencia de archivos con el programa de transferencia segura de archivos

Para cargar o descargar archivos de forma interactiva desde un servidor SSH, use el programa de transferencia de archivos, **sftp**. Una sesión con el comando `sftp` usa el mecanismo de **autenticación segura** y la transferencia de **datos cifrados** desde y hacia el servidor SSH.

Al igual que con el comando `scp`, el comando `sftp` usa `[user@]host` para identificar el sistema de **destino** y el **nombre** de usuario. Si no especifica un usuario, el comando intentará iniciar sesión utilizando su nombre de usuario local como nombre de usuario remoto. A continuación, se le presentará un prompt `sftp>`.

```
[user@host ~]$ sftp remoteuser@remotehost
remoteuser@remotehost's password: password
Connected to remotehost.
sftp>
```

La sesión **sftp** interactiva acepta diversos comandos que funcionan de la misma manera en el sistema de archivos remoto que en el sistema de archivos local, como **ls**, **cd**, **mkdir**, **rmdir** y **pwd**. El comando **put** carga un archivo en el sistema remoto. El comando **get** descarga un archivo desde el sistema remoto. El comando **exit** finaliza la sesión sftp.

Sincronizar archivos y directorios con rsync

El uso del comando **rsync** es otra forma de copiar archivos en forma segura de un sistema a otro. La herramienta usa un **algoritmo** que minimiza la cantidad de datos copiados sincronizando solo las partes que se modificaron de los archivos.

Se diferencia de **scp** en que, si dos archivos o directorios son similares entre dos sistemas, rsync copia las **diferencias** entre los sistemas de archivos, mientras que scp copiaría todo.

Una **ventaja** de rsync es que puede **copiar** archivos entre un sistema local y un sistema remoto en forma **segura y eficiente**. Cuando una sincronización inicial de un directorio demora prácticamente el mismo tiempo que el copiado, las sincronizaciones posteriores solo requerirán que se copien las diferencias a través de la red, lo que acelera sustancialmente las actualizaciones.

Una opción importante de rsync es la **opción -n** para realizar un **simulacro**. Un simulacro es una imitación de lo que sucede cuando el comando se ejecuta. El simulacro muestra los cambios que rsync ejecutaría cuando el comando se ejecuta normalmente. Realice un simulacro antes de la operación de rsync real para garantizar que no se sobrescriban ni eliminen archivos importantes.

Las dos **opciones más comunes** cuando se sincronizan archivos y directorios con rsync son **-v** y **-a**.

La opción `-v` o `--verbose` proporciona un resultado más detallado. Esto es útil para solucionar problemas y para ver el progreso en vivo.

Puede usar **rsync** para sincronizar el contenido de un archivo o directorio local con un archivo o directorio en una **máquina remota**, utilizando cualquiera de las dos máquinas como origen. También puede sincronizar el contenido de dos archivos o directorios locales.

Por ejemplo, para sincronizar contenido del directorio `/var/log` con el directorio `/tmp`:

```
[user@host ~]$ su -
Password: password
[root@host ~]# rsync -av /var/log /tmp
receiving incremental file list
log/
log/README
log/boot.log
...output omitted...
log/tuned/tuned.log

sent 11,592,423 bytes  received 779 bytes  23,186,404.00 bytes/sec
total size is 11,586,755  speedup is 1.00
[user@host ~]$ ls /tmp
log  ssh-RLjDdarkKiW1
[user@host ~]$
```

Una barra al final del directorio de origen sincroniza el contenido de un directorio sin crear el subdirectorio en el directorio de destino. En este ejemplo, el directorio `log` no se crea en el directorio `/tmp`, solo el contenido de `/var/log/` se sincroniza en `/tmp`.

```
[root@host ~]# rsync -av /var/log/ /tmp
sending incremental file list
./
README
boot.log
...output omitted...
tuned/tuned.log

sent 11,592,389 bytes  received 778 bytes  23,186,334.00 bytes/sec
total size is 11,586,755  speedup is 1.00
[root@host ~]# ls /tmp
anaconda                dnf.rpm.log-20190318  private
audit                   dnf.rpm.log-20190324  qemu-ga
boot.log                 dnf.rpm.log-20190331  README
...output omitted...
```

Importante

Cuando se ingrese el directorio de origen en el comando **rsync**, es importante si está presente la barra final en el nombre del directorio. Esto determina si el directorio o solo el contenido del directorio se sincroniza en el destino.

La terminación con **tab** de Bash agrega automáticamente una **barra** al final de los nombres de directorios.

Al igual que con los comandos **scp** y **sftp**, **rsync** especifica **ubicaciones remotas** usando el formato `[user@]host:/path`. La ubicación remota puede ser el sistema de origen o destino, pero una de las dos máquinas debe ser local.

Para preservar la propiedad del archivo, debe ser root en el sistema de destino. Si el destino es **remoto**, autentiquese como **root**. Si el destino es **local**, debe ejecutar **rsync** como root.

2.10. Acceso a los sistemas de archivos de Linux.

Actualizaciones

Conceptos de la administración del almacenamiento

Conceptos de la administración del almacenamiento

RedHat (2022) afirma que se accede a los archivos en un servidor Linux a través de la **jerarquía** de sistemas de archivos, un solo **árbol de directorios invertido**. Esta jerarquía del sistema de archivos se ensambla desde sistemas de archivos provistos por los dispositivos de almacenamiento disponibles para su sistema. Cada **sistema de archivos** es un dispositivo de almacenamiento que ha sido formateado para almacenar archivos.

En cierto sentido, la jerarquía de sistemas de archivos Linux presenta una colección de sistemas de archivos en dispositivos de almacenamiento separados como si fuera un conjunto de archivos en un dispositivo de **almacenamiento gigante** en el que puede navegar. La mayoría de las veces no necesita saber en qué dispositivo de almacenamiento se encuentra un archivo en particular, solo necesita saber en qué **directorio** se encuentra el archivo.

Sin embargo, a veces puede ser importante. Es posible que deba determinar qué tan lleno está un dispositivo de almacenamiento y qué directorios en la jerarquía de sistemas de archivos son afectados. Es posible que haya **errores en los registros** de un dispositivo de almacenamiento y, por lo tanto, necesita saber qué sistemas de archivos están en riesgo. Es posible que solo desee crear un **vínculo rígido** entre dos archivos y necesita saber si están en el mismo sistema de archivos para determinar si es posible.

Sistemas de archivos y puntos de montaje

Para que los contenidos de un sistema de archivos estén disponibles en la jerarquía de sistemas de archivos, se debe montar en un **directorio vacío**. Este directorio se llama **punto de montaje**. Una vez montado, si usa **ls** para enumerar ese directorio, verá el contenido del sistema de archivos montado y puede acceder y, luego, usar esos archivos normalmente. Muchos sistemas de archivos se montan **automáticamente** como parte del proceso de arranque.

Si solo ha trabajado con letras de unidad de Microsoft Windows, este es un concepto fundamentalmente diferente. Es algo similar a la característica de carpetas montadas de NTFS.

Sistemas de archivos, almacenamiento y dispositivos de bloque

El acceso de **bajo nivel** a los dispositivos de almacenamiento en Linux se proporciona mediante un tipo especial de archivo llamado **dispositivo de bloque**. Estos dispositivos de bloque deben **formatearse** con un sistema de archivos para poder montarlos.

Los archivos de dispositivos de bloque se almacenan en el **directorio /dev**, junto con otros archivos de dispositivo. Los archivos del dispositivo son creados automáticamente por el sistema operativo. En muchas distribuciones Linux el **primer disco duro** SATA/PATA, SAS, SCSI o USB detectado se denomina **/dev/sda**, el segundo **/dev/sdb**, y así sucesivamente. Estos nombres representan el disco duro en su totalidad.

Otros tipos de almacenamiento tendrán otras formas de asignar nombres.

Tipo de dispositivo	Patrón de nomenclatura de dispositivos
Almacenamiento adjunto a SATA/SAS/USB.	/dev/sda , /dev/sdb ...
Almacenamiento paravirtualizado virtio-blk (algunas máquinas virtuales).	/dev/vda , /dev/vdb ...
Almacenamiento adjunto a NVMe (muchos SSD).	/dev/nvme0, /dev/nvme1 ...
Almacenamiento SD/MMC/eMMC (tarjetas SD).	/dev/mmcblk0, /dev/mmcblk1 ...

Tabla 6. Nomenclatura de dispositivos de bloque. Fuente: elaboración propia.

Nota

Muchas máquinas virtuales utilizan el almacenamiento paravirtualizado virtio-scsi más nuevo que tendrá la nomenclatura de estilo /dev/sd*.

Particiones del disco

Normalmente, no hace que todo el dispositivo de almacenamiento se convierta en un sistema de archivos. Los dispositivos de almacenamiento normalmente se dividen en **fragmentos** más pequeños llamados **particiones**.

Las particiones le permiten **compartimentar** un disco; las diferentes particiones pueden formatearse con diferentes sistemas de archivos o usarse con fines distintos. Por ejemplo, una partición puede contener directorios de inicio de un usuario mientras que otra puede contener registros y datos del sistema. Si un usuario llena la partición del directorio de inicio con datos, la partición del sistema puede seguir teniendo espacio disponible.

Las particiones son **dispositivos de bloque** por derecho propio. En el almacenamiento adjunto a **SATA**, la primera partición en el primer disco es /dev/sda1. La tercera partición en el segundo disco es /dev/sdb3 y así sucesivamente. Los dispositivos de almacenamiento paravirtualizados tienen un sistema de nomenclatura similar.

Los nombres del dispositivo **SSD** adjunto a NVMe nombra sus particiones de manera diferente. En ese caso, la primera partición en el primer disco es `/dev/nvme0p1`. La tercera partición en el segundo disco es `/dev/nvme1p3` y así sucesivamente. Las tarjetas **SD o MMC** tienen un sistema de nomenclatura similar.

Un listado extenso del archivo de dispositivo `/dev/sda1` en host revela que su tipo de archivo especial es **b**, que significa «dispositivo de bloque».

```
[user@host ~]$ ls -l /dev/sda1  
brw-rw----. 1 root disk 8, 1 Feb 22 08:00 /dev/sda1
```

Volúmenes lógicos

Otra manera de **organizar discos y particiones** es mediante la administración de volúmenes lógicos (**LVM**). Con la administración de volúmenes lógicos, uno o más dispositivos de bloque pueden agregarse a un conjunto de almacenamiento denominado **grupo de volúmenes**. El espacio en disco en el grupo de volúmenes se divide en uno o más volúmenes lógicos, que son el equivalente funcional de una partición que reside en un disco físico.

El sistema LVM **asigna nombres** a los grupos de volúmenes y volúmenes lógicos en el momento de la creación. LVM crea un **directorio** en `/dev` que coincide con el nombre del grupo y, luego, crea un **enlace simbólico** dentro de ese nuevo directorio con el mismo nombre que el volumen lógico. Ese archivo de volumen lógico está disponible para ser montado.

Por ejemplo, si un grupo de volúmenes se llama `myvg` y el volumen lógico dentro de este se llama `mylv`, entonces el nombre completo de la ruta al archivo de dispositivo de volumen lógico es `/dev/myvg/mylv`.

La forma del nombre del dispositivo de volumen lógico mencionado anteriormente se implementa, en realidad, como un enlace simbólico al archivo de dispositivo real

utilizado para acceder a este que puede variar entre los arranques. Hay otra forma de nombre de dispositivo de volumen lógico vinculado desde archivos en **/dev/mapper** que se utilizan **a menudo** y, también, son enlaces simbólicos al archivo del dispositivo real.

Examen de sistemas de archivos

Para obtener una descripción general de los dispositivos de sistemas de archivos locales y remotos y la cantidad de espacio libre disponible, ejecute el **comando df**. Cuando el comando **df** se ejecuta sin argumentos, **arroja un informe** con el espacio en disco total, el espacio en disco usado, el espacio en disco libre y el porcentaje del espacio total en disco usado en todos los sistemas de archivos regulares montados. Informa sobre sistemas de **archivos locales y remotos**.

En el siguiente ejemplo, se muestran los sistemas de archivos y los puntos de montaje en host.

```
[user@host ~]$ df
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
devtmpfs	912584	0	912584	0%	/dev
tmpfs	936516	0	936516	0%	/dev/shm
tmpfs	936516	16812	919704	2%	/run
tmpfs	936516	0	936516	0%	/sys/fs/cgroup
/dev/vda3	8377344	1411332	6966012	17%	/
/dev/vda1	1038336	169896	868440	17%	/boot
tmpfs	187300	0	187300	0%	/run/user/1000

Las particiones en el sistema **host** muestran dos sistemas de archivos físicos, que se montan en **/** y **/boot**. Esto es frecuente en el caso de **máquinas virtuales**. Los dispositivos **tmpfs** y **devtmpfs** son sistemas de archivos en la memoria del sistema. Todos los archivos escritos en **tmpfs** o en **devtmpfs** desaparecen después de que se reinicia el sistema.

A fin de **mejorar la legibilidad** de los tamaños de las salidas, hay dos opciones diferentes legibles por el ojo humano: **-h** o **-H**. La diferencia entre estas dos opciones es que **-h** informa en KiB (210), MiB (220), o GiB (230), mientras que la opción **-H** informa en unidades SI: KB (103), MB (106), GB (109). Los fabricantes de discos duros normalmente usan las unidades **SI** cuando anuncian sus productos.

Muestre un informe sobre los sistemas de archivos en el sistema host con todas las unidades convertidas a formato legible por el ojo humano:

```
[user@host ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        892M   0    892M   0% /dev
tmpfs           915M   0    915M   0% /dev/shm
tmpfs           915M  17M   899M   2% /run
tmpfs           915M   0    915M   0% /sys/fs/cgroup
/dev/vda3       8.0G  1.4G   6.7G  17% /
/dev/vda1      1014M  166M   849M  17% /boot
tmpfs          183M   0    183M   0% /run/user/1000
```

Para obtener información más detallada sobre el **espacio usado** por un árbol de directorios en particular, use el comando **du**. El comando **du** ofrece las opciones **-h** y **-H** para convertir la salida a formato legible por el ojo humano. El comando **du** muestra el **tamaño** de todos los **archivos** en el árbol de directorios actual de modo recursivo.

Muestre un informe sobre el uso del disco para el directorio **/usr/share** en host:

```
[root@host ~]# du /usr/share
...output omitted...
176 /usr/share/smartmontools
184 /usr/share/nano
8 /usr/share/cmake/bash-completion
8 /usr/share/cmake
356676 /usr/share
```

Muestre un informe sobre el uso del disco en formato legible por el ojo humano, para el directorio /usr/share en host:

```
[root@host ~]# du -h /var/log
...output omitted...
176K  /usr/share/smartmontools
184K  /usr/share/nano
8.0K  /usr/share/cmake/bash-completion
8.0K  /usr/share/cmake
369M  /usr/share
```

Montaje manual de sistemas de archivos

Un sistema de archivos que reside en un dispositivo de almacenamiento extraíble debe montarse para acceder a este. El comando **mount** permite que el usuario root monte **manualmente** un sistema de archivos. El primer argumento del comando mount especifica el sistema de archivos que se debe montar. El segundo argumento especifica el directorio que se usará como punto de montaje en la jerarquía de sistemas de archivos.

Hay dos formas comunes de especificar el sistema de archivos en una partición de disco para el comando mount:

Con el nombre del archivo del dispositivo en /dev que contiene el sistema de archivos.

Con el UUID escrito en el sistema de archivos, un identificador universalmente único.

El montaje de un dispositivo es relativamente simple. Debe **identificar** el dispositivo que desea montar, **asegurarse** de que exista el punto de montaje y **montar** el dispositivo en el punto de montaje.

Identificación del dispositivo de bloque

Un dispositivo de almacenamiento conectable en funcionamiento, ya sea como unidad de disco duro (HDD) o dispositivo de estado sólido (SSD) o un portadiscos de almacenamiento USB, pueden conectarse a un **puerto diferente** cada vez que se conectan a un sistema.

Use el comando **lsblk** para enumerar los detalles de un dispositivo de bloque especificado o todos los dispositivos disponibles.

```
[root@host ~]# lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
vda	253:0	0	12G	0	disk	
└─vda1	253:1	0	1G	0	part	/boot
└─vda2	253:2	0	1G	0	part	[SWAP]
└─vda3	253:3	0	11G	0	part	/
vdb	253:16	0	64G	0	disk	
└─vdb1	253:17	0	64G	0	part	

Si sabe que acaba de agregar un dispositivo de almacenamiento de 64 GB con una partición, puede deducir a partir del resultado anterior que `/dev/vdb1` es la partición que desea montar.

Montaje por nombre de dispositivo de bloque

El siguiente ejemplo monta el sistema de archivos en la partición `/dev/vdb1` en el directorio `/mnt/data`.

```
[root@host ~]# mount /dev/vdb1 /mnt/data
```

Para montar un sistema de archivos, ya **debe existir** el directorio de destino. El directorio `/mnt` existe de forma predeterminada y está destinado a ser utilizado como punto de montaje temporal.

Puedes usar el **directorio /mnt**, o mejor aún crear un **subdirectorio de /mnt** para usar como punto de montaje temporal, a menos que tenga una buena razón para montarlo en una ubicación específica en la jerarquía de sistemas de archivos.

Importante

Si el directorio que funciona como punto de montaje no está vacío, no se puede acceder a los archivos copiados en ese directorio antes de que el sistema de archivos se montara hasta que el sistema de archivos se desmonte nuevamente.

Este enfoque funciona bien a **corto plazo**. Sin embargo, el orden en que el sistema operativo detecta discos puede cambiar si se agregan o eliminan dispositivos del sistema. Esto cambiará el nombre del dispositivo asociado con ese dispositivo de almacenamiento. Un mejor enfoque sería montar por alguna **característica** incorporada en el sistema de archivos.

Montaje por UUID de sistema de archivos

Un identificador estable que está asociado con un sistema de archivos es su UUID, un **número hexadecimal** muy largo que actúa como **identificador único universal**. Este UUID es parte del sistema de archivos y permanece igual siempre que el sistema de archivos no se recree.

El comando **lsblk -fp** detalla la ruta completa del dispositivo, junto con los UUID y los puntos de montajes, así como el tipo de sistema de archivos en la partición. Si el sistema de archivos no está montado, el punto de montaje estará en blanco.

```
[root@host ~]# lsblk -fp
```

NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
/dev/vda				
└─/dev/vda1	xf		23ea8803-a396-494a-8e95-1538a53b821c	/boot
└─/dev/vda2	swap		cdf61ded-534c-4bd6-b458-cab18b1a72ea	[SWAP]
└─/dev/vda3	xf		44330f15-2f9d-4745-ae2e-20844f22762d	/
/dev/vdb				
└─/dev/vdb1	xf		46f543fd-78c9-4526-a857-244811be2d88	

Monte el sistema de archivos por el UUID del sistema de archivos.

```
[root@host ~]# mount UUID="46f543fd-78c9-4526-a857-244811be2d88" /mnt/data
```

Montaje automático de dispositivos de almacenamiento extraíbles

Si ha iniciado sesión y está utilizando el entorno de escritorio gráfico, montará automáticamente cualquier medio de almacenamiento extraíble cuando se inserte.

El dispositivo de almacenamiento extraíble se monta en **/run/media/USERNAME/LABEL** donde NOMBRE DE USUARIO es el nombre del usuario que ha iniciado sesión en el entorno gráfico y ETIQUETA es un identificador, a menudo, el nombre que recibe el sistema de archivos al crearse, si estaba disponible.

Antes de retirar el dispositivo, debe desmontarlo manualmente.

Desmontaje de sistemas de archivos

Los procedimientos de apagado y reinicio desmontan todos los sistemas de archivos **automáticamente**. Como parte de este proceso, todos los datos del sistema de archivos almacenados en la memoria caché se **transfieren** al dispositivo de almacenamiento, lo que **garantiza** que el sistema de archivos no sufra daños en los datos.

Aviso

Los datos del sistema de archivos son a menudo almacenados en la memoria caché.

Por lo tanto, para evitar dañar los datos en el disco, es esencial que desmonte las unidades extraíbles antes de desenchufarlas. El procedimiento de desmontaje sincroniza los datos antes de liberar la unidad, lo que garantiza la integridad de los datos.

Para desmontar un sistema de archivos, el comando **umount** espera el punto de montaje como argumento.

```
[root@host ~]# umount /mnt/data
```

No se puede desmontar si el sistema de archivos montado está en **uso**. Para que el comando **umount** se ejecute correctamente, todos los procesos deben **dejar de acceder** a los datos en el punto de montaje.

En el siguiente ejemplo, el **umount** falla porque el sistema de archivos está en uso (la shell está usando **/mnt/data** como su directorio de trabajo actual) y genera un mensaje de error.

```
[root@host ~]# cd /mnt/data
[root@host data]# umount /mnt/data
umount: /mnt/data: target is busy.
```

El comando **lsof** enumera todos los archivos abiertos y el proceso que accede a ellos en el directorio proporcionado. Resulta útil identificar los procesos que actualmente impiden un correcto desmontaje del sistema de archivos.

```
[root@host data]# lsof /mnt/data
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	1593	root	cwd	DIR	253,17		6 128	/mnt/data
lsof	2532	root	cwd	DIR	253,17		19 128	/mnt/data
lsof	2533	root	cwd	DIR	253,17		19 128	/mnt/data

Una vez que se identifican los procesos, puede tomarse una medida como esperar a que finalice el proceso o enviar una señal SIGTERM o SIGKILL al proceso. En este caso, basta con **cambiar el directorio** en funcionamiento actual por un directorio fuera del punto de montaje.

```
[root@host data]# cd
```

```
[root@host ~]# umount /mnt/data
```

Una razón común para que los sistemas de archivos no puedan desmontarse es que una shell Bash está usando el punto de montaje o un subdirectorio como directorio de trabajo actual. Use el **comando cd** para cambiar el sistema de archivos para resolver este problema.

Búsqueda de archivos

Un administrador de sistemas necesita herramientas para buscar archivos que coincidan con ciertos criterios en el sistema de archivos. En esta sección, se analizan dos comandos que pueden buscar archivos en la jerarquía de sistemas de archivos.

El comando **locate** busca una base de datos generada previamente para nombres de archivo o rutas de archivos y arroja los resultados instantáneamente.

El comando **find** busca archivos en tiempo real mediante un rastreo de la jerarquía de sistemas de archivos.

Localización de archivos por nombre

El comando **locate** busca archivos en función del **nombre** o la **ruta al archivo**. Es rápido porque busca esta información de la base de datos **mlocate**. Sin embargo, esta

base de datos no se actualiza en tiempo real y debe actualizarse con frecuencia para que los resultados sean precisos. Esto también significa que locate no encontrará los archivos que se han creado desde la última actualización de la base de datos.

La base de datos locate se actualiza automáticamente todos los días. Sin embargo, en cualquier momento el usuario root puede emitir el comando **updatedb** para forzar una actualización inmediata.

```
[root@host ~]# updatedb
```

El comando locate **restringe** los resultados para los usuarios sin privilegios. Para ver el nombre del archivo resultante, el usuario debe tener **permiso de búsqueda** en el directorio en el que reside el archivo.

Busque archivos con **passwd** en el nombre o en la ruta en los árboles de directorios legibles por el usuario en host.

```
[user@host ~]$ locate passwd
/etc/passwd
/etc/passwd-
/etc/pam.d/passwd
/etc/security/opasswd
/usr/bin/gpasswd
/usr/bin/grub2-mkpasswd-pbkdf2
/usr/bin/lppasswd
/usr/bin/passwd
...output omitted...
```

Los resultados se presentan incluso cuando la ruta o el nombre de archivo es solo una coincidencia parcial con la consulta de búsqueda.

```
[root@host ~]# locate image
/etc/selinux/targeted/contexts/virtual_image_context
/usr/bin/grub2-mkimage
/usr/lib/sysimage
/usr/lib/dracut/dracut.conf.d/02-generic-image.conf
/usr/lib/firewalld/services/ovirt-imageio.xml
/usr/lib/grub/i386-pc/lnxboot.image
...output omitted...
```

La opción **-i** realiza una búsqueda que distingue entre mayúsculas y minúsculas. Con esta opción, todas las **combinaciones** posibles de letras en mayúsculas y minúsculas coinciden con la búsqueda.

```
[user@host ~]$ locate -i messages
...output omitted...
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES/vim.mo
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES/vim.mo
/usr/share/vim/vim80/syntax/messages.vim
/usr/share/vim/vim80/syntax/msmessages.vim
/var/log/messages
```

La opción **-n limita** el número de resultados de búsqueda arrojados por el comando **locate**. En el siguiente ejemplo, se limitan los resultados de búsqueda arrojados por locate a las primeras cinco coincidencias.

```
[user@host ~]$ locate -n 5 snow.png
/usr/share/icons/HighContrast/16x16/status/weather-snow.png
/usr/share/icons/HighContrast/22x22/status/weather-snow.png
/usr/share/icons/HighContrast/24x24/status/weather-snow.png
/usr/share/icons/HighContrast/256x256/status/weather-snow.png
/usr/share/icons/HighContrast/32x32/status/weather-snow.png
```

Búsqueda de archivos en tiempo real

El comando **find** localiza los archivos mediante la realización de una búsqueda en **tiempo real** en la jerarquía de sistemas de archivos. Es más lento que **locate**, pero más **preciso**. También puede buscar archivos según criterios que no sean el nombre del archivo, como los permisos del archivo, el tipo de archivo, su tamaño o la hora de su modificación.

El comando **find** examina los archivos en el sistema de archivos usando la cuenta de usuario que ejecutó la búsqueda. El **usuario** que invoca el comando **find** debe tener **permiso de lectura y ejecución** en un directorio para examinar su contenido.

El primer argumento para el comando **find** es el directorio en que se realizará la búsqueda. Si el argumento del directorio se omite, **find** comienza la búsqueda en el directorio actual y busca coincidencias en los subdirectorios.

Para buscar archivos por nombre de archivo, use la opción **-name FILENAME**. Con esta opción, **find** arroja la ruta a los archivos que coinciden exactamente con el NOMBRE DE ARCHIVO. Por ejemplo, para buscar archivos con el nombre **sshd_config** comenzando con el directorio **/**, ejecute el siguiente comando:

```
[root@host ~]# find / -name sshd_config
/etc/ssh/sshd_config
```

Nota

Con el comando **find**, las opciones de palabra completa usan un solo guion y las opciones siguen el argumento del nombre de la ruta, a diferencia de la mayoría de otros comandos de Linux.

Los **comodines** están disponibles para buscar el nombre de un archivo y arrojan todos los resultados que son coincidencias parciales. Al usar comodines, es importante poner **entre comillas** el nombre del archivo para evitar que el terminal interprete el comodín.

En el siguiente ejemplo, busque archivos comenzando en el directorio / que finalizan en .txt:

```
[root@host ~]# find / -name '*.txt'
/etc/pki/nssdb/pkcs11.txt
/etc/brltty/brl-lt-all.txt
/etc/brltty/brl-mb-all.txt
/etc/brltty/brl-md-all.txt
/etc/brltty/brl-mn-all.txt
...output omitted...
```

Para buscar archivos en el **directorio /etc/** que contienen la palabra **pass** en cualquier parte de sus nombres en host, ejecute el siguiente comando:

```
[root@host ~]# find /etc -name '*pass*'
/etc/security/opasswd
/etc/pam.d/passwd
/etc/pam.d/password-auth
/etc/passwd-
/etc/passwd
/etc/authselect/password-auth
```

Para realizar una búsqueda que no distinga entre mayúsculas y minúsculas de un nombre de archivo determinado, use la opción **-iname** seguida del nombre del archivo que desea buscar. Para buscar archivos con texto que no distinga entre mayúsculas y minúsculas, **messages**, en sus nombres en el directorio / en host, ejecute el siguiente comando:

```
[root@host ~]# find / -iname '*messages*'
...output omitted...
/usr/share/vim/vim80/lang/zh_CN.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_CN.cp936/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW/LC_MESSAGES
/usr/share/vim/vim80/lang/zh_TW.UTF-8/LC_MESSAGES
/usr/share/vim/vim80/syntax/messages.vim
/usr/share/vim/vim80/syntax/msmessages.vim
```

Administración de paquetes de software con yum

Existen infinidad de gestores de paquetes de software. Vamos a hablar de uno de los principales como es Yum.

Yum está diseñado para ser un mejor sistema de gestión de instalación y actualizaciones de software basado en **RPM**. El comando yum le permite **instalar, actualizar, eliminar y obtener información** sobre los paquetes de software y sus dependencias. Puede obtener un historial de transacciones realizadas y trabajar con múltiples repositorios de software.

Algunas de las opciones principales de yum son las siguientes:

yum help muestra la información de uso.

yum list muestra los paquetes instalados y aquellos disponibles.

```
[user@host ~]$ yum list 'http*'
```

Available Packages

http-parser.i686	2.8.0-2.el8	rhel8-appstream
http-parser.x86_64	2.8.0-2.el8	rhel8-appstream
httpcomponents-client.noarch	4.5.5-4.module+el8+2452+b359bfcd	rhel8-appstream
httpcomponents-core.noarch	4.4.10-3.module+el8+2452+b359bfcd	rhel8-appstream
httpd.x86_64	2.4.37-7.module+el8+2443+605475b7	rhel8-appstream
httpd-devel.x86_64	2.4.37-7.module+el8+2443+605475b7	rhel8-appstream
httpd-filesystem.noarch	2.4.37-7.module+el8+2443+605475b7	rhel8-appstream
httpd-manual.noarch	2.4.37-7.module+el8+2443+605475b7	rhel8-appstream
httpd-tools.x86_64	2.4.37-7.module+el8+2443+605475b7	rhel8-appstream

yum info PACKAGENAME arroja información detallada sobre un paquete que incluye el espacio en disco necesario para la instalación.

Para obtener información sobre el servidor HTTP Apache:


```
[user@host ~]$ yum info httpd
```

Available Packages

```
Name       : httpd
Version    : 2.4.37
Release    : 7.module+el8+2443+605475b7
Arch       : x86_64
Size       : 1.4 M
Source     : httpd-2.4.37-7.module+el8+2443+605475b7.src.rpm
Repo       : rhel8-appstream
Summary    : Apache HTTP Server
URL        : https://httpd.apache.org/
License    : ASL 2.0
Description: The Apache HTTP Server is a powerful, efficient, and
            extensible
            : web server.
```

yum install PACKAGENAME obtiene e instala un paquete de software junto con cualquier tipo de dependencia.

yum update PACKAGENAME obtiene e instala una versión más reciente del paquete especificado, incluidas las dependencias. Generalmente, el proceso intenta preservar los archivos de configuración, pero en algunos casos se les cambiará el nombre si el empaquetador considera que el anterior no funcionará después de la actualización. Si no se especifica el PACKAGENAME, instala todas las actualizaciones relevantes.

```
[user@host ~]$ sudo yum update
```

En el siguiente vídeo *Principales comandos Linux* vamos a realizar un repaso de más de treinta comandos que un administrador de sistemas Linux debe conocer.



Accede al vídeo

2.11. Referencias bibliográficas

Copyleft. (2022, agosto 13). En *Wikipedia*. <https://es.wikipedia.org/wiki/Copyleft>

Velasco, R. (2015, febrero 21). *Las mejores 20 distribuciones de Linux para el día a día*. Redes zone. <https://www.redeszone.net/2015/02/21/las-mejores-20-distribuciones-de-linux-para-el-dia-dia/>

RedHat. (2021). *Train the trainers*. Manuscrito no publicado.

Desdelinux.net

Alejandro (a.k.a KZKG^Gaara). (28 de marzo de 2019). Tips: Más de 400 comandos para GNU/Linux que deberías conocer :D. *Desde Linux*. https://blog.desdelinux.net/mas-de-400-comandos-para-gnulinix-que-deberias-conocer/?utm_source=destacado-inside

Según sucesivas encuestas en StackOverflow, Linux es el sistema operativo más utilizado por los desarrolladores profesionales, con un impresionante 55,9 % de la cuota de mercado. No es solo una coincidencia. Linux es gratuito y de código abierto, tiene mejor seguridad que sus competidores y cuenta con una potente línea de comandos que hace que los desarrolladores y usuarios avanzados sean más eficaces.

Tanto si eres un administrador de sistemas con experiencia como si eres un recién llegado a Linux, puedes aprovechar esta guía.

RedHat.com

Página de Red Hat. <https://www.redhat.com/en>

Red Hat Enterprise Linux es la plataforma empresarial Linux líder en el mundo. Se trata de un sistema operativo (SO) *open source*. Es lo que permite ampliar las aplicaciones actuales e implementar tecnologías nuevas en equipos sin sistema operativo, entornos virtuales, contenedores y todo tipo de entornos de nube.

Puedes consultar más información en la web.

The Linux Foundation

The Linux Foundation. (2022). *OPEN SOURCE GUIDES. Open Source Best Practices for the Enterprise*. <https://www.linuxfoundation.org/resources/open-source-guides/>

El objetivo de The Linux Foundation es democratizar el código y escalar la adopción para todos los proyectos por igual. El esfuerzo es para crear nuevas categorías de tecnología mediante la identificación de nuevas tendencias y la aceleración del crecimiento de las tecnologías emergentes mediante la eliminación de las barreras a la adopción.

Podemos ampliar un montón de información en todas las secciones, pero la sesión de guías tiene un montón de información sobre los temas tratados en las secciones anteriores.

El software de código abierto es software con código fuente que cualquiera:

- A. Puede vender y modificar.
- B. Puede usar, estudiar y modificar con la licencia Creative Commons.
- ☐ C. Puede usar, estudiar, modificar y compartir.
- D. Puede usar, estudiar y modificar si lo da de alta en The Linux Foundation.

El software de código abierto es software con código fuente que cualquiera puede usar, estudiar, modificar y compartir.

¿Cuál no es un beneficio del código abierto para el usuario?

- A. Control.
- ☐ B. Libre de virus.
- C. Seguridad.
- D. Estabilidad.

Control, formación, seguridad y estabilidad.

Las licencias copyleft o de «uso compartido para todos» requieren que cualquier persona que distribuya el código fuente:

- ☐ A. También deba extender esta libertad para que los demás también puedan copiar, cambiar y distribuir el código.
- B. No puede extender esta libertad para que los demás también puedan copiar, cambiar y distribuir el código.
- C. También deba extender esta libertad para que los particulares que puedan copiar, cambiar y distribuir el código.
- D. Ninguna de las anteriores.

Las licencias copyleft o de «uso compartido para todos» requieren que cualquier persona que distribuya el código fuente, con o sin cambios, también deba extender esta libertad para que los demás también puedan copiar, cambiar y distribuir el código.

Los comandos ingresados en el prompt de shell están compuestos por partes básicas que son:

- _ A. Comando, opciones y argumentos.
- B. Comando, argumentos y parámetros.
- C. Parámetros y comando.
- D. Plantillas, comandos y parámetros.

Comando que se ejecuta, opciones que permiten ajustar el comportamiento del comando y argumentos que, generalmente, son destinos del comando.

La continuación de un comando largo en otra línea de un Shell de Linux se realiza con el carácter siguiente:

- A. \$
- B. #
- C. |
- _ D. \

Para ello, utilizará un carácter al que se conoce como carácter de escape, para ignorar el significado del carácter que se encuentra inmediatamente después él.

En la estructura de ficheros de un sistema Linux tenemos la ruta /etc que guarda:

A. Datos variables específicos de este sistema que deberían conservarse entre los arranques.

_ B. Archivos de configuración específicos para este sistema.

C. Datos de tiempo de ejecución para procesos que se iniciaron desde el último arranque.

D. Software instalado, bibliotecas compartidas, incluye archivos y datos de programa de solo lectura.

Archivos de configuración específicos para este sistema, frente a otras ramas como son /var, /run o /usr que se utilizan para otros usos de sistema operativo.

Para ver la información de todos los procesos y la información relacionada con un usuario, el comando que utilizará será:

- A. ps -user.
- B. ps -ut.
- _ C ps -au.
- D. ps -alluser.

Para ver la información del proceso, use el comando ps. La opción predeterminada es mostrar solo los procesos que están en la shell actual. Agregue la opción a para ver todos los procesos con un terminal. Para ver el usuario relacionado con un proceso, incluya la opción u.

El reinicio de un servicio sshd lo realizaría con el comando siguiente:

- A. system restart sshd.
- B. restart service sshd.
- C. restart sshd.service reload.
- _ D. systemctl restart sshd.service.

Durante el reinicio de un servicio en ejecución, el servicio se detiene y, luego, se inicia. Al reiniciar el servicio, la ID del proceso cambia y una nueva ID del proceso se asocia durante el arranque. Para reiniciar un servicio en ejecución, use el argumento restart con el comando systemctl.

Para ver la monitorización de eventos de seguridad utilizaríamos el comando:

- A. secure -h /etc/tail.
- _ B. tail -f /var/log/secure.
- C. tail -r /etc/home.
- D. tail -top /var/etc.

Para monitorear los intentos fallidos de inicio de sesión, ejecute el comando tail en un terminal y, luego, en otro terminal, ejecute el comando ssh como el usuario root mientras un usuario intenta iniciar sesión en el sistema.

Puede especificarse un nombre del host estático en el archivo:

- A. /system32/drivers/etc.
- B. /var/etc/hosts.
- C. /home/System/hostname.
- _ D. /etc/hostname.

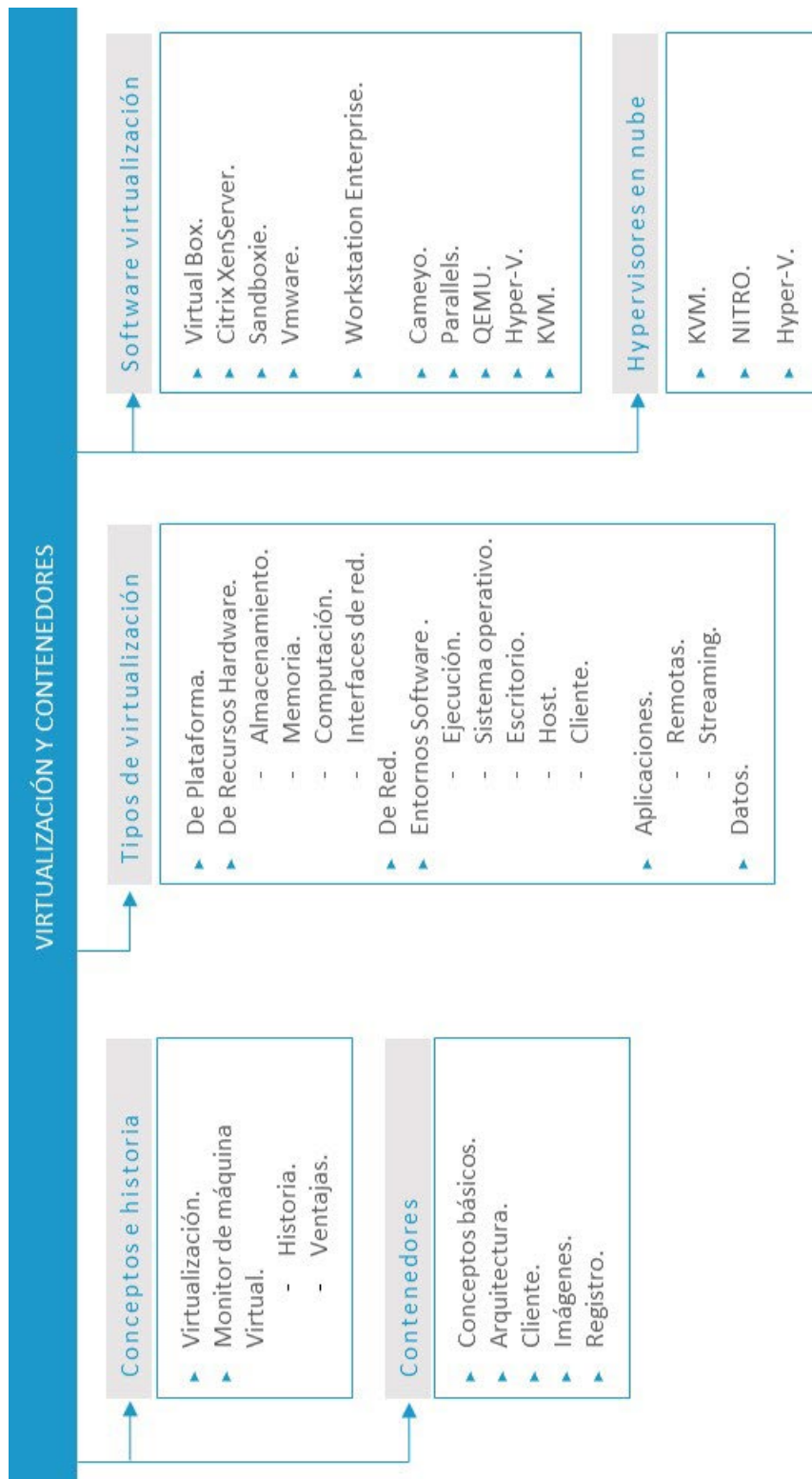
Puede especificarse un nombre del host estático en el archivo /etc/hostname. Se usa el comando hostnamectl para modificar este archivo y puede usarse para ver el estado del nombre del host totalmente calificado del sistema.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Virtualización y contenedores

Índice

Esquema	3
Ideas clave	4
3.1. Introducción y objetivos	4
3.2. Conceptos e historia	5
3.3. Tipos de virtualización	13
3.4. Software de virtualización	16
3.5. Hipervisores en la nube	23
3.6. Contenedores	31
3.7. Referencias bibliográficas	35
A fondo	37
Test	38



Esquema

3.1. Introducción y objetivos

En este tema vamos a estudiar los conceptos asociados a la **virtualización de infraestructuras**.

Empezaremos conociendo un poco de **historia y principios básicos** de la tecnología de la virtualización. Haremos posteriormente un repaso de los **principales softwares** de virtualización y que podemos utilizar en nuestros centros de datos.

Finalmente veremos los **hypervisores** que utilizan los grandes proveedores de nube. También realizaremos **prácticas** de esta tecnología tanto en nuestros ordenadores como en la nube de Amazon (AWS) y Microsoft (Azure)

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Aprender los conceptos principales sobre la virtualización.

Diferenciar los diferentes hypervisores que hay en el mercado.

Aprender a instalar algunos de los principales hypervisores.

Conocer los hypervisores de los grandes proveedores de nube.

Utilizar los hypervisores de AWS y Azure.

En los **vídeos** de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Instalación Windows Server. Después de la instalación limpia de un Windows Server crearemos un directorio activo.

Vídeo 2. Gestionaremos usuarios y máquinas en el directorio activo creado en el vídeo anterior. Así mismo instalaremos nuevos roles como el role DHCP.

3.2. Conceptos e historia

Virtualización

La virtualización es la **creación** de una **versión virtual** de un sistema operativo, de una plataforma, de un sistema de almacenamiento, etc. de tal manera que se crea una **capa de abstracción** entre la máquina física y el sistema que se virtualiza. En una misma máquina física se puede tener varias máquinas virtualizadas, independientes unas de otras, cuya administración y gestión se realiza desde la misma interfaz en la capa de abstracción.

La virtualización de **máquinas** permite poder **ejecutar** varios sistemas operativos y aplicaciones distintos con toda la **seguridad** en la misma máquina física y compartiendo los recursos.

En la virtualización de **servidores**, el servidor se divide en varios servidores virtuales más pequeños. Esto proporciona una mayor facilidad para **gestionarlos**, ya que los usuarios pueden acceder de manera independiente a cada uno de los servidores virtuales.

Por lo tanto, es una gran **ventaja** que tienen los **proveedores** de hosting quienes pueden maximizar la capacidad y el rendimiento de los servidores utilizando servidores virtualizados a **servidores dedicados** (servidores no virtualizados), ya que para dar acceso a los clientes bastaría con darles acceso al servidor virtual que hayan contratado. Para los clientes, la máquina física y los demás recursos instalados en el servidor son totalmente invisibles.

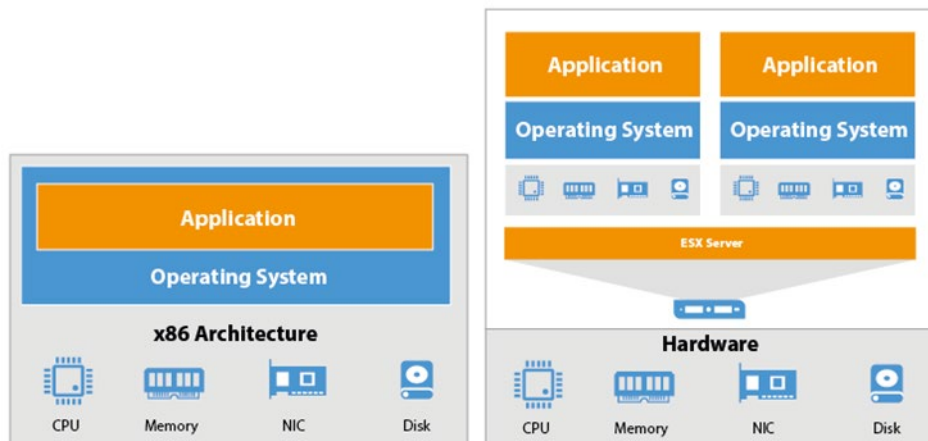


Figura 1. Máquina física vs. máquina virtual. Fuente: Vanover, 2017.

Monitor de máquina virtual o hypervisor

La capa de abstracción se conoce como monitor de máquina virtual (virtual machine monitor, VMM) o hypervisor.

Gracias a esta capa se puede **gestionar** y **administrar** los recursos principales de la máquina física (CPU, conexiones de red, dispositivos periféricos y memoria) y se pueden **distribuir** los recursos entre las máquinas virtuales.

Según las necesidades, cada máquina virtual podrá utilizar los recursos disponibles de la máquina física de una manera más eficiente para aprovecharlos al máximo.

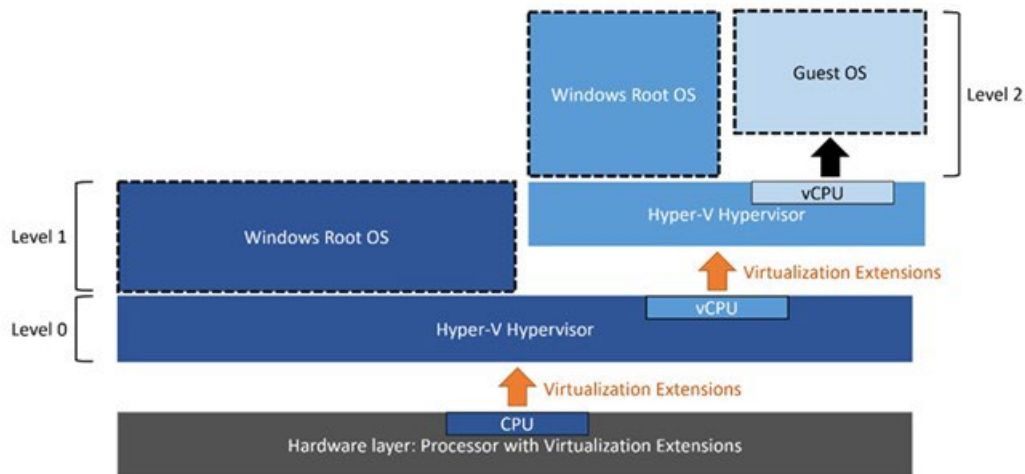


Figura 2: Hypervisor. Fuente: Microsoft, 2022.

Máquina virtual

Una máquina virtual es un **software** que simula un ordenador físico por lo tanto su comportamiento es el mismo que el de una **máquina real** y tiene su propia CPU virtual, memoria, disco duro y tarjeta de red. Esta característica permite que se **instalen y ejecuten programas** como si hubieran sido instalados en una máquina real con la única limitación de los recursos del ordenador físico que se haya configurado en la instalación del software.

Historia

La primera vez que se utiliza el concepto de virtualización es en el año **1960** para realizar **particiones en los mainframes** (ordenadores grandes y potentes utilizados para el procesamiento de una cantidad elevada de datos). Estas particiones permitían al ordenador poder realizar varias tareas con gran carga de procesamiento de manera paralela y ejecutar varias aplicaciones o procesos de manera simultánea.

Los mainframes eran sistemas muy **caros** porque se diseñaron para que pudieran realizar estas particiones y, de esta manera, poder obtener el máximo rendimiento y beneficio de ellos, aprovechando todo el potencial del que se disponía.

En los años **1980 y 1990** se dejó la virtualización, cuando establecieron el **modelo de informática distribuida** a partir de las aplicaciones cliente-servidor y los servidores x86 y escritorios económicos.

En los años noventa con la gran acogida que tuvo **Windows** entre los usuarios y la aparición emergente de **Linux** consiguió convertir los servidores x86 en el modelo a utilizar en el sector. El aumento del uso de **servidores x86** y escritorios destapó nuevos problemas, hasta el momento desconocidos, de tipo operacional y sobre la infraestructura de TI.

Los **problemas** que surgieron son:

Infrautilización de los recursos: solo se ejecutaba una aplicación por servidor para mantener la integridad y seguridad de la aplicación ante posibles inconsistencias.

Aumento de los costes de infraestructura: los equipos y su mantenimiento cada vez es más elevado.

Aumento de los costes de gestión de TI: este tipo de servidores conlleva un gran número horas de mano de obra.

Escasa protección ante desastres o fallos: lo que ha motivado la importancia de la continuidad después de un desastre.

Aumento de costes de seguridad de los equipos personales: el mantenimiento de los equipos personales conlleva un elevado coste de mantenimiento, ya que hay que realizar tareas de mantenimiento, actualización o instalación de parches que son obligatorios para mantener la integridad y seguridad de los equipos.

Como todos estos problemas suponían un inconveniente en la evolución de la virtualización hubo una empresa que se dedicó a la investigación de las mejoras de la virtualización hasta que consiguió solucionar todos los problemas que implicaba hasta el momento.

VMware presentó en **1999** la solución a muchos de estos problemas con la **virtualización en equipos x86** y convertir estos sistemas en un recurso compartido, aunque su uso sea general y que, además, garantiza el aislamiento, la movilidad y la posibilidad de poder elegir el sistema operativo deseado.

Las máquinas x86 no habían sido creadas para soportar una virtualización completa, como los mainframes que sí la tenían, por lo que VMware tiene que enfrentarse a un gran desafío para crear máquinas virtuales teniendo como base los ordenadores x86.

Tanto en los ordenadores personales como en los mainframes, la **CPU** tiene como función principal la ejecución de secuencias de instrucciones, es decir, la **ejecución** de software. VMware descubre que hay un problema por el que se muestra un aviso de **error** y que hace que el programa se cierre. Este problema es provocado por diecisiete instrucciones que son **incompatibles** o que provoca errores en la virtualización por lo que estas instrucciones son el principal reto al que se enfrentan para conseguir que la virtualización funcione correctamente en sistemas x86.

La **técnica** que utiliza VMware para conseguir que la virtualización se realice correctamente en una arquitectura x86 es detectar cuando se genera una de las instrucciones conflictivas y **convertirlas** en **instrucciones seguras**. Esto permite que se pueda realizar una virtualización del sistema de una manera segura y permite al resto de instrucciones (no conflictivas) virtualizarse y ejecutarse sin generar ningún problema.

El resultado de toda esta evolución y trabajo es una máquina virtual de alto rendimiento, con una compatibilidad total del software y que se adapta completamente al hardware en el que se ejecute.

Ventajas

Las ventajas que ofrece un sistema virtualizado frente a un sistema dedicado son tanto de referidas al rendimiento, como de costes o de recuperación, por lo que la virtualización nos ofrece más ventajas **estratégicas y económicas** que un servicio dedicado.

Reducción de infraestructura física necesaria: al tener los equipos virtualizados en una misma máquina se pueden tener varios sistemas distintos ejecutándose a la vez, por lo que se reduce en gran medida el espacio físico necesario para almacenar los equipos. Los centros de procesamiento de datos (CPD) de hoy en día ocupan una ínfima parte a los CPDs existentes antes de usar la virtualización de equipos.

Aumento de tolerancia a fallos: con la virtualización es muy sencillo tener una copia del sistema completo e incluso tener una imagen real que sirva de respaldo por si ocurriera algún contratiempo en el sistema principal.

Centralización de la administración: la administración de las máquinas es mucho más sencilla y rápida, ya que desde el mismo sistema se pueden gestionar (actualizar, mantener, etc.) varias máquinas, reduciendo tiempos y coste de mantenimiento.

Mejorar el rendimiento: con la virtualización se pueden asignar los recursos justos y necesarios del servidor mejorando el rendimiento de los sistemas virtualizados como el de la máquina física.

Recuperación ante desastres: al reducir los costes de gestión, mantenimiento y los equipos ser más ligeros, es más factible para las empresas invertir parte del dinero ahorrado gracias a la virtualización. Al tener redundancia de los servidores en otras localidades con características medioambientales distintas, frente al caso que ocurra algún desastre, podrá restablecer el servicio lo más rápidamente posible y podrá seguir ofreciendo los servicios a los clientes. Este servicio ininterrumpido

hace que las empresas no tengan pérdidas económicas ni de clientes, por lo que es un beneficio extra que consiguen las empresas cuando implantan un sistema de recuperación ante desastres.

Por todo esto, hay muchos motivos que pueden incentivarnos a **elegir la virtualización** frente a los ordenadores o servidores dedicados, independientemente del tamaño de la empresa y de la infraestructura que se vaya a necesitar. Hoy en día, miles de empresas, incluyendo las más grandes y punteras en el sector TI, se han decantado por la virtualización para beneficiarse de todas las **ventajas** que presenta.

Las principales razones por las que las empresas, independientemente del sector al que se dediquen, se decantan por la virtualización son:

Aprovechar al máximo los recursos que dispone: se pueden agrupar los recursos comunes en los servidores y abandonar de manera definitiva el modelo utilizado anteriormente de aplicación-servidor, en el cual cada servidor solo ejecutaba y dedicaba a una única aplicación. Las ventajas que aporta este motivo a la empresa son:

Reducción en más del 50 % de los costes tanto operativos como de hardware por cada sistema virtualizado.

Reducción en un 80 % de los costes energéticos por cada sistema virtualizado.

Reducción de hasta un 70 % del tiempo para el abastecimiento de nuevos equipos.

Reducción de las interrupciones de servicio y, por lo tanto, aumento de la fiabilidad.

Aumento del aprovechamiento del hardware, pasa del 15 % al 80 %. Se reducen los requisitos de hardware, ya que la virtualización se realiza sobre máquinas estándar y se pueden configurar los recursos necesarios de manera independiente.

Administración de la infraestructura centralizada en un único punto, lo que tiene como consecuencia la disminución de los tiempos de abastecimiento, gestión de las máquinas desde una única central y una supervisión de rendimiento y de host de las máquinas desde una única ubicación.

Reducción de costes del CPD disminuyendo la infraestructura física y aumentando la ratio de servidores por administrador. Al reducir el número de máquinas físicas se reduce el personal necesario para administrar las máquinas y además una cantidad inferior de máquinas físicas implica: menos espacio, menos consumo energético y menos consumo de refrigeración.

Aumento de la disponibilidad del hardware y el software para garantizar la continuidad del negocio. Se realizan copias de seguridad (backups) de los sistemas completos y se pueden migrar sin que el servicio se interrumpa a los clientes/usuarios. Por lo tanto, se reducen las interrupciones de servicio y la recuperación ante imprevisto es casi inmediata.

Flexibilidad. La virtualización ofrece una gran flexibilidad y puede adaptarse a los cambios de mercado que pueden ir surgiendo.

Mejora la gestión y la seguridad de los escritorios. Al poder acceder de manera local o remota, mejora la capacidad de gestión y supervisión de los escritorios.

3.3. Tipos de virtualización

Tal y como podemos leer en Wikipedia (2022), podemos clasificar la virtualización según lo que se quiere virtualizar en:

Virtualización de plataforma. También llamado virtualización de servidor o virtualización de hardware. Se tienen máquinas virtuales que son softwares huésped ejecutado sobre un software especial llamado Hipervisor o VMM (Virtual Machine Monitor). Esta VMM crea una capa de abstracción del hardware de la máquina física (host) y se la ofrece al sistema operativo de la máquina virtual.

Virtualización de recursos hardware. Por ejemplo, según el tipo de recurso hardware que queremos virtualizar tenemos:

Virtualización de almacenamiento. Es lo que hacen, por ejemplo, los sistemas RAID, LVM, SAN y NAS

Virtualización de memoria. Es lo que hacen, por ejemplo, los sistemas de memoria virtual.

Virtualización de recursos de computación. Es lo que hacen, por ejemplo, los clústeres de computación.

Virtualización de interfaces de red. Por ejemplo, la Agregación de enlaces simula un dispositivo de red único con gran ancho de banda utilizando múltiples interfaces de red independientes.

Virtualización de red. Es lo que hacen las VPN, VLAN, Virtual IP y SDN.

Según Red Hat, la virtualización de las funciones de red (NFV) separa las funciones clave de una red (como los servicios de directorio, el uso compartido de archivos y la configuración de IP) para distribuir las entre los entornos. Cuando las funciones del software se independizan de las máquinas físicas donde se alojaban, las funciones específicas pueden empaquetarse en una nueva red y asignarse a un entorno. La virtualización de redes, que se utiliza con frecuencia en el sector de las

telecomunicaciones, reduce la cantidad de elementos físicos (como conmutadores, enrutadores, servidores, cables y centrales) que se necesitan para crear varias redes independientes.

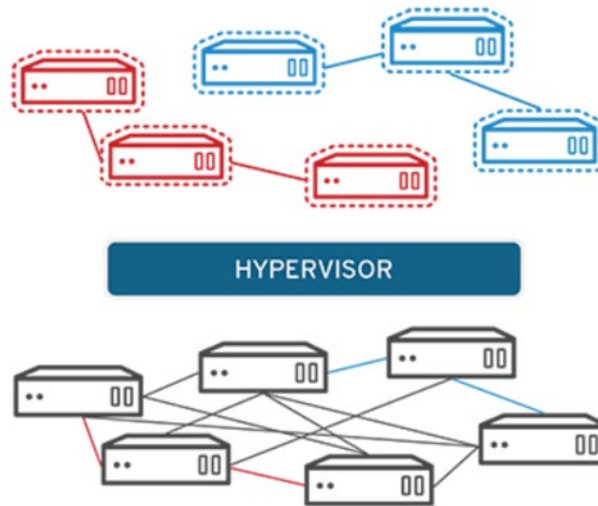


Figura 3. Virtualización de red. Fuente: Red Hat, 2018.

Virtualización de entornos software de ejecución. Dependiendo del entorno de ejecución que virtualizamos:

Virtualización a nivel de sistema operativo o contenedorización. El kernel del sistema operativo subyacente permite ejecutar de forma paralela instancias de espacio de usuario, aisladas unas de otras. Todas las instancias comparten el kernel del sistema operativo subyacente. En cada instancia, llamada contenedor, el usuario puede instalar una o más aplicaciones. Al programa que ofrece alojamiento a contenedores se le llama motor de contenedores.

Virtualización de escritorio. Implementan un escritorio como servicio basándose en una estructura cliente-servidor. Permite una administración centralizada y, en entornos de trabajo estandarizados, es posible reducir de forma significativa los costes de gestión y mantenimiento. Dependiendo de dónde se aplique la

potencia de procesamiento para crear un escritorio virtual, se puede diferenciar entre distintos enfoques:

Basada en el host. Los escritorios virtuales se ejecutan directamente en el servidor y los clientes tienen acceso a los escritorios virtuales a través de la red (ej. VNC, RDP). Al usar máquinas virtuales para proporcionar y gestionar escritorios virtuales se le llama infraestructura de escritorios virtuales o VDI (del inglés Virtual Desktop Infrastructure). La VDI aloja entornos de escritorio en un servidor centralizado y los distribuye a los usuarios finales según las necesidades.

Basada en el cliente. El dispositivo cliente es el que proporciona los recursos necesarios para gestionar el entorno de escritorio y cada escritorio virtual se sincroniza con el servidor. Ejemplos de este tipo de virtualización es lo que hace Microsoft Enterprise Desktop Virtualization (MED-V), VMware ACE y SolidIce.

Virtualización de aplicaciones. Consiste en usar aplicaciones que aparentemente se ejecutan en local. Se pueden implementar, por ejemplo:

Con aplicaciones remotas. Las aplicaciones se ejecutan en un servidor remoto y el cliente se conecta a ellas a través de protocolos de visualización como VNC o RDP.

Streaming de aplicaciones. Las aplicaciones se ejecutan en local, pero ciertos componentes se descargan de forma que es posible funcionar sin tener conexión a la red. De esta forma solo las partes esenciales del código de una aplicación deben instalarse en la computadora. Mientras el usuario final realiza acciones en la aplicación, el código y los archivos necesarios se entregan a través de la red cuando y como se requieren. De esta forma se permite ejecutar los programas en entornos aislados y distribuirlos en diferentes sistemas sin que sea necesario realizar cambios en el sistema local. Esto permite proteger el sistema operativo subyacente de código malicioso. Ejemplo de sistemas para

proporcionar este tipo de servicios: VMware ThinApp, Microsoft Application Virtualization y XenApp (Citrix).

Virtualización de datos. Consiste en integrar datos de fuentes dispersas, en distintas localizaciones y formatos, sin replicar los datos, para construir una capa de datos virtual que facilita la provisión de servicios de datos unificados para dar soporte a múltiples aplicaciones y usuarios. Para ello proporciona una capa de abstracción y una capa de servicios de datos.



Figura 4. Virtualización de datos. Fuente: Red Hat, 2018.

3.4. Software de virtualización

En la actualidad hay muchos softwares y sistemas de virtualización con distintas características y aplicaciones. Puede ser de gran ayuda conocer las **diferencias** para poder seleccionar el sistema que más se adecue a las **necesidades** que se tienen que cubrir.

Este tema presenta una gran variedad de productos y sus características para que el alumno obtenga los conocimientos necesarios y, en el caso en el que tenga que realizar una virtualización, sepa cómo se realiza y qué producto del mercado es el que más le conviene utilizar.

Se recomienda estudiar las Ideas clave, que se visiten las páginas de los **productos** que se exponen y de las **empresas** que los comercializan para comprobar que, además de los expuestos, hay muchos más que podrían serles de gran utilidad en el futuro.

Además, se recomienda al alumno que intente **instalar** en su ordenador personal una máquina virtual utilizando al menos dos de los productos desarrollados para que pueda **comprobar** las diferencias y sacar sus primeras conclusiones a cerca de la virtualización y los distintos métodos que existen para realizarla.

Según podemos leer en [Profesional review](#), vamos a ver una serie de **hipervisores** que podremos utilizar en nuestro centro de datos, dejando para el siguiente punto los tres hipervisores que utilizan los grandes proveedores de nube como son AWS, Azure y GCP.

VirtualBox

Para muchos VirtualBox está considerado como el mejor software de virtualización para **pequeñas empresas** o en **casa**. Una de sus grandes ventajas es que está disponible de forma libre mediante licencia GNU de **uso público**. Es una herramienta que se puede ejecutar en sistemas operativos Windows, Mac OS, Linux y Solaris. Es **muy versátil y sencillo de usar** y cuenta con funciones muy interesantes para usuarios principiantes, como la posibilidad de ejecutar las máquinas virtuales a pantalla completa o de intercambiar el puntero del ratón entre el software virtual y el equipo físico.



Figura 5. Virtual Box. Fuente: López,2022.

Citrix XenServer

Citrix XenServer es un software para virtualización que se ha destacado como uno de los **mejores** del mercado. Es una plataforma para virtualización de servidores que se integra con la **tecnología Xen** y que proporciona un entorno de administración seguro y fiable. Se puede ejecutar en sistemas operativos Windows y Linux y cuenta con dos versiones, una **gratuita** y otra de **pago** que incluye funcionalidades más avanzadas.

Sandboxie

Sandboxie es, para muchos, el mejor software para virtualización si lo que quieres es realizar pruebas con **programas o código maliciosos**. Puedes crear un espacio aislado y protegido en tu equipo para generar un entorno de pruebas en el que ejecutar software malicioso sin que afecte al resto del equipo, lo que se denomina **sandboxing**. Una vez que finalices la sesión, todo lo que haya en ese entorno de pruebas será eliminado y el equipo volverá a su funcionamiento normal.

VMware

VMware es un de las soluciones más potentes y con más paquetes de software disponibles para virtualización. La empresa propietaria dispone de productos de pago que abarcan absolutamente todos los ámbitos de virtualización.

En cuando a las soluciones de las que disponemos para **entornos de escritorio** disponemos en gran parte de aplicaciones de **licencia pagada** y no precisamente barata. Sin embargo, también da la oportunidad con alguna que otra **herramienta gratuita para empresas**.

vSphere Hypervisor: es un Hypervisor gratuito que está destinado a la virtualización en entornos de servidores fundamentalmente.

vCenter Converter: esta aplicación lo que hace es convertir una máquina física Linux o Windows y otras máquinas virtuales en una máquina virtual VMware.

En cuanto a las **aplicaciones** de virtualización para usuarios normales tenemos fundamentalmente dos:

VMware Workstation Player

Es el software más económico que podemos encontrar para nuestras necesidades. Este Hypervisor es capaz de virtualizar un sistema operativo completamente funcional en nuestro PC. Dispondrá de funcionalidades para compartir carpetas en red y comunicar nuestras máquinas virtuales con la máquina física. Disponemos de una versión de prueba. La podremos comprar por algo más de 160 euros y se ejecuta en Windows, Mac y Linux.

VMware Workstation Pro

Digamos que es la versión completa del anterior. En este caso podremos virtualizar más de un sistema operativo dentro de nuestro PC. Además, podremos conectarnos a servidores virtuales corriente en vSphere. Su adquisición sale por 275 euros y se ejecuta en Windows, Mac y Linux.

Si hablamos de **entornos empresariales** y virtualización de servidores tendremos la siguiente:

vSphere Enterprise

Es una aplicación que proporciona un entorno completo para virtualización y es el más utilizado y potente de las herramientas que dispone la compañía. Ofrece virtualización completa, por hardware, migraciones en caliente, monitorización en tiempo real, etc.



Figura 6. VMware Workstation 16 Player. Fuente: VMware, 2022.

Cameyo

Cameyo es un sistema de virtualización que permite ejecutar un programa virtual dentro de un paquete virtualizado. De esta forma, el programa se ejecuta dentro de ese **paquete virtual**, como si lo hiciera en el equipo. Es una solución ideal para convertir los programas normales en **programas portables** que se puedan ejecutar en distintos equipos sin necesidad de instalación.

Parallels

Parallels es un software de virtualización para Mac, cuya principal ventaja es su capacidad para **ejecutar otros sistemas operativos** en equipos de la marca Apple. Una de las mejores soluciones para aquellos que cuentan con ordenadores Apple, pero quieren ejecutar programas o entornos que solo están disponibles para otros sistemas operativos.

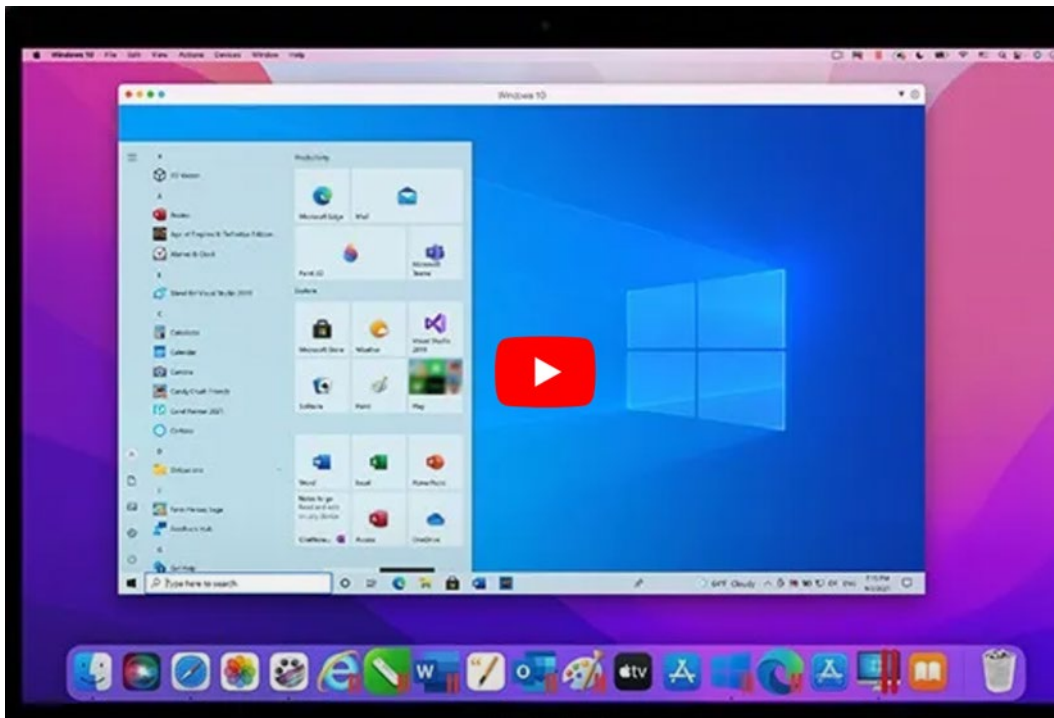


Figura 7. Parallels Desktop 18 for Mac. Fuente: Parallels, 2022.

QEMU

QEMU es un software de virtualización gratuito que se ha erigido como una de las grandes **alternativas a VirtualBox**. Se trata de una herramienta **open source** que está disponible para sistemas operativos Windows, Mac OS y Linux. Aunque no dispone de una interfaz gráfica de usuario propia, es posible descargarla como extensión para Windows y Mac. El software UTM para virtualizar en Mac y que es gratuito está basado en este hypervisor.

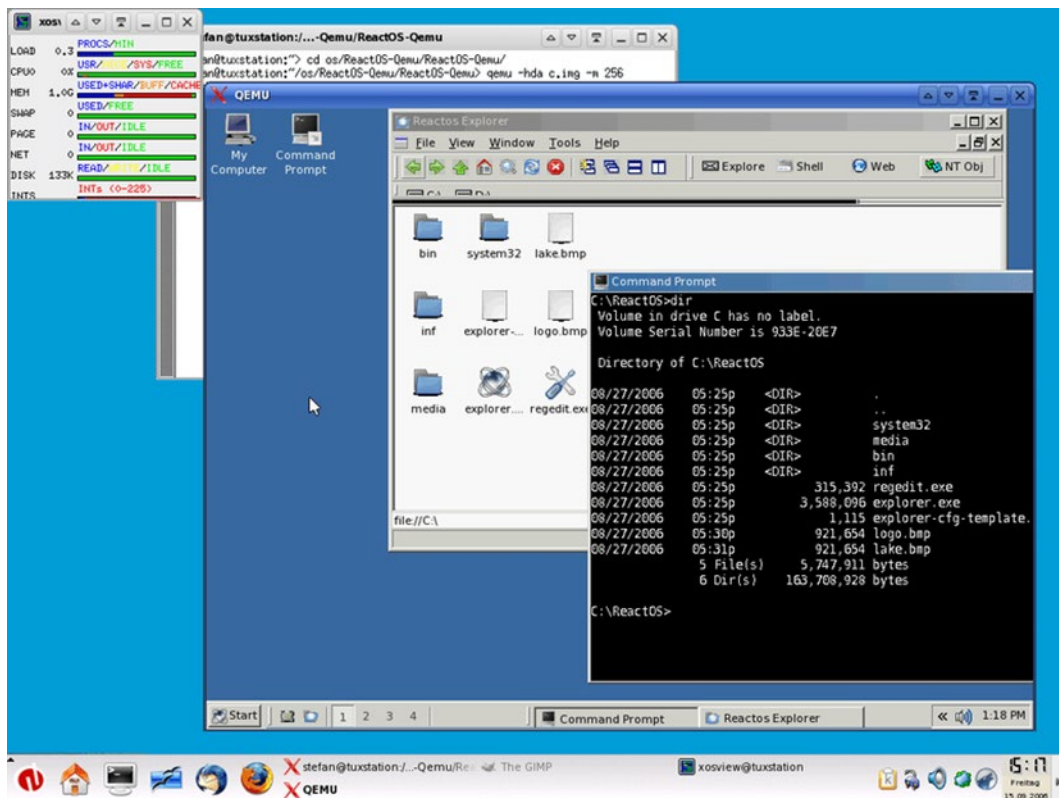


Figura 8. QEMU ejecutando el sistema operativo ReactOS en Linux. Fuente: QEMU, s.f.

En la siguiente tabla podemos ver un resumen del software de virtualización y su característica de tipo de licenciamiento.

Hypervisor	Tipo de licencia
QEMU	Libre
Parallels	Privativo
Vmware	Privativo
KVM	Libre
Hyper-V	Privativo
Xen	Privativo
Virtual Box	GNU, libre.

Tabla 2. Hypervisor y tipo de licencia. Fuente: elaboración propia.

3.5. Hipervisores en la nube

En esta sección vamos a ver con más detalle los hipervisores utilizados por los grandes **proveedores de nube** y que usan para la construcción de sus infraestructuras.

Veremos el **hypervisor NITRO** que usa AWS que es una modificación realizada de KVM. Este hypervisor también lo utiliza Google Cloud Platform en su nube. Finalmente veremos el hipervisor **Hyper-V** que utiliza Microsoft tanto en Windows 10/11, Windows server y Azure.

KVM

Según [RedHat](#) una máquina virtual basada en el kernel (KVM) es una tecnología de virtualización **open source** integrada a **Linux®**. En concreto, con las KVM puede convertir a Linux en un hipervisor que permite que una máquina host ejecute varios entornos virtuales aislados llamados **máquinas virtuales** (VM) o guests.

Las KVM forman parte de Linux, así que están disponibles en **Linux 2.6.20** o una versión posterior. Se anunciaron por primera vez en **2006** y, un año después, se incorporaron a la versión principal del kernel de Linux. Dado que las KVM forman parte del **código actual** de Linux, aprovechan de forma inmediata todas sus mejoras, correcciones y funciones nuevas, sin requerir ningún tipo de ingeniería adicional.

Funcionamiento de las KVM

Las KVM convierten a Linux en un hipervisor de tipo 1, es decir, de **servidor dedicado** (bare metal). Todos los hipervisores necesitan algunos elementos del sistema operativo (por ejemplo, el administrador de memoria, el programador de procesos, la pila de entrada o salida [E/S], los controladores de dispositivos, el administrador de seguridad, la pila de red, entre otros) para ejecutar las máquinas virtuales. Las

KVM tienen todos estos elementos porque forman parte del **kernel de Linux**. Cada máquina virtual se implementa como un proceso habitual de Linux, el cual se establece mediante el programador estándar, con hardware virtual exclusivo, como tarjetas de red, adaptadores de gráficos, CPU, memorias y discos.

Implementación de las KVM

En resumen, debe ejecutar una versión de Linux que se haya lanzado después de **2007**, y debe instalarla en sistemas de **hardware X86** que respalden las funciones de virtualización. Si ya lo hizo, entonces solo debe cargar dos módulos actuales (un módulo host del kernel y uno específico del procesador), un emulador y cualquier controlador que le permita ejecutar sistemas adicionales.

Sin embargo, si se implementan las KVM en una distribución de Linux compatible, como RedHat Enterprise Linux, se **amplían sus funciones**, lo cual le permite intercambiar los recursos entre los **guests**, compartir bibliotecas comunes, optimizar el rendimiento del sistema y mucho más.

Funciones de las KVM

Las KVM forman parte de Linux, y Linux forma parte de las KVM. Ambas soluciones ofrecen las mismas características, pero las KVM tienen funciones **específicas** que las convierten en el hipervisor preferido para las empresas.

Seguridad

Las KVM utilizan una combinación de **Security-Enhanced Linux (SELinux)** y **Secure Virtualization (sVirt)** para mejorar la seguridad y el aislamiento de las máquinas virtuales. SELinux establece límites de seguridad en las máquinas virtuales, mientras que sVirt amplía las funciones de SELinux, lo cual permite que se aplique la seguridad del control de acceso obligatorio (MAC) a las máquinas virtuales guest y previene los errores del etiquetado manual.

Almacenamiento

Las KVM pueden usar todos los tipos de almacenamiento **compatibles con Linux**, lo cual incluye algunos discos locales y el almacenamiento adjunto a la red (NAS). La entrada y salida de varias rutas mejora el almacenamiento y proporciona constancia. Las KVM también admiten los sistemas de **archivos compartidos**, para que se puedan compartir las imágenes de máquinas virtuales entre varios hosts. Las imágenes del disco son compatibles con el método de implementación fina, mediante el cual se asigna el almacenamiento según se requiera en lugar de hacerlo por adelantado.

Sin embargo, si se implementan las KVM en una distribución de Linux compatible, como **RedHat Enterprise Linux**, se amplían sus funciones, lo cual le permite intercambiar los recursos entre los guests, compartir bibliotecas comunes, optimizar el rendimiento del sistema y mucho más.

Compatibilidad con los sistemas de hardware

Las KVM pueden usar una amplia variedad de plataformas de hardware **certificadas y compatibles** con Linux. Dado que los proveedores de hardware contribuyen al desarrollo del kernel con frecuencia, las funciones más recientes de hardware suelen adoptarse rápidamente en el kernel de Linux.

Gestión de la memoria

Las KVM cuentan con las **funciones de Linux** para la gestión de la memoria, como el acceso a la memoria no uniforme y la memoria compartida del kernel. La memoria de las máquinas virtuales se puede intercambiar —con el respaldo de volúmenes grandes de datos para obtener un mejor rendimiento— y se puede compartir o respaldar con un archivo de disco.

Migración activa

Las KVM **respaldan** la migración activa: la capacidad para trasladar una máquina virtual en ejecución entre hosts físicos sin interrumpir el servicio. Eso permite que la máquina virtual y las conexiones de red sigan **activas** y que las aplicaciones se sigan **ejecutando** durante el proceso de migración. También se guarda su estado actual para que se pueda almacenar y reanudar luego.

Rendimiento y capacidad de ajuste

Las KVM incorporan el desempeño de Linux y se expanden para **satisfacer la demanda** si aumenta la cantidad de máquinas guest y de solicitudes. También, permiten que se virtualicen las cargas de trabajo más exigentes de las aplicaciones y constituyen la **base** para muchas configuraciones de virtualización empresarial, como los centros de datos y las nubes privadas (mediante OpenStack®).

Programación y control de recursos

En el modelo de KVM, una máquina virtual es un proceso de Linux que el **kernel programa y gestiona**. El programador de Linux brinda un control detallado sobre los recursos que se asignan a un proceso y garantiza la calidad del servicio para un proceso particular. Con las KVM, se **incluye el programador** completamente justo, los grupos de control, los espacios de nombre de las redes y las extensiones inmediatas.

Latencia más baja y mayor priorización

El kernel de Linux incluye **extensiones inmediatas** que permiten que las aplicaciones basadas en máquinas virtuales se ejecuten con una latencia más baja y un mejor orden prioritario —en comparación con los servidores dedicados (bare metal)—. El kernel también **divide** los procesos que requieren largos períodos de computación en elementos más pequeños que luego se programan y se procesan debidamente.

Gestión de las KVM

Es posible gestionar de forma manual varias máquinas virtuales que están activas en una sola estación de trabajo sin una herramienta de gestión. Las grandes empresas usan un **software de gestión** de la virtualización que interactúa con los entornos virtuales y el hardware físico subyacente para **simplificar** la administración de los recursos, **mejorar** el análisis de los datos y **optimizar** las operaciones. RedHat creó **RedHat Virtualization**, precisamente, con ese fin.

Nitro

Funcionamiento

Según Koop (2021), el sistema AWS Nitro es una **combinación** de hardware y software especialmente diseñado para proporcionar **rendimiento y seguridad**. Las máquinas virtuales (instancias) de última generación, incluidas las familias de instancias populares con las cargas de trabajo de HPC, y muchas otras se basan en el sistema Nitro.



Figura 9. Nitro. Fuente: Koop, 2021.

Como se muestra en la Figura 9 el sistema Nitro de AWS se compone de **tres componentes** principales: las **tarjetas Nitro**, el **chip de seguridad Nitro** y el **hipervisor Nitro**.

Las tarjetas Nitro proporcionan **controladores** para el plano de datos de VPC (acceso a la red), el acceso a Amazon Elastic Block Store (Amazon EBS), el almacenamiento de instancias (NVMe local), así como la coordinación general para el host. Al descargar estas capacidades a las tarjetas Nitro, se elimina la necesidad de utilizar los recursos del procesador host para implementar estas funciones, además de ofrecer beneficios de seguridad.

El chip de seguridad Nitro proporciona una raíz de **confianza de hardware** y un **arranque seguro**, entre otras características para ayudar con la seguridad del sistema. El hipervisor Nitro es un hipervisor liviano que administra la asignación de memoria y CPU.

Con este diseño, el sistema host ya no tiene acceso directo a los recursos de AWS. Solo las **tarjetas Nitro reforzadas** pueden acceder a otros recursos y, cada una de ellas, proporciona dispositivos de hardware definidos por software que son los únicos **puntos de acceso** desde el dispositivo host. Con los accesos de E/S manejados por tarjetas Nitro permite que el último componente, el hipervisor Nitro, sea liviano y tenga un impacto mínimo en las cargas de trabajo que se ejecutan en el host.

El hipervisor Nitro solo tiene las funciones necesarias, con el objetivo de diseño de estar **inactivo**, lo que significa que nunca debe activarse a menos que esté trabajando para una instancia que lo solicitó. Esto también significa que no hay tareas en segundo plano que consumen recursos cuando no se necesitan.

La arquitectura del sistema Nitro también permite que AWS ofrezca instancias y acceso directo al «bare metal» del host.

Desde su presentación inicial en 2017, muchas familias de instancias ofrecen variantes bare, que brindan acceso directo al **hardware subyacente** y no al hipervisor. Como en el caso de que se utilice el hipervisor Nitro, las tarjetas Nitro siguen siendo los únicos **puntos de acceso** a los recursos **fuera del host**. Estas instancias se usan con más frecuencia para cargas de trabajo que no pueden ejecutarse en un entorno

virtualizado debido a requisitos de licencia o aquellas que necesitan funciones de hardware específicas que solo se brindan mediante acceso directo.

Tanto la opción de instancias «sin sistema operativo» como las instancias virtualizadas de Nitro, esto proporciona un método para mostrar la **diferencia de rendimiento** entre el rendimiento de la aplicación HPC en modo sin sistema operativo y la ejecución en el hipervisor AWS Nitro.

En el siguiente vídeo *Administración Hyper-V* veremos la instalación del hipervisor de Microsoft tanto en estaciones de trabajo como en servidores. Aprenderemos a realizar una instalación de máquina virtual con este hipervisor y veremos también la gestión de alta disponibilidad en un entorno real.



Accede al vídeo

Microsoft Hyper-V Server

Funcionamiento de Hyper-V

Según Microsoft (2012) Hyper-V es una **tecnología de virtualización** basada en **hipervisor**. Hyper-V usa el Windows que requiere un **procesador físico** con características específicas.

En la mayoría de los casos, el hipervisor administra las interacciones entre el hardware y las máquinas virtuales.

Este acceso controlado por hipervisor al hardware proporciona a las máquinas virtuales el **entorno aislado** en el que se ejecutan. En algunas configuraciones, una máquina virtual o el sistema operativo que se ejecuta en la máquina virtual tiene **acceso directo** a gráficos, redes o hardware de almacenamiento.

¿En qué consiste Hyper-V?

Hyper-V tiene **elementos** necesarios que funcionan conjuntamente para que pueda **crear y ejecutar** máquinas virtuales. Juntas, estas partes se denominan **plataforma de virtualización**. Se instalan como un conjunto al instalar el rol de Hyper-V. Entre las partes necesarias se incluyen Windows hipervisor, el servicio de administración de máquinas virtuales de Hyper-V, el proveedor WMI de virtualización, el bus de máquina virtual (VMBus), el proveedor de servicios de virtualización (VSP) y el controlador de infraestructura virtual (VID).

Hyper-V también tiene herramientas para la **administración** y la **conectividad**. Puede instalar estos en el mismo equipo en el que está instalado el rol de Hyper-V y en equipos sin el rol de Hyper-V instalado. Estas herramientas son:

Administrador de Hyper-V.

Módulo de Hyper-V para Windows PowerShell.

Conexión a máquina virtual (a veces denominado VMConnect).

Windows PowerShell Direct.

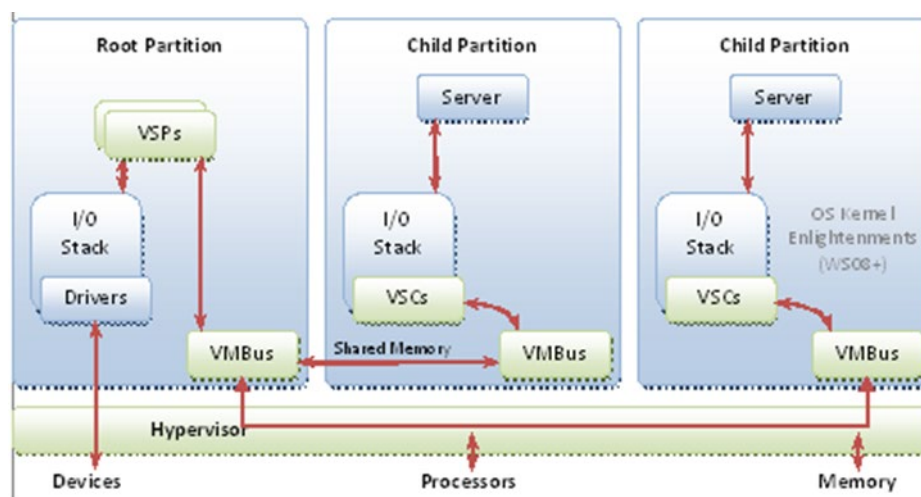


Figura 10. Arquitectura Hyper-V. Fuente: Microsoft, 2022.

3.6. Contenedores

Un contenedor es una **unidad** de software estandarizada que **empaqueta** el código de una aplicación y todas sus dependencias para permitir su ejecución de manera rápida y consistente, independientemente del entorno en el que se ejecute.

Docker es una plataforma de código abierto para desarrollar, distribuir y desplegar aplicaciones en entornos aislados y seguros denominados contenedores.

Los contenedores tienen la característica de ser **livianos**, ya que no necesitan la carga adicional de un hipervisor como ocurre con las máquinas virtuales, sino que se ejecutan directamente dentro del **núcleo** de la máquina host. Esto significa que con un determinado hardware podremos ejecutar mayor número de contenedores que si estuviéramos utilizando varias máquinas virtuales.

La plataforma **Docker** nos permite empaquetar nuestras aplicaciones con todo lo necesario para su ejecución (librerías, código, herramientas, configuraciones), eliminando así dependencias del sistema operativo de la máquina host y facilitando que un despliegue muy rápido de nuestras aplicaciones.

Cómo funcionan los contenedores

Los contenedores tienen una filosofía totalmente diferente a la de las máquinas virtuales. En lugar de tener un sistema operativo completo, los contenedores **comparten** el mismo **kernel o núcleo del sistema operativo host** sobre el que se ejecutan.

El motor de Docker será el encargado de **desplegar y gestionar** los contenedores de nuestras aplicaciones, pero en lugar de reservar parte de los recursos de hardware de la máquina para cada contenedor, lo que hace es **compartirlos** entre todos los

contenedores, permitiendo optimizar su uso y eliminando la necesidad de tener sistemas operativos separados para conseguir el aislamiento.

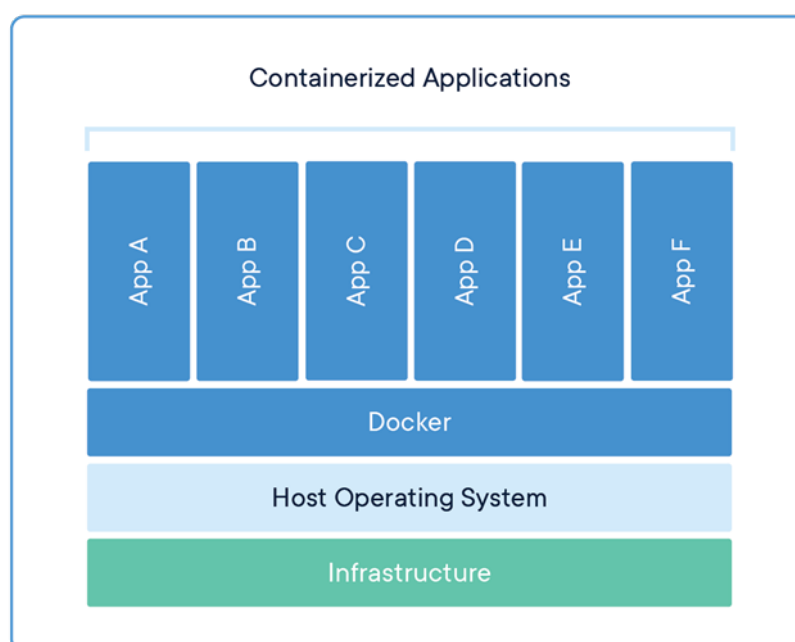


Figura 11. Los contenedores comparten los recursos del host. Fuente: Docker, 2022.

En resumen, los contenedores **comparten los recursos** subyacentes del host sobre el que se ejecutan. Además, los desarrolladores construyen una **imagen** que incluye exactamente lo que necesitan para ejecutar su aplicación o servicio, comenzando por lo básico y agregando solo lo necesario.

Las **máquinas virtuales** se construyen en la dirección opuesta, se empieza por un sistema operativo completo y, dependiendo de las necesidades de aplicación, iremos añadiendo o eliminando componentes.

Arquitectura docker

La arquitectura de Docker sigue un **modelo cliente-servidor**. En la siguiente imagen vemos los principales componentes de la arquitectura de Docker.

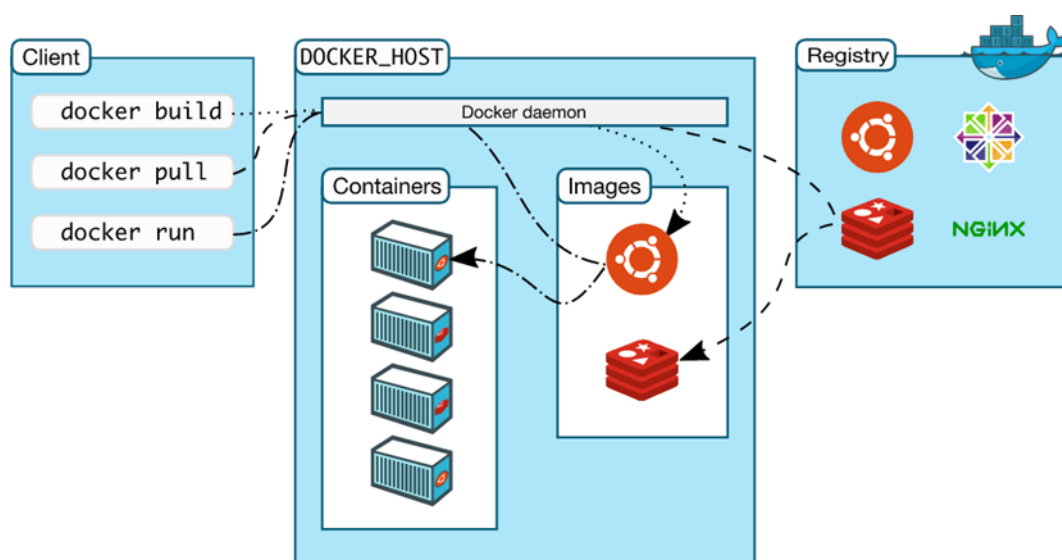


Figura 12. Arquitectura de Docker. Fuente: Docker docs, 2022.

El demonio de Docker

El demonio de Docker (proceso docker) escucha las solicitudes del API y es el responsable de **administrar los objetos** de Docker, como imágenes, contenedores, redes y volúmenes. Un demonio también puede comunicarse con otros demonios para **administrar** los servicios de Docker de **forma distribuida**.

El cliente Docker

El cliente Docker (comando docker) permite a los usuarios **interactuar** con el demonio de Docker utilizando la **línea de comandos**. El cliente Docker y el demonio de Docker se comunican mediante un **API REST**, ya sea a través de sockets UNIX o mediante una interfaz de red. Ambos no tienen por qué ejecutarse en la misma máquina, ya que el cliente Docker podrá conectarse a un demonio de Docker de un sistema remoto.

Imágenes

Las imágenes son **objetos** de Docker que nos servirán para **empaquetar una aplicación o servicio** junto a todo lo necesario para su funcionamiento: código de la aplicación, librerías dependientes, configuraciones, etcétera.

Contenedores

Los contenedores son **instancias o ejecuciones de la imagen** de nuestra aplicación o servicio. Los contenedores se comportan como **entornos aislados y seguros**, por lo que en una misma máquina podemos tener varios contenedores en ejecución de la misma imagen.

Para entender mejor la **diferencia entre imágenes y contenedores** podemos ver su **paralelismo** con un fichero ejecutable y el proceso que lo ejecuta. La imagen sería el equivalente al ejecutable y el contenedor similar a los distintos procesos que hay en ejecución del ejecutable.

Registros de Docker

Los registros de Docker permiten **almacenar y distribuir imágenes** de Docker que luego serán utilizadas para la creación de los contenedores. **Docker Hub** es un registro **público** que puede ser utilizado por los usuarios para compartir imágenes.

Por defecto Docker buscará imágenes en Docker Hub, aunque como veremos existen otros repositorios públicos en la nube. Además, podremos disponer de nuestro propio registro de imágenes privado.

En el vídeo *Contenedores (Dockers)* veremos la instalación del servicio de contenedores en una máquina Linux y el uso básico de contenedores. Utilizaremos tres contenedores de un registro público, los bajaremos a nuestra máquina Linux y lanzaremos los tres contenedores en la misma máquina virtual.



Accede al vídeo

3.7. Referencias bibliográficas

Docker docs. (2022). *Docker overview*. <https://docs.docker.com/get-started/overview/#docker-architecture>

Docker. (2022). *Use containers to Build, Share and Run your applications*. <https://www.docker.com/resources/what-container>

Koop, M. (2021, agosto 5). Bare metal performance with the AWS Nitro System. *AWS HPC Blog*. <https://aws.amazon.com/blogs/hpc/bare-metal-performance-with-the-aws-nitro-system/>

López, P. (2019, octubre 15). *Cómo usar VirtualBox para crear una máquina virtual*. Geeknetic. <https://www.geeknetic.es/Noticia/17411/Como-usar-VirtualBox-para-crear-una-maquina-virtual.html>

Microsoft. (2022, septiembre 19). *Arquitectura de Hyper-V*. <https://docs.microsoft.com/es-es/windows-server/administration/performance-tuning/role/hyper-v-server/architecture>

Microsoft. (2022, septiembre 22). *Ejecución de Hyper-V en una máquina virtual con la virtualización anidada*. <https://docs.microsoft.com/es-es/virtualization/hyper-v-on-windows/user-guide/nested-virtualization>

Parallels. (2022). *Parallels Desktop 18 for Mac*. <https://www.parallels.com/eu/products/desktop/>

QEMU. (s.f.). *QEMU. A generic and open source machine emulator and virtualizer*. <https://www.qemu.org/>

Red Hat. (2018, marzo 2). *¿Qué es la virtualización?* <https://www.redhat.com/es/topics/virtualization/what-is-virtualization#%C2%BFpor-qu%C3%A9-deber%C3%ADa-trasladarse-a-red%C2%A0hat>

Vanover, R. (2017, marzo 28). *Servidores físicos vs. Máquinas virtuales*. *Veeam Blog*. <https://www.veeam.com/blog/es-lat/why-virtual-machine-backups-different.html>

Virtualización. (2022, octubre 23). En *Wikipedia*. <https://es.wikipedia.org/wiki/Virtualizaci%C3%B3n>

VMware. (2022). *VMware Workstation 16 Player*. <https://www.vmware.com/latam/products/workstation-player/workstation-player-evaluation.html>

Información general sobre la tecnología Hyper-V

Microsoft. (2022) *Hyper-V en Windows Server*. <https://docs.microsoft.com/es-es/windows-server/virtualization/hyper-v/hyper-v-on-windows-server>

Centro de recursos multimedia de Microsoft donde puedes ampliar información sobre todos los temas de esta unidad.

Documentación Amazon Web Services

AWS. (2022). *AWS Nitro System*. <https://aws.amazon.com/es/ec2/nitro/>

Centro de recursos propios sobre el fabricante de nube AWS.

Cómo comenzar con Docker

Dockercon. (2020, mayo 28). *How to get started with Docker*. <https://docker.events.cube365.net/docker/dockercon/content/Videos/hgMFTyX5kYKmTPWZo>

Charla inicial de la conferencia DockerCon 2020 en la que Peter McKee nos cuenta por qué los contenedores son esenciales en el mundo del desarrollo hoy en día. Nos detalla los primeros pasos con Docker y Kubernetes, así como algunas de las mejores prácticas para configurar un entorno de desarrollo.

¿Qué es la virtualización?

- A. El uso de un hypervisor.
- B. La creación de recursos virtuales.
- ☐ C. La creación de una versión virtual de un sistema operativo.
- D. Ninguna de las anteriores.

La virtualización es la creación de una versión virtual de un sistema operativo, de una plataforma, de un sistema de almacenamiento, etc.

¿Qué definición podríamos aplicar a «hypervisor»?

- A. Software libre que se pone en servidores.
- ☐ B. La capa de abstracción se conoce como monitor de máquina virtual.
- C. Versión del software KVM de Linux.
- D. Microprocesador que deja crear máquinas virtuales en bare metal.

La capa de abstracción se conoce como monitor de máquina virtual (virtual machine monitor, VMM) o hypervisor, gracias a esta capa se puede gestionar y administrar los recursos principales de la máquina física.

¿Cuál no es ventaja de la virtualización?

- ☐ A. Uso gratuito del software que virtualizamos.
- B. Reducción de infraestructura física.
- C. Aumento de tolerancia a fallos.
- D. Mejora del rendimiento.

La capa de abstracción se conoce como monitor de máquina virtual (virtual machine monitor, VMM) o hypervisor, gracias a esta capa se puede gestionar y administrar los recursos principales de la máquina física.

¿Qué podemos considerar como un «contenedorización»?

- ☐ A. Virtualización de nivel de sistema operativo.
- B. Virtualización de red.
- C. Virtualización de aplicaciones.
- D. Virtualización de centro de datos.

Virtualización a nivel de sistema operativo o contenedorización. El kernel del sistema operativo subyacente permite ejecutar de forma paralela instancias de espacio de usuario, aisladas unas de otras.

Una ventaja del Virtual Box es:

- A. Tiene soporte gratuito ilimitado.
- B. Es el único que se puede usar en los centros de datos empresariales.
- C. Gestiona mejor la memoria que otros hypervisores.
- ☐ D. Es de licencia GNU, libre.

Una de sus grandes ventajas es que está disponible de forma libre mediante licencia GNU de uso público.

¿La versión de VmWare empresarial es denominada?

- A. Workstation.
- ☐ B. VSphere Enterprise.
- C. Hyper-V.
- D. Vmware Business.

Es una aplicación que proporciona un entorno completo para virtualización y es el más utilizado y potente de las herramientas que dispone la compañía.

El KVM permite:

- A. Manejar un teclado monitor y pantalla múltiple (keyboard, vídeo, *mouse*).
- B. Es un sistema de nube privada.
- ☐ C. Convertir a Linux en un Hypervisor para alojar máquinas invitadas.
- D. Es un orquestador de contenedores en Linux.

Las KVM puede convertir a Linux en un hipervisor que permite que una máquina host ejecute varios entornos virtuales aislados llamados máquinas virtuales (VM) o guests.

¿Cuál no es una propiedad de KVM?

- A. Gestión de la memoria.
- B. Migración activa.
- C. Almacenamiento.
- _ D. Gestión de licencias.

Entre las propiedades de KVM tenemos, migración activa, gestión de la memoria, almacenamiento.

¿Los registros de Docker sirven para:

- A. Mantener un control de los contenedores en ejecución.
- _ B. Almacenar y distribuir imágenes de Docker.
- C. Permitir la comunicación entre dos contenedores Docker.
- D. Instalar host de Docker remotos.

Un registro de Docker nos permite almacenar de manera centralizada las imágenes que utilizará nuestro sistema. Las imágenes se descargarán automáticamente del registro cuando se necesiten y no estén disponibles localmente.

Los contenedores de Docker:

- A. Son más pesados en tamaño que las máquinas virtuales.
- B. No son convenientes para trabajar con microservicios.
- C. Se inician más lento que las máquinas virtuales, pero son más seguros.
- _ D. Permiten aprovechar mejor el hardware de la máquina host.

Los contenedores son más ligeros y rápidos al desplegarse que las máquinas virtuales, lo que nos permite ejecutar más cargas de trabajo en el mismo hardware.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Fundamentos de computación en la nube

Índice

Esquema	3
Ideas clave	4
4.1. Introducción y objetivos	4
4.2. Introducción a la informática en la nube	5
4.3. Introducción y ventajas de la informática en la nube	11
4.4. Marco de adopción de la nube	21
4.5. Costo total de propiedad	25
4.6. Prácticas recomendadas para crear soluciones en la nube	36
4.7. Infraestructura global	39
4.8. Referencias bibliográficas	45
A fondo	46
Test	47

Esquema



4.1. Introducción y objetivos

La computación en la nube es una realidad desde hace ya varios años. Es importante conocer diferentes **tipos de proveedores** de nube, así como conocer las principales **ventajas** de la informática en la nube.

Todos los proveedores de nube tienen los **servicios categorizados** por categorías que son importantes conocer, así como diferenciar sus principales **servicios**.

Existen también varios **marcos de trabajo** relacionados con la nube. El primer marco de referencia nos ayuda a movernos a la nube teniendo presente todas las unidades de negocio de las compañías. En el segundo marco nos centraremos en la creación de buenas arquitecturas en la nube y aplicaremos las buenas prácticas que los proveedores nos proporcionan.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Introducción a la informática en la nube.

Ventajas de la informática en la nube.

Introducción a Amazon Web Services (AWS), Azure y GCP.

Marco de adopción de la nube.

Marco de creación de buenas arquitecturas en la nube.

En los vídeos de este tema podremos aprender los siguientes conceptos:

Vídeo 1. Veremos los diferentes modelos de servicio en la nube centrándonos en los tres principales IaaS, PaaS y SaaS con ejemplos en una de las principales nubes públicas.

Vídeo 2. Introducción a los marcos de trabajo Cloud Adoption Framework y Well Architected Framework para aproximación a nube pública.

4.2. Introducción a la informática en la nube

Según afirma Amazon Web Services (2021), la informática en la nube es la **entrega bajo demanda** de potencia de cómputo, bases de datos, almacenamiento, aplicaciones y otros recursos de TI a través de **Internet** con un sistema de precios de **pago** por uso.

Estos recursos se ejecutan en equipos de servidores ubicados en grandes centros de datos en diferentes partes del mundo. Cuando se utiliza un **proveedor** de servicios en la nube, ese proveedor de servicios es el **propietario de los equipos** que se utilizan. Estos recursos se pueden utilizar juntos como piezas fundamentales para crear soluciones que ayuden a alcanzar los objetivos empresariales y a cumplir los requisitos tecnológicos.

La informática en la nube permite dejar de considerar la infraestructura como hardware y, en cambio, verla (y usarla) como software. ¿Qué quiere decir esto?

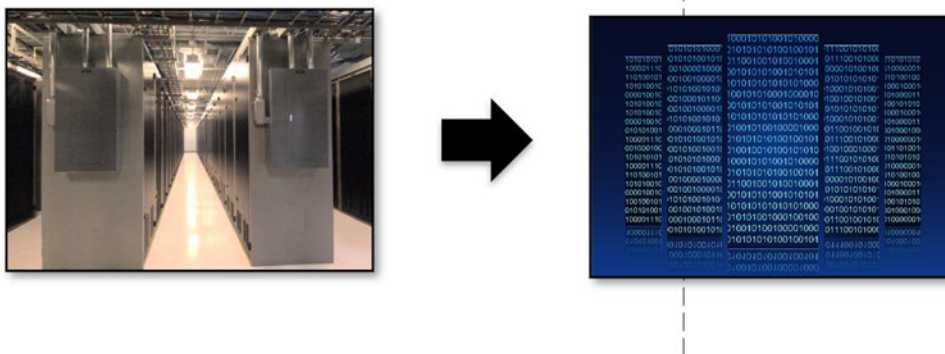


Figura 1. Datacenter vs. nube. Fuente: AWS, 2021.

En el modelo de **informática tradicional**, la infraestructura se considera **hardware**. Las soluciones de hardware son **físicas**, lo que significa que requieren espacio, personal, seguridad física, planificación e inversión de capital.

Otro aspecto prohibitivo de la informática tradicional, además de la importante inversión inicial, es el largo ciclo de obtención de hardware que implica la **adquisición**, el **aprovisionamiento** y el **mantenimiento** de la infraestructura en las instalaciones.

Con una solución de hardware, es necesario preguntarse si hay **capacidad de recursos o almacenamiento suficientes** para satisfacer sus necesidades y aprovisionar capacidad mediante la predicción de los picos máximos teóricos. Si no alcanza el pico máximo previsto, tendrá que pagar por recursos costosos que permanecen inactivos. Si supera el pico máximo previsto, no tendrá capacidad suficiente para satisfacer sus necesidades. Además, si sus necesidades cambian, debe dedicar el tiempo, el esfuerzo y el dinero necesarios para implementar una solución nueva.

Por ejemplo, si desea aprovisionar un nuevo sitio web, necesitaría comprar el hardware, montarlo en bastidores y apilarlo, colocarlo en un centro de datos y, luego, administrarlo o contratar a alguien que lo haga por usted. Este enfoque es costoso y requiere mucho tiempo.

En cambio, la informática en la nube le permite ver su infraestructura como software. Las **soluciones** de software son **flexibles**. Puede seleccionar los servicios en la nube que mejor se adapten a sus necesidades, aprovisionar y terminar esos recursos bajo demanda, y pagar por lo que utilice. Puede escalar los recursos en forma ascendente y descendente de manera elástica y automática. Con el modelo de informática en la nube, puede tratar a los recursos como temporales y desechables. La flexibilidad que ofrece la informática en la nube permite a los negocios implementar **soluciones nuevas** rápidamente y con **costos iniciales bajos**.

En comparación con las soluciones de hardware, las soluciones de software pueden cambiar de manera mucho más rápida, sencilla y rentable.

La informática en la nube ayuda a los desarrolladores y departamentos de TI a evitar arduas tareas, como el aprovisionamiento, el mantenimiento y la planificación de la capacidad, lo que les permite concentrarse en lo que más importa.

A medida que ha incrementado la popularidad de la informática en la nube, se han desarrollado varios **modelos de servicios y estrategias** de implementación para satisfacer las necesidades de los distintos usuarios. Cada tipo de modelo de servicio en la nube y estrategia de implementación le ofrece **distintos niveles de control, flexibilidad y administración**. Comprender las diferencias entre estos modelos de servicios en la nube y las estrategias de implementación puede ayudarlo a decidir qué conjunto de servicios es adecuado para sus necesidades.

Existen tres **modelos principales de servicios en la nube**. Cada modelo representa una parte diferente de la pila de informática en la nube y le ofrece un nivel diferente de control sobre sus recursos de TI:

Infraestructura como servicio (IaaS): los servicios en esta categoría son la base para la TI en la nube. Por lo general, permiten acceder a las características de redes, a los equipos (virtuales o en hardware dedicado) y al espacio de almacenamiento de datos. La IaaS le ofrece el mayor nivel de flexibilidad y control de administración en relación con sus recursos de TI. Es lo más parecido a los recursos de TI existentes con los que están familiarizados muchos departamentos y desarrolladores de TI en la actualidad.

Plataforma como servicio (PaaS): los servicios de esta categoría reducen la necesidad de administrar la infraestructura subyacente (normalmente, el hardware y los sistemas operativos) y le permiten centrarse en la implementación y la administración de sus aplicaciones.

Software como servicio (SaaS): los servicios de esta categoría le ofrecen un producto completo que el proveedor de servicios ejecuta y administra. En la mayoría de los casos, el software como servicio hace referencia a las aplicaciones de usuario final. Con una solución basada en SaaS, ya no debe pensar en cómo mantener el servicio ni en cómo administrar la infraestructura subyacente. Tiene que pensar solo en cómo planea utilizar ese software en particular. Un ejemplo común de una aplicación SaaS es un programa de correo electrónico basado en la web que le permite enviar y recibir mensajes sin tener que administrar la incorporación de características al producto del correo ni mantener los servidores y los sistemas operativos en los que se ejecuta el programa.

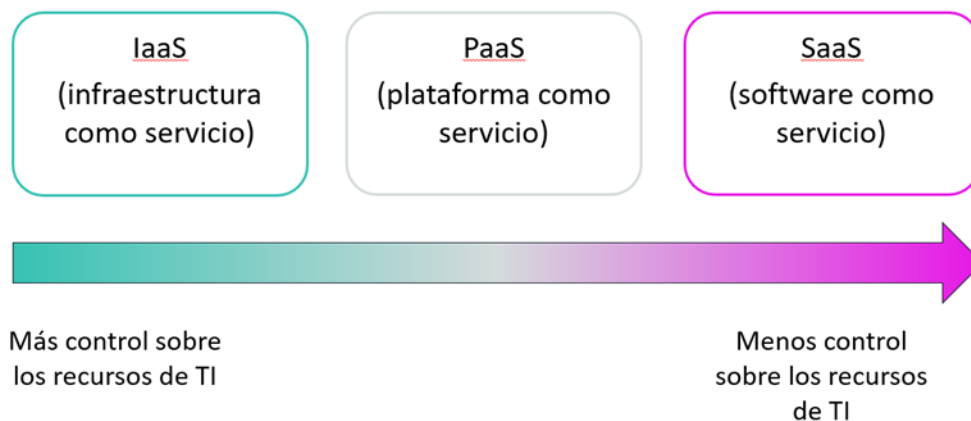


Figura 2. IaaS vs. PaaS vs. SaaS. Fuente: AWS, 2021.

En el vídeo *Modelos de servicio en la nube* vamos a ver los modelos de servicios en la nube tradicional con un pequeño ejemplo. Los tres modelos tradicionales son infraestructura como servicio (IaaS), plataforma como servicio (PaaS) y software como servicio (SaaS). También se ven nuevos modelos como pueden ser entorno de servicio como servicio (IDEaaS) o función como servicio (FaaS).



Accede al vídeo

Existen **tres modelos** principales de **implementación de informática en la nube**, que representan los entornos en la nube en los que se pueden implementar sus aplicaciones:

1. **Nube:** una aplicación basada en la nube se encuentra completamente implementada en la nube de modo que todas las partes de la aplicación se ejecutan en la nube. Las aplicaciones en la nube se crearon en la nube o las migraron de una infraestructura existente para aprovechar los beneficios de la informática en la nube. Las aplicaciones basadas en la nube se pueden crear en partes de infraestructura de bajo nivel o pueden utilizar servicios de nivel superior que permiten abstraerse de los requisitos de administración, arquitectura y escalado de la infraestructura principal.

Híbrida: una implementación híbrida es una manera de conectar la infraestructura y las aplicaciones entre los recursos basados en la nube y los recursos existentes situados fuera de ella. El método más común de implementación híbrida es entre la infraestructura en la nube y la que ya se encuentra en las instalaciones. Este modelo permite a una organización extender y aumentar su infraestructura en la nube mientras conecta los recursos de la nube a los sistemas internos.

En las instalaciones: la implementación de recursos en las instalaciones mediante herramientas de administración de recursos y virtualización se denomina a veces nube privada. A pesar de que la implementación en las instalaciones no ofrece muchos de los beneficios de la informática en la nube, en algunas ocasiones se utiliza por su capacidad de proporcionar recursos dedicados. En la mayoría de los casos, este modelo de implementación es idéntico al de la infraestructura de TI antigua, pero también podría utilizar tecnologías de virtualización y administración de aplicaciones para incrementar el uso de los recursos.

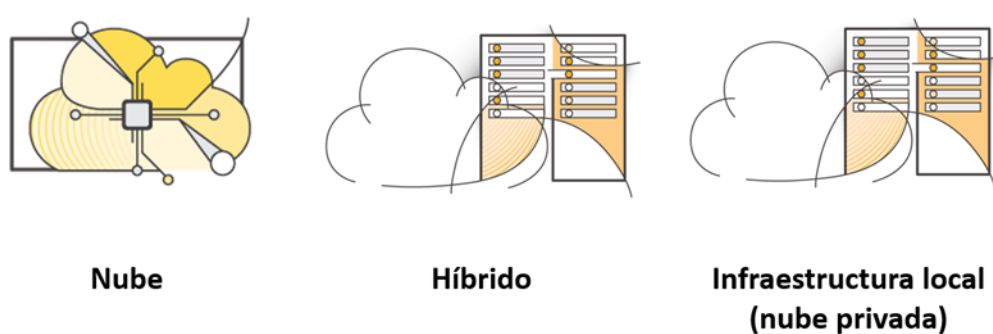


Figura 3. Tipos de nube. Fuente: AWS, 2021.

Hay muchas similitudes entre la nube y el espacio de TI en la instalación tradicional:

Los grupos de seguridad, las listas de control de acceso a la red (ACL de red) y son similares a los firewalls, las listas de control de acceso (ACL) y los administradores.

Elastic Load Balancing y VPC, conceptos de red, son similares a los enrutadores, las canalizaciones de red y los conmutadores.



Figura 4. Infraestructura tradicional. Fuente: AWS, 2021.

4.3. Introducción y ventajas de la informática en la nube

Según afirma Amazon Web Services (2021), por lo general, un **servicio web** es cualquier **software** que está disponible en **Internet** o en redes privadas (intranet).

Un servicio web utiliza un **formato estandarizado** (como el lenguaje de marcado extensible [XML] o la notación de objetos de JavaScript [JSON]) para la solicitud y la respuesta de una interacción con la interfaz de programación de aplicaciones (API). No está vinculado a ningún sistema operativo o lenguaje de programación. Es **autodescriptivo** a través de un archivo de definición de interfaz y es detectable.

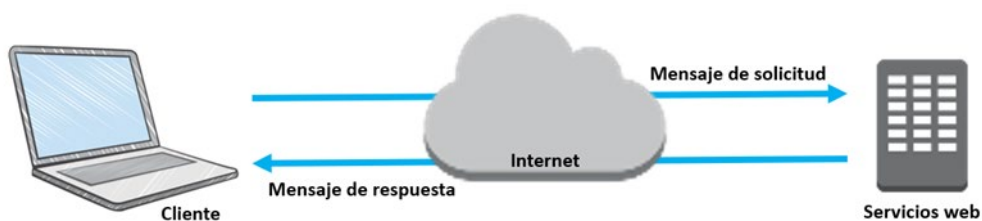


Figura 5. Funcionamiento de Internet. Fuente: AWS, 2021.

Amazon Web Services (AWS), Azure y [GCP](#) son plataformas en la nube segura que ofrece un amplio conjunto de **productos globales** basados en la nube. Como estos productos se entregan a través de Internet, tiene acceso **bajo demanda** a los recursos informáticos, de almacenamiento, de red, de base de datos y otros recursos de TI que podría necesitar para sus proyectos, así como a las herramientas para administrarlos. Inmediatamente, puede aprovisionar y lanzar recursos. Los recursos están listos para que los utilice en cuestión de minutos.

Ofrecen **flexibilidad** y sus entornos puede volver a configurarse y actualizarse bajo demanda, escalarse de forma ascendente o descendente automáticamente para cumplir con los patrones de uso y optimizar el gasto, o bien, cerrarse de forma temporal o permanente. La facturación de los servicios se convierte en un **gasto operativo** en lugar de un gasto de capital.

Los servicios están diseñados para trabajar **juntos** a fin de admitir prácticamente cualquier tipo de aplicación o carga de trabajo.

En la Figura 6 vea a los **servicios** como **bloques de creación** que puede juntar rápidamente para crear soluciones sofisticadas y escalables y, luego, ajustarlas a medida que cambien sus necesidades.

Los servicios se clasifican en **diferentes categorías** y cada una contiene uno o varios servicios. Puede seleccionar los servicios que desea de estas categorías diferentes para crear sus soluciones.



Figura 6. Servicios en AWS. Fuente: AWS, 2021.

Servicios web y creación de aplicación

Supongamos que está creando una aplicación de base de datos. Es posible que los clientes envíen datos a sus instancias que es un servicio en la categoría de informática. Estos servidores agrupan los datos por lotes en incrementos de un minuto y agregan un objeto por cliente al servicio de almacenamiento que eligió utilizar.

A continuación, puede utilizar una base de datos no relacional, con el fin de impulsar su aplicación para, por ejemplo, crear un índice que le permita encontrar todos los objetos de un cliente específico que se recopilaron durante un período determinado. Es posible que decida ejecutar estos servicios dentro de una Virtual Private Cloud, que es un servicio en la categoría de redes.

El objetivo de este ejemplo sencillo es mostrar que puede seleccionar servicios web de diferentes categorías y utilizarlos juntos para crear una solución (en este caso, una aplicación de base de datos). Por supuesto, las soluciones que crea pueden ser bastante complejas.

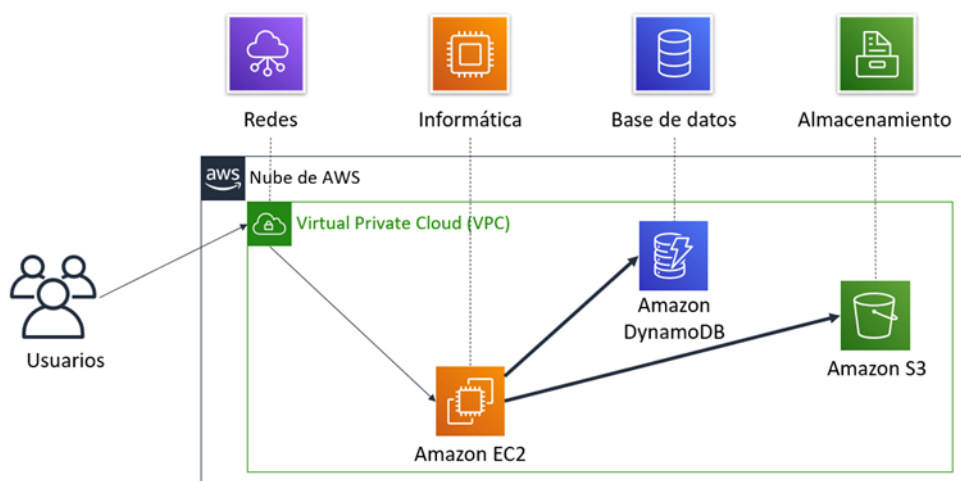


Figura 7. Sencilla arquitectura en AWS. Fuente: AWS, 2021.

El servicio que elija utilizar dependerá de sus objetivos empresariales y requisitos tecnológicos. En el ejemplo que acaba de ver, la solución utilizó máquinas virtuales como servicio informático. Sin embargo, este es solo uno de los muchos servicios de informática que ofrecen las nubes.

A continuación, se muestran otras **ofertas de informática** que podría elegir utilizar para los siguientes casos de uso de ejemplo:

Máquinas virtuales: desea tener un control absoluto de sus recursos informáticos.

Función como código: desea ejecutar el código y no administrar ni aprovisionar servidores.

Plataforma como servicio: desea un servicio que implemente, administre y escale sus aplicaciones web en su lugar.

Webs estáticas: necesita una plataforma en la nube ligera para una aplicación web sencilla.

Procesos por lotes: necesita ejecutar cientos de miles de cargas de trabajo por lotes.

Servicios nube en local: desea ejecutar la infraestructura de AWS en su centro de datos en las instalaciones.

Contenedores: desea implementar una arquitectura de microservicios o contenedores.

Del mismo modo, existen diversos servicios entre los que puede elegir en las demás categorías y el número de ofertas sigue creciendo.

La variedad de servicios de nube puede resultar intimidante cuando se comienza el traspaso hacia la nube. En los próximos temas veremos algunos de los servicios más comunes en las siguientes categorías de servicios: informática, almacenamiento, base de datos, redes y entrega de contenido, seguridad, identidad y conformidad, administración y gobernanza, y administración de costos.

SERVICES	aws	Azure	Google Cloud Platform
Virtual Servers	Elastic Cloud Compute	Virtual Machines	Google Compute Engine
Serverless Computing	Lambda	Azure Functions	Cloud Functions
Kubernetes Management	Elastic Kubernetes Service	Kubernetes Service	Kubernetes Engine
Object Storage	Simple Storage Service	Azure Blob	Cloud Storage
File Storage	Elastic File Storage	Azure Files	Filestore
Block Storage	Elastic Block Storage	Azure Disk	Persistent Disk
Relational Database	Relational Database Service	SQL Database	Cloud SQL
NoSQL Database	DynamoDB	Cosmos DB	Firestore
Virtual Network	Virtual Private Cloud	Azure VNet	Virtual Private Network
Content Delivery Network	CloudFront	Azure CDN	Cloud CDN
DNS Service	Route 53	Traffic Manager	Cloud DNS
Authentication and Authorization	IAM	Azure Active Directory	Cloud IAM
Key Management	KMS	Azure Key Vault	KMS
Network Security	AWS WAF	Application Gateway	Cloud Armor

Figura 8. Comparación de servicios. Fuente: Tutorials Dojo, 2022.

Es posible que se pregunte cómo obtener acceso a la amplia gama de servicios que ofrece. Existen tres **formas de crear y administrar** los recursos en la nube:

Consola de administración: la consola proporciona una interfaz gráfica completa para la mayoría de las características (en ocasiones, es posible que las características nuevas no tengan todas sus capacidades integradas en la consola cuando se lance inicialmente la característica).

Interfaz de línea de comandos (CLI): la CLI ofrece un conjunto de utilidades que se pueden lanzar desde un script de comandos en Linux, macOS o Microsoft Windows.

Kits de desarrollo de software (SDK): los proveedores ofrecen paquetes que permiten obtener acceso a una variedad de lenguajes de programación conocidos. Esto facilita el uso de sus aplicaciones existentes y, también, permite crear aplicaciones que implementan y monitorean sistemas complejos en su totalidad a través de código.

Las tres opciones se basan en APIs para acceder a los servicios de las nubes:



Figura 9. Acceso a los servicios de AWS. Fuente: AWS, 2021.

Los **aprendizajes clave** de esta sección del módulo son los siguientes:

AWS es una plataforma en la nube segura que ofrece un amplio conjunto de productos globales basados en la nube denominados servicios que están diseñados para trabajar juntos.

Existen muchas categorías de servicios de AWS y cada categoría tiene, a su vez, muchos servicios entre los cuales elegir.

Elija un servicio en función de sus objetivos empresariales y requisitos tecnológicos.

Existen tres formas de interactuar con los servicios de AWS.

A continuación, veamos unas **ventajas** sobre la computación en la **nube**:

1. **Cambiar sus gastos de capital por gastos variables:** los gastos de capital (capex) son los fondos que utiliza una empresa para adquirir, actualizar y mantener activos los recursos físicos como los inmuebles, edificios industriales o equipos. ¿Recuerda el ejemplo de centro de datos en el modelo de informática tradicional en el que necesitaba montar en bastidores el hardware y apilarlo y, luego, administrarlo todo? Tiene que pagar por todo lo que hay en el centro de datos, lo use o no.

En cambio, un gasto variable es un gasto que la persona que asume el costo puede modificar o evitar fácilmente. En lugar de hacer grandes inversiones en centros de datos y servidores antes de saber qué uso les va a dar, puede pagar solo cuando consuma recursos y únicamente por la cantidad que consuma. Por lo tanto, ahorra dinero en tecnología. También le permite adaptarse a aplicaciones nuevas con todo el espacio que necesite en cuestión de minutos, en lugar de semanas o días. El mantenimiento es reducido, por lo que puede prestar más atención a los objetivos principales de su negocio.



Figura 10. Inversión vs. gasto. Fuente: AWS, 2021.

Beneficiarse de las grandes economías de escala: mediante el uso de la informática en la nube, puede conseguir un costo variable inferior al que puede obtener por su cuenta. En la nube se suman los consumos realizados por cientos de miles de usuarios. De esta forma, los proveedores, pueden aplicar mayores economías de escala que se traducen en precios por uso más bajos.



Figura 11. Economía de escala. Fuente: AWS, 2021.

Evitar asumir estimaciones sobre capacidad: elimine las conjeturas sobre las necesidades de capacidad de su infraestructura. Cuando toma una decisión respecto a la capacidad antes de implementar una aplicación, a menudo termina con recursos inactivos costosos o con una capacidad limitada. Con la informática en la nube, estos problemas desaparecen. Puede acceder a los recursos que necesite, sean muchos o pocos, y escalar de forma ascendente o descendente, según sea necesario, con solo avisar unos minutos.

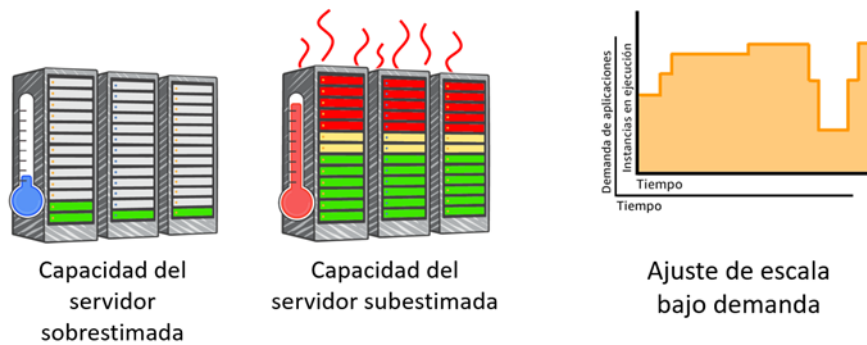


Figura 12. Estimaciones de capacidad. Fuente: AWS, 2021.

Aumentar la velocidad y la agilidad: en un entorno de informática en la nube, se puede acceder a recursos de TI nuevos con tan solo un clic, por lo que el tiempo que se tarda en poner esos recursos a disposición de sus desarrolladores se reduce de semanas a tan solo minutos. El resultado es un aumento radical del nivel de agilidad de la organización, ya que se reducen de manera significativa el costo y el tiempo para realizar tareas de desarrollo y experimentación.

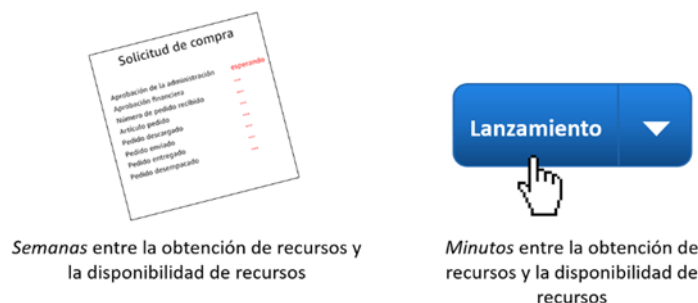


Figura 13. Velocidad y agilidad. Fuente: AWS, 2021.

Dejar de gastar dinero en la ejecución y el mantenimiento de centros de datos: concéntrese en proyectos que diferencien su negocio en lugar de centrarse en la infraestructura. La informática en la nube le permite centrarse en sus propios clientes, en lugar de la tediosa tarea de montar servidores en bastidores, apilarlos y alimentar servidores.

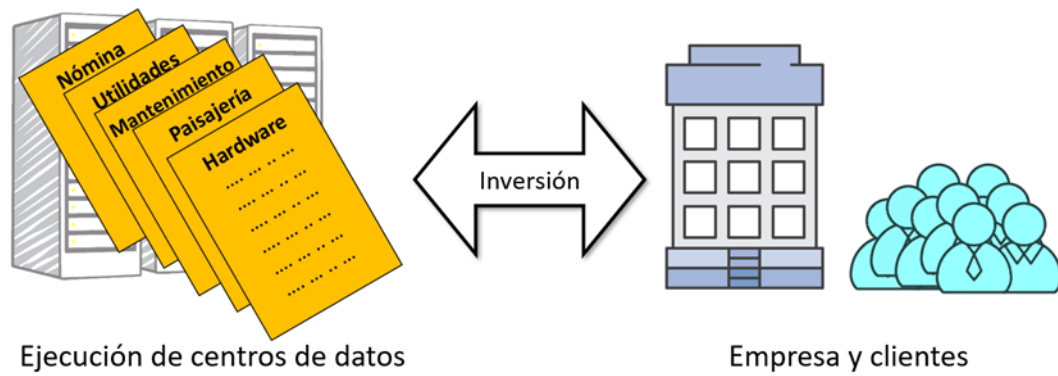


Figura 14. Inversión en data center. Fuente: AWS, 2021.

Adquirir escala mundial en cuestión de minutos: puede implementar su aplicación en varias regiones de AWS de todo el mundo con tan solo unos clics. Como resultado, puede ofrecer una latencia menor y una mejor experiencia a sus clientes de forma sencilla y con un costo mínimo.

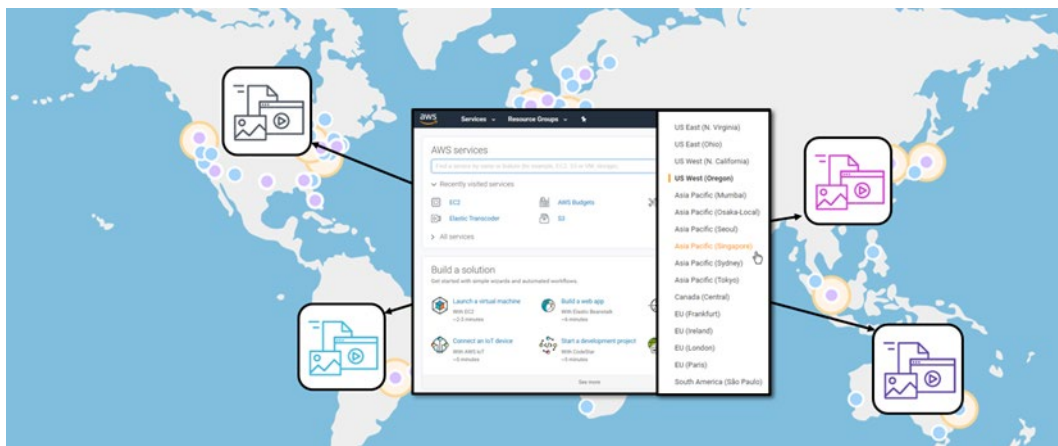


Figura 15. Escala mundial de la nube. Fuente: AWS, 2021.

4.4. Marco de adopción de la nube

Según afirma Amazon Web Services (2021), como ya ha aprendido, la informática en la nube ofrece muchas ventajas con respecto al modelo tradicional. Sin embargo, para la mayoría de las organizaciones, la adopción de la nube no se realiza al instante. La tecnología es una cosa, pero una organización también se compone de personas y procesos y estos tres elementos deben estar en consonancia para una adopción satisfactoria de la nube.

La informática en la nube introduce un cambio significativo en la forma en que se obtiene, utiliza y administra la tecnología.

También cambia la forma en que las organizaciones presupuestan y pagan los servicios tecnológicos. La adopción de la nube requiere que los **cambios** fundamentales se **debatan** y se tengan en cuenta en toda la organización. También requiere que las partes interesadas de todas las unidades organizativas, tanto dentro como fuera de la TI, **respalden** estos nuevos cambios.

Según Amazon Web Services (2022), Cloud Adoption Framework es un marco de buenas prácticas para la migración a la nube basado en 6 capacidades que podemos ver en la siguiente figura:

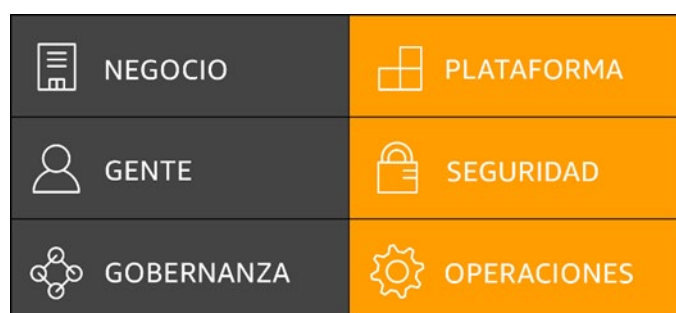


Figura 16. Marco CAF. Fuente: AWS, 2021.

El proceso de adopción de la nube de cada organización es único. Sin embargo, para que cualquier organización pueda migrar con éxito su portafolio de TI a la nube, tres elementos (personas, procesos y tecnología) deben estar en **consonancia**. Los líderes empresariales y tecnológicos de una organización deben comprender el estado actual de la organización, el estado del objetivo y la transición necesaria para alcanzar el estado del objetivo, de manera que puedan definir objetivos y crear procesos para el personal.

El marco de adopción de la nube ofrece orientación y prácticas recomendadas para ayudar a las organizaciones a identificar deficiencias en las habilidades y los procesos.

También ayuda a las organizaciones a crear un enfoque integral para la informática en la nube, en toda la organización y durante todo el ciclo de vida de la TI, con el fin de acelerar la adopción satisfactoria de la nube.

En el máximo nivel, **CAF** organiza la orientación en seis áreas de enfoque, son denominadas **perspectivas**. Las perspectivas abarcan personas, procesos y tecnología. Cada perspectiva se compone de un conjunto de **capacidades**, que abarca responsabilidades particulares que son propiedad o cuentan con la administración de partes interesadas relacionadas a nivel funcional.

Las capacidades que incluye cada perspectiva se utilizan para identificar qué áreas de su organización requieren atención. Mediante la **identificación de deficiencias**, se pueden crear flujos de trabajo prescriptivos que ayudan a que el traspaso a la nube sea exitoso.

En general, las perspectivas de la empresa, el personal y la gobernanza se centran en las capacidades empresariales; mientras que las perspectivas de plataforma, seguridad y operaciones se centran en las capacidades técnicas.



Figura 17. División capacidades de CAF. Fuente: AWS, 2021.

Las partes interesadas desde la perspectiva de los **negocios** (por ejemplo, gerentes comerciales, directores de finanzas, responsables del presupuesto y partes interesadas en la estrategia) pueden utilizar el **CAF** con el fin de crear un **argumento empresarial sólido** para la adopción de la nube y de priorizar las iniciativas de adopción de la nube. Las partes interesadas deben asegurarse de que las estrategias y los objetivos empresariales de una organización estén en consonancia con las **estrategias y objetivos de la TI**.

Las partes interesadas desde la perspectiva del **personal** (por ejemplo, directores de recursos humanos y personal) pueden utilizar el CAF para evaluar funciones y estructuras organizativas, requisitos de habilidades y procesos nuevos y, así, **identificar deficiencias**. Realizar un análisis de las necesidades y las deficiencias puede ayudar a priorizar la formación técnica, la dotación de personal y los cambios organizativos para crear una **organización ágil**.

Las partes interesadas desde la perspectiva de la **gobernanza** (por ejemplo, el director de información, los directores de programas, los arquitectos empresariales, los analistas de negocios y los administradores de portafolios) pueden utilizar el **CAF** para concentrarse en las habilidades y los procesos necesarios para poner en consonancia la **estrategia y los objetivos de la TI** con la estrategia y los **objetivos**

empresariales. Este enfoque ayuda a la organización a maximizar el valor de negocio de su inversión en TI y minimizar los riesgos comerciales.

Las partes interesadas desde la perspectiva de la **plataforma** (por ejemplo, director de tecnología, directores de TI y arquitectos de soluciones) utilizan una variedad de dimensiones y modelos arquitectónicos para **comprender y comunicar** la naturaleza de los sistemas de TI y sus relaciones. Deben poder describir la arquitectura del entorno de estado de destino detalladamente. El CAF incluye principios y patrones para la implementación de nuevas soluciones en la nube y para la migración de cargas de trabajo en las instalaciones a la nube.

Las partes interesadas desde la perspectiva de la **seguridad** (por ejemplo, director de seguridad de la información, administradores de seguridad de TI y analistas de seguridad de TI) deben **garantizar** que la organización cumpla los **objetivos** de seguridad en materia de visibilidad, auditoría, control y agilidad. Las partes interesadas desde la perspectiva de la seguridad pueden utilizar el CAF para estructurar la selección y la implementación de **controles de seguridad** que satisfagan las necesidades de la organización.

Las partes interesadas desde la perspectiva de las **operaciones** (por ejemplo, los directores de operaciones de TI y los directores de soporte de TI) definen cómo se realizan los negocios diarios, trimestrales y anuales. Las partes interesadas desde la perspectiva de las operaciones se **alinean** a las **operaciones del negocio** y las **respaldan**. El CAF ayuda a estas partes interesadas a definir los procedimientos operativos actuales. También los ayuda a identificar la formación técnica y los cambios en los procesos necesarios para implementar una adopción satisfactoria de la nube.

4.5. Costo total de propiedad

Las nubes tienen tres **generadores** fundamentales **de costos**: informática, almacenamiento y transferencia de datos de salida. Estas características varían ligeramente en función del producto y el modelo de precios que elija.

En la mayoría de los casos, no se aplican **cargos** por la transferencia de datos de entrada ni por la transferencia de datos entre otros servicios dentro de la misma región. Existen algunas excepciones, así que asegúrese de verificar las tarifas de transferencia de datos antes de empezar a utilizar el **servicio de nube**.

Las transferencias de datos de salida se **acumulan** entre todos los servicios y, luego, se cobran según la tarifa para ese tipo de transferencia. Este monto aparece en el **extracto mensual**.

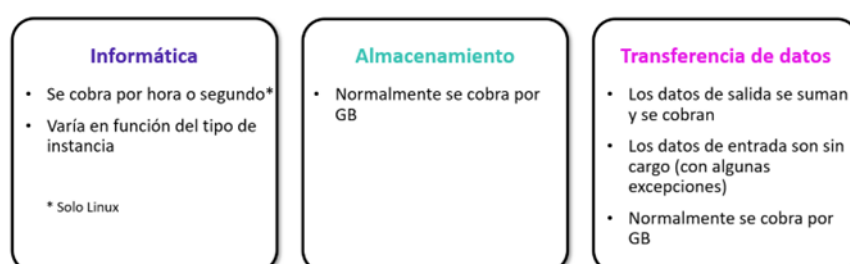


Figura 18. Precios en nube. Fuente: AWS, 2021.

Mientras que la cantidad y los tipos de servicios han aumentado de forma considerable, la filosofía de precios no ha cambiado. Así, al final de cada mes, pagará por lo que **utiliza**. Puede comenzar a utilizar un producto o dejar de hacerlo en cualquier momento. No se necesitan contratos a largo plazo. Solo se paga por la **cantidad exacta de recursos** que realmente necesita para cada uno de estos servicios.

Este **modelo de precios**, similar al de los servicios básicos, implica lo siguiente:

Pagar por lo que utilice.

Pague menos al reservar.

Pague menos cuando utilice más.

Pagar aún menos a medida que crece el proveedor.

Examinará en mayor profundidad estos conceptos básicos de precios.



Figura 19. Pago por uso. Fuente: AWS, 2021.

A menos que se dedique a crear centros de datos, quizás ya ha invertido demasiado tiempo y dinero en crearlos. Con la nube, paga solo por los servicios que utiliza, sin grandes gastos iniciales. Puede reducir los costos variables y, así, ya no necesita destinar valiosos recursos para construir una infraestructura costosa, lo que incluye la compra de servidores y licencias de software y el alquiler de instalaciones.

Puede adaptarse a los cambios en las necesidades de un negocio y redirigir su enfoque a la **innovación** y la **inversión** si paga solo por lo que utiliza y durante el tiempo que lo necesita. Todos los **servicios** se encuentran disponibles **bajo demanda** y no requieren contratos a largo plazo ni dependen de licencias complejas.

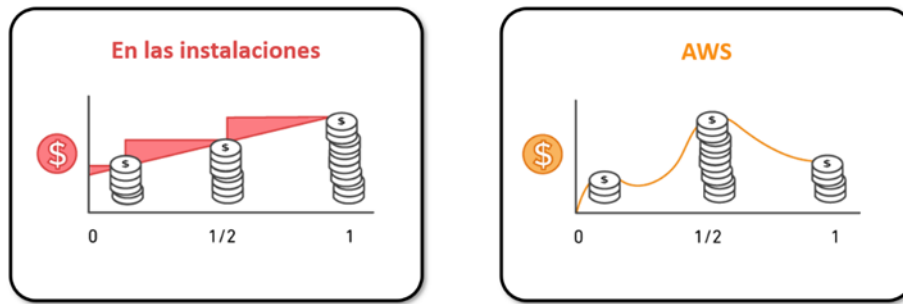


Figura 20. Pago en local vs. pago en nube. Fuente: AWS, 2021.

Para ciertos servicios, puede invertir en capacidad reservada. Con las **instancias reservadas**, puede ahorrar hasta un 75 % en comparación con la capacidad bajo demanda equivalente. Las instancias reservadas están disponibles en **tres opciones**:

Instancia reservada con pago inicial completo (AURI).

Instancia reservada con pago inicial parcial (PURI).

Instancia reservada sin pago inicial (NURI).

Cuando compra **instancias reservadas**, mientras más grande sea el pago inicial, mayor será el **descuento**. Para maximizar el ahorro, puede efectuar un pago inicial completo y recibir el descuento mayor. Las instancias reservadas con pago inicial parcial ofrecen menores descuentos, pero le dan la opción de pagar menos por adelantado. Por último, puede optar por no hacer ningún pago inicial y recibir un descuento más reducido, lo que le permite contar con capital para invertir en otros proyectos.



Figura 21. Tipos de pago por instancias. Fuente: AWS, 2021.

Puede obtener **descuentos** basados en el **volumen** y ahorrar cantidades significativas a medida que incrementa su uso. Además, la transferencia de datos entrante es siempre gratuita. El uso de varios servicios de almacenamiento ofrece menores costos según sus necesidades. Como resultado, a medida que aumenten sus necesidades de uso, se beneficiará de las economías de escala que le permiten incrementar la adopción y mantener los costos bajo control.

A medida que su **organización evolucione**, también dispondrá de opciones para adquirir servicios que lo ayuden a satisfacer las necesidades del negocio. Por ejemplo, el portafolio de servicios de almacenamiento ofrece opciones que lo ayudan a reducir los precios en función de la **frecuencia** con la que accede a los datos y el rendimiento necesario para recuperarlos. Para optimizar el ahorro, puede elegir la combinación adecuada de **soluciones de almacenamiento** que lo ayuden a reducir los costos y, al mismo tiempo, mantener el rendimiento, la seguridad y la durabilidad.

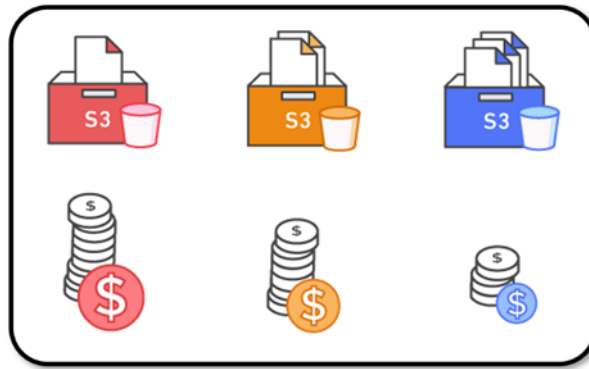


Figura 22. Pago por almacenamiento. Fuente: AWS, 2021.

Los proveedores de nube se centran constantemente en reducir los costos de hardware del centro de datos, mejorar la eficiencia operativa, disminuir el consumo de energía y reducir los costos empresariales de forma general.

Estas optimizaciones y las economías de escala importantes y en crecimiento de nube le **devuelven sus ahorros** en forma de precios reducidos.

Otro beneficio del crecimiento es que nuevos recursos de mejor rendimiento **reemplazan** a los **recursos actuales** sin que se apliquen cargos adicionales.

Las necesidades de cada cliente son diferentes. Si ninguno de los modelos de precios de nube se aplica a su proyecto, cuenta con precios personalizados disponibles para proyectos de gran volumen y con requisitos únicos.

Para ayudar a los nuevos clientes a comenzar a utilizar la nube, se ofrece una capa de uso gratuita. La **capa gratuita** se aplica a determinados servicios y opciones.

También se ofrecen una variedad de **servicios sin cargo** adicional:

Virtual Private Cloud: le permite aprovisionar una sección de la nube aislada de forma lógica donde puede lanzar recursos en una red virtual que usted defina.

Identity and Access Management (IAM): controla el acceso de los usuarios a los servicios y los recursos.

Facturación unificada.

Plataforma como servicio: es una manera aún más sencilla de administrar e implementar rápidamente aplicaciones en la nube.

Infraestructura como código: ofrece a desarrolladores y administradores de sistemas un método sencillo para crear una colección de recursos relacionados entre sí y aprovisionarlos de una manera ordenada y predecible.

En resumen, mientras que la cantidad y los tipos de servicios han aumentado de forma considerable, la filosofía de precios no ha cambiado. Al final de cada mes, paga solo por lo que **utiliza** y puede comenzar a utilizar un producto o dejar de hacerlo en cualquier momento. No se necesitan contratos a largo plazo.

La mejor forma de calcular el costo es **analizar** las características fundamentales de cada **servicio**, calcular el **uso** de ellas y, luego, **relacionar** ese uso con los precios publicados en el sitio web del proveedor. La estrategia de precios por servicio le da la flexibilidad de elegir el servicio que necesita para cada proyecto y pagar solo por lo que utiliza.

Si bien los servicios son gratuitos, los recursos que aprovisionan pueden no serlo. Además, la transferencia de datos de entrada o la transferencia de datos entre servicios dentro de la misma región no tiene costo. Sin embargo, los cargos por la transferencia de datos de salida se aplican por niveles.

Muchas empresas se preguntan cuál es la **diferencia** entre una infraestructura en las instalaciones y una infraestructura en la nube. La diferencia entre estas dos opciones se encuentra en la **forma** en que se **implementan**.

Las infraestructuras en las instalaciones se instalan **localmente** en los propios equipos y servidores de una empresa. Existen varios costos fijos, también conocidos como **gastos de capital**, que están asociados a la infraestructura tradicional. Los gastos de capital incluyen gastos de instalaciones, hardware, licencias y personal de mantenimiento. El escalado ascendente puede resultar costoso y llevar mucho tiempo. El escalado descendente no reduce los costos fijos.

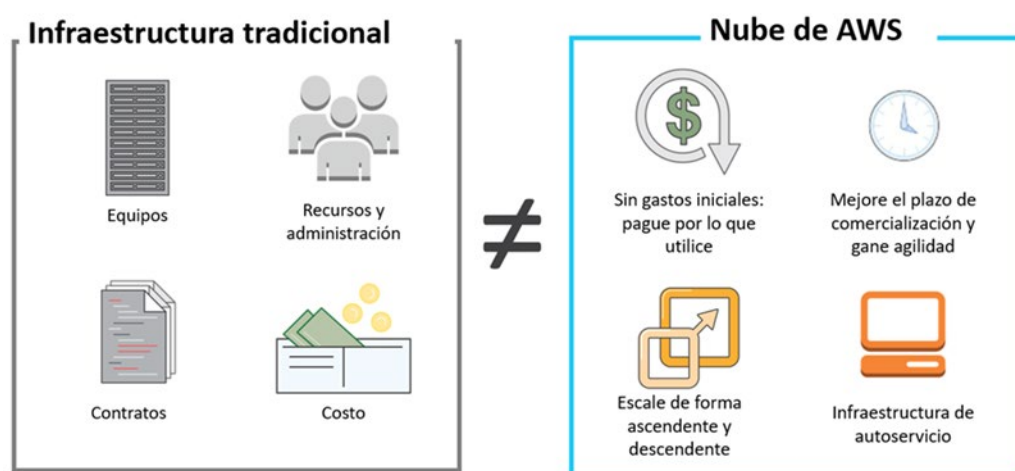


Figura 23. Coste infraestructura tradicional vs. nube. Fuente: AWS, 2021.

La **infraestructura** en la nube se obtiene de un **proveedor de servicios** que diseña y mantiene las instalaciones y el hardware y se ocupa del personal de mantenimiento. El cliente paga por lo que utiliza. El escalado ascendente o descendente es sencillo. Los costos son fáciles de calcular, ya que dependen del uso del servicio.

Es difícil comparar un modelo de suministro de TI en las instalaciones con un modelo en la nube. Los dos son diferentes porque utilizan **diferentes** conceptos y términos. El uso de **TI** en las instalaciones implica un debate que se basa en gastos de capital, ciclos de planificación prolongados y varios componentes para comprar, crear, administrar y actualizar recursos a lo largo del tiempo. El uso de la **nube** implica un debate sobre la flexibilidad, la agilidad y los costos basados en el consumo.

La respuesta está en **comparar la solución** en las instalaciones con una solución en la nube. El costo total de propiedad (o TCO) es una herramienta que puede utilizarse para realizar dicha comparación. Se trata de una estimación financiera que ayuda a los compradores y propietarios a determinar los costos directos e indirectos de un producto o sistema. El **costo total de propiedad** incluye el costo de un servicio más todos los costos relacionados con la propiedad de dicho servicio.

En el entorno de la nube, el costo total de propiedad se utiliza para comparar los **costos de la ejecución** de un entorno de infraestructura completo para una carga de trabajo específica en las instalaciones o en una ubicación con los costos de la ejecución de la misma carga en una infraestructura en la nube. Esta comparación se realiza para determinar un **presupuesto** o desarrollar un caso de negocio de **toma de decisiones** de la empresa con respecto a la solución de implementación óptima.

Algunos de los **costos** asociados con la **administración** de un centro de datos son los siguientes:

Costos de servidor para hardware y software, además de costos de las instalaciones para alojar los equipos.

Costos de almacenamiento para el hardware, la administración y las instalaciones.

Costos de red para el hardware, la administración y las instalaciones.

Costos de mano de obra de TI que se necesitan para administrar toda la solución.

Cuando se compara una solución de infraestructura en las instalaciones con una en la nube, es importante evaluar correctamente los verdaderos costos de ambas opciones.

En la **nube**, la mayoría de los **costos** son **iniciales** y pueden calcularse fácilmente.

Costos en la nube

Los proveedores de servicios en la nube ofrecen precios transparentes basados en diferentes métricas de uso como RAM, almacenamiento y ancho de banda, entre otras. Usualmente, los precios se determinan por unidad de tiempo.

Los clientes ganan confianza con respecto a los precios y, por lo tanto, son capaces de calcular fácilmente los costos en función de diversas estimaciones de uso.



Figura 24. Costes generales en T.I. Fuente: AWS, 2021.

La calculadora de costo total de propiedad de los proveedores de nube te permite **evaluar** el costo total de propiedad de una **solución**. Puede reducir el costo total de propiedad reduciendo su inversión en grandes gastos de capital (o capex) y utilizando un modelo de pago por uso para que pueda invertir en la capacidad que necesita cuando la necesite.

La **calculadora de costo total** de propiedad es una herramienta que le **permite** hacer lo siguiente:

Calcular el ahorro de costos cuando utiliza.

Revisar un conjunto de informes detallados que pueden utilizarse en presentaciones ejecutivas.

Modificar los supuestos que mejor se ajustan a sus necesidades.

Otro beneficio de la calculadora es que tiene la capacidad de **ponderar** los aspectos financieros de poseer y **operar** un centro de datos con los de usar una infraestructura en la nube. Además, la calculadora de costo total de propiedad permite **explicar** los supuestos y la metodología detrás de los cálculos.

Los **beneficios tangibles** incluyen la reducción de los gastos de informática, almacenamiento, redes y seguridad. También, incluyen la disminución de compras de hardware y software y reducciones de costos operativos, copias de seguridad, recuperación de desastres y personal operativo.

El **costo total de propiedad de la nube** determina cuál será el costo de la **tecnología** después de adquirirla (o el costo de ejecutar la solución). Por lo general, para un análisis del costo total de propiedad, se tiene en cuenta la **infraestructura** en las instalaciones tal como está y se la compara con el costo de cómo será la infraestructura en la nube. Aunque esta diferencia puede ser fácil de calcular, solo puede presentar un panorama reducido del impacto financiero total del traslado a la nube.

Se puede utilizar un **análisis de retorno de la inversión** (ROI) para determinar el valor que se genera y tener en cuenta el **gasto** y el **ahorro**. Este análisis comienza con la identificación de los beneficios tangibles en relación con las reducciones de costos directos y visibles y con las mejoras en la eficiencia.

Luego, se identifican los ahorros intangibles. Estos son valores que resultan difíciles de calcular con exactitud, pero que pueden ser más valiosos que los ahorros tangibles. Es importante que entienda el concepto tanto de los beneficios tangibles como de los intangibles para comprender el **valor total de la nube**.

Entre los beneficios intangibles se incluyen los siguientes:

Reutilización de los servicios y las aplicaciones que le permiten definir y redefinir soluciones con el mismo servicio en la nube.

Mejora de la productividad de los desarrolladores.

Mayor satisfacción del cliente.

Procesos empresariales ágiles capaces de dar una respuesta rápida a las oportunidades nuevas y emergentes.

Mayor alcance global.

Utilice la **calculadora de costo mensual** para lo siguiente:

Calcular los costos mensuales.

Identificar oportunidades para reducir los costos mensuales.

Utilizar plantillas para comparar servicios y modelos de implementación.

Utilice la **calculadora de costo total de propiedad** para lo siguiente:

Analizar informes detallados que muestran una comparación del costo total de propiedad por categorías de costos durante tres años.

Observar informes que puedan incluirse en presentaciones ejecutivas.

Modificar los supuestos en función de las necesidades empresariales.

4.6. Prácticas recomendadas para crear soluciones en la nube

Según Microsoft (2022) un Well-Architected Framework es «una guía diseñada para ayudarlo a crear la infraestructura más segura, resiliente, eficiente y de alto rendimiento posible para sus aplicaciones y cargas de trabajo en la nube».

Ofrece un enfoque uniforme para evaluar arquitecturas en la nube e instrucciones para ayudarlo a implementar diseños. Documenta una serie de preguntas básicas y prácticas recomendadas que le permiten comprender si una arquitectura determinada sigue las prácticas recomendadas de la nube.

Un Well-Architected Framework comprende cinco **pilares**: excelencia operativa, seguridad, fiabilidad, eficacia del rendimiento y optimización de costos.

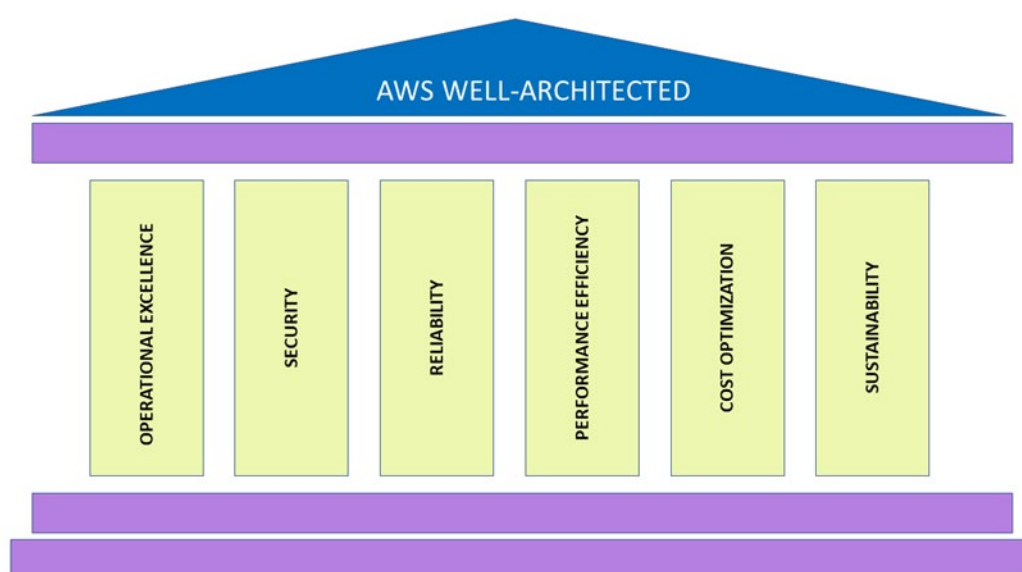


Figura 25. Well Architected Framework. Fuente: Subbarao, 2022.

El **pilar de seguridad** trata sobre la capacidad de proteger la información, los sistemas y los activos, mientras que se ofrece valor de negocio mediante evaluaciones de **riesgo y estrategias de mitigación**.

Su arquitectura presentará una presencia de seguridad mucho más sólida si implementa una gran base de identidades, habilita la trazabilidad, aplica seguridad en todas las capas, automatiza las prácticas recomendadas de seguridad y protege los datos en tránsito y en reposo.

El **pilar de excelencia operativa** aborda la capacidad de ejecutar sistemas y obtener información sobre sus operaciones para ofrecer valor de negocio. También aborda la capacidad de mejorar continuamente los procesos y procedimientos de soporte.

Cuando se diseña una carga de trabajo para operaciones, debes tener en cuenta cómo se implementará, actualizará y operará. Implementa **prácticas de ingeniería** que se alineen con las reducciones de defectos y las correcciones rápidas y seguras. Habilita la observación con registros, instrumentación y métricas técnicas y empresariales para que pueda recibir información sobre lo que está sucediendo dentro de su arquitectura.

Puedes ver toda su carga de trabajo en los **paneles de control** del proveedor de nube (aplicaciones, infraestructura, políticas, gobernanza y operaciones) como **código**. Todo se puede definir y actualizar mediante el uso de código. Esto significa que puede aplicar la misma disciplina de ingeniería que utiliza para el código de las aplicaciones a cada elemento de su pila.

En el pilar de **fiabilidad** y el de **eficacia** de rendimiento se abordan las capacidades de un sistema para recuperarse de interrupciones de infraestructura o servicio y adquirir, de manera dinámica, recursos informáticos para satisfacer la demanda. También, aborda la capacidad de un sistema para mitigar interrupciones como configuraciones erróneas o problemas transitorios de red.

Puede ser difícil garantizar la fiabilidad en un **entorno tradicional**. Los problemas surgen de puntos únicos de error, falta de automatización y falta de elasticidad. Si aplica las prácticas recomendadas descritas en el pilar de fiabilidad, puede evitar muchos de estos problemas. Esto ayudará tanto a usted y a sus clientes a tener una

arquitectura correctamente diseñada con respecto a la alta disponibilidad, la tolerancia a errores y la redundancia general.

Cuando considera el **rendimiento**, debe maximizarlo mediante el uso eficiente de los **recursos informáticos**. También debe mantener esa eficiencia a medida que cambie la demanda.

Además, es importante democratizar las **tecnologías avanzadas**. En situaciones en las que resulta difícil implementar la tecnología por su cuenta, considere la posibilidad de utilizar un **proveedor**. Cuando el proveedor implementa la tecnología por usted, se hace cargo de la complejidad y el conocimiento, lo que libera a su equipo para que se concentre en hacer un trabajo de mayor valor agregado.

La optimización de costos es un **requisito continuo** de cualquier buen diseño de arquitectura. El proceso es iterativo y debe perfeccionarse y mejorarse durante toda la vida útil de su producción.

Comprender cuán eficiente es su arquitectura actual en relación con sus objetivos puede eliminar gastos innecesarios.

Considere el uso de servicios administrados, ya que estos operan a escala de nube y pueden ofrecer un costo menor por transacción o servicio.

En el siguiente vídeo, *Well Architected Framework*, daremos un repaso a los dos marcos de aproximación a la nube que hemos comentado. Por un lado, veremos el Cloud Adoption Framework que es utilizado para iniciar el viaje a la nube, basado en seis capacidades de alto nivel dentro de los modelos de estrategia de las empresas.



Una vez que hemos decidido empezar a utilizar la nube pública, usaremos el Well-Architecting Framework para demostrar bien a los clientes o para uso interno que las arquitecturas que estamos realizando están siendo construidas con las principios y buenas prácticas adecuadas.

4.7. Infraestructura global

Según afirma Amazon Web Services (2021), el diagrama muestra las regiones actuales de cualquier proveedor de nube.



Figura 26. Infraestructura global de AWS. Fuente: AWS, 2021.

La infraestructura de la nube se basa en **regiones**. Una región de AWS es una ubicación geográfica física con una o varias zonas de **disponibilidad**. Las zonas de disponibilidad, a su vez, constan de uno o varios **centros de datos**.

Para lograr la **tolerancia a errores y estabilidad**, las regiones están **aisladas** unas de otras. Los recursos en una región no se replican automáticamente en otras regiones. Cuando almacena datos en una región específica, no se replica fuera de esa región.

Si las necesidades del negocio así lo requieren, es su responsabilidad replicar los datos en las regiones.

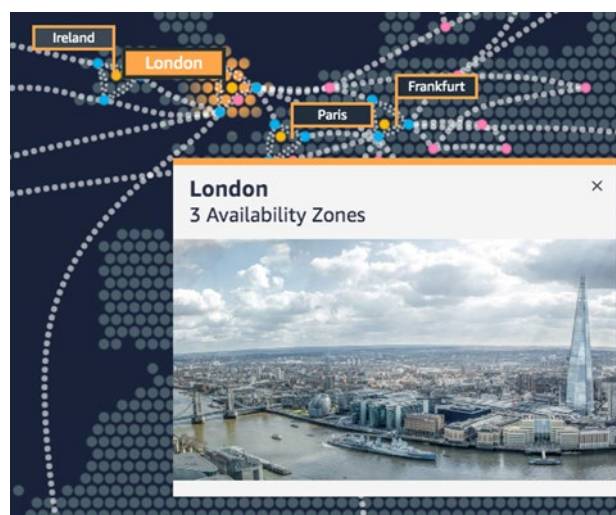


Figura 27. Zonas de disponibilidad. Fuente: AWS, 2021.

Existen algunos **factores** que debe tener en cuenta al momento de **seleccionar la región** o las regiones óptimas para almacenar datos y utilizar los servicios de nube. Una consideración fundamental es la **gobernanza de los datos** y los **requisitos legales**. Las leyes locales podrían exigir que determinada información se mantenga dentro de límites geográficos. Dichas leyes podrían restringir las regiones en las que puede ofrecer contenido o servicios. Por ejemplo, considere la Directiva de protección de datos de la Unión Europea (UE).

Si no es ese el caso, por lo general, se recomienda que ejecute las aplicaciones y almacene los datos en una región lo más cercana posible al usuario y los sistemas que obtendrán acceso a ellos. Esto lo ayudará a reducir la latencia. Por ejemplo, **CloudPing** es un sitio web que puede utilizar para probar la latencia entre su

ubicación y todas las regiones de AWS. Para obtener más información acerca de CloudPing, consulte [aquí](#).



Figura 28. Factores de elección de una región. Fuente: AWS, 2021.

Cada región tiene varias ubicaciones aisladas, conocidas como zonas de disponibilidad.

Cada zona de disponibilidad ofrece la posibilidad de operar bases de datos y aplicaciones con disponibilidad, tolerancia a errores y escalabilidad mayores que las que ofrecería un único centro de datos. Cada zona de disponibilidad puede incluir **varios centros de datos** (normalmente tres) y, a escala completa, pueden incluir cientos de miles de servidores.

Son particiones completamente **aisladas** de la infraestructura global. Las zonas de disponibilidad tienen su **propia infraestructura** de energía y están físicamente separadas por muchos kilómetros de otras zonas de disponibilidad, aunque todas las zonas de disponibilidad están a menos de cien kilómetros una de la otra.

Todas las zonas de disponibilidad están interconectadas con **redes de alto ancho de banda** y baja latencia a través de fibra dedicada completamente redundante que proporciona un alto rendimiento entre las zonas de disponibilidad. La red realiza la replicación simultánea entre zonas de disponibilidad.

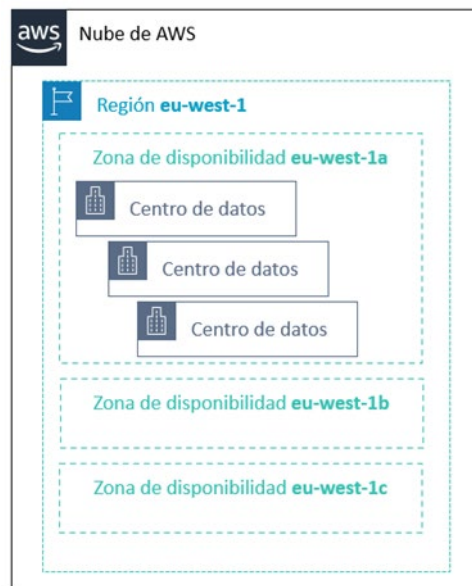


Figura 29. Región vs. zona de disponibilidad. Fuente: AWS, 2021.

Las zonas de disponibilidad ayudan a crear **aplicaciones de alta disponibilidad**. Cuando una aplicación se divide entre las zonas de disponibilidad, las empresas quedan mejor aisladas y protegidas de problemas como rayos, tornados, terremotos y más.

Usted es responsable de seleccionar las zonas de disponibilidad donde se encontrarán sus sistemas. Los sistemas pueden abarcar múltiples zonas de disponibilidad. Para lograr resiliencia, los proveedores de nube recomiendan la **replicación** entre las zonas de disponibilidad. Debe diseñar sus sistemas para que resistan fallas temporales o prolongadas de una zona de disponibilidad si ocurre un desastre.

La base de la **infraestructura de AWS** son los **centros de datos**. Los clientes no especifican un centro de datos para la implementación de recursos. Por el contrario, el nivel más detallado que el cliente puede especificar es el de las zonas de disponibilidad. Sin embargo, el centro de datos es la **ubicación** donde residen los **datos reales**. Amazon opera centros de datos de vanguardia de alta disponibilidad. Aunque es poco frecuente, pueden ocurrir fallas que afecten la disponibilidad de las

instancias que están en la misma ubicación. Si aloja todas las instancias en una misma ubicación y se produce una falla en ella, ninguna de las instancias estará disponible.

Los centros de datos están diseñados de manera segura, teniendo en cuenta varios **factores**:

Cada ubicación se evalúa con cuidado a fin de mitigar el riesgo ambiental.

Los centros de datos tienen un diseño redundante que anticipa y tolera los errores, al mismo tiempo que mantiene los niveles de servicio.

Para garantizar la disponibilidad, se realiza una copia de seguridad de los componentes del sistema críticos en varias zonas de disponibilidad.

Para garantizar la capacidad, AWS monitorea en todo momento el uso del servicio para implementar la infraestructura con el fin de respaldar los compromisos y requisitos de disponibilidad.

No se divulgan las ubicaciones de los centros de datos y está restringido todo acceso a ellas.

En caso de error, los procesos automatizados desvían el tráfico de datos de la zona afectada.

Las **cachés de borde regionales** se utilizan de forma predeterminada para distribuir contenido por todo el mundo. Se utilizan cuando posee **contenido** al que **no se accede** con la frecuencia suficiente como para que permanezca en una ubicación de borde. Las cachés de borde regionales absorben este contenido y proporcionan una alternativa a este que se debe recuperar del servidor de origen.

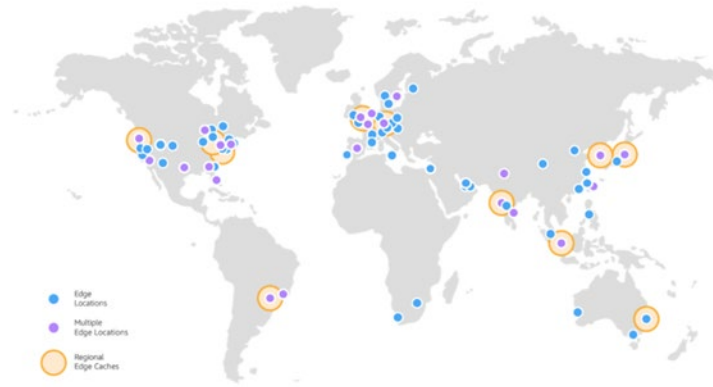


Figura 30. Cachés de borde. Fuente: AWS, 2021.

Ahora que conoce bien los componentes principales de la infraestructura global de nube, vamos a considerar los **beneficios** que aporta esta infraestructura.

La infraestructura global tiene varias características importantes:

En primer lugar, es **elástica y escalable**. Esto significa que los recursos se pueden adaptar de manera dinámica a los aumentos o reducciones de los requisitos de capacidad. La infraestructura también se puede adaptar rápidamente para admitir el crecimiento.

En segundo lugar, es **tolerante a errores**, es decir, sus componentes tienen redundancia integrada, lo que permite que la infraestructura continúe funcionando a pesar de que haya un error en un componente.

Por último, no requiere casi intervención humana y proporciona **alta disponibilidad** con tiempo de inactividad mínimo.

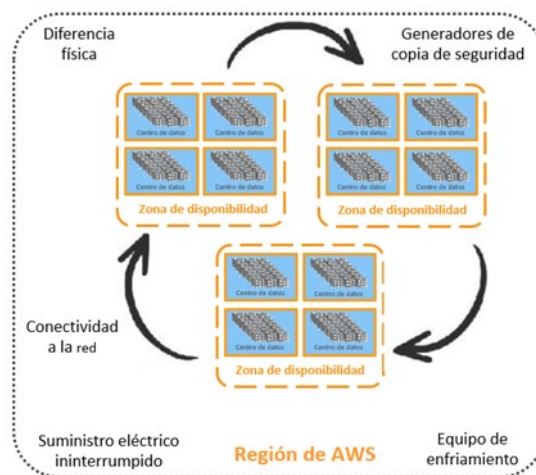


Figura 31. Partes de una región. Fuente: AWS, 2021.

4.8. Referencias bibliográficas

Amazon Web Services. (2022). *AWS Cloud Adoption Framework (AWS CAF)*.

<https://aws.amazon.com/es/professional-services/CAF/>

AWS. (2021, diciembre). *Training Track: Cloud Architecting*. Manuscrito no publicado.

Microsoft. (2022) *Marco de buena arquitectura de Microsoft Azure*.

<https://docs.microsoft.com/es-es/azure/architecture/framework/>

Subbarao, G. (2022, febrero 4). *Building a better architecture using AWS Well Architected Best Practices - Part 1*. [Publicación]. LinkedIn.

<https://www.linkedin.com/pulse/building-better-architecture-using-aws-well-best-guruprakash-subbarao/>

Tutorials Dojo. (2022). *AWS vs Azure vs GCP – Which One Should I Learn?*

<https://tutorialsdojo.com/aws-vs-azure-vs-gcp-which-one-should-i-learn/>

Microsoft Azure

Microsoft. (2022) *Documentación de Azure*. <https://docs.microsoft.com/es-es/azure/?product=popular>

Centro de recursos multimedia de Microsoft donde puedes ampliar información sobre todos los temas de esta unidad relativos a la nube de este proveedor.

Amazon Web Services

AWS. (2022). *AWS Documentation*. <https://docs.aws.amazon.com/>

Como Azure y GCP, AWS tiene un web de documentación con todo tipo de documentos entre ellos guías rápidas, *Hands on lab*, guías específicas de todos los servicios y arquitecturas de soluciones.

Google Cloud Platform

Google. (2022). *Productos de Google Cloud*. <https://cloud.google.com/products>

Desde este enlace, podemos consultar todos los servicios y productos del proveedor de nube Google.

1. La informática en la nube es:

- A. Es solo posible en un proveedor.
- B. Es una nube híbrida.
- ☐ C. La entrega bajo demanda de servicios.
- D. Ninguna de las anteriores.

La informática en la nube es la entrega bajo demanda de potencia de cómputo, bases de datos, almacenamiento, aplicaciones y otros recursos de TI a través de Internet.

¿Qué modelo de servicio no existe?

- A. IaaS.
- ☐ B. RaaS.
- C. PaaS.
- D. SaaS.

Existen tres modelos principales de servicios en la nube. Cada modelo representa una parte diferente de la pila de informática en la nube y le ofrece un nivel diferente de control sobre sus recursos de TI.

La infraestructura como servicio tiene más:

- ☐ A. Control de los recursos.
- B. Control sobre el software de TI.
- C. Menos control sobre los recursos.
- D. Igual de control sobre los recursos.

Existen tres modelos principales de servicios en la nube. Cada modelo representa una parte diferente de la pila de informática en la nube y le ofrece un nivel diferente de control sobre sus recursos de TI. IaaS tiene más control y SaaS menos control de los recursos.

¿Qué modelo en nube no existe?

- ☐ A. Externo.
- B. Híbrido.
- C. Nube.
- D. Local.

Existen tres modelos principales de implementación de informática en la nube que representan los entornos en la nube en los que se pueden implementar sus aplicaciones: híbrido, local y en nube.

Una ventaja de la informática de la nube es:

- A. Finanzas todo con el proveedor.
- B. Usas el opex.
- C. Capex y opex no se pueden utilizar en la nube.
- ☐ D. Usas el opex.

Cambiar sus gastos de capital por gastos variables: los gastos de capital (capex) son los fondos que utiliza una empresa para adquirir, actualizar y mantener activos los recursos físicos como los inmuebles, edificios industriales o equipos.

La nube te permite escalar mundialmente en:

- A. Segundos.
- ☐ B. Minutos.
- C. Horas.
- D. Días.

Adquirir escala mundial de este modo, puede implementar su aplicación en varias regiones de AWS de todo el mundo con tan solo unos clics. Como resultado, puede ofrecer una latencia menor y una mejor experiencia a sus clientes de forma sencilla y con un costo mínimo.

Señala la que no es forma de administrar recursos en la nube.

- A. CLI.
- B. Consola.
- ☐ C. Python.
- D. SDK.

Es posible que se pregunte cómo obtener acceso a la amplia gama de servicios que ofrece la nube. Existen otras tres formas de crear y administrar los recursos en la nube.

Señala una capacidad empresarial del marco de adopción de la nube.

- A. Operaciones.
- B. Plataforma.
- C. Seguridad.
- ☐ D. Gobernanza.

Es la opción correcta, ya que junto a las perspectivas de la empresa y el personal se centran en las capacidades empresariales mientras que las otras opciones se centran en las capacidades técnicas.

¿Qué es una zona de disponibilidad?

- A. Son particiones múltiples de la infraestructura global.
- ☐ B. Son particiones completamente aisladas de la infraestructura global.
- C. Son particiones conectadas de la infraestructura global.
- D. Son particiones híbridas de la infraestructura global.

Las zonas de disponibilidad tienen su propia infraestructura de energía y están físicamente separadas por muchos kilómetros de otras zonas de disponibilidad, aunque todas las zonas de disponibilidad están a menos de cien kilómetros una de la otra.

¿Qué servicio no tiene coste?

- A. Base de datos.
- B. Máquinas virtuales.
- C. Almacenamiento.
- _ D. Infraestructura como código.

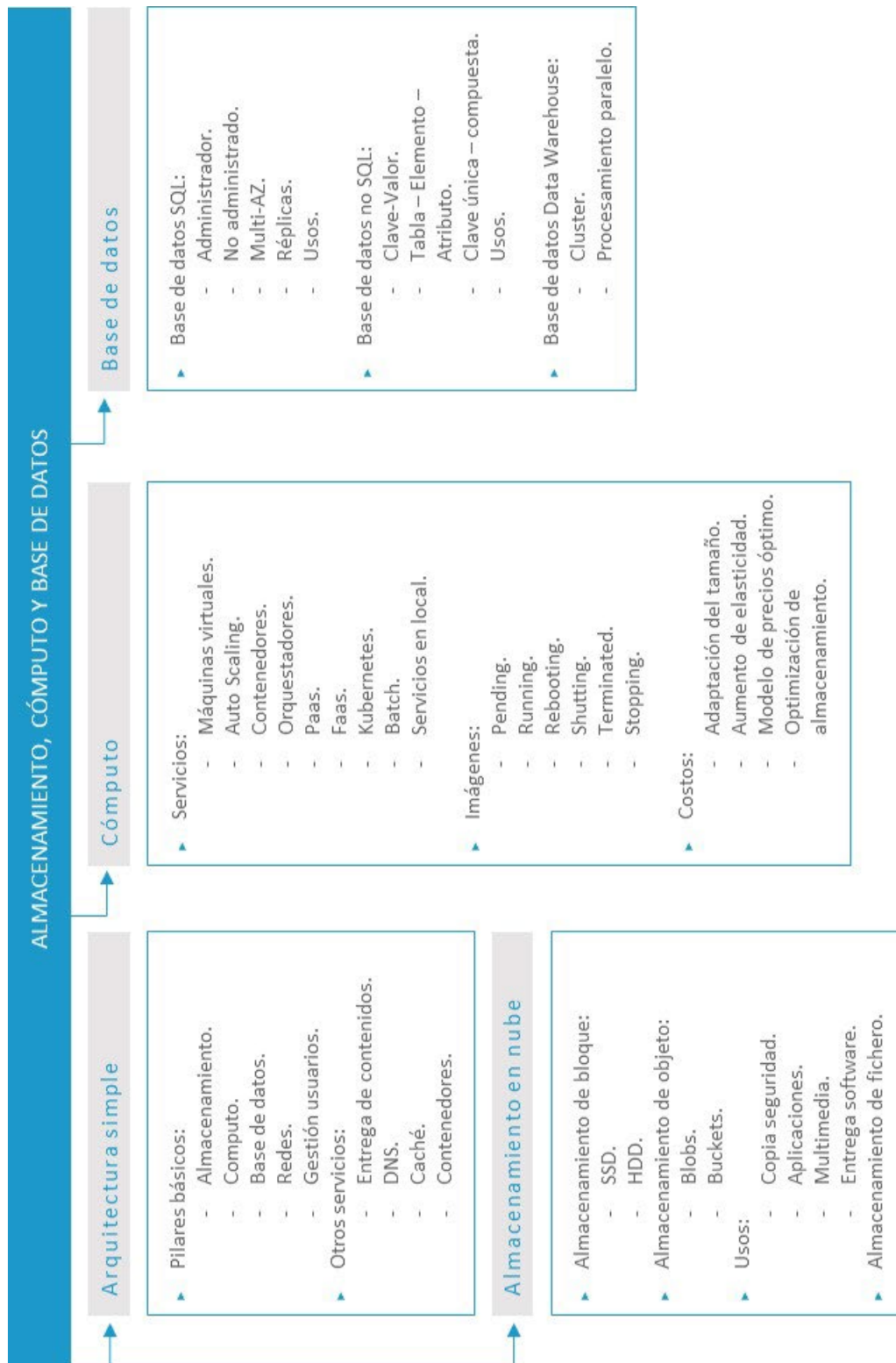
Además de este servicio, las redes proveen servicios gratuitos como son la identidad y la red.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Almacenamiento, cómputo y base de datos

Índice

Esquema	3
Ideas clave	4
5.1. Introducción y objetivos	4
5.2. Ejemplo de arquitectura simple	5
5.3. Almacenamiento de datos	6
5.4. Cómputo	15
5.5. Bases de datos SQL y no SQL	26
5.6. Referencias bibliográficas	45
A fondo	46
Test	47



5.1. Introducción y objetivos

Los proveedores de nube tienen categorizado los servicios en sus **catálogos de servicios**. Es imprescindible conocer, en primer lugar, los servicios base principales para poder, por ejemplo, leer y saber interpretar esquemas de arquitecturas de nube.

En primer lugar, veremos una **arquitectura de nube** y se verán tanto los grupos de servicios base como otros servicios.

Posteriormente, nos centraremos en los **servicios principales** de la arquitectura como son almacenamiento, cómputo y base de datos.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Interpretar y leer arquitecturas de alto nivel en nube.

Conocer los tres tipos de almacenamiento. Fichero, objeto y bloque.

Conocer los grupos de servicios de cómputo.

Desplegar máquinas virtuales y conocer el ciclo de vida de las máquinas.

Diferenciar servicio administrado y no administrado de base de datos.

Conocer las bases de datos SQL y no SQL.

Conocer las bases de datos *data warehouse*.

En los vídeos de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Aprenderemos la instalación de varias máquinas virtuales en nube pública.

Vídeo 2. Instalaremos varios tipos de base de datos relacionales, tanto en instancia única como en multi zona de disponibilidad.

5.2. Ejemplo de arquitectura simple

Como punto de partida, vamos a pensar en esta arquitectura que podemos observar en la siguiente figura y que podemos consultar en el centro de arquitectura de AWS <https://aws.amazon.com/architecture/>.

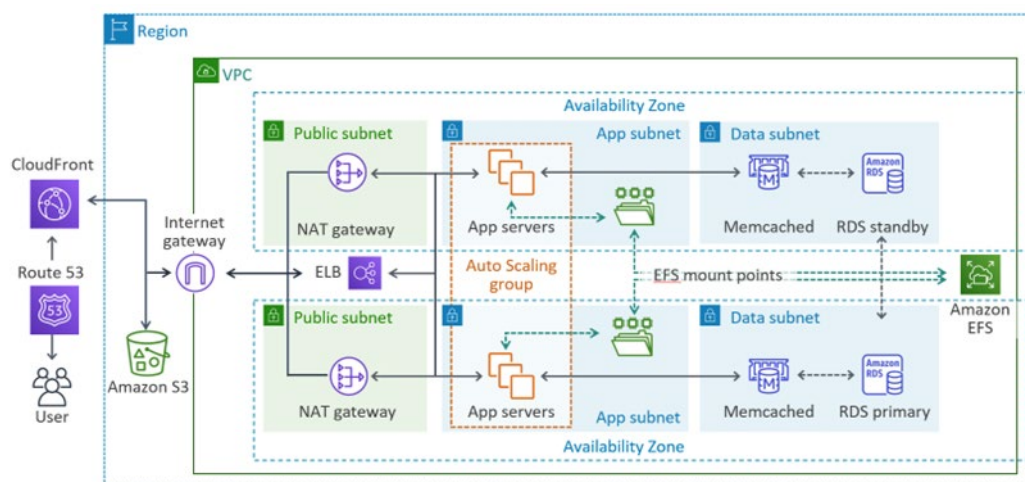


Figura 1, Arquitectura en nube. Fuente: AWS, 2021.

Esta arquitectura está basada en los **pilares fundamentales de nube** que vamos a ver en los próximos temas. Entre ellos podemos destacar:

Almacenamiento.

Cómputo.

Bases de datos.

Redes.

Gestión de usuarios.

Así mismo, en el esquema existen otros **servicios de red** que veremos posteriormente como son:

Entrega de contenidos.

DNS.

Caché.

Contenedores.

5.3. Almacenamiento de datos

Según afirma Amazon Web Services (2021), el almacenamiento es otra **categoría principal** de los servicios de nube. Algunas categorías amplias de almacenamiento incluyen el **almacén de instancias** (almacenamiento efímero), almacenamiento bloque, fichero, objeto y archivado.

Almacén de instancias: o almacenamiento efímero, es un almacenamiento temporal que se agrega a las máquinas virtuales.

Almacenamiento de bloque: es una forma de almacenamiento persistente y aplicable, que puede montarse como un dispositivo en una máquina virtual. En esta categoría, cada archivo se convierte en un objeto y está disponible a través de un localizador uniforme de recursos (URL). Además, se puede acceder a este servicio desde cualquier lugar.

Almacenamiento de fichero: es un sistema de archivos de uso compartido que varias máquinas virtuales pueden montar al mismo tiempo.

Almacenamiento de archivado: sirve para el almacenamiento en frío de datos a los que no se accede con frecuencia (por ejemplo, cuando necesita almacenamiento de datos a largo plazo por motivos de archivo o conformidad).

Almacenamiento de bloque

Ofrece **volúmenes persistentes** de almacenamiento en bloques para utilizarlos con máquinas virtuales.

Se denomina almacenamiento persistente a cualquier dispositivo de almacenamiento de datos que retenga datos una vez que se apaga la alimentación de ese dispositivo.

Algunas veces, también se lo denomina **almacenamiento no volátil**.

Todos los volúmenes se **replican** automáticamente dentro de su zona de disponibilidad para protegerlo de cualquier error que se produzca en los componentes. Está diseñado para ofrecer alta **disponibilidad** y **durabilidad**. Los volúmenes ofrecen el rendimiento uniforme y de baja latencia que se necesita para ejecutar sus cargas de trabajo.

Puede ampliar o reducir el uso en cuestión de minutos, mientras paga un precio reducido solo por lo que aprovisiona.

Permite crear volúmenes de almacenamiento **individuales** y asociarlos a unas máquinas virtuales. Ofrece almacenamiento a nivel de bloques, en el cual sus volúmenes se replican automáticamente dentro de su zona de disponibilidad. Está diseñado para proporcionar almacenamiento a nivel de **bloques duradero y desmontable** (que es como una unidad de disco duro externo) para sus máquinas virtuales. Como se encuentran asociados de forma directa a las instancias, pueden proporcionar baja latencia entre la ubicación en la que están guardados los datos y en la que podrían usarse en la instancia.

Entre los usos de los volúmenes, se incluyen los siguientes:

Volúmenes de arranque y almacenamiento para máquinas virtuales.

Almacenamiento de datos con un sistema de archivos.

Hosts de bases de datos.

Aplicaciones empresariales.

Encontrar y utilizar la tecnología adecuada para su carga de trabajo es una de las prácticas recomendadas para reducir los costos de almacenamiento. Los volúmenes con **tecnología SSD de IOPS** provisionadas pueden ofrecerle el **máximo rendimiento**.

Sin embargo, si la aplicación no requiere o no utiliza un rendimiento tan alto, el SSD de uso general suele ser suficiente. Solo se pueden utilizar SSD como volúmenes de arranque para máquinas virtuales. Las opciones de menor costo podrían ser una solución para almacenamiento adicional o casos de uso distintos a los volúmenes de arranque.

	Unidades de estado sólido (SSD)		Unidades de disco duro (HDD)	
	Uso general	IOPS provisionadas	Optimizado para rendimiento	Frío
Tamaño máximo del volumen	16 TiB	16 TiB	16 TiB	16 TiB
Cantidad máxima de IOPS por volumen	16 000	64 000	500	250
Rendimiento máximo por volumen	250 MiB/s	1000 MiB/s	500 MiB/s	250 MiB/s

Figura 2. Almacenamiento de bloque. Tipos de discos. Fuente: AWS, 2021.

Como se menciona con anterioridad, un **volumen de Amazon EBS** es un dispositivo de almacenamiento a nivel de bloques duradero que se puede asociar a una única instancia **EC2**. Puede usar volúmenes de Amazon EBS como almacenamiento principal de los datos que requieren actualizaciones frecuentes, como la unidad del sistema de una instancia o el almacenamiento de una aplicación de base de datos.

También, se pueden utilizar con las aplicaciones que requieren un **nivel alto de rendimiento** y que realicen análisis continuos de los discos. Los volúmenes de Amazon EBS persisten, independientemente de la vida de ejecución de una instancia EC2.

Los casos de uso de **EBS** varían en función del **tipo de almacenamiento** utilizado y de si se utiliza almacenamiento de uso general de las IOPS provisionadas

Unidades de estado sólido (SSD)		Unidades de disco duro (HDD)	
Uso general	IOPS provisionadas	Optimizado para rendimiento	Frío
• Este tipo se recomienda para la mayoría de las cargas de trabajo • Volúmenes de arranque del sistema • Escritorios virtuales • Aplicaciones interactivas de baja latencia • Entornos de desarrollo y pruebas	• Aplicaciones de negocio críticas que necesitan rendimiento de IOPS constante o más de 16 000 IOPS o 250 MiB/s de rendimiento por volumen • Cargas de trabajo de base de datos grande	• Cargas de trabajo de <u>streaming</u> que requieren un rendimiento uniforme y rápido a un precio bajo • Big data • Almacenes de datos • Procesamiento de registros • No puede ser un volumen de arranque	• Almacenamiento orientado al rendimiento para grandes volúmenes de datos con acceso poco frecuente • Situaciones en las que es importante el costo de almacenamiento más bajo • No puede ser un volumen de arranque

Figura 3. Características almacenamiento de bloque. Fuente: AWS, 2021.

Almacenamiento de objeto

Las empresas deben contar con la capacidad de recopilar, almacenar y analizar sus datos de manera simple y segura a gran escala. Almacenamiento de bloque es un servicio de almacenamiento de objetos creado para almacenar y recuperar cualquier cantidad de **datos** desde **cualquier ubicación**: sitios web y aplicaciones móviles, aplicaciones corporativas y datos de sensores o dispositivos de Internet de las cosas (IoT).

Es un servicio de almacenamiento a **nivel de objetos**, lo que significa que, si desea cambiar una parte de un archivo, tiene que realizar la modificación y luego volver a cargar todo el archivo modificado.

Este servicio almacena datos como objetos dentro de recursos conocidos como blobs o buckets.

Es una solución administrada de **almacenamiento en la nube** que se diseñó para brindar un escalado sin problemas y 99,999999999 % (once nueves) de durabilidad. Además de poder almacenar prácticamente todos los objetos que desee dentro de un bucket, le permite realizar operaciones de escritura, lectura y eliminación de los objetos almacenados en el bucket.

Los nombres de los buckets son **universales** y deben ser **únicos** entre todos los nombres de buckets existentes. Los objetos pueden ser de hasta 5 TB. De forma predeterminada, los datos se almacenan de forma redundante en varias instalaciones y en diferentes dispositivos de cada instalación.

Los datos que almacena no están asociados a ningún servidor en particular. Además, no necesita administrar ninguna infraestructura por su cuenta. Puede colocar tantos objetos como quiera.

Los objetos pueden ser prácticamente **cualquier archivo** de datos, como imágenes, vídeos o registros del servidor. Admite objetos de hasta varios terabytes de tamaño, le permite, incluso, almacenar instantáneas de bases de datos como objetos.

También ofrece acceso de baja latencia a los datos a través de Internet mediante el **protocolo de transferencia de hipertexto** (HTTP) o el HTTP seguro (HTTPS), para que pueda recuperar datos en cualquier momento y desde cualquier lugar. También puede acceder de forma privada a través de un punto de enlace de nube virtual privad.

Obtendrá un control detallado sobre quién puede acceder a sus datos a través de las políticas de Identity and Access Management (IAM), las políticas de bucket e, incluso, las listas de control de acceso por objeto.

De forma predeterminada, no se comparte ninguno de sus datos públicamente. También puede **cifrar** los datos en tránsito y elegir **habilitar** el cifrado del lado del servidor en sus objetos.

Puede acceder a través de la consola de administración basada en la web, de forma programática a través de la API y los SDK o con soluciones de terceros que utilizan la API o los SDK.

Tiene **notificaciones** de eventos que le permiten configurar notificaciones automáticas cuando se producen determinados eventos, como la carga o la eliminación de un objeto en un bucket específico. Se le pueden enviar estas notificaciones o pueden utilizarse para desencadenar otros procesos, como funciones de código *serverless*.

Con el análisis de la clase de almacenamiento puede evaluar los **patrones de acceso** al almacenamiento y transferir los datos correctos a la clase de almacenamiento adecuada. La característica de análisis identifica automáticamente la **política de ciclo de vida** óptima para transferir el almacenamiento al que se accede con menos frecuencia a estándar-acceso poco frecuente. Puede configurar una política de análisis de la clase de almacenamiento para monitorear un bucket completo, un prefijo o una etiqueta de objeto.

Una vez que se observa un patrón de acceso poco frecuente, se puede crear con facilidad una nueva política de ciclo de vida en función de los resultados. El análisis de clases de almacenamiento también proporciona **visualizaciones diarias** del uso del almacenamiento en la consola de administración de nube.

El almacenamiento de bloque ofrece una variedad de **clases de almacenamiento a nivel** de objetos que están diseñadas para diferentes casos de uso. Entre estas otras clases se incluyen las siguientes:

Estándar.

Una zona de disponibilidad.

Archivado.

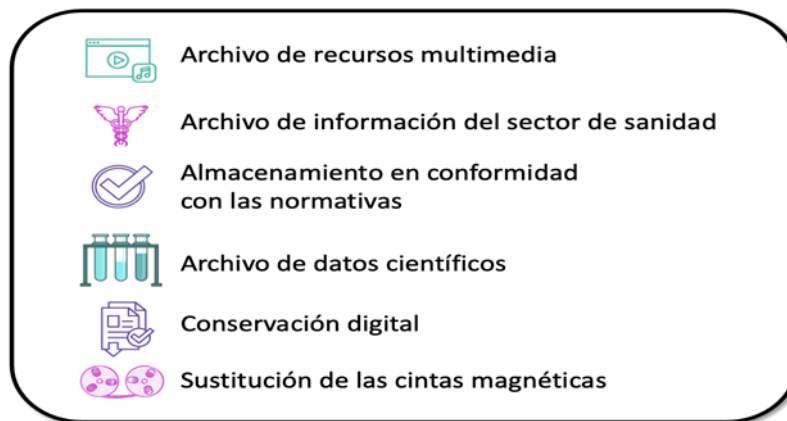


Figura 4. Uso de almacenamiento de objeto. Fuente: AWS, 2021.

Los **usos** más frecuentes son:

Copia de seguridad y almacenamiento: preste servicios de copia de seguridad y almacenamiento de datos a terceros.

Alojamiento de aplicaciones: preste servicios que implementen, instalen y administren aplicaciones web.

Alojamiento multimedia: cree una infraestructura redundante, escalable y de alta disponibilidad que aloje cargas y descargas de vídeos, fotos o música.

Entrega de software: aloje sus aplicaciones de software para que los clientes puedan descargarlas.

Los **costos específicos** varían en función de la región y de las solicitudes específicas que se realizaron. Solo paga por lo que utiliza, lo que incluye gigabytes por mes, transferencias desde otras regiones y solicitudes PUT, COPY, POST, LIST y GET.

Como regla general, solo paga por las transferencias que cruzan el límite de su región, lo que significa que no paga por las transferencias entrantes.

Almacenamiento de fichero

Implementa almacenamiento para las máquinas virtuales a las que pueden acceder desde varias de ellas de forma **simultánea**. Se ha implementado como un sistema de archivos de **uso compartido** que utiliza el protocolo de sistemas de archivos de red (NFS).

Proporciona un almacenamiento de archivos **simple, escalable y elástico** para utilizarlo con los servicios de nube y los recursos disponibles en las instalaciones. Ofrece una **interfaz sencilla** que le permite crear y configurar sistemas de archivos de forma rápida y simple.

Está diseñado para escalar de manera dinámica bajo **demanda** sin interrumpir las aplicaciones, por lo que se ampliará y reducirá de forma automática a medida que agregue o elimine archivos. Está diseñado para que sus aplicaciones dispongan del almacenamiento que necesiten, cuando lo necesiten.

Es un servicio completamente administrado que facilita la configuración y el escalado del almacenamiento de archivos en la nube. Puede utilizarlo para crear un sistema de archivos para *big data* y análisis, flujos de trabajo de procesamiento multimedia, administración de contenido, servidores web y directorios principales.

Puede crear sistemas de archivos que sean accesibles para las máquinas virtuales a través de una interfaz de sistema de archivos (mediante las API estándares de operaciones de E/S de archivos del sistema operativo). Estos sistemas de archivos admiten **semántica de acceso completo** al sistema de archivos, como la consistencia sólida y el bloqueo de archivos.

Los sistemas de archivos de archivos pueden escalar automáticamente desde gigabytes hasta petabytes de datos sin necesidad de aprovisionar almacenamiento. Este servicio, además, está diseñado para proporcionar un rendimiento uniforme a cada una de estas máquinas virtuales.

Puede acceder a su sistema de archivos desde las máquinas virtuales en su red al mismo tiempo, de manera que las aplicaciones que escalen más allá de una conexión única puedan acceder al sistema de archivos. Las máquinas virtuales que se ejecutan en varias zonas de disponibilidad dentro de la misma región cuentan con acceso al sistema de archivos, de manera que muchos usuarios pueden acceder a un origen de datos común y compartirlo.

En el diagrama, la red tiene tres **zonas de disponibilidad** y cada una tiene un **destino de montaje** creado en ella. Le recomendamos que acceda al sistema de archivos desde un destino de montaje dentro de la misma zona de disponibilidad. Una de las zonas de disponibilidad tiene dos subredes. Sin embargo, se crea un destino de montaje solo en una de las subredes.

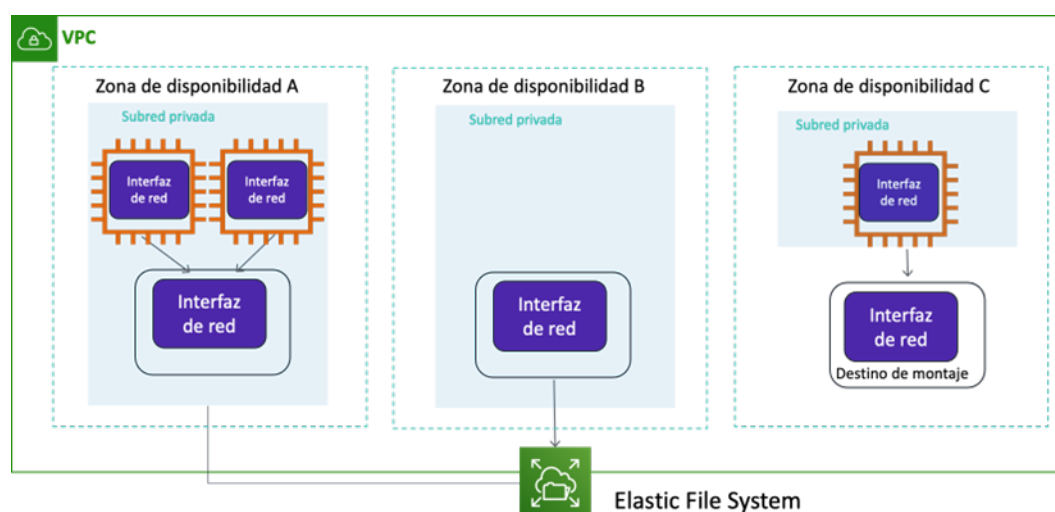


Figura 5. Almacenamiento de ficheros. Fuente: AWS, 2021.

5.4. Cómputo

Según afirma Amazon Web Services (2021), los proveedores de nube ofrecen muchos servicios informáticos. Este es un breve resumen de lo que brinda cada uno de ellos:

Máquinas virtuales cuyo tamaño se puede modificar.

Auto Scaling: respalda la disponibilidad de las aplicaciones, ya que le permite definir las condiciones que lanzarán o terminarán máquinas virtuales.

Registro de contenedores: se utiliza para almacenar y recuperar imágenes de Docker.

Orquestador de contenedores: es una herramienta o sistema que automatiza el despliegue, la gestión, el escalado, la interconexión y la disponibilidad de nuestras aplicaciones basadas en contenedores.

Plataforma como servicio: ofrece una forma sencilla de ejecutar y administrar aplicaciones web.



Figura 6. Servicios de cómputo. Fuente: AWS, 2021.

Función como servicio: es una solución informática sin servidor. Solo paga por el tiempo que utiliza los recursos informáticos.

Kubernetes: contenedores con orquestador *open source*.

Proceso por lotes: proporciona una herramienta para ejecutar trabajos por lotes a cualquier escala.

Servicios en instalación local: ofrece una forma de ejecutar determinados servicios de nube en su centro de datos en las instalaciones.

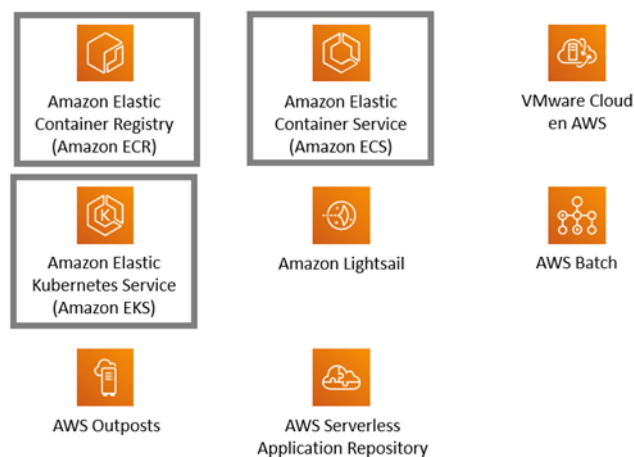


Figura 7. Fuente: AWS, 2021.

Se puede considerar que cada servicio de informática de nube pertenece a una de cuatro **categorías** amplias: máquinas virtuales (VM) que proporcionan infraestructura como servicio (IaaS), tecnología sin servidor, tecnología basada en contenedores y plataforma como servicio (PaaS).

Los **servicios de VM** proporcionan **máquinas virtuales** y se puede considerar una forma de **infraestructura** como servicio (**IaaS**). Los servicios de IaaS ofrecen flexibilidad y dejan muchas de las responsabilidades de administración del servidor en sus manos. Puede **elegir** el sistema operativo, así como el tamaño y las capacidades de los recursos de los servidores que lance. El concepto de las máquinas virtuales resulta familiar para los profesionales de la TI con experiencia en el uso de la informática en las instalaciones.

La función como código es una plataforma de informática que no requiere ningún tipo de administración.

Te permite ejecutar código sin necesidad de aprovisionar ni administrar servidores. Solo paga por el tiempo de recursos informáticos que consume. Este concepto de **tecnología sin servidor** es relativamente nuevo para muchos profesionales de la TI. Sin embargo, es cada vez más popular porque admite arquitecturas nativas en la nube, lo cual posibilita una **escalabilidad** masiva a un **costo menor** que el de la ejecución ininterrumpida de servidores para respaldar las mismas cargas de trabajo.

Los servicios basados en contenedores te permiten ejecutar varias cargas de trabajo en un solo sistema operativo. Los contenedores se implementan con mayor rapidez que las máquinas virtuales, lo cual implica capacidad de respuesta. Las soluciones basadas en contenedores son cada vez más populares.

Por último, la plataforma como servicio (**PaaS**) facilita la implementación rápida de las aplicaciones que se crean al prestar todos los servicios de aplicaciones necesarios. El proveedor administra el sistema operativo, el servidor de aplicaciones y los demás componentes de la infraestructura para que usted pueda centrarse en desarrollar el código de la aplicación.

Generalmente, la **arquitectura informática** que se utiliza se basa en el **código heredado**. Sin embargo, eso no significa que no se pueda modificar la arquitectura para aprovechar los diseños probados nativos en la nube.

Estas son algunas de las prácticas recomendadas:

Evaluar las opciones de informática disponibles.

Comprender las opciones de configuración informática disponibles.

Recopilar métricas relacionadas con el equipo.

Utilizar la elasticidad de los recursos disponibles.

Volver a evaluar las necesidades informáticas en función de las métricas.

Ejecutar servidores en las instalaciones es una tarea costosa.

Se deben adquirir sistemas de hardware que podrían basarse en los planes de proyecto en lugar del uso real de los servidores. Resulta costoso crear un centro de datos, así como mantenerlo o dotarlo de personal. Las organizaciones también deben aprovisionar, de forma permanente, la cantidad suficiente de sistemas de hardware para gestionar los picos de tráfico y las cargas de trabajo máximas. Una vez creadas las implementaciones tradicionales en las instalaciones, la capacidad del servidor podría permanecer sin uso e inactiva durante gran parte del tiempo de ejecución de los servidores, lo que implica un desperdicio.

Ejemplos de usos de instancias de Amazon EC2

- ✓ Servidor de aplicaciones
- ✓ Servidor web
- ✓ Servidor de bases de datos
- ✓ Servidor para videojuegos
- ✓ Servidor de correo
- ✓ Servidor de contenido multimedia
- ✓ Servidor de catálogos
- ✓ Servidor de archivos
- ✓ Servidor de cómputos
- ✓ Servidor proxy

Figura 8. Usos de máquinas virtuales. Fuente: AWS, 2021.

La nube proporciona máquinas virtuales en las que puede alojar el mismo tipo de aplicaciones que podría ejecutar en un servidor tradicional en las instalaciones. Ofrece capacidad de cómputo segura y de tamaño ajustable en la nube.

Características de las máquinas virtuales en nube:

Elastic se refiere a la capacidad para aumentar o reducir fácilmente la cantidad de servidores que se ejecutan para admitir una aplicación de manera automática, así como para aumentar o reducir el tamaño de los servidores existentes.

Compute hace referencia al motivo principal por el que la mayoría de los usuarios ejecutan servidores: alojar aplicaciones en ejecución o procesar datos. Estas acciones requieren recursos informáticos, incluida la capacidad de procesamiento (CPU) y la memoria (RAM).

Cloud se refiere al hecho de que las máquinas virtuales ejecutadas se alojan en la nube.

Amazon EC2 en AWS, VM en Azure y VM GCP proporciona máquinas virtuales en la nube y le ofrece control administrativo total sobre el sistema operativo Windows o Linux que se ejecute en la instancia.

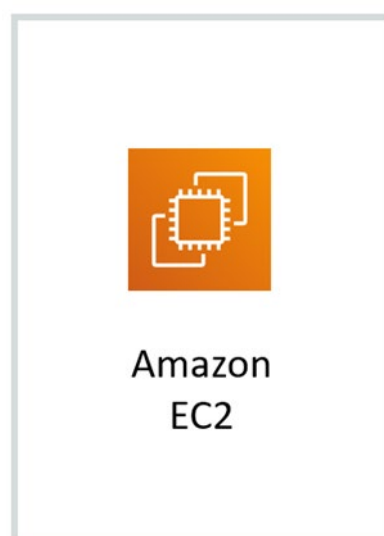


Figura 9. Servicio de EC2 en AWS. Fuente: AWS, 2021.

Un sistema operativo que se ejecuta en una máquina virtual suele denominarse **sistema operativo invitado** para distinguirlo del sistema operativo *host*. El sistema operativo *host* se instala directamente en cualquier hardware de servidor que aloje una o más máquinas virtuales.

Puede lanzar cualquier cantidad de **instancias** de todo tipo de tamaños en cualquier zona de disponibilidad del mundo y puede lograr esto en cuestión de minutos. Las instancias se lanzan a partir de **imágenes** que son plantillas de máquinas virtuales.

El **asistente de lanzamiento de instancias** facilita el lanzamiento de una máquina virtual. Por ejemplo, si decide aceptar todos los ajustes predeterminados, podrá omitir la mayoría de los pasos que ofrece el asistente y lanzar una máquina con tan solo unos clics.

Sin embargo, en la mayoría de las implementaciones, es conveniente modificar la configuración predeterminada para que los servidores que lance se implementen según sus necesidades específicas.

Una imagen proporciona la información necesaria para lanzar una máquina virtual. Debe especificar una imagen de origen al lanzar una VM.

Las imágenes incluyen los siguientes **componentes**:

Una **plantilla para el volumen raíz** de la instancia. Generalmente, un volumen raíz contiene un sistema operativo y todo lo que se instaló en él (aplicaciones, bibliotecas, etc.).

Permisos de lanzamiento que controlan qué cuentas pueden usar la imagen.

La **asignación de dispositivos de bloques** que especifica los volúmenes que deben asociarse a la instancia en su lanzamiento, si corresponde.

Elecciones que se toman con el asistente de lanzamiento de instancias:

1. **AMI**
2. **Tipo de instancia**
3. **Configuración de red**
4. **Rol de IAM**
5. **Datos de usuario**
6. **Opciones de almacenamiento**
7. **Etiquetas**
8. **Grupo de seguridad**
9. **Par de claves**

Figura 10. Fases de lanzamiento de máquinas virtuales. Fuente: AWS, 2021.

En el siguiente vídeo *Instalación de instancia en EC2-AWS* aprenderemos a instalar una máquina virtual Windows como Linux en una nube pública, en este caso del proveedor AWS. Veremos todas las opciones de instalación y nos conectaremos a las dos máquinas virtuales creadas.



Accede al vídeo

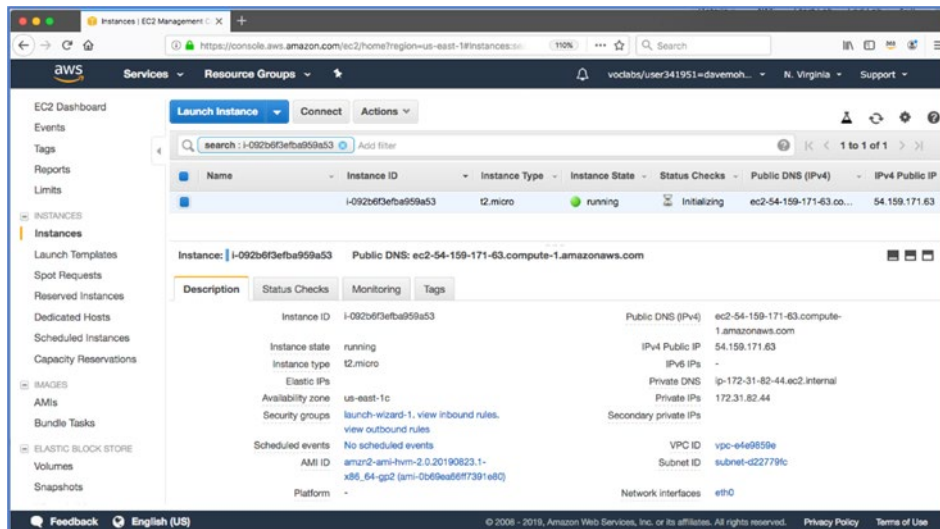


Figura 11. Consola de AWS mostrando las máquinas virtuales. Fuente: AWS, 2021.

También puede lanzar máquinas virtuales mediante programación, ya sea a través de la interfaz de línea de comandos o uno de los kits de desarrollo de software (SDK).

Este es el **ciclo de vida** de una máquina virtual. Las flechas muestran las **acciones** que puede realizar y las casillas indican el **estado** que tendrá la instancia después de dicha acción. Las instancias pueden tener uno de los siguientes estados:

Pending (pendiente): cuando se lanza una máquina por primera vez desde una imagen o cuando se activa una instancia detenida, esta pasa al estado pending cuando arranca y se implementa en un equipo de alojamiento. El tipo de instancia que especificó durante el lanzamiento determinará el hardware del equipo de alojamiento para la instancia.

Running (en ejecución): cuando ya arrancó la instancia por completo y está lista, sale del estado pending y pasa al estado running. Puede conectarse a través de Internet a la instancia en ejecución.

Rebooting (reiniciada): reiniciar las máquinas con la consola, la CLI o los SDK, en lugar de invocar una acción de reinicio desde el sistema operativo invitado. Una instancia reiniciada permanece en el mismo *host* físico, mantiene el mismo nombre DNS público y la misma dirección IP pública y, si tiene volúmenes del almacén de instancias, conserva los datos en ellos.

Shutting down (en proceso de cierre): este es un estado intermedio entre running y terminated.

Terminated (terminada): las instancias terminadas permanecen visibles en la consola durante un tiempo antes de que se elimine la máquina virtual. Sin embargo, no es posible conectarse a una instancia terminada ni recuperarla.

Stopping (en proceso de detención): las instancias que cuentan con el respaldo de almacenamiento de bloque se pueden detener. Entran en el estado stopping antes de alcanzar el estado stopped por completo.

Stopped (detenida): una instancia en el estado stopped no generará los mismos costos que una instancia en el estado running. Si se inicia una instancia en el estado stopped, esta vuelve al estado pending y se traslada a una nueva máquina de alojamiento.

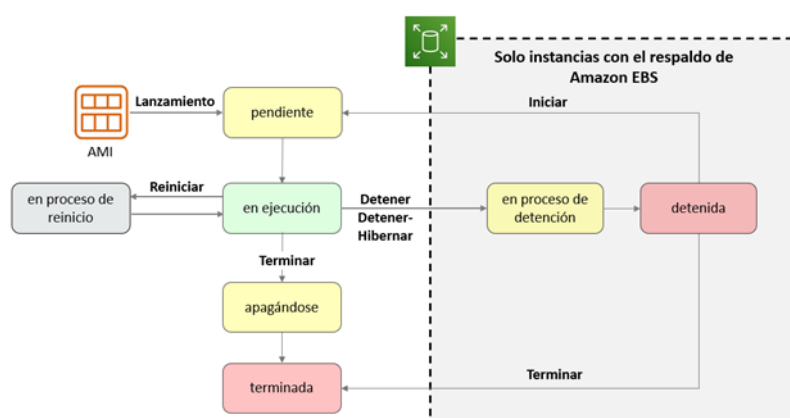


Figura 12. Ciclo de vida de una máquina virtual. Fuente: AWS, 2021.

Una **dirección IP pública** es una dirección IPv4 a la que se puede acceder desde Internet. A cada instancia que recibe una dirección IP pública se le asigna también un **nombre de host DNS externo**. Por ejemplo, si la dirección IP pública asignada a la máquina es 203.0.113.25, el nombre de host DNS externo podría ser ec2-203-0-113-25.compute-1.amazonaws.com.

Si especifica que se debe asignar una dirección IP pública a la máquina, esta se asigna desde el grupo de **direcciones IPv4 públicas de AWS**. La dirección IP pública no está asociada a su cuenta de nube. Cuando se desvincula una dirección IP pública de la VM, se vuelve a liberar en el grupo de direcciones IPv4 públicas y no podrá indicar que desea volver a utilizarla. El proveedor libera la dirección IP pública de la instancia cuando la instancia se detiene o se termina. La **instancia detenida** recibe una dirección IP pública nueva cuando se reinicia.

Si necesita una dirección IP pública permanente, le recomendamos asociar una dirección IP elástica a la instancia o máquina virtual.

Para hacerlo, primero debe asignar una nueva dirección IP elástica en la región donde se encuentra la instancia y la utilice para configurar las aplicaciones o el sistema operativo.

Puede monitorear las instancias con el servicio de monitorización, el cual recopila y procesa los datos, y los convierte en métricas legibles casi en tiempo real. Estas estadísticas se registran durante un período de quince meses, de forma que pueda acceder a la información histórica y obtener una mejor perspectiva acerca del rendimiento de su servicio o aplicación web.

De forma predeterminada, hay un **monitoreo básico**, que envía datos de métricas en intervalos de cinco minutos.

Opciones de precios

Los precios de las **máquinas bajo demanda** funcionan bien para las cargas de trabajo con **picos de demanda**, o si solo necesita probar o ejecutar una aplicación durante poco tiempo (por ejemplo, durante el desarrollo o las pruebas de aplicaciones). Cuando las cargas de trabajo son **impredecibles**, las máquinas virtuales bajo demanda son una buena opción.

Las **máquinas virtuales spot** son una buena opción si sus aplicaciones pueden tolerar interrupciones con una notificación de advertencia de dos minutos. De forma predeterminada, las máquinas virtuales se terminan, pero puede configurarlas para que se detengan o hibernen. Entre los casos de uso comunes, se incluyen aplicaciones tolerantes a errores, como servidores web, backends de API y procesamiento de *big data*. Las cargas de trabajo que almacenan datos constantemente en almacenamiento persistente también son una buena alternativa.

Las **máquinas virtuales reservadas** son una buena opción cuando tiene cargas de trabajo a largo plazo con patrones de uso predecibles, como cuando sabe que desea ejecutar los servidores de manera uniforme durante muchos meses.

Los *hosts* dedicados son una buena opción cuando dispone de licencias de software por conector, por núcleo o por máquina virtual, o cuando debe cumplir requisitos normativos y de conformidad corporativa específicos.

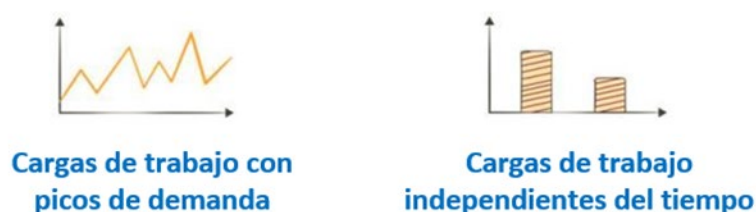


Figura 13. Cargas de trabajo. Fuente: AWS, 2021.

Para optimizar los costos, debe tener en cuenta cuatro **factores constantes** y muy importantes:

Adaptación del tamaño: elija el equilibrio adecuado de los tipos de máquinas virtuales. Tenga en cuenta los casos en que los servidores pueden desactivarse o reducirse y seguir cumpliendo con sus requisitos de rendimiento.

Aumento de elasticidad: diseñe sus implementaciones para reducir la capacidad del servidor que está inactiva con implementaciones que son elásticas, como las que utilizan el escalado automático para administrar picos de cargas.

Modelo de precios óptimo: identifique las opciones de precios disponibles. Analice sus patrones de uso para poder ejecutar máquinas virtuales con la combinación adecuada de opciones de precios.

Optimización de las opciones de almacenamiento: analice los requisitos de almacenamiento de sus implementaciones. Reduzca la sobrecarga de almacenamiento sin utilizar siempre que sea posible y elija las opciones de almacenamiento más económicas si aún pueden satisfacer sus requisitos de rendimiento de almacenamiento.



Figura 14. Optimización de costos. Fuente: AWS, 2021.

5.5. Bases de datos SQL y no SQL

Base de datos SQL

Según afirma Amazon Web Services (2021), le damos la bienvenida a una introducción a los servicios de bases de datos fundamentales que se encuentran disponibles en **Amazon Web Services (AWS)**.

Este módulo comienza con Amazon Relational Database Service (Amazon RDS).

Esta sección comienza con la revisión de las **diferencias** entre un servicio administrado y uno no administrado en relación con Amazon RDS.



Figura 15. Icono de servicio RDS en AWS. Fuente: AWS, 2021.

Las soluciones de base de datos suelen corresponder a una de dos **categorías: administradas o no administradas.**

En general, los **servicios no administrados** se aprovisionan en partes **discretas** según lo que especifica el usuario. Debe administrar el modo en que el servicio responde a los cambios en la carga, los errores y las situaciones por los que los recursos dejan de estar disponibles. Supongamos que lanza un servidor web en una máquina virtual. Dado que es una solución no administrada, dicho servidor web no escalará para tolerar el aumento de la carga de tráfico ni sustituirá las instancias en mal estado por otras en buen estado, a menos que especifique que debe utilizar una solución de escalado, como **Auto Scaling**.

El **beneficio** de usar un servicio no administrado es que se tiene un **control más estricto** sobre el modo en que la solución gestiona los cambios en la carga, los errores y las situaciones por los que los recursos dejan de estar disponibles.

Los **servicios administrados** requieren **configuración** por parte del usuario. Por ejemplo, el usuario crea un almacenamiento de objeto y, a continuación, establece los permisos para él. Sin embargo, los servicios administrados suelen requerir menos configuración.

Supongamos que tiene un sitio web estático que aloja en una solución de almacenamiento basada en la nube. El sitio web estático no tiene un servidor web. Sin embargo, el almacenamiento de objeto es una solución administrada. Este

servicio se encargará de gestionar automática e internamente ciertas características, como el escalado, la tolerancia a errores y la disponibilidad.

Ahora, consideraremos los desafíos de ejecutar una base de datos relacional independiente y no administrada. Luego, aprenderá cómo los servicios de RDS abordan estos desafíos.

Cuando ejecuta su propia base de datos relacional, es responsable de varias **tareas administrativas**, como el mantenimiento del servidor y la huella energética, el software, la instalación, la implementación de parches y las copias de seguridad de la base de datos. También se encarga de **garantizar** una alta disponibilidad, de planificar la escalabilidad y la seguridad de los datos y de instalar el sistema operativo e implementar parches. Todas estas tareas ocupan recursos que podrían dedicarse a otros elementos de su lista de asuntos pendientes y requieren experiencia en varias áreas.

Mantenimiento del servidor y huella energética.

Instalación y parches de software.

Copias de seguridad y alta disponibilidad de la base de datos.

Límites en la escalabilidad.

Seguridad de los datos.

Instalación y parches del sistema operativo.

Para enfrentar los desafíos de ejecutar una base de datos relacional independiente y no administrada, el proveedor de nube proporciona un servicio que configura, opera y escala la base de datos relacional sin necesidad de administrar constantemente. Proporciona **capacidad rentable** y de **tamaño modificable**, a la vez que automatiza las tareas administrativas que consumen mucho tiempo.

Le permite centrarse en sus aplicaciones para poder proporcionarles el **rendimiento**, la alta **disponibilidad**, la **seguridad** y la **compatibilidad** que necesitan. Con este servicio, puede enfocarse, principalmente, en los datos y en optimizar su aplicación.

¿Qué significa el término servicios administrados?

Cuando la base de datos está en las instalaciones, su administrador es el responsable de todo. Las tareas de administración de las bases de datos incluyen la **optimización** de las aplicaciones y las consultas, la configuración del hardware, la implementación de parches en el hardware, la configuración de las redes y la energía; y la administración de la calefacción, la ventilación y el aire acondicionado (HVAC).

Si se traslada a una base de datos que se ejecute en máquinas virtuales, ya no tendrá que administrar el hardware subyacente ni las operaciones del centro de datos. Sin embargo, seguirá siendo responsable de la aplicación de parches al sistema operativo y de la administración de todas las operaciones de software y copia de seguridad.

Si instala su base de datos en **servicios administrados de RDS** reduce sus responsabilidades administrativas. Si migra sus recursos a la nube, puede escalar su base de datos, habilitar la alta disponibilidad, administrar las copias de seguridad e implementar los parches de manera automática. Por lo tanto, podrá centrarse en lo que realmente importa más: **optimizar su aplicación**.

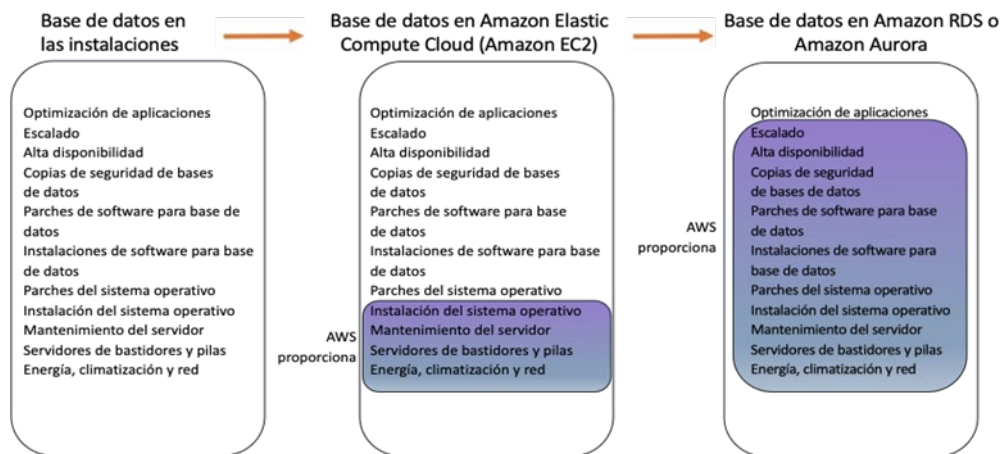


Figura 16. Diferencias entre una base de datos en AWS administrada y no administrada. Fuente: AWS, 2021.

Usted administra la optimización de su aplicación. El **proveedor** se encarga de instalar el sistema operativo y el software de base de datos, y también de aplicarles parches, así como de administrar las copias de seguridad automáticas y la alta disponibilidad.

El proveedor también escala los recursos, administra la energía y los servidores y realiza tareas de mantenimiento.

Si encarga estas operaciones al servicio administrado, se reducen su carga de trabajo operativa y los costos asociados a su base de datos relacional. Ahora, revisaremos brevemente la información general sobre el servicio y algunos casos de uso posibles.

El bloque de creación básico es la máquina virtual de base de datos.

Una máquina virtual de base de datos es un entorno de base de datos aislado que puede contener varias bases de datos creadas por el usuario.

Se puede acceder a él utilizando las mismas herramientas y aplicaciones que utiliza con una instancia de base de datos independiente. Los **recursos** que se encuentran en una instancia de base de datos se definen en función de la clase de instancia de base de datos y el tipo de almacenamiento se determina por el tipo de disco.

Las máquinas virtuales y el almacenamiento de base de datos difieren en cuanto a las características de **rendimiento** y al **precio**, lo que le permite adaptar el costo y el rendimiento a las necesidades de su base de datos.

Cuando elige crear una instancia de base de datos, primero tiene que especificar qué **motor de base** de datos ejecutar. Actualmente, Amazon RDS admite seis bases de datos: MySQL, Amazon Aurora, Microsoft SQL Server, PostgreSQL, MariaDB y Oracle.

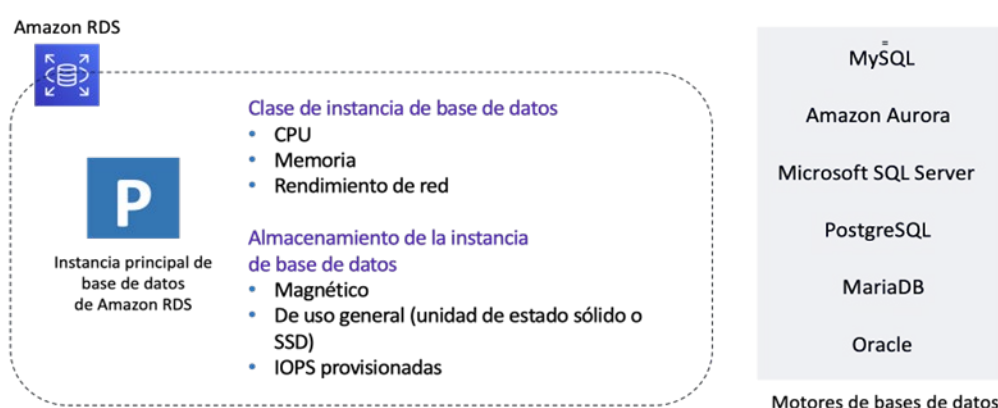


Figura 17. Servicio RDS en AWS. Fuente: AWS, 2021.

Puede seleccionar su propio intervalo de direcciones IP, así como la propia red que tengamos en la nube, crear subredes y configurar las listas de control de acceso (ACL) y el direccionamiento.

En general, la **máquina de base de datos** se encuentra **aislada** en una **subred privada** y solo pueden acceder directamente a ella las instancias de aplicación determinadas. Las subredes de una red están asociadas a una **única zona de disponibilidad**, por lo que, cuando selecciona la subred, también elige la zona de disponibilidad (o la ubicación física) de su instancia de base de datos.

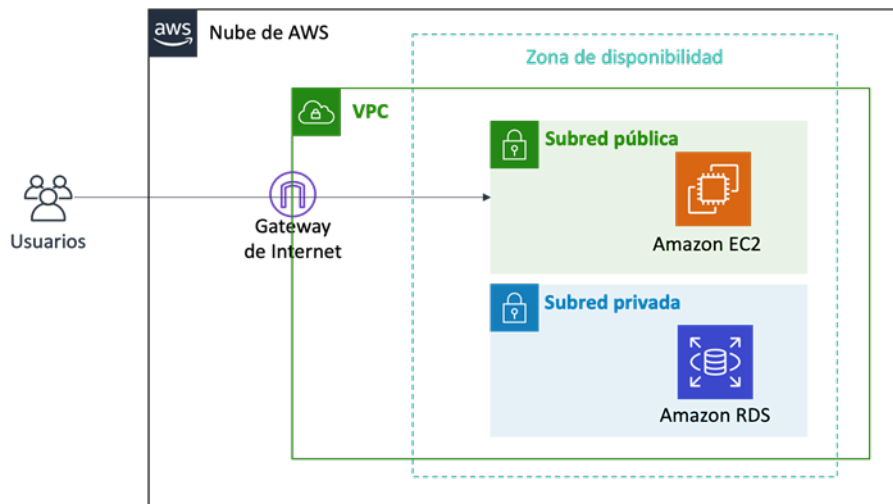


Figura 18. Arquitectura de red de dos capas con base de datos. Fuente: AWS, 2021.

Una de las características más importantes de RDS es la capacidad de **configurar** su instancia de base de datos para una **alta disponibilidad** con una implementación **Multi-AZ**. Una vez configurada la implementación Multi-AZ, RDS genera de manera automática una copia en espera de la instancia de base de datos en otra zona de disponibilidad dentro de la misma red. Después de propagar la copia de la base de datos, las transacciones se replican de forma sincrónica a la copia en espera.

La ejecución de una instancia de base de datos en una implementación Multi-AZ puede **mejorar la disponibilidad** durante el mantenimiento programado del sistema y ayudar a **proteger** sus bases de datos contra los errores de las instancias de base de datos y la interrupción de la zona de disponibilidad.

Por lo tanto, si la instancia de base de datos principal falla en una implementación Multi-AZ, RDS activa automáticamente la instancia de base de datos en espera como la nueva instancia principal. La replicación sincrónica minimiza la posibilidad de que haya pérdida de datos. Dado que sus aplicaciones hacen referencia a la base de datos por su nombre mediante el punto de enlace del sistema de nombres de dominio (DNS) de RDS, no tiene que cambiar nada en el código de las aplicaciones a fin de utilizar la copia en espera para la conmutación por error.

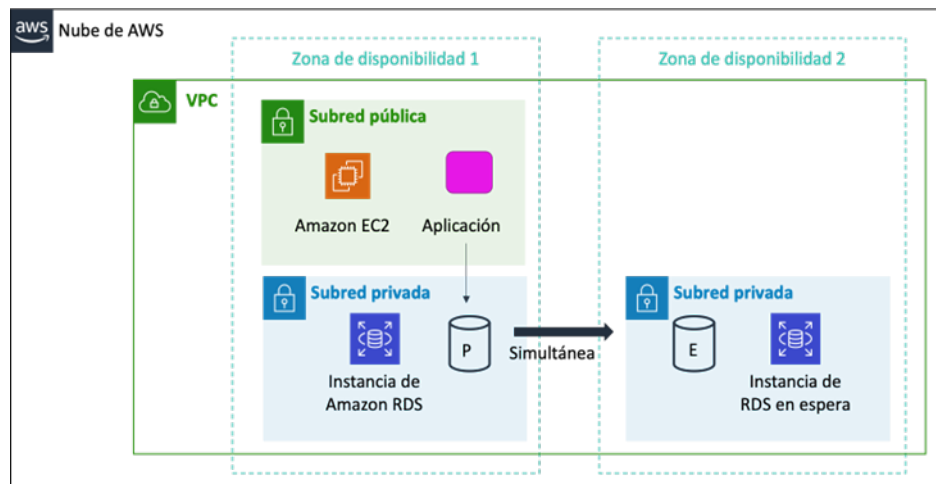


Figura 19. Arquitectura de red de dos capas con base de datos en multi-AZ. Fuente: AWS, 2021.

Si se utilizan **réplicas de lectura**, también puede realizar un escalado horizontal más allá de los límites de capacidad de una única instancia de base de datos para cargas de trabajo de base de datos con operaciones intensivas de lectura. Las réplicas de lectura también pueden convertirse en la instancia de base de datos principal, pero, debido a la replicación asíncrona, esto debe hacerse de forma manual.

Las réplicas de lectura pueden crearse en una región diferente a la utilizada por la base de datos principal.

Esta característica puede ayudar a cumplir los requisitos de recuperación de desastres o a disminuir la latencia al dirigir las lecturas a una réplica de lectura que esté más cerca del usuario.

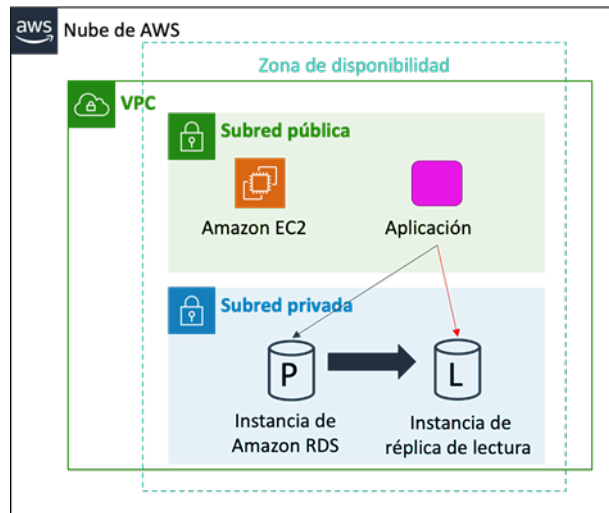


Figura 20. Arquitectura de red de dos capas con base de datos máster y replica. Fuente: AWS, 2021.

RDS es ideal para las **aplicaciones web y móviles** que necesitan una base de datos con alto rendimiento, enorme escalabilidad en el almacenamiento y alta disponibilidad. RDS ofrece a las empresas de comercio electrónico grandes y pequeñas una solución de base de datos flexible, segura y de bajo costo para la venta minorista y el comercio en línea. Los juegos móviles y en línea requieren una plataforma de base de datos con alto rendimiento y disponibilidad. RDS administra la infraestructura de la base de datos, de manera que los desarrolladores de videojuegos no tengan que preocuparse por aprovisionar, escalar o monitorear los servidores de base de datos.

Aplicaciones web y móviles	✓ Alto rendimiento ✓ Escalabilidad de almacenamiento masivo ✓ Alta disponibilidad
Aplicaciones de comercio electrónico	✓ Base de datos de bajo costo ✓ Seguridad de los datos ✓ Solución completamente administrada
Juegos móviles y online	✓ Capacidad de crecimiento rápido ✓ Escalado automático ✓ Monitoreo de bases de datos

Figura 21. Uso de bases de datos SQL. Fuente: AWS, 2021.

En los casos en que no deba utilizar RDS, considere la posibilidad de utilizar una solución de base de datos **NoSQL** o ejecutar su motor de base de datos relacional en máquinas virtuales, lo cual le proporcionará más opciones para personalizar la base de datos.

Cuando comience a calcular el costo de RDS, debe tener en cuenta las **horas de reloj del servicio**, que son recursos que generan cargos cuando se ejecutan (por ejemplo, desde el momento en que lanza una instancia de base de datos hasta que la termina).

También hay que tener en cuenta las **características** de la base de datos. La capacidad física de la base de datos que elija tendrá un efecto sobre cuánto se le cobre. Las características de la base de datos varían según el motor, el tamaño y la clase de memoria de la base de datos.

Tenga en cuenta el **tipo de compra** de base de datos. Cuando utiliza máquinas virtuales bajo demanda, paga la capacidad de cómputo por cada hora en que se ejecuta su instancia de base de datos, sin necesidad de compromisos mínimos. Con las instancias reservadas, puede realizar un pequeño pago inicial único por cada instancia de base de datos que desee reservar por un período de uno o tres años.

Considere el **almacenamiento aprovisionado**. No hay cargos adicionales por el almacenamiento para copias de seguridad de hasta el 100 % de su almacenamiento

de base de datos aprovisionado para una instancia de base de datos activa. Una vez que se termina la instancia de base de datos, el almacenamiento para copias de seguridad se factura por GB por mes.

También tenga en cuenta la cantidad de almacenamiento para copias de seguridad que se suma a la cantidad de almacenamiento aprovisionado, la cual se factura por GB por mes.

Además, considere la cantidad de **solicitudes de entrada y de salida** que se realizaron a la base de datos.

Considere el **tipo de implementación**. Puede implementar su instancia de base de datos en una única zona de disponibilidad (lo que equivale a un centro de datos independiente) o en varias zonas de disponibilidad (lo que equivale a un centro de datos secundario para disponer de mayor disponibilidad y durabilidad). Los cargos por almacenamiento y operaciones de E/S varían según la cantidad de zonas de disponibilidad en las que se realiza la implementación.

Por último, tenga en cuenta la **transferencia de datos**. La transferencia entrante de datos es gratuita y los costos por la transferencia saliente se aplican por niveles.

Según las necesidades de la aplicación, es posible optimizar los costos de las máquinas virtuales de base de datos de RDS mediante la adquisición de máquinas virtuales reservadas.

En el vídeo que tenemos a continuación, *Instalación de base de datos RDS en AWS*, aprenderemos el uso de un servicio en nube pública en el cuál podremos instalar versiones de base de datos SQL. La característica de estos servicios es que el proveedor se encarga de toda la infraestructura y nosotros nos centraremos en los datos, los modelos de entidad-relación o las restricciones. Realizaremos una instalación en una sola zona de disponibilidad y, posteriormente, realizaremos una

instalación en múltiples zonas de disponibilidad para tener tolerancia a fallos y alta disponibilidad en nuestros despliegues web en la nube pública.



Accede al vídeo

Base de datos no SQL

A continuación, se muestra una revisión de las diferencias entre estos los tipos de bases de datos SQL y no SQL. Según Microsoft (2022), podemos decir que:

Las **bases de datos relacionales (RDB)** funcionan con **datos estructurados** que se organizan en tablas, registros y columnas. En las RDB, se establece una relación bien definida entre las tablas de la base de datos. Las RDB utilizan el **lenguaje de consulta estructurado (SQL)**, la cual es una aplicación de usuario estándar que proporciona una interfaz de programación para la interacción con la base de datos. Las bases de datos relacionales pueden enfrentar dificultades para escalar horizontalmente o para trabajar con datos semiestructurados y, también, pueden requerir muchas combinaciones para los datos normalizados.

Las **bases de datos no relacionales** son aquellas que no siguen el modelo relacional proporcionado por los sistemas de administración de bases de datos relacionales (RDBMS) tradicionales. Las bases de datos no relacionales han ganado popularidad porque se diseñaron para **superar las limitaciones** de las bases de datos relacionales en la gestión de las demandas de datos estructurados variables. Las bases de datos no relacionales pueden escalar de forma horizontal y funcionar **con datos no estructurados y semiestructurados**.

A continuación, se muestra un resumen de lo que ofrece una base de datos no SQL de tipo clave-valor:

	Relacional (SQL)	No relacional												
Almacenamiento de datos	Filas y columnas	Clave-valor, documento, gráfico												
Esquemas	Fijos	Dinámicos												
Consultas	Utiliza SQL	Se centra en la recopilación de documentos												
Escalabilidad	Vertical	Horizontal												
Ejemplo	<table><thead><tr><th>ISBN</th><th>Título</th><th>Autor</th><th>Formato</th></tr></thead><tbody><tr><td>3111111223439</td><td>Withering Depths</td><td>Jackson, Mateo</td><td>Libro de tapa blanda</td></tr><tr><td>3122222223439</td><td>Wily Willy</td><td>Wang, Xiulan</td><td>Libro electrónico</td></tr></tbody></table>	ISBN	Título	Autor	Formato	3111111223439	Withering Depths	Jackson, Mateo	Libro de tapa blanda	3122222223439	Wily Willy	Wang, Xiulan	Libro electrónico	{ ISBN: 3111111223439, Título: "Withering Depths", Autor: "Jackson, Mateo", Formato: "libro de tapa blanda" }
ISBN	Título	Autor	Formato											
3111111223439	Withering Depths	Jackson, Mateo	Libro de tapa blanda											
3122222223439	Wily Willy	Wang, Xiulan	Libro electrónico											

Figura 22. Diferencias entre bases de datos SQL y no SQL. Fuente: AWS, 2021.

El servicio de base de datos NoSQL basado en bases de datos de tipo clave-valor es rápido y flexible para todas las aplicaciones que requieren una latencia uniforme de milisegundos de un solo dígito a cualquier escala.

El proveedor administra toda la infraestructura subyacente de datos para este servicio y almacena los datos de manera redundante en varias instalaciones dentro de una región nativa de EE. UU., como parte de la arquitectura tolerante a errores. Puede crear tablas y elementos. Puede agregar elementos a una tabla.

El sistema **particiona sus datos automáticamente** y cuenta con el almacenamiento de tablas necesario para cumplir con los requisitos de carga de trabajo. No existe ningún límite práctico respecto de la cantidad de elementos que se pueden almacenar en una tabla. Por ejemplo, algunos clientes tienen tablas de producción con miles de millones de elementos.

Uno de los beneficios de las bases de datos NoSQL es que los elementos de la misma tabla pueden tener **atributos diferentes**. Esto le da **flexibilidad** para agregar atributos a medida que la aplicación evoluciona. Puede almacenar elementos con formatos más nuevos junto a otros con formatos más antiguos en la misma tabla, sin tener que realizar migraciones de esquema.

A medida que su aplicación se vuelve más popular y los usuarios continúan interactuando con ella, el almacenamiento puede **crecer según las necesidades** de la aplicación. Todos los datos se almacenan en unidades de estado sólido (SSD) y su lenguaje de consulta simple permite un rendimiento de las consultas uniforme y de baja latencia.

Además de escalar el almacenamiento, le permite aprovisionar el volumen del **rendimiento de lectura o escritura** que necesita para su tabla. A medida que aumenta la cantidad de usuarios de la aplicación, las tablas se pueden escalar para admitir el incremento de las solicitudes de lectura y escritura mediante el aprovisionamiento manual.

De forma alternativa, puede habilitar el **escalado automático** para bases de datos NoSQL: monitorear la carga de la tabla e incrementar o disminuir el rendimiento aprovisionado de manera automática.

Los **componentes principales** de una base de datos no SQL de tipo clave-valor son los elementos, las tablas y los atributos.

Una **tabla** es un conjunto de datos.

Los **elementos** son un grupo de atributos que se puede identificar de forma exclusiva entre todos los demás elementos.

Un **atributo** es un elemento de datos fundamental que no es preciso seguir dividiendo.

Admite dos tipos distintos de **claves principales**:

La **clave de partición** es una clave principal simple que consta de un atributo denominado clave de ordenamiento.

La clave de partición y de ordenamiento, también conocidas como **clave principal compuesta**, está conformada por dos atributos.

A medida que aumenta el volumen de datos, la clave principal particiona e indexa los datos de la tabla.

Puede recuperar los datos de una tabla de dos formas distintas:

Según el primer método, la operación de consultas aprovecha la partición para localizar de manera eficaz los elementos por medio de la clave principal.

El segundo método se lleva a cabo mediante un escaneo que le permite localizar los elementos de la tabla a partir de las coincidencias con las condiciones en los atributos que no son clave. El segundo método le da la flexibilidad necesaria para localizar elementos por medio de otros atributos.

Para aprovechar al máximo las operaciones de consulta es importante que la clave utilizada identifique de forma exclusiva los elementos de la tabla. Puede configurar una clave principal simple basada en un único atributo de los valores de los datos con una distribución uniforme, como el **Identificador Único Global (GUID)** u otros identificadores aleatorios.

Por ejemplo, si quisiera modelar una tabla con productos, podría utilizar algunos **atributos**, como el ID de producto. De forma alternativa, puede especificar una clave compuesta, que incluye una clave de partición y una clave secundaria. En este ejemplo, si tuviera una tabla con libros, podría utilizar la combinación de autor y título para identificar de forma exclusiva los elementos de la tabla. Este método podría ser útil si espera evaluar libros por autor con frecuencia, ya que entonces podría utilizar la consulta.

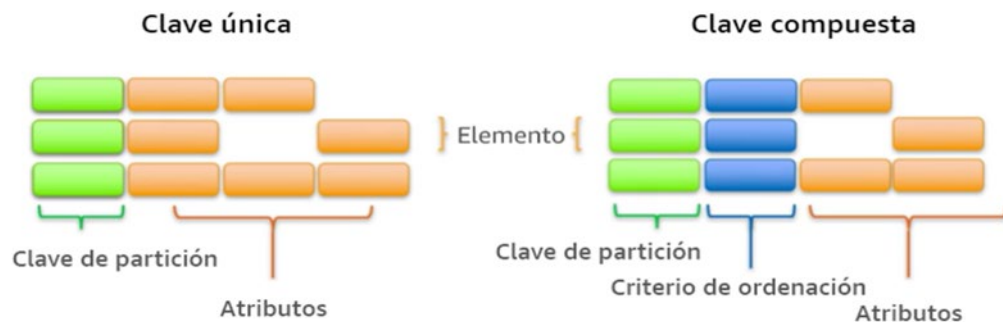


Figura 23. Clave única vs clave compuesta. Fuente: AWS, 2021.

Aprendizajes

La base de datos se ejecuta exclusivamente en **SSD** y admite modelos de almacenamiento de **clave-valor** y **documentos**.

Funciona bien con las aplicaciones móviles, web, de videojuegos, de tecnología publicitaria y de Internet de las cosas (IoT). Se puede acceder a este servicio a través de la consola, de la CLI y de las llamadas a la API.

La capacidad de escalar sus tablas en términos de almacenamiento y rendimiento de aprovisionamiento hace que sea una buena opción para los **datos estructurados** de las aplicaciones web, móviles y de IoT. Por ejemplo, puede tener una gran cantidad de clientes que generan datos de manera continua y realizan numerosas solicitudes por segundo. En este caso, el escalado del rendimiento proporciona un rendimiento uniforme a sus clientes.

También se utiliza en **aplicaciones** que se ven **afectadas por la latencia**. El rendimiento predecible de las consultas, incluso en tablas grandes, lo hace útil para los casos en los que la latencia variable podría causar un impacto significativo en la experiencia del usuario o en los objetivos empresariales, como en el ámbito de la tecnología publicitaria o los videojuegos.

Con la característica de las tablas globales, se reduce la tarea de replicar los datos entre las regiones y de solucionar los conflictos de actualización. Con esta característica, se replican automáticamente las tablas en las regiones que elija. Las tablas globales pueden ayudar a que las aplicaciones mantengan la disponibilidad y el rendimiento necesarios para la continuidad del negocio.

Base de datos Data Warehouse

Data Warehouse es un servicio de almacenamiento de datos **rápido** y completamente **administrado** que permite analizar todos los datos de forma sencilla y rentable mediante el uso de **SQL estándar** y de las herramientas de inteligencia empresarial existentes.

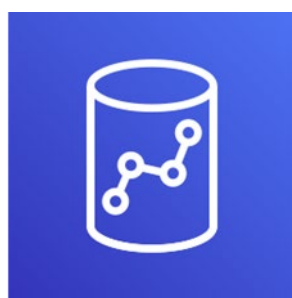


Figura 24. Icono servicio Redshift, un data warehouse en AWS. Fuente: AWS, 2021.

En la actualidad, la analítica es importante para las empresas, pero construir un almacenamiento de datos es complejo y costoso. El almacenamiento de datos puede demandar meses y una considerable cantidad de recursos financieros para configurarse.

Es una forma de **almacenamiento de datos rápida, potente y completamente administrada** que se puede configurar, utilizar y escalar de manera simple y rentable. Permite ejecutar consultas analíticas complejas en petabytes de datos estructurados utilizando una sofisticada optimización de consultas, un almacenamiento en columnas en los discos locales de alto rendimiento y una ejecución masiva de consultas en paralelo. La mayoría de los resultados se obtienen en segundos.

A continuación, revisaremos un poco más en detalle las **características principales de un data warehouse** y algunos casos de uso comunes.

El **nodo principal** administra las comunicaciones con los programas cliente y todas las comunicaciones con los nodos informáticos. Analiza y desarrolla los planes de ejecución para llevar a cabo las operaciones de bases de datos, en específico, la serie de pasos que se necesitan para obtener resultados de consultas complejas. Además, compila código para los elementos individuales del plan de ejecución y lo asigna a los nodos informáticos individuales. Los **nodos informáticos** ejecutan el código compilado y devuelven resultados intermedios al nodo principal para su agregación final.

Al igual que con otros servicios, solo paga por lo que usa.

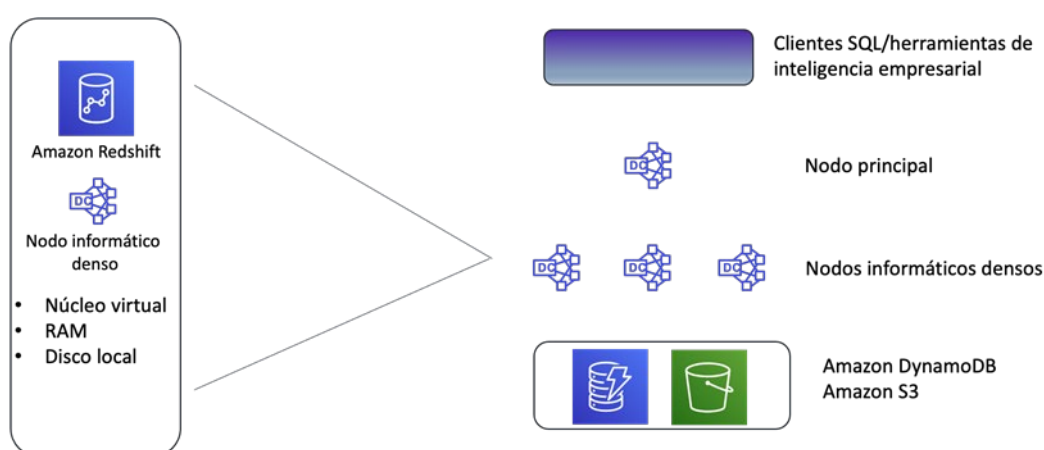


Figura 25. Arquitectura procesamiento paralelo. Fuente: AWS, 2021.

Resulta sencillo **automatizar** la mayoría de las **tareas administrativas** comunes para administrar, monitorear y escalar su clúster, lo que le permite enfocarse en sus datos y su negocio.

La **escalabilidad** es inherente. El clúster puede escalarse de manera ascendente y descendente a medida que cambian sus necesidades, con apenas unos pocos clics en la consola.

La seguridad es la máxima prioridad.

La seguridad está integrada y está diseñada para proporcionar un cifrado sólido de sus datos, tanto en reposo como en tránsito.

Los clientes de *big data* tienen una característica en común: las enormes cantidades de datos que llevan los sistemas hasta el límite. Es posible que los clientes más pequeños no cuenten con los recursos suficientes para adquirir el hardware y la experiencia necesarios para ejecutar estos sistemas. Los clientes más pequeños pueden configurar y utilizar de manera rápida un espacio de almacenamiento de datos a un punto de precio bajo en comparación.

Como servicio administrado, se ocupa de muchas de las tareas de implementación y mantenimiento continuo que suelen requerir los servicios de un administrador de base de datos. Esto permite a los clientes centrarse en consultar y analizar sus datos.

Los clientes de software como servicio (SaaS) pueden aprovechar las características escalables y fáciles de administrar que ofrece. Algunos clientes utilizan el data warehouse para brindar capacidades analíticas a sus aplicaciones. Otros usuarios implementan un clúster por cliente y utilizan el etiquetado para simplificar y administrar la facturación y los acuerdos de nivel de servicio (SLA).

En resumen, es un servicio de almacenamiento de datos **rápido y completamente administrado**. A medida que el negocio crece, puede escalar de manera sencilla y sin tiempo de inactividad, ya que solo necesita agregar más nodos.

Este servicio está diseñado para ofrecer **alto rendimiento de manera uniforme**. Utiliza almacenamiento en columnas y una arquitectura de procesamiento en paralelo masivo. Estas características ubican en paralelo y distribuyen los datos y las consultas entre varios nodos. También el proveedor monitorea su clúster y realiza copias de seguridad de los datos de forma automática para que pueda restaurarlos con facilidad si es necesario. El cifrado está integrado, solo tiene que habilitarlo.

5.6. Referencias bibliográficas

Amazon Web Services (2022). *Centro de arquitectura de AWS*.
<https://aws.amazon.com/architecture/>

AWS. (2021, diciembre). *Training Track: Cloud Architecting*. Manuscrito no publicado.

Microsoft (2022). *Azure Cosmos DB*. <https://azure.microsoft.com/en-us/services/cosmos-db/>

Microsoft Azure

Microsoft. (2022) *Documentación de Azure*. <https://docs.microsoft.com/es-es/azure/?product=popular>

Centro de recursos multimedia de Microsoft donde puedes ampliar información sobre todos los temas de esta unidad relativos a la nube de este proveedor.

Amazon Web Services

Amazon Web Services. (2022). *AWS Documentation*. <https://docs.aws.amazon.com/>

Como Azure y GCP, AWS tiene un web de documentación con todo tipo de documentos entre ellos: guías rápidas, *hands on lab*, guías específicas de todos los servicios y arquitecturas de soluciones.

Google Cloud Platform

Google Cloud. (s.f). *Productos de Google Cloud*. <https://cloud.google.com/products>

Podemos consultar todos los servicios y productos del proveedor de nube Google desde este enlace.

¿Cuál no es un servicio base en la nube?

- A. Gestión de usuarios.
- B. Cómputo.
- ☐ C. Entrega de contenidos.
- D. Almacenamiento.

Esta arquitectura está basada en los pilares fundamentales de nube que vamos a ver en los próximos temas. Entre ellos podemos destacar: almacenamiento, cómputo, bases de datos, redes y gestión de usuarios.

¿Qué almacenamiento es una forma de almacenamiento persistente?

- A. Almacenamiento de objeto.
- ☐ B. Almacenamiento de bloque.
- C. Almacenamiento de fichero.
- D. Almacenamiento de archivado.

Almacenamiento de bloque es una forma de almacenamiento persistente y aplicable que puede montarse como un dispositivo en una máquina virtual.

¿Cuál no es uso de un volumen?

- ☐ A. Datalakes.
- B. *Host* de base de datos
- C. Sistemas de archivos.
- D. Aplicaciones empresariales.

Entre los usos de los volúmenes, se incluyen los siguientes: volúmenes de arranque y almacenamiento para máquinas virtuales, almacenamiento de datos con un sistema de archivos, *hosts* de bases de datos y aplicaciones empresariales.

La durabilidad de un almacenamiento de objeto es:

- ☐ A. Once nueves.
- B. Cinco nueves.
- C. Siete nueves.
- D. Ninguna de las anteriores.

Es una solución administrada de almacenamiento en la nube que se diseñó para brindar un escalado sin problemas y 99,999999999 % de durabilidad.

Una ventaja del almacenamiento de fichero es:

- A. Es solo para sistemas Windows.
- B. Es multi región.
- C. Sirve para arrancar máquinas virtuales.
- ☐ D. Sistema de fichero compartido entre varias máquinas virtuales.

Implementa almacenamiento para las máquinas virtuales a las que pueden acceder varias máquinas virtuales de forma simultánea. Se ha implementado como un sistema de archivos de uso compartido que utiliza el protocolo de sistemas de archivos de red.

¿Cuál no es servicio de cómputo en las nubes?

- A. Data Warehouse.
- ☐ B. Auto scaling.
- C. IoT.
- D. Ficheros.

Los proveedores de nube ofrecen muchos servicios informáticos. Este es un breve resumen de lo que brinda cada uno de ellos: máquinas virtuales cuyo tamaño se puede modificar, Auto Scaling (respalda la disponibilidad de las aplicaciones, ya que le permite definir las condiciones que lanzarán o terminarán máquinas virtuales), registro de contenedores (se utiliza para almacenar y recuperar imágenes de Docker), orquestador de contenedores (servicio de organización) y plataforma como servicio que ofrece una forma sencilla de ejecutar y administrar las aplicaciones web.

Una imagen proporciona información necesaria:

- A. Para base de datos.
- B. De los ficheros que contiene.
- ☐ C. Para lanzar una máquina virtual.
- D. De todos los servicios de cómputo.

Una imagen proporciona la información necesaria para lanzar una máquina virtual.

Debe especificar una imagen de origen al lanzar una VM.

¿Cuál no es un estado en el ciclo de vida de una máquina virtual?

- A. Pending.
- B. Running.
- C. Stopping.
- ☐ D. Standby.

Este es el ciclo de vida de una máquina virtual. Las flechas muestran las acciones que puede realizar y las casillas indican el estado que tendrá la instancia después de dicha acción.

Los servicios no administrados:

- A. Se aprovisionan automáticamente.
- ☐ B. Se aprovisionan en partes discretas.
- C. Se aprovisionan solo para base de datos.
- D. No existen en proveedores de nube actuales.

En general, los servicios no administrados se aprovisionan en partes discretas, según lo que especifica el usuario.

El servicio de base de datos NoSQL basado en bases de datos de tipo clave-valor es:

A. Rápido para todas las aplicaciones que requieren una latencia grande de un solo dígito a cualquier escala.

B. Flexible para todas las aplicaciones.

C. Rápido, pero poco fiable.

_ D. Rápido y flexible para todas las aplicaciones que requieren una latencia uniforme de milisegundos de un solo dígito a cualquier escala.

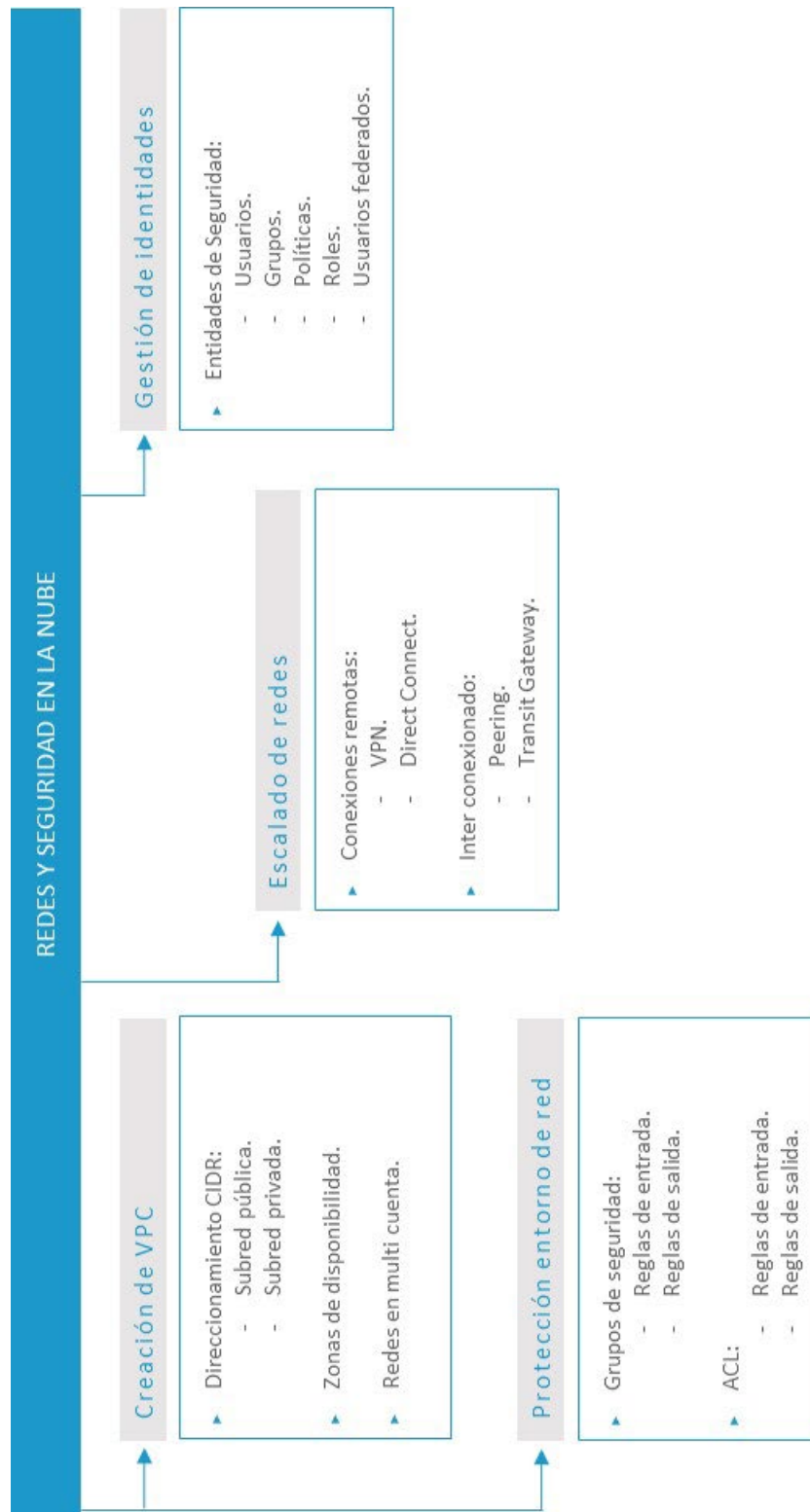
El servicio de base de datos NoSQL basado en bases de datos de tipo clave-valor es rápido y flexible para todas las aplicaciones que requieren una latencia uniforme de milisegundos de un solo dígito a cualquier escala.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Redes y seguridad en la nube

Índice

Esquema	3
Ideas clave	4
6.1. Introducción y objetivos	4
6.2. Creación y conexionado de un entorno de red	5
6.3. Protección del entorno de red	14
6.4. Conexión a la red remota	18
6.5. Escalado de redes	22
6.6. Gestión de identidad y seguridad en la nube	25
6.7. Referencias bibliográficas	33
A fondo	34
Test	35



6.1. Introducción y objetivos

En este tema vamos a abordar dos de los pilares fundamentales de un **proveedor de nube pública**.

En primer lugar, vamos a ver como definimos la **arquitectura de red en la nube** para hacer funcionar todos nuestros servicios a nivel de red. Veremos las diferencias entre las redes en nuestros *data center* en local y la nube pública. Estudiaremos desde la definición del direccionamiento y subredes, así como las configuraciones de seguridad. Por último, veremos la posibilidad de realizar **conexiones VPN** con el proveedor de nube desde nuestras empresas.

El segundo gran apartado de este tema es la **seguridad**. Si bien el apartado de seguridad es muy amplio, nos centraremos en el apartado de gestión de identidades y gestión multi cuenta en los proveedores de nube. No abordaremos en este tema servicios de seguridad específicos o servicios de ciberseguridad.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

- ▶ Explicar el rol básico de una nube virtual privada (VPC)
- ▶ Identificar cómo conectar su entorno de red en nube pública a Internet
- ▶ Describir cómo aislar los recursos dentro de su entorno de red pública.
- ▶ Crear una red con subredes, una gateway de Internet, tablas de enrutamiento y un grupo de seguridad
- ▶ Describir cómo conectar una red en las instalaciones a la red de la nube.
- ▶ Describir cómo conectar las VPC en la nube.
- ▶ Describir cómo escalar las VPC en la nube.
- ▶ Explicar el propósito de los usuarios, los grupos y los roles.

- ▶ Describir cómo se permite la federación de usuarios dentro de una arquitectura.
- ▶ Describir cómo administrar varias cuentas.

En los vídeos de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Gestión de redes en nube pública. Crearemos redes con subredes públicas y privadas y desplegaremos servicios en las nuevas redes.

Vídeo 2. Veremos la gestión de entidades de seguridad y la creación de políticas.

6.2. Creación y conexionado de un entorno de red

Virtual Private Cloud

Según afirma Amazon Web Services (2021), vamos a aprender a **diseñar una red en nube** y crear una **red con subredes**. Para ello, partiremos de un esquema general como el siguiente y veremos las diferentes partes del diseño a lo largo de la unidad.

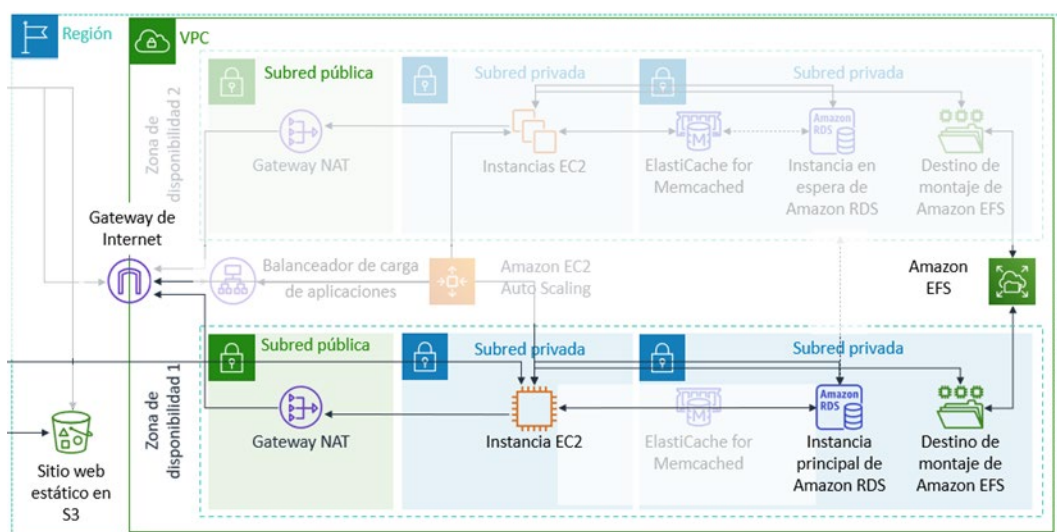


Figura 1. Arquitectura en nube. Fuente: AWS, 2021.

Un **VPC** (Virtual Private Cloud) o **red en nube pública** es un servicio que le permite aprovisionar una sección de la nube aislada lógicamente (denominada nube virtual privada o VPC) donde puede lanzar sus recursos de nube.

Te permite **controlar sus recursos** de red virtuales. Por ejemplo, se puede seleccionar su propio intervalo de direcciones IP, crear subredes y configurar tablas de enrutamiento y gateways de red. Puede usar IPv4 e IPv6 en su VPC para acceder de forma segura a los recursos y las aplicaciones.

También se puede **personalizar la configuración de red** de la VPC. Por ejemplo, se puede crear una subred pública para los servidores web que tengan acceso a Internet público. Puede colocar los **sistemas de backend** (como los servidores de aplicaciones o bases de datos) en una subred privada sin acceso a Internet público. Por último, se puede utilizar varias **capas de seguridad** para ayudar a controlar el acceso a las máquinas virtuales en cada subred. Estas capas de seguridad incluyen grupos de seguridad y listas de control de acceso a la red (ACL de red).

Cada VPC pertenece a una sola región del proveedor de nube. Una VPC abarca todas las zonas de disponibilidad de una región, de modo que puede alojar recursos compatibles desde cualquier zona de disponibilidad dentro de su región.



Figura 2. Las VPC pueden alojar recursos compatibles desde cualquier zona de disponibilidad dentro de su región. Fuente: AWS, 2021.

Cuando se crea una VPC, proporciona el **conjunto de direcciones IP privadas** que desea que utilicen las instancias de la VPC. Este conjunto de direcciones se especifica como un bloque de direccionamiento entre dominios sin clases (CIDR), por ejemplo, 10.0.0.0/16. Se trata del bloque de CIDR principal de la VPC. Puede asignar tamaños de bloque de entre /28 (16 direcciones IP) y /16 (65 536 direcciones IP).

VPC admite la utilización de **direccionamiento IPv4 e IPv6** y tiene distintos límites de tamaño de bloque de CIDR para cada uno. De forma predeterminada, todas las VPC y las subredes deben tener bloques de CIDR IPv4. Este comportamiento no se puede modificar. De forma opcional, puede asociar un bloque de CIDR IPv6 a su VPC.

El VPC puede funcionar en modo de pila doble, lo que implica que los recursos se pueden comunicar mediante IPv4, IPv6 o ambos.

Las direcciones IPv4 e IPv6 son independientes entre sí. Por lo tanto, debe configurar el direccionamiento y la seguridad de su VPC de forma individual para IPv4 e IPv6.

Se puede dividir una VPC en una **subred** o más. Una subred es un segmento o una partición del intervalo de direcciones IP de una VPC, donde se puede asignar un grupo

de recursos. Es importante recordar que las subredes no son límites de aislamiento en torno a su aplicación. En cambio, son **contenedores para políticas de direccionamiento**, sobre las que aprenderá en la siguiente sección de este módulo.

Cuando se crea una subred, se debe especificar el **bloque de CIDR** que le corresponde, el cual es un subconjunto del bloque de CIDR de la VPC. Los bloques de CIDR de las subredes no pueden superponerse.

Aunque cada subred debe estar dentro de una zona de disponibilidad por completo y no puede abarcar zonas, cada zona de disponibilidad puede tener una o más subredes. Puede elegir agregar subredes en una zona local. Cuando crea una subred en una zona local, la VPC también se extiende a esa zona local.

Como las subredes de la VPC se asignan a zonas de disponibilidad específicas, la ubicación de las subredes implica una forma de garantizar que los servicios se distribuyan de forma adecuada entre distintas ubicaciones.



Figura 3. Ejemplo de una VPC con CIDR /22 que incluye un total de 1024 direcciones IP. Fuente: AWS, 2021.

Cuando configure cualquier red de equipos, tenga en cuenta los siguientes **principios universales** para el **diseño de redes**:

- ▶ Cree una subred por zona de disponibilidad utilizable por cada grupo de alojamientos que tenga requisitos de direccionamiento exclusivos.
- ▶ Divida el intervalo de red de la VPC de manera equitativa entre todas las zonas de disponibilidad utilizables en una región.
- ▶ No asigne todas las direcciones de red a la vez. En cambio, asegúrese de reservar un espacio de direcciones para su uso en el futuro.
- ▶ Ajuste el tamaño del CIDR de su VPC y las subredes de modo que admitan un crecimiento significativo de las cargas de trabajo esperadas.
- ▶ Asegúrese de que el intervalo de red de VPC (bloque de CIDR) no se superponga con los demás intervalos de red privada de su organización.

Cuando diseña y crea su entorno de red, hay un número limitado de **casos de uso** en los que el entorno **de una sola VPC** podría ser adecuado:

- ▶ Pequeñas aplicaciones individuales que administra un equipo reducido.
- ▶ Entornos de informática de alto rendimiento (HPC) (como simulaciones de física): un entorno de una sola VPC tiene una latencia más baja que la de los entornos distribuidos entre varias VPC.
- ▶ Entornos de administración de identidades: una sola VPC puede proporcionar el mejor nivel de seguridad.

La opción de **múltiples VPC** es más adecuada para un solo equipo o una organización que disponga de control absoluto sobre el aprovisionamiento y la administración de todos los recursos en cada entorno de aplicación. Por ejemplo, piense en un solo equipo que desarrolla una gran aplicación de comercio electrónico. Pueden usar este patrón cuando los desarrolladores tienen acceso total a los entornos de desarrollo y producción.

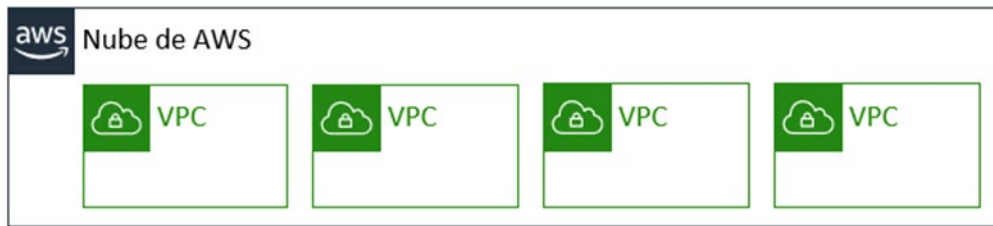


Figura 4. Esquema de red en AWS con varios VPC en una misma cuenta. Fuente: AWS, 2021.

Como se mencionó, puede crear varias VPC en la misma cuenta de la nube o en cuentas diferentes.

Los **patrones de cuentas múltiples** son más adecuados para clientes u organizaciones empresariales que implementan aplicaciones administradas en **varios equipos**. Por ejemplo, piense en una organización que respalda a dos o más equipos. Se puede utilizar este patrón para respaldar a los desarrolladores que disponen de acceso total a los recursos del entorno de desarrollo, pero acceso limitado o nulo al entorno de producción.



Figura 5. Esquema de red en AWS con varias cuentas y un único VPC en cada cuenta. Fuente: AWS, 2021.

Una gateway de Internet es un componente de VPC que permite la comunicación entre los recursos en la VPC y el Internet. Se escala de forma horizontal, es redundante y cuenta con alta disponibilidad.

Una gateway de Internet admite el tráfico IPv4 e IPv6. Además, cumple dos propósitos. En primer lugar, proporciona un objetivo en las tablas de enrutamiento de la VPC para el tráfico que se puede llevar a Internet. En segundo lugar, la gateway

de Internet realiza la conversión de las direcciones de red (NAT) para instancias a las que se asignaron direcciones IPv4 públicas.

Para convertir una subred en pública, primero debe crear una gateway de Internet y asociarla a la VPC.



Figura 6. Esquema de red de un gateway de Internet. Fuente: AWS, 2021.

A continuación, se debe actualizar la tabla de enrutamiento asociada a la subred que desea conectar a Internet. Una **tabla de enrutamiento** contiene un conjunto de reglas denominadas rutas. Las rutas se utilizan para determinar hacia dónde se dirige el **tráfico de red**.

Cuando se crea una VPC, se le asocia automáticamente una tabla de enrutamiento principal. En un principio, la **tabla de enrutamiento principal** (y todas las de una VPC) contiene solo una ruta local con la que se puede establecer la comunicación de los recursos dentro de la VPC. No se puede modificar la ruta local en una tabla de enrutamiento. Al lanzar una instancia en la VPC, la ruta local alcanza dicha instancia automáticamente. No es necesario agregar la nueva instancia a una tabla de enrutamiento. Se pueden crear más tablas de enrutamiento personalizadas para la VPC.

Cada **subred** de la VPC debe estar asociada a una tabla de enrutamiento, con la que se controla el direccionamiento de la subred. Si no asocia de forma explícita una subred a una tabla de enrutamiento particular, la subred se asocia de forma implícita a la tabla de enrutamiento principal y hace uso de ella. Una subred puede asociarse a una sola tabla de enrutamiento por vez, pero pueden asociarse varias subredes a la misma tabla de enrutamiento.

Puede crear tablas de enrutamiento personalizadas para cada subred con el fin de habilitar el direccionamiento detallado de los destinos.

Para enviar tráfico no local a través de la gateway de Internet hacia Internet, cree una ruta con **destino 0.0.0.0/0** y **objetivo <igw-id>** en la tabla de enrutamiento asociada a la subred.

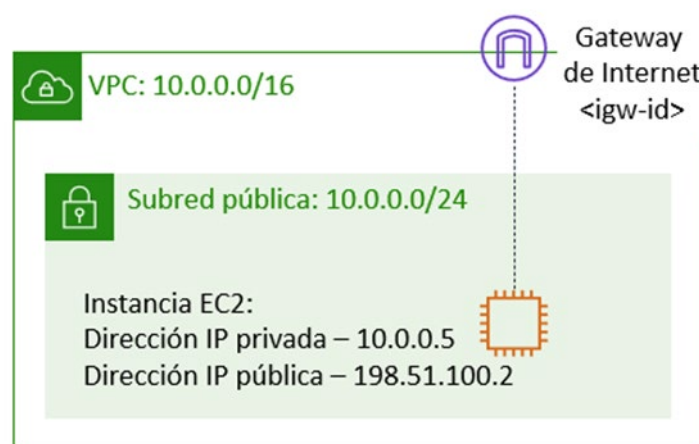


Tabla de enrutamiento pública

Destino	Objetivo
10.0.0.0/16	local
0.0.0.0/0	<igw-id>

Figura 7. Tablas de enrutamiento en una VPC. Fuente: AWS, 2021.

Un servidor bastión es un servidor que proporciona acceso a una red privada desde una red externa, como Internet.

Se puede usar un servidor bastión para **minimizar las posibilidades de intrusión y ataque** en los recursos de su red privada.

Por ejemplo, supongamos que se desea permitir la conexión desde una red externa a máquinas Linux en una subred privada de su VPC a través de Secure Shell o SSH.

Se puede utilizar un servidor bastión para mitigar el riesgo de permitir estas conexiones SSH externas a las instancias de la subred privada. Un servidor bastión normalmente se ejecuta en una máquina virtual de una subred pública de la VPC, como se muestra en la siguiente figura.

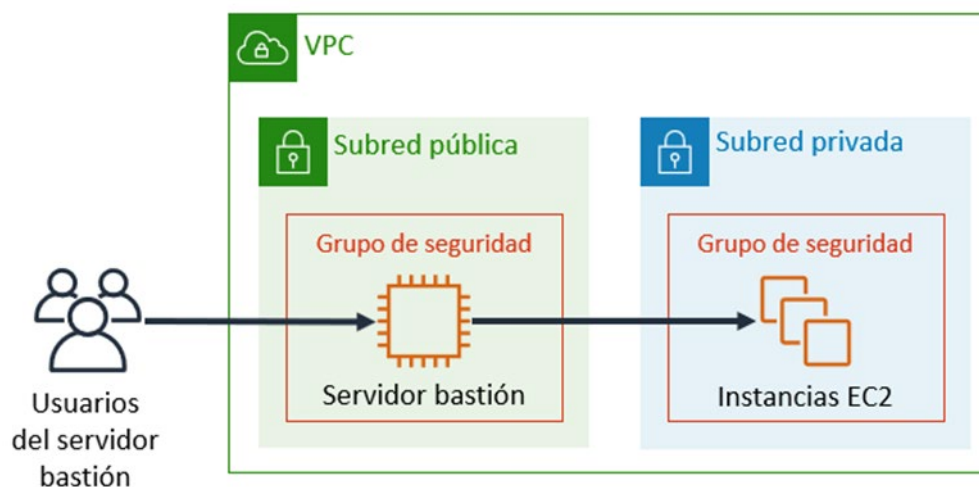


Figura 8. Esquema de un servidor bastión. Fuente: AWS, 2021.

Las máquinas Linux en la subred privada se encuentran en un grupo de seguridad que permite el acceso SSH desde el grupo de seguridad asociado al servidor bastión. Los usuarios del servidor bastión se conectan a dicho servidor para poder establecer contacto con las máquinas de Linux.

Aunque se puede adaptar esta arquitectura para que cumpla sus propios requisitos, el servidor bastión debería ser la única fuente de tráfico SSH para las máquinas virtuales (instancias) de Linux.

6.3. Protección del entorno de red

Según afirma Amazon Web Services (2021), después de diseñar e implementar un entorno de red, además de conectarlo a Internet, se debe aislar sus aplicaciones y cargas de trabajo.

Se puede lograr el aislamiento mediante la implementación de las **instancias EC2** que alojan la aplicación o la carga de trabajo en un grupo de seguridad asociado a la VPC.

Los grupos de seguridad representan *firewall* con estado que actúa en el nivel de la instancia o la interfaz de red.



Figura 9. Esquema de los grupos de seguridad. Fuente: AWS, 2021.

Con estado significa que el tráfico de **retorno** se admite de manera **automática**, independientemente de las reglas. Por ejemplo, supongamos que inicia un comando ping del protocolo de mensaje de control de Internet (ICMP) en la instancia desde su equipo personal. Si las reglas de entrada del grupo de seguridad permiten el tráfico

de ICMP, se realiza un seguimiento de la información sobre la conexión (incluida la información del puerto). No se realiza un seguimiento del tráfico de respuesta de la instancia por el comando ping como una nueva solicitud.

En cambio, se realiza un seguimiento de dicho tráfico como una conexión establecida. Está permitido el tráfico saliente de la instancia, incluso si las reglas de salida del grupo de seguridad restringen el tráfico ICMP saliente.

Las **reglas del grupo de seguridad** controlan el tráfico entrante y saliente correspondiente a sus recursos de nube. Debe configurar estas reglas de manera muy estricta para restringir el tráfico y permitir el acceso solo según sea necesario. El tráfico se puede restringir con cualquier protocolo de Internet, puerto de servicio y dirección IP de origen o destino (dirección IP individual o bloque de CIDR).

Cuando crea un grupo de seguridad, este carece de reglas de entrada. Esto significa que debe agregar reglas de entrada al grupo de seguridad para **permitir el tráfico entrante** que proceda de otro alojamiento hacia su instancia. De forma predeterminada, en los grupos de seguridad se incluye una **regla de salida** con la que se permite todo el tráfico saliente. Es posible quitar esta regla y agregar reglas de salida que permitan solo tráfico saliente específico. Si el grupo de seguridad no tiene reglas de salida, no se permitirá el tráfico saliente que proceda de su instancia.

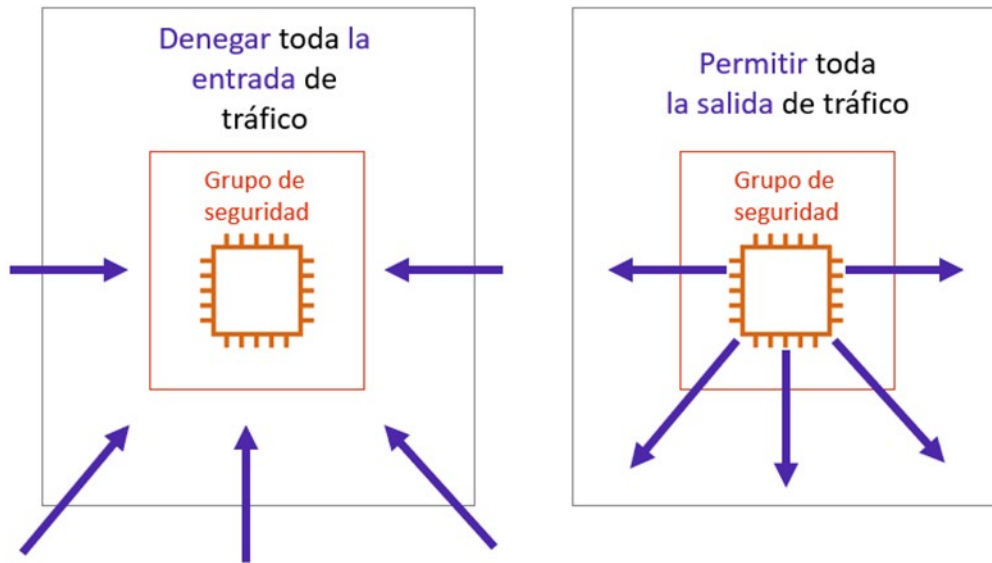


Figura 10. Comportamiento de grupos de seguridad. Fuente: AWS, 2021.

Una lista de control de acceso a la red (ACL de red) es una capa de seguridad opcional para su VPC.

Actúa como *firewall* para controlar el tráfico que entra y sale de una o más subredes. Para agregar una capa de seguridad adicional a su VPC, puede configurar ACL de red con reglas similares a sus grupos de seguridad.

Cada **subred** de su VPC debe estar **asociada a una ACL de red**. Si no se asocia una subred de forma explícita a una ACL de red, la subred se asociará automáticamente a la ACL de red predeterminada. Se puede asociar una ACL de red a varias subredes. Sin embargo, una subred solo puede asociarse a una ACL de red a la vez. Cuando se asocia una ACL de red a una subred, se elimina la asociación anterior.

Una **ACL** de red tiene **reglas de entrada y de salida** independientes y cada regla puede permitir o denegar tráfico. Su VPC incluye automáticamente una ACL de red predeterminada y modificable. De forma predeterminada, permite todo el tráfico IPv4 de entrada y salida, además, si corresponde, del tráfico IPv6.

Las ACL de red no tienen estado, lo que significa que no se mantiene información sobre las solicitudes después de procesarlas. Las reglas deben permitir de forma explícita el tráfico de retorno.

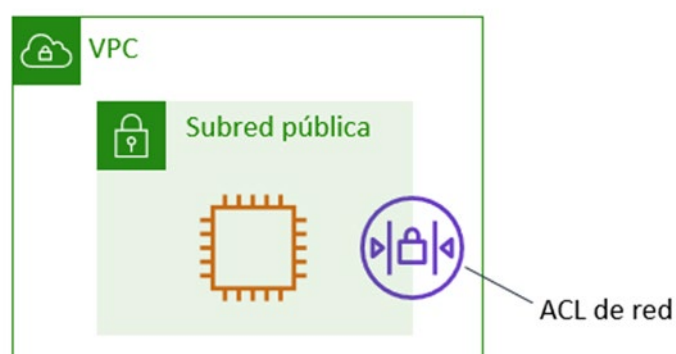


Figura 11. Posición de una ACL en una VPC. Fuente: AWS, 2021.

Como práctica recomendada, se debe **proteger** su infraestructura con **varias capas de defensa**. Cuando ejecuta su infraestructura en una VPC, puede controlar qué instancias están expuestas a Internet. Se puede definir grupos de seguridad y ACL de red para proteger aún más la infraestructura en los niveles de infraestructura y subred, respectivamente. Además, debería proteger las instancias con un *firewall* al nivel del sistema operativo y seguir otras prácticas recomendadas para la seguridad.

Cuando se implementa tanto las ACL de red como los grupos de seguridad como una forma de defensa pormenorizada para controlar el tráfico, **un error** en la configuración de uno de estos controles **no expondrá el alojamiento** a tráfico no deseado.

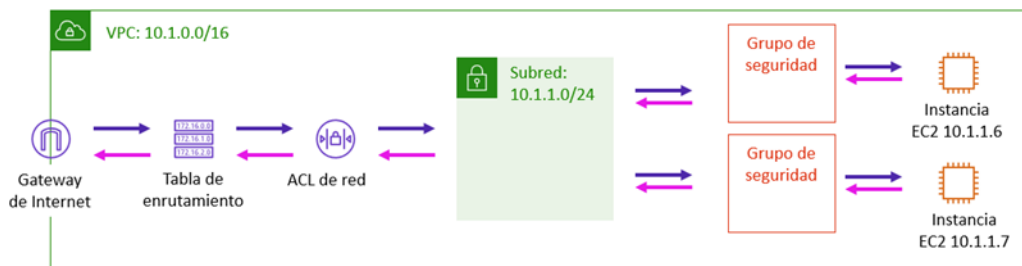


Figura 12. Estructura de una infraestructura con varias capas de defensa. Fuente: AWS, 2021.

En el vídeo *Gestión de redes en nube pública* aprenderemos a crear redes para nuestras infraestructuras en nube pública. Diferenciaremos entre subredes públicas y privadas y modificaremos las tablas de enrutamiento para tener salida a Internet.



6.4. Conexión a la red remota

De forma predeterminada, los servicios que se lanzan en una nube virtual privada (VPC) en la nube no pueden comunicarse con la red en las instalaciones.

Puede utilizar **Virtual Private Network** para conectar de forma segura la red en las instalaciones o el sitio de su sucursal a la VPC. Cada conexión de VPN utiliza comunicaciones de seguridad del protocolo de Internet (IPSec) para crear túneles de VPN cifrados entre dos ubicaciones.

Un **túnel de VPN** es un enlace cifrado por el que los datos pueden viajar entre la red del cliente y la nube. El lado en la nube de la conexión es la gateway privada virtual. El lado en las instalaciones de la conexión es la gateway de cliente.

Un **Site-to-Site VPN** proporciona dos túneles de VPN entre varias zonas de disponibilidad que puede utilizar simultáneamente y así obtener una alta disponibilidad. Se puede transmitir el tráfico principal a través del primer túnel y utilizar el segundo para lograr redundancia. Si un túnel deja de funcionar, el tráfico llegará igualmente a su VPC.

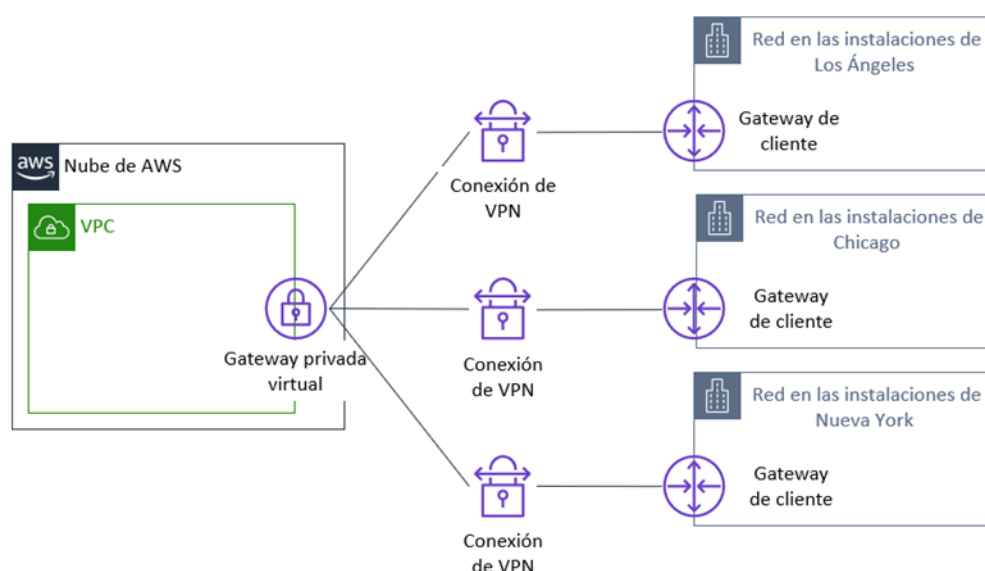


Figura 13. Conexión entre nube y centro de datos privado. Fuente: AWS, 2021.

Cuando crea una conexión de Site-to-Site VPN, debe especificar el **tipo de direccionamiento** que planea utilizar y debe actualizar la **tabla de enrutamiento** de su subred.

Un Site-to-Site VPN admite dos **tipos de direccionamiento**. El tipo de direccionamiento que seleccione depende de la marca y el modelo de los dispositivos de VPN:

- Si el dispositivo de VPN admite el protocolo de gateway fronteriza (BGP), especifique direccionamiento dinámico al configurar la conexión de Site-to-Site VPN. El direccionamiento dinámico utiliza el BGP para anunciar rutas a la gateway privada virtual. Además, admite un máximo de cien rutas propagadas por tabla de enrutamiento.

- ▶ Si el dispositivo de VPN no admite BGP, especifique el direccionamiento estático. El direccionamiento estático requiere que especifique las rutas (es decir, los prefijos de IP) para la red que deberían comunicarse con la gateway privada virtual.

Se recomienda utilizar **dispositivos que admitan BGP**, ya que dicho protocolo ofrece comprobaciones de detección de conexión sólidas que pueden ayudar en la conmutación por error al segundo túnel de VPN en caso de una falla en el primero. Los dispositivos que no admiten BGP también pueden realizar comprobaciones de estado para asistir en la conmutación por error al segundo túnel cuando se necesiten.

Direct Connect (DX) es otra solución que se extiende más allá de la simple conectividad a través de Internet.

DX utiliza redes de área local virtual (VLAN) de estándar abierto 802.1q para que pueda establecer una conexión de red privada exclusiva entre las instalaciones y la nube. Esta conexión privada puede reducir los costos de red, incrementar el rendimiento del ancho de banda y proporcionar una experiencia de red más uniforme que las conexiones basadas en Internet.

Las conexiones exclusivas están disponibles con capacidades de 1 Gbps y 10 Gbps.

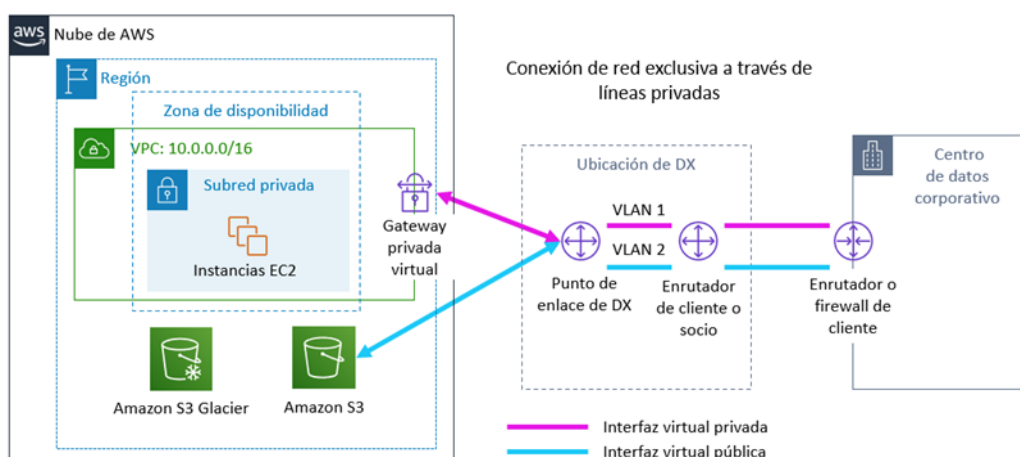


Figura 14. Ejemplo de conectividad entre el proveedor AWS y un data center local. Fuente: AWS, 2021.

DX es útil para varios casos, por ejemplo:

- ▶ **Entornos híbridos:** para aplicaciones que requieren acceso a equipos existentes del centro de datos, como una base de datos en las instalaciones, DX le permite crear un entorno híbrido que ofrece la oportunidad de aprovechar la elasticidad y los beneficios económicos de la nube.
- ▶ **Transferencia de grandes conjuntos de datos:** para aplicaciones que operan en grandes conjuntos de datos, como aplicaciones de informática de alto rendimiento (HPC), transferir grandes conjuntos de datos a través de Internet entre el centro de datos y la nube que puede llevar mucho tiempo y ser costoso. En el caso de dichas aplicaciones, conectarse a la nube con DX es una buena solución debido a lo siguiente:
 - Las transferencias por medio de la red no competirán por ancho de banda de Internet en su centro de datos.
 - El enlace con ancho de banda elevado reduce la posibilidad de que se produzcan congestiones en la red y disminuya el rendimiento de las aplicaciones.
- ▶ **Mayor rendimiento en las aplicaciones:** las aplicaciones que necesitan rendimiento de red predecible también se pueden beneficiar del uso de DX. Entre los ejemplos, se pueden incluir las aplicaciones que funcionan en fuentes de datos en tiempo real, como las transmisiones de audio o vídeo. En dichos casos, una conexión de red exclusiva puede ofrecer un rendimiento de red más estable que la conectividad de Internet estándar.
- ▶ **Seguridad y conformidad:** las políticas normativas o de seguridad empresariales a veces precisan que las aplicaciones alojadas en la nube de AWS solo sean accesibles a través de circuitos de red privados. DX es una solución lógica para este requisito porque el tráfico entre el centro de datos y la aplicación se transmite mediante la conexión de red privada exclusiva.

6.5. Escalado de redes

VPC Peering

Según afirma Amazon Web Services (2021), por lo general, aislar las cargas de trabajo en **VPC individuales** se considera una práctica recomendada. Por ejemplo, cuando su empresa o arquitectura sean lo suficientemente grandes, es posible que deba separar elementos lógicos, ya sea por seguridad o necesidades de la arquitectura, o bien, por simplicidad. Sin embargo, puede ser muy conveniente tener conectividad entre las VPC para situaciones en las que necesite transferir datos entre ellas.

Una interconexión de VPC es una conexión de red directa entre dos VPC que permite dirigir el tráfico entre ellas de forma privada.

Los servicios de cualquiera de las VPC pueden comunicarse entre sí como si estuvieran en la misma red. Puede crear una **interconexión de VPC entre sus propias VPC**, con una VPC de otra cuenta del proveedor de nube o con una VPC de otra región.

Puede establecer relaciones de interconexión entre VPC de diferentes regiones. La **interconexión de VPC entre regiones** ofrece una forma sencilla y rentable de compartir recursos entre regiones o de replicar datos para conseguir redundancia geográfica. Los datos transferidos a través de interconexiones de VPC entre regiones se cobran según las tarifas estándar de transferencia de datos entre regiones.

Para establecer una interconexión de VPC, el propietario de la VPC solicitante (o VPC local) envía una **solicitud al propietario** de la VPC del mismo nivel. Para activar la conexión, el propietario de la VPC del mismo nivel debe aceptar la solicitud de interconexión de VPC.

Para permitir el **flujo del tráfico** entre las VPC del mismo nivel utilizando direcciones IP privadas, debe agregar una ruta a una o más tablas de enrutamiento de la VPC.

Esta ruta debe llevar al intervalo de direcciones IP de la VPC del mismo nivel. El propietario de la VPC del mismo nivel agrega una ruta a una de sus tablas de enrutamiento de la VPC que lleva al intervalo de direcciones IP de la VPC.

Es posible que también tenga que **actualizar las reglas del grupo de seguridad** que se han asociado a la instancia para que el tráfico desde y hacia la VPC del mismo nivel no se vea restringido.

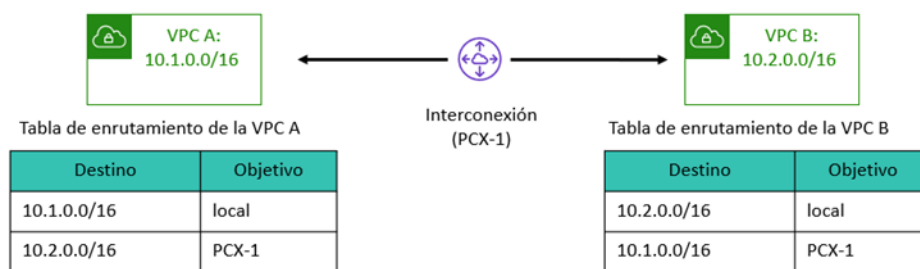


Figura 15. Configuración de interconexión entre VPCs. Fuente: AWS, 2021.

Transit Gateway

A medida que aumente el número de cargas de trabajo que se ejecutan en la nube, deberá ser capaz de **ajustar la escala de sus redes** en varias cuentas y VPC para seguir el ritmo de crecimiento. Se puede usar la interconexión de VPC para conectar pares de VPC.

Sin embargo, **administrar la conectividad** de un punto a otro en muchas VPC, sin la posibilidad de gestionar las políticas de conectividad de forma centralizada, puede resultar una tarea engorrosa y costosa desde un punto de vista operativo. Para la conectividad en las instalaciones, debe **asociar su VPN a cada VPC**. Esta solución puede requerir mucho tiempo en la etapa de creación, además de ser difícil de administrar cuando la cantidad de VPC alcance los tres dígitos.

Es importante tener en cuenta cuán grande puede llegar a ser su entorno con el paso del tiempo, qué tan bien ajustará su escala y cómo organizará las VPC. Si desea solucionar este problema, puede utilizar un **Transit Gateway** para simplificar su modelo de red.

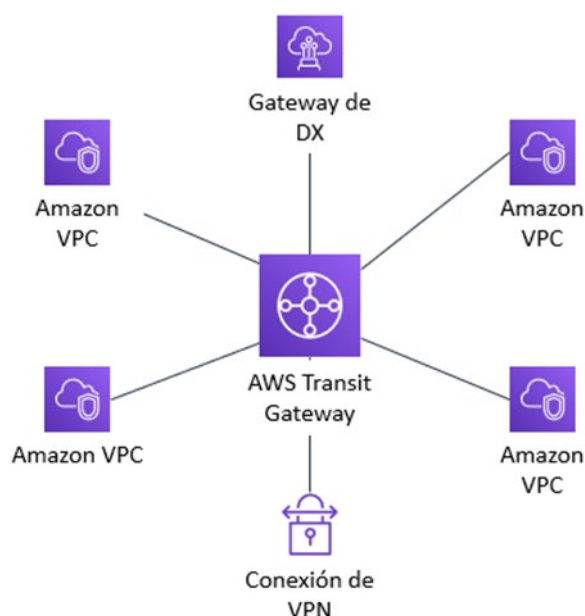


Figura 16. Esquema del servicio Transit Gateway. Fuente: AWS, 2021.

Transit Gateway es un servicio que le permite conectar las VPC y las redes en las instalaciones a una única gateway (llamada gateway de tránsito).

Con Transit Gateway, solo tiene que **crear y administrar una única conexión** desde la gateway central para cada VPC, centro de datos en las instalaciones u oficina remota de su red.

Transit Gateway utiliza un **sistema radial**. Este sistema radial simplifica de forma significativa la administración y reduce los costos operativos porque cada red solo tiene que conectarse a la gateway de tránsito y no a todas las demás redes. Todas las VPC nuevas se conectan a la gateway de tránsito y luego quedan disponibles automáticamente para cualquier otra red que esté conectada a la gateway de

tránsito. Esta facilidad en la conectividad simplifica el escalado de la red para acompañar su crecimiento.

6.6. Gestión de identidad y seguridad en la nube

Usuarios y políticas

Según afirma Amazon Web Services (2021), cuando se crea una cuenta en un proveedor de nube, se comienza con un **usuario raíz**. Este usuario puede iniciar sesión en la consola de administración con la dirección de correo electrónico que se utilizó para crear la cuenta.

El usuario raíz de la cuenta tiene **acceso completo** a todos los recursos de la cuenta, incluidos la información de facturación, los datos personales del perfil de usuario y todos los recursos creados en cualquier servicio de la cuenta. No se puede controlar los privilegios de las credenciales de usuario raíz.

Se recomienda enfáticamente que no se utilice las **credenciales del usuario raíz** para las interacciones cotidianas. En cambio, cree uno o más usuarios en el servicio IAM (Identity Access Management). Guarde las credenciales del usuario raíz en una ubicación segura. Para la mayoría de las tareas continuas de administración y acceso de la cuenta, puede utilizar las credenciales de usuario de IAM.

Cómo IAM determina los permisos en el momento en que se realiza la solicitud:



Figura 17. Determinación de permisos en IAM. Fuente: AWS, 2021.

En IAM, los permisos se definen en los **documentos de políticas de IAM**. Las políticas le permiten ajustar los privilegios que se conceden a las entidades principales. Ejemplos de entidades principales son los usuarios de IAM, los roles de IAM u otros servicios de AWS.

Cuando IAM determina si se concede un permiso, comprueba primero si existe alguna política de denegación explícita que se pueda aplicar. Si no existe ninguna indicación de denegación explícita, comprueba si existe alguna política de permiso explícito que se pueda aplicar. Si no existe una política de denegación explícita ni una de permiso explícito, IAM vuelve al comportamiento predeterminado y deniega el acceso. Este proceso se denomina **denegación implícita**. El usuario podrá realizar la acción solo si la acción solicitada no está denegada de forma explícita y está permitida de forma explícita.

Las políticas basadas en identidades son políticas de permisos que puede asociar a una entidad principal (o identidad), como un usuario, un rol o un grupo de IAM.

Estas políticas controlan qué acciones puede realizar dicha identidad, en qué recursos y de acuerdo con qué condiciones.

Las políticas basadas en **identidades** se pueden clasificar como administradas por el proveedor de nube, administradas por el cliente o insertadas. Se crea y administra las políticas administradas por el proveedor y usted puede asociarlas a varios usuarios, grupos y roles de su cuenta de nube. Se recomienda comenzar con las políticas administradas. Las políticas administradas por el **cliente** son políticas que el usuario crea y administra en su cuenta.

Las **políticas administradas por el cliente** ofrecen un nivel de control más preciso sobre las políticas que las políticas administradas por el proveedor. Puede crear y editar una política de IAM en el editor visual o mediante la creación del documento de política **JSON** directamente.

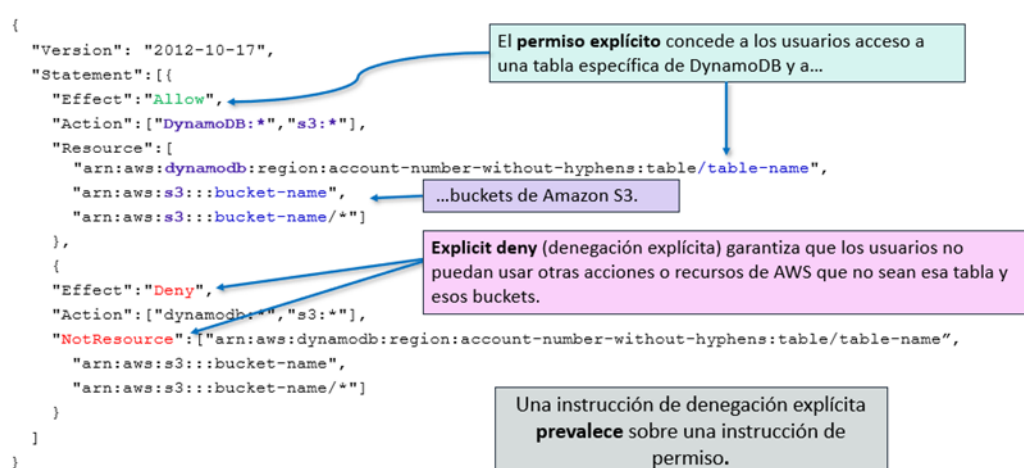


Figura 18. Estructura de un fichero .json de políticas IAM. Fuente: AWS, 2021.

La **política de IAM** de este ejemplo concede a los usuarios acceso únicamente a los siguientes **recursos**:

- ▶ La tabla de AWS Amazon DynamoDB, cuyo nombre está representado por *table-name*.
- ▶ El bucket de AWS S3, cuyo nombre está representado por *bucket-name* y todos los objetos que contiene.

La política de IAM también incluye un elemento de denegación explícita («Effect»:«Deny»). El elemento **NotResource** ayuda a garantizar que los usuarios no

puedan utilizar ninguna otra acción o recurso de **DynamoDB** o **S3**, solo las acciones y los recursos especificados en la política. Esto se respeta incluso si se han concedido permisos en otra política. Una instrucción de denegación explícita prevalece sobre una instrucción de permiso.

Grupos (IAM)

Un grupo de IAM es un conjunto de **usuarios de IAM**. Los grupos representan una ventaja, ya que facilitan la administración de permisos para un conjunto de usuarios. No es necesario administrar los permisos de cada usuario individual.

La **suscripción** a grupos se administra por medio de una sencilla lista:

- ▶ Agregue usuarios a un grupo o elimínelos de uno.
- ▶ Un usuario puede pertenecer a varios grupos.
- ▶ Los grupos no pueden pertenecer a otros grupos.
- ▶ Se pueden conceder permisos a los grupos mediante las políticas de control de acceso.
- ▶ Los grupos no tienen credenciales de seguridad y no pueden acceder directamente a los servicios web. Existen solo para facilitar la administración de los permisos de los usuarios.

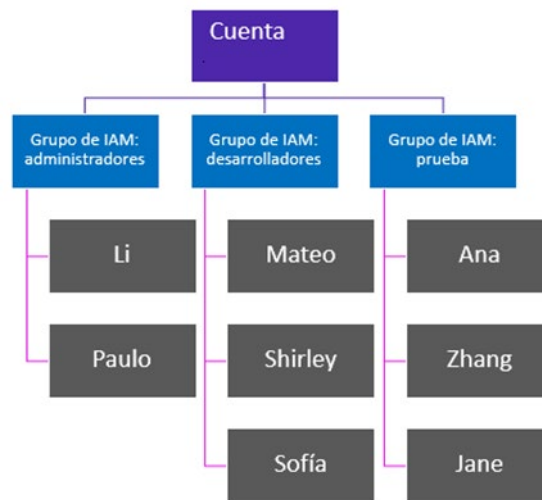


Figura 19. Estructura de grupos de usuarios. Fuente: AWS, 2021.

Roles

Un rol de IAM le permite definir un conjunto de **permisos** para acceder a los recursos que necesita un usuario o un servicio. Sin embargo, los permisos no se asocian ni a un usuario ni a un grupo de IAM. En cambio, **se asocian a un rol** y el usuario o el servicio asumen el rol.

Cuando un usuario asume un rol, los permisos anteriores del usuario se olvidan temporalmente.

Si se usan roles de IAM, no es necesario compartir credenciales de seguridad de largo plazo por cada entidad que requiera acceso a un recurso, como la creación de un usuario de IAM.

La **entidad principal** que asume el rol también podría ser un usuario, un grupo o un rol de IAM de otra cuenta, incluidas las cuentas que no le pertenecen.

Al crear un rol para el **acceso a cuentas externas**, no es necesario administrar nombres de usuario ni contraseñas de terceros. Si ya no desea que alguien o algún sistema tenga acceso, puede modificar o eliminar el rol. Por lo tanto, no necesita crear ni administrar cuentas de personas ajenas a su organización.

Federación de identidades

Vamos a entender ahora los **usuarios autenticados externamente**.

IAM admite la federación de identidad para el acceso delegado a la consola de administración. Con la federación de identidad, se concede a las identidades externas acceso seguro a los recursos de la cuenta sin tener que crear usuarios de IAM.

La figura que vemos a continuación muestra los cuatro pasos principales que se producen cuando recurre a un **proveedor de identidades (IdP)** a la hora de crear credenciales temporales para un usuario o una aplicación.



Figura 20. Validación con proveedor de identidad externo. Fuente: AWS, 2021.

Cuentas múltiples

La **federación de identidad** se puede lograr de una de **tres maneras**. La primera forma es utilizar un IdP corporativo (como Microsoft Active Directory) o una aplicación personalizada de agente de identidades. El segundo enfoque es crear una integración que utilice el lenguaje de marcado para confirmaciones de seguridad (SAML). El tercer enfoque consiste en utilizar un proveedor de identidades web de cada uno de los proveedores de nube.

Cuando se utiliza la **nube para dar soporte** a los diferentes equipos y departamentos de una organización, se puede elegir entre **dos patrones** de arquitectura generales para aislar y separar los recursos que utiliza cada equipo.

El **primer patrón** consiste en definir varias nubes virtuales privadas (VPC) en una sola cuenta. Si se prefiere una administración centralizada de la seguridad de la información con sobrecarga mínima, puede optar por usar una sola cuenta.

El **segundo patrón** consiste en crear varias cuentas y definir una VPC en cada cuenta. En la práctica, las organizaciones grandes y pequeñas tienden a crear varias cuentas para sus organizaciones. Por ejemplo, se podrían crear cuentas individuales para distintas unidades de negocio. También se podrían crear cuentas separadas para sus recursos de desarrollo, prueba y producción.

Cuando los clientes utilizan **cuentas diferentes** (generalmente con facturación consolidada) para recursos de desarrollo y producción, pueden separar diferentes tipos de recursos de forma prolija. Esto también puede presentar algunos **beneficios** en materia de seguridad.

Como alternativa, con el desarrollo y las pruebas, se podría configurar tres cuentas y contar con una para cada entorno. Además, si tiene varios departamentos autónomos, también podría crear cuentas diferentes para cada parte autónoma de la organización.

Cuando utiliza varias cuentas, una **estrategia más eficiente** consiste en crear una sola cuenta para los recursos comunes de un proyecto. Los **recursos comunes** podrían incluir servicios de sistema de nombres de dominio (DNS), Microsoft Active Directory y sistemas de administración de contenido (CMS). También podría separar las cuentas de los proyectos o los departamentos autónomos. Esta estrategia le permite asignar permisos y políticas en cada cuenta de departamento o proyecto y conceder acceso a los recursos en todas las cuentas.

Varias VPC en una sola cuenta patrón de arquitectura



Varias cuentas, una VPC en cada cuenta patrón de arquitectura



Figura 21. Estructuras de multi cuenta en el proveedor AWS. Fuente: AWS, 2021.

Aunque la mayoría de las organizaciones optan por utilizar **varias cuentas**, esta opción presenta algunos **desafíos**.

En primer lugar, debe determinar cómo administrar eficazmente la **seguridad** en todas sus cuentas. Si replica las políticas de IAM definidas en todas las cuentas para garantizar la coherencia, esto podría implicar automatización personalizada, esfuerzo manual o ambas cosas.

Además, es posible que se le pida todo el tiempo que cree más cuentas. Lleva tiempo realizar esta tarea de forma manual. También podría ser difícil rastrear todas las cuentas y el propósito de cada una.

También podría representar un desafío determinar qué **centro de costos** de la organización debería recibir la factura correspondiente a cada recurso en las distintas cuentas. Finalmente, es posible que también desee lograr la **gobernanza centralizada** que se necesita para garantizar la coherencia.

En el siguiente vídeo *Gestión de accesos e identidades* vamos a ver cómo se gestionan las entidades de seguridad como son los usuarios, grupos, roles y usuarios federados. Veremos que los permisos sobre servicios en la nube se gestionan con ficheros en formato JSON y aprenderemos la estructura y uso de dichas políticas de seguridad.



Accede al vídeo

6.7. Referencias bibliográficas

Amazon Web Services. (2022). *Amazon Virtual Private Cloud (Amazon VPC)* <https://aws.amazon.com/es/vpc/>

AWS. (2021, diciembre). *Training Track: Cloud Architecting*. Manuscrito no publicado.

Google Cloud. (2022). *Nube privada (VPC)*. <https://cloud.google.com/vpc?hl=es>

Microsoft (2022). *¿Qué es Azure Virtual Network?* <https://docs.microsoft.com/es-es/azure/virtual-network/virtual-networks-overview>

Google Cloud Platform

Google Cloud. (2022). *Gestión de identidades y accesos (IAM)*
<https://cloud.google.com/iam?hl=es>

Centro de recursos del proveedor de nube Google. En ella podemos encontrar toda la información ampliada con respecto a la gestión de identidades en la nube, así como también información sobre la infraestructura de red que hemos visto a lo largo de este tema.

Amazon Web Services-VPC

Amazon Web Services. (2022). *Amazon Virtual Private Cloud (Amazon VPC)*
<https://aws.amazon.com/es/vpc/>

En el siguiente recurso encontrarás todo sobre redes del fabricante Amazon.

Microsoft

Microsoft. (2022). *Azure Virtual Network*. <https://azure.microsoft.com/es-es/services/virtual-network/#overview>

Desde este enlace, podemos consultar todos los servicios de red y seguridad del proveedor de nube Microsoft Azure.

1. Cada VPC pertenece a:

- A. A dos regiones del proveedor para conseguir disponibilidad.
- B. A dos *data centers* dentro de la misma región.
- ☐ C. Una sola región.
- D. Ninguna de las anteriores.

Cada VPC pertenece a una sola región del proveedor de nube. Una VPC abarca todas las zonas de disponibilidad de una región, de modo que puede alojar recursos compatibles desde cualquier zona de disponibilidad dentro de su región.

2. Cuando se crea una subred:

- A. Se deben crear tres subredes obligatoriamente.
- ☐ B. Se debe especificar el bloque de CIDR que le corresponde.
- C. Puedo superponer direccionamientos CIDR.
- D. El direccionamiento CIDR debe coincidir con el CIDR de tu empresa.

Cuando se crea una subred, se debe especificar el bloque de CIDR que le corresponde, el cual es un subconjunto del bloque de CIDR de la VPC.

3. La opción de múltiples VPC es más adecuada para:

- ☐ A. Un solo equipo o una organización.
- B. Microempresas.
- C. Para cuentas de desarrollo y no de producción.
- D. Para *partners* de proveedores de nube.

La opción de múltiples VPC es más adecuada para un solo equipo o una organización que disponga de control absoluto sobre el aprovisionamiento y la administración de todos los recursos en cada entorno de aplicación.

4. Una gateway de Internet es un componente de:

- ☐ A. VPC que permite la comunicación entre los recursos en la VPC y el Internet.
- B. Red para base de datos.
- C. VPC que da acceso a internet a subredes privadas.
- D. VPC para interconectar la red pública con la empresa.

Un gateway de Internet es un componente de VPC que permite la comunicación entre los recursos en la VPC y el Internet. Se escala de forma horizontal, es redundante y cuenta con alta disponibilidad.

5. Un servidor bastión es un servidor:

- A. De Windows donde instalamos aplicaciones.
- B. De Linux donde instalamos programas de seguridad.
- C. Que proporciona aislamiento de los servicios web.
- ☐ D. Que proporciona acceso a una red privada desde una red externa.

Un servidor bastión es un servidor que proporciona acceso a una red privada desde una red externa, como Internet. Se puede usar un servidor bastión para minimizar las posibilidades de intrusión y ataque en los recursos de su red privada.

6. Las reglas del grupo de seguridad controlan el tráfico:

- A. Entrante correspondiente a sus recursos de nube.
- ☐ B. Saliente correspondiente a sus recursos de nube.
- C. Entrante y saliente correspondiente a sus recursos de nube.
- D. Ninguna de las anteriores.

Las reglas del grupo de seguridad controlan el tráfico entrante y saliente correspondiente a sus recursos de nube.

7. Una lista de control de acceso a la red ACL:

- A. Es otra forma de llamar los grupos de seguridad.
- B. Es una capa de seguridad obligatoria para su VPC.
- ☐ C. Es una capa de seguridad opcional para su VPC.
- D. Son reglas de entrada y salida a tu región de nube.

Una lista de control de acceso a la red (ACL de red) es una capa de seguridad opcional para su VPC. Actúa como firewall para controlar el tráfico que entra y sale de una o más subredes.

8. Las políticas basadas en identidades son políticas de permisos que:

- A. Gestionan recursos del proveedor de nube.
- B. Son ficheros planos en formato XML.
- C. Se asocian para gestionar proveedores de identidad externos al proveedor de nube.
- ☐ D. Puede asociar a una entidad principal (o identidad), como un usuario, un rol o un grupo de IAM.

Las políticas basadas en identidades son políticas de permisos que puede asociar a una entidad principal (o identidad), como un usuario, un rol o un grupo de IAM.

9. Un rol de IAM le permite definir:

- A. Un conjunto de permisos para acceder a nuestra empresa desde el proveedor de nube.
- ☐ B. Un conjunto de permisos para acceder a los recursos que necesita un usuario o un servicio.
- C. Un conjunto de permisos para acceder desde nuestra casa cuando teletrabajamos.
- D. Ninguna de las anteriores.

Un rol de IAM le permite definir un conjunto de permisos para acceder a los recursos que necesita un usuario o un servicio.

10. Si se usan roles de IAM:

A. No es necesario compartir ningún recurso.

B. No es necesario compartir los grupos de seguridad de largo plazo por cada entidad que requiera acceso a un recurso.

C. Es necesario compartir credenciales de seguridad de largo plazo por cada entidad que requiera acceso a un recurso.

☐ D. No es necesario compartir credenciales de seguridad de largo plazo por cada entidad que requiera acceso a un recurso.

Si se usan roles de IAM, no es necesario compartir credenciales de seguridad de largo plazo por cada entidad que requiera acceso a un recurso, como la creación de un usuario de IAM.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Monitorización, elasticidad y alta disponibilidad en la nube

Índice

Esquema	3
Ideas clave	4
7.1. Introducción y objetivos	4
7.2. Escalado de los recursos informáticos	5
7.3. Escalado de las bases de datos	12
7.4. Entornos de alta disponibilidad	17
7.5. Monitorización y observabilidad	26
7.6. Referencias bibliográficas	33
A fondo	34
Test	35



7.1. Introducción y objetivos

En este tema veremos cómo implementar una **arquitectura reactiva** que sea elástica, resiliente y con capacidad de respuesta.

Se analizan los **componentes** que hacen que esta arquitectura sea escalable y tenga disponibilidad alta (como la segunda zona de disponibilidad, el balanceador de carga de aplicaciones, Auto Scaling y servidor de nombre de dominio).

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

- ▶ Utilizar Auto Scaling dentro de una arquitectura para fomentar la elasticidad.
- ▶ Explicar cómo escalar los recursos de las bases de datos.
- ▶ Implementar un balanceador de carga de aplicaciones para crear un entorno de alta disponibilidad.
- ▶ Utilizar DNS para la conmutación por error de sistemas de nombres de dominio
- ▶ Crear un entorno de alta disponibilidad.
- ▶ Diseñar arquitecturas que utilicen Amazon CloudWatch para monitorear los recursos y reaccionar en consecuencia.

En los vídeos de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Crearemos una infraestructura de alta disponibilidad con balanceadores de carga y grupos de auto escalado.

Vídeo 2. Veremos las opciones de monitorización y observabilidad de los servicios desplegados en nube pública.

7.2. Escalado de los recursos informáticos

Auto escalado y balanceadores

Según afirma Amazon Web Services (2021), las aplicaciones modernas deben poder manejar cantidades masivas de datos sin tiempo de inactividad y con tiempos de respuesta que estén por debajo del segundo. Para cumplir estos requisitos, puede implementar un **sistema reactivo**, que sea elástico, resiliente, con capacidad de respuesta y que esté basado en mensajes.

Una arquitectura reactiva bien diseñada puede ahorrarle dinero y ofrecer una mejor experiencia a sus usuarios.

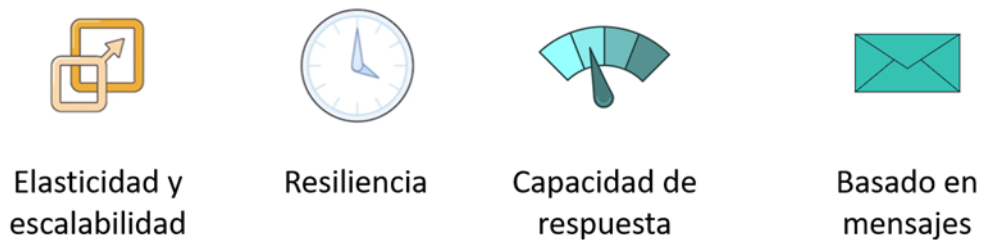


Figura 1. Arquitecturas reactivas. Fuente: AWS, 2021.

Una característica de una arquitectura reactiva es la **elasticidad**.

La elasticidad significa que la infraestructura puede ampliarse o contraerse si cambian las necesidades de capacidad.

Puede adquirir recursos cuando los necesite y liberar recursos cuando no los necesite.

La elasticidad le permite lo siguiente:

- ▶ Aumentar la cantidad de servidores web si el tráfico a la aplicación se incrementa.
- ▶ Reducir la capacidad de escritura en la base de datos si el tráfico disminuye.
- ▶ Manejar la fluctuación diaria de la demanda en toda la arquitectura.

El **escalado** es una técnica que se utiliza para lograr elasticidad. El escalado es la capacidad de aumentar o reducir la capacidad de cómputo de una aplicación. Hay dos **tipos** de escalado:

- ▶ El **escalado horizontal** ocurre si se agregan o se eliminan recursos. Por ejemplo, es posible que necesite agregar más discos duros a una matriz de almacenamiento o agregar más servidores para admitir una aplicación. Agregar recursos se denomina escalado ascendente y terminar recursos se denomina escalado descendente. El escalado horizontal es una buena manera de crear aplicaciones a escala de Internet que aprovechen la elasticidad de la informática en la nube.
- ▶ El **escalado vertical** ocurre si aumenta o disminuye las especificaciones de un recurso determinado. Por ejemplo, podría actualizar un servidor para que tenga un disco duro más grande o una CPU más rápida. El escalado vertical puede llegar en algún momento a un límite, y no siempre es un enfoque eficiente o de disponibilidad alta. Sin embargo, es fácil de implementar y puede ser suficiente para muchos casos de uso, sobre todo a corto plazo.

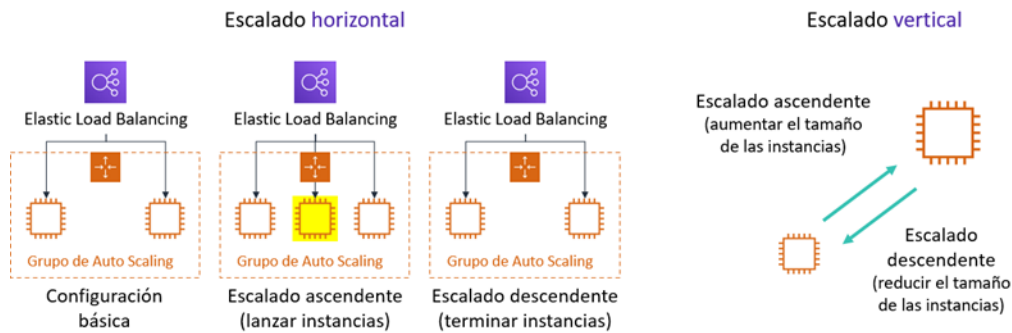


Figura 2. Escalado horizontal vs. escalado horizontal. Fuente: AWS, 2021.

En la nube, el escalado se puede manejar automáticamente. El **auto escalado** (Auto Scaling) ayuda a mantener la disponibilidad de la aplicación y le permite agregar o eliminar máquinas virtuales de forma automática según las políticas definidas, las programaciones y las comprobaciones de estado. Si especifica las políticas de escalado, Auto Scaling puede **lanzar o terminar instancias** en función del aumento o la disminución de la demanda de la aplicación.

El Auto Scaling se integra con **Elastic Load Balancing**: registra automáticamente nuevas instancias con balanceadores de carga para distribuir el tráfico entrante entre las instancias.

El Auto Scaling le permite crear arquitecturas de disponibilidad alta que abarcan varias zonas de disponibilidad en una región.

Si una zona de disponibilidad pasa a estar en mal estado o no está disponible, Auto Scaling lanza **nuevas instancias** en una zona de disponibilidad que no se haya visto afectada. Cuando la zona de disponibilidad en mal estado vuelve a tener un estado correcto, Auto Scaling redistribuye automáticamente las instancias de la aplicación de manera **uniforme** entre todas las zonas de disponibilidad designadas.

Con **Auto Scaling** se ofrecen varias **opciones de escalado** para atender las necesidades de las aplicaciones de la mejor manera.

- ▶ **Escalado programado.** Con el escalado programado, las acciones de escalado se efectúan automáticamente en función de la **fecha y la hora**. Esta característica resulta útil para las cargas de trabajo predecibles si sabe exactamente cuándo aumentar o reducir la cantidad de instancias del grupo. Por ejemplo, imaginemos que todas las semanas el tráfico a la aplicación web comienza a aumentar el miércoles, permanece alto el jueves y comienza a disminuir el viernes. Puede programar las acciones de escalado en función de los patrones de tráfico predecibles de la aplicación web. Para implementar el escalado programado, cree una acción programada.
- ▶ **Escalado dinámico bajo demanda.** Este enfoque es una forma más **avanzada** de escalar los recursos. Permite definir **parámetros** con que controlar el proceso de escalado. Por ejemplo, suponga que tiene una aplicación web que se ejecuta en dos máquinas virtuales. Desea que la utilización de la CPU del grupo de Auto Scaling permanezca cerca del 50 % si se modifica la carga de la aplicación. Esta opción resulta útil si el escalado responde a cambios de las condiciones, pero no se sabe en qué momento cambiarán esas condiciones. El escalado dinámico le da capacidad adicional para manejar los picos de tráfico sin tener que mantener una cantidad excesiva de recursos inactivos. Puede configurar el grupo de Auto Scaling a fin de que el escalado se efectúe automáticamente para atender esa necesidad.
- ▶ **Escalado predictivo.** Auto Scaling puede implementar el escalado predictivo, por el cual la capacidad escala en función de la demanda prevista. El escalado predictivo utiliza **datos** que se recopilan a partir del uso real de **máquinas virtuales**, y estos, además, incorporan miles de millones de puntos de datos que se extraen de las observaciones del proveedor de nube. Este utiliza **modelos de aprendizaje** automático bien entrenados para predecir el tráfico por esperar incluidos los patrones diarios y semanales. El modelo necesita datos históricos de al menos un día para comenzar a hacer predicciones. El modelo se revalúa cada

veinticuatro horas para crear una predicción de las siguientes 48 horas. El proceso de predicción genera un plan de escalado con el que se puede impulsar uno o más grupos de máquinas virtuales de escalado automático.

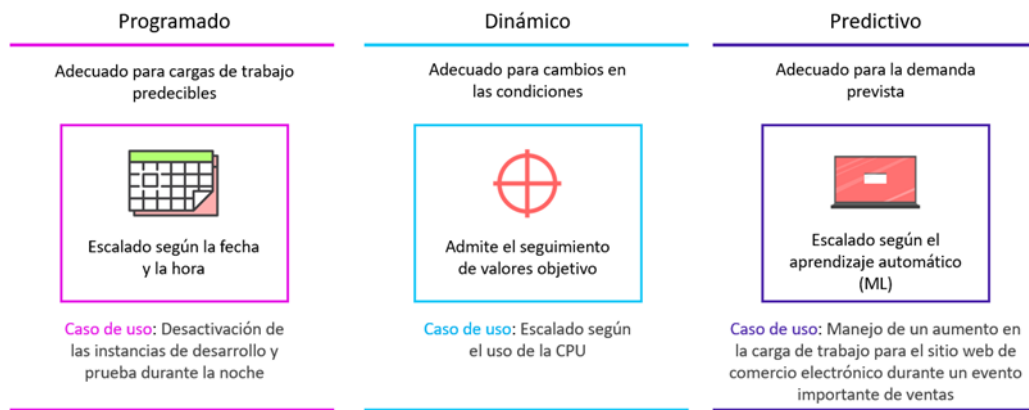


Figura 3. Tipos de auto escalado. Fuente: AWS, 2021.

El escalado dinámico significa ajustar la capacidad de su aplicación para afrontar cambios en la demanda a fin de poder optimizar la disponibilidad, el rendimiento y los costos.

El **tipo de política de auto escalado** determina cómo se efectúa la acción de escalado:

- Con las políticas de **escalado por pasos y escalado sencillo**, puede elegir las métricas de escalado y los valores de límite de las alarmas de monitorización que activan el proceso de escalado. También puede definir cómo debe escalarse el grupo de Auto Scaling si se supera un límite durante un número determinado de períodos de evaluación. La principal diferencia es que con el escalado por pasos la capacidad actual del grupo de Auto Scaling aumenta o disminuye en función de un conjunto de ajustes de escalado, llamados ajustes por pasos, que varían en función del tamaño de la interrupción de alarma.

- ▶ Con las políticas de **escalado de seguimiento de valores objetivo**, se aumenta o se reduce la capacidad actual del grupo en función de un valor objetivo de una métrica determinada. Este tipo de escalado se parece a la manera en que los termostatos mantienen la temperatura del hogar: el usuario selecciona una temperatura y el termostato hace el resto del trabajo. Las políticas de escalado de seguimiento de valores objetivo le permiten **seleccionar una métrica** de escalado y establecer un valor objetivo.

Con Auto Scaling, se crean y se administran las alarmas de monitorización que activan la política de escalado y calculan el ajuste de escalado en función del valor **objetivo y la métrica**. Con la política de escalado, se aumenta o se reduce la capacidad en función de las necesidades para mantener la métrica en el valor objetivo determinado o en un valor próximo. Además de mantener la métrica próxima al valor objetivo, la política de escalado de seguimiento de valores objetivo también se ajusta a los cambios de la métrica que se produzcan por una carga de trabajo que cambia.

Auto Scaling lo ayuda a asegurarse de que cuenta con la cantidad correcta de máquinas anteriores para controlar la carga de la aplicación.

Crea colecciones de máquinas virtuales denominadas grupos de Auto Scaling. Puede especificar la **cantidad mínima** de instancias en cada grupo de Auto Scaling y ayuda a que el grupo nunca tenga menos de esa cantidad de instancias. Puede especificar la **cantidad máxima** de instancias en cada grupo de Auto Scaling y ayuda a que el grupo nunca tenga menos de esa cantidad de instancias. Si especifica la **capacidad deseada**, ya sea cuando crea el grupo o después, ayuda a que el grupo tenga esa cantidad de instancias.

Tenga en cuenta que la **capacidad deseada** es una configuración basada en desencadenador y que puede fluctuar en respuesta a eventos como un límite que se ha superado. Refleja la cantidad de instancias que se están ejecutando en ese momento. Nunca puede ser menor que el valor mínimo o mayor que el valor máximo. El rol de la política de escalado es actuar como **representante automatizado** y tomar decisiones sobre cómo ajustar la capacidad deseada.

Auto Scaling le permite hacer **escalados ascendentes y descendentes** en la infraestructura en respuesta a cambios en las condiciones. Cuando configura un grupo de Auto Scaling, puede especificar los tipos de máquinas virtuales que utiliza. También puede especificar qué **porcentaje** de la capacidad deseada se debe atender con instancias bajo demanda, instancias reservadas e instancias de spot. Con Auto Scaling, después se aprovisiona de la combinación de instancias de precio más bajo para atender la capacidad deseada en función de estas preferencias.

Solo puede utilizar un tipo de instancia. Sin embargo, una práctica recomendada es utilizar unos cuantos tipos para no intentar iniciarlas desde grupos de instancias que tengan capacidad insuficiente.

Si la solicitud del grupo de Auto Scaling no se puede atender en un grupo de instancias de spot, el grupo de Auto Scaling sigue intentándolo en otros grupos de instancias de spot en lugar de lanzar instancias bajo demanda.

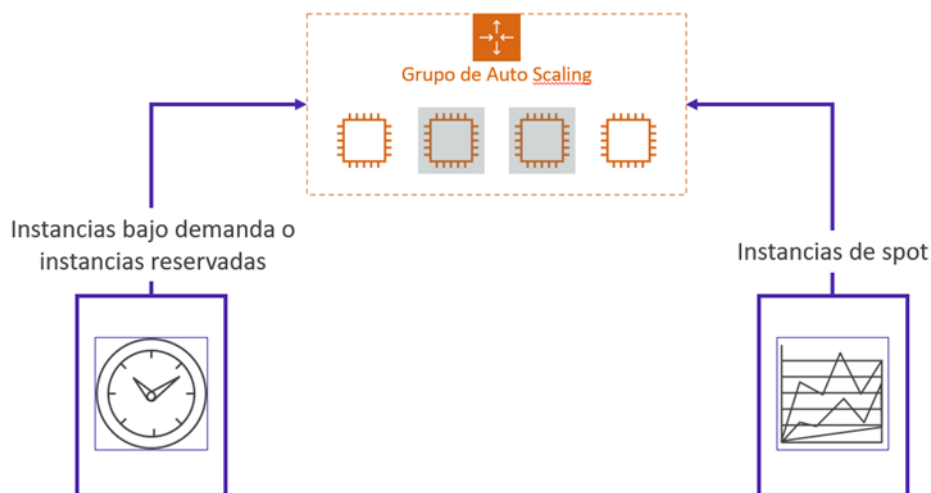


Figura 4. Opciones de compra en grupos de auto escalado. Fuente: AWS, 2021.

En el vídeo *Grupos de auto escalado y alta disponibilidad*, que podemos ver a continuación, veremos los conceptos de balanceador y de grupo de auto escalado. Son los dos servicios más importantes que podemos utilizar para realizar arquitecturas que se ajustan a la demanda en nuestros despliegues en nube pública.



Accede al vídeo

7.3. Escalado de las bases de datos

Según afirma Amazon Web Services (2021), en primer lugar, se plantea escalar las **bases de datos relacionales**. Como servicio administrado, los proveedores de nube escalan la base de datos relacional para que pueda seguir el ritmo a la demanda creciente de su aplicación.

Puede **escalar verticalmente** la instancia de base de datos de RDS cambiando el tipo de máquina virtual RDS. Sin embargo, si decide hacer un escalado ascendente o descendente de los recursos informáticos disponibles para la instancia de base de datos, tenga en cuenta que la base de datos dejará de estar disponible temporalmente mientras se modifica la clase de la instancia de base de datos. Este período de indisponibilidad suele durar solo unos minutos. Se produce durante el intervalo de mantenimiento de la instancia de base de datos, a menos que especifique que la modificación debe aplicarse inmediatamente.

Si hace un **escalado ascendente o descendente** de la instancia de base de datos, el tamaño de almacenamiento permanece igual y no se ve afectado por el cambio. Puede modificar **por separado** la instancia de base de datos para aumentar el espacio de almacenamiento asignado o mejorar el rendimiento cambiando el tipo de almacenamiento: por ejemplo, de una unidad de estado sólido (SSD) de uso general a una SSD de operaciones de entrada/salida por segundo (IOPS) aprovisionada.

En lugar de aprovisionar manualmente el almacenamiento, también puede utilizar el **escalado automático de almacenamiento de RDS** para escalar automáticamente la capacidad de almacenamiento en respuesta a las crecientes cargas de trabajo de la base de datos.

Además de verticalmente, puede **escalar horizontalmente** la base de datos. RDS utiliza la funcionalidad de replicación integrada de los motores de base de datos MariaDB, MySQL, Oracle y PostgreSQL para crear un tipo especial de instancia de base de datos, llamada **réplica de lectura**, a partir de una instancia de base de datos de origen. Las actualizaciones que se hacen en la instancia de base de datos de origen se copian de forma asíncrona en la réplica de lectura. Dirija las consultas de lectura de las aplicaciones a la réplica de lectura para reducir la carga sobre la instancia de base de datos de origen.

Para **mejorar el rendimiento** de las cargas de trabajo de base de datos de lectura intensiva, puede utilizar réplicas de lectura para escalar horizontalmente la instancia de base de datos de origen. Puede **crear una o más** (hasta cinco) réplicas de una instancia de base de datos de origen determinada y atender el alto volumen de tráfico de lectura de la aplicación desde distintas copias de los datos, lo que aumenta el total del rendimiento de lectura.

Si se produce un desastre, puede aumentar la disponibilidad de la base de datos promoviendo una réplica de lectura a una instancia de base de datos independiente.

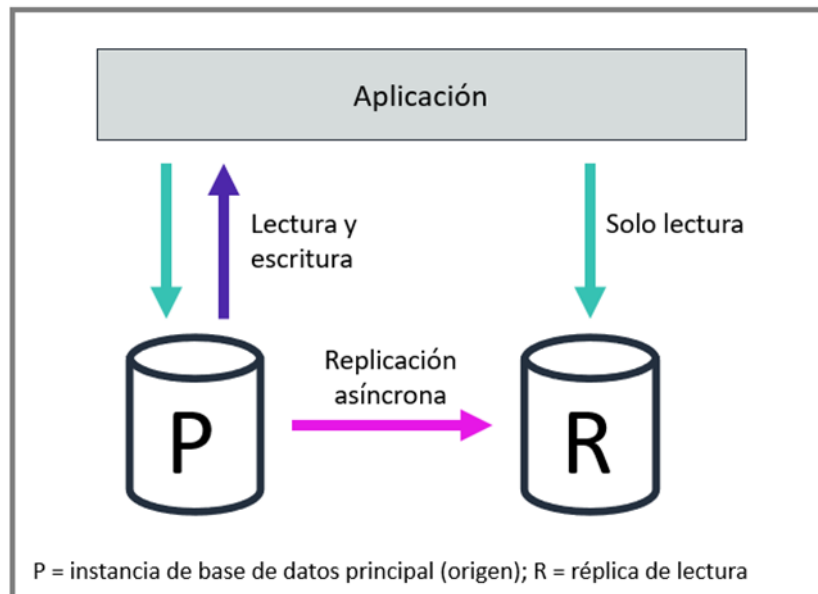


Figura 5. Réplicas de lectura en RDS. Fuente: AWS, 2021.

Sin embargo, en algunos entornos, las **cargas de trabajo** pueden ser **intermitentes** e **impredecibles**. Puede haber períodos de cargas de trabajo intensas que duren desde solo unos minutos hasta horas y períodos largos de actividad reducida o incluso nula.

Ejemplos de ello son los sitios web comerciales en que hay eventos de ventas intermitentes, las bases de datos de generación de informes que los producen cuando se necesitan, los entornos de desarrollo y prueba o las aplicaciones nuevas cuyos requisitos son inciertos. En estos casos y muchos otros, puede ser difícil establecer la capacidad correcta en el momento oportuno. Esta situación puede dar lugar a costos más elevados si hay que pagar una capacidad que no se ha utilizado.

Serverless RDS

Es una configuración de **escalado automático bajo demanda** para alguna base de datos en nube. Permite que con la base de datos se inicie, se apague y se aumente o reduzca la capacidad de forma automática según las necesidades de la aplicación. Le permite ejecutar la base de datos en la nube sin necesidad de administrar instancias

de base de datos. Se puede utilizar para cargas de trabajo poco frecuentes, intermitentes o impredecibles.

La **partición**, también llamada fragmentación horizontal, es un enfoque popular de escalabilidad ascendente para bases de datos relacionales con el cual se mejora el rendimiento de escritura.

La **partición** es una técnica mediante la cual se **dividen datos** en subconjuntos más pequeños y se los distribuye entre una serie de servidores de bases de datos separados físicamente. Cada **servidor** es la partición de una base de datos. Las particiones de bases de datos suelen tener el mismo tipo de hardware, motor de base de datos y estructura de datos para generar un nivel de rendimiento similar. Sin embargo, no tienen ninguna relación entre sí, que es la **característica clave** que diferencia la partición de otros enfoques de escalabilidad ascendente, como la agrupación en clústeres de bases de datos o la replicación.

Una arquitectura de base de datos fragmentada ofrece escalabilidad y tolerancia a errores.

Mediante la partición, los **datos** se pueden distribuir entre tantos servidores de bases de datos como sea necesario. Además, si una partición de base de datos tiene un problema de hardware o una conmutación por error, ninguna otra partición se ve afectada, pues el punto único de error o desaceleración está físicamente **aislado**. El **inconveniente** de este enfoque es que, como los datos se distribuyen entre particiones, la lógica de mapeo y direccionamiento de **datos** debe diseñarse específicamente para leer o unir datos de varias particiones. Estos tipos de consultas pueden suponer una **latencia mayor** en comparación con una base de datos no particionada.

Base de datos NoSQL

Para cargas de trabajo impredecibles que requieran una base de datos no relacional, la opción *serverless* bajo demanda es una opción de **facturación flexible** para atender miles de solicitudes por segundo sin planificar la capacidad. Supone un modelo de precios de **pago por solicitud**, en lugar de un modelo de precios de aprovisionamiento.

Con este tipo de base de datos NoSQL bajo demanda, se puede observar cualquier **aumento o escalado** del nivel de tráfico. Si el **nivel de tráfico** alcanza un nuevo máximo, se adapta rápidamente para recibir la carga de trabajo. Esta característica es útil si la carga de trabajo es difícil de predecir o si tiene picos grandes que duran un período de tiempo corto.

Puede cambiar una tabla de capacidad aprovisionada a capacidad bajo demanda una vez al día. Puede pasar de la capacidad bajo demanda a la capacidad aprovisionada tantas veces como desee.

También tiene una característica denominada escalado automático que está habilitada de forma predeterminada. Con la característica de escalado automático se ajusta automáticamente la capacidad de rendimiento de lectura y escritura en respuesta al cambio dinámico de los volúmenes de solicitudes, sin tiempo de inactividad.

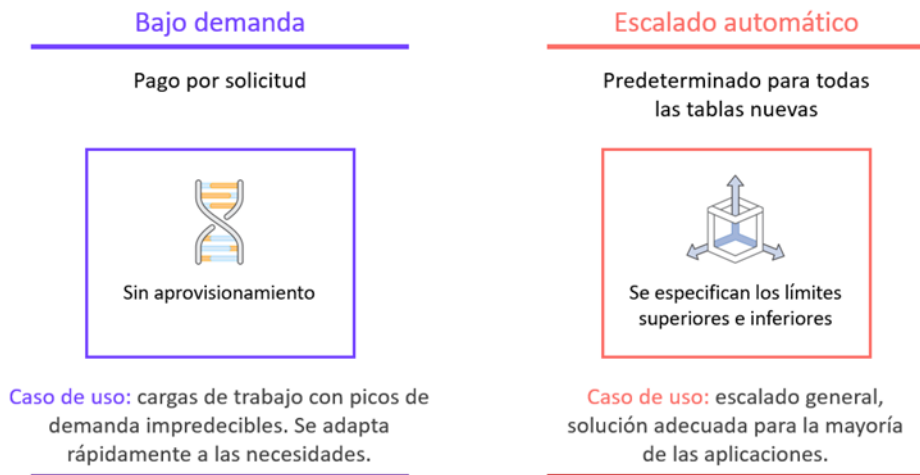


Figura 6. Escalado bajo demanda vs. automático. Fuente: AWS, 2021.

7.4. Entornos de alta disponibilidad

Según afirma Amazon Web Services (2021), es una práctica recomendada **evitar puntos únicos de error** a la hora de diseñar y desarrollar soluciones. Para seguir esta práctica recomendada, se debería diseñar la arquitectura para que tenga disponibilidad alta.

Un sistema de disponibilidad alta es el que puede soportar cierto grado de degradación sin dejar de estar disponible. En un sistema de disponibilidad alta, el tiempo de inactividad se minimiza tanto como es posible y se requiere una intervención humana mínima.

Un sistema de disponibilidad alta permite la **resiliencia** en una **arquitectura reactiva**. Una carga de trabajo resiliente puede recuperarse si está estresada por la carga (más solicitudes de servicio), ataques o errores de componentes. Una carga de trabajo resiliente se recupera de un error o pasa a un origen secundario antes que pase una cantidad aceptable de tiempo de rendimiento.

Elastic Load Balancing (ELB) es un **componente clave** de la creación de una arquitectura de disponibilidad alta. Con ELB se distribuye automáticamente el tráfico de aplicaciones entrante entre varios destinos, tales como máquinas virtuales, contenedores, direcciones IP y funciones serverless. Puede **controlar la carga variable del tráfico** de la aplicación en una o más zonas de disponibilidad.

ELB ofrece **tres tipos** de balanceadores de carga. Los balanceadores de carga pueden ser externos y distribuir tráfico público entrante. También pueden ser internos y distribuir tráfico privado. Cada balanceador de carga recibe un nombre de sistema de **nombres de dominio (DNS)** predeterminado.

Por último, con ELB se distribuye automáticamente el tráfico entrante entre varios destinos en varias zonas de disponibilidad y solo se envía tráfico a destinos que estén en buen estado. Para detectar la disponibilidad de las máquinas virtuales, los balanceadores de carga, de forma periódica, envían *pings*, intentan establecer una conexión o envían solicitudes para probar las máquinas virtuales. Estas pruebas se denominan **comprobaciones de estado**. Todas máquinas virtuales registradas deben responder al objetivo de la comprobación de estado con un **código de estado HTTP 200** para que el balanceador de carga estime que están en buen estado.

ELB ofrece varios **tipos de balanceadores de carga**, que cuentan con la disponibilidad alta, el escalado automático y la seguridad sólida necesarios para que las aplicaciones sean tolerantes a errores.

Veamos los dos **modelos** de balanceadores de carga más habituales:

- El **balanceador de carga de aplicaciones** opera a nivel de la aplicación (capa 7 del modelo de interconexión de sistemas abiertos u OSI). Direcciona el tráfico a los objetivos. ELB ofrece tres tipos de balanceadores de carga, que cuentan con la disponibilidad alta, el escalado automático y la seguridad sólida necesarios para que las aplicaciones sean tolerantes a errores.

- Un **balanceador de carga de red** opera a nivel del transporte de red (capa 4 del modelo de OSI), direccionando conexiones a objetivos como máquinas virtuales, contenedores y direcciones IP. Funciona bien para balancear la carga del tráfico del protocolo de control de transmisión (TCP) y del protocolo de datagramas de usuario (UDP). Con el balanceador de carga de red se pueden manejar millones de solicitudes por segundo y mantener latencias muy bajas. El balanceador de carga de red está optimizado para gestionar patrones de tráfico de red repentinos y volátiles.

Mediante una **interconexión de VPC**, puede acceder a balanceadores de carga **internos** (incluidos los balanceadores de carga clásicos, los balanceadores de carga de aplicaciones y los balanceadores de carga de red) desde otra VPC. Las interconexiones de VPC están disponibles para la conectividad dentro de la región y entre regiones para VPC locales o entre cuentas.

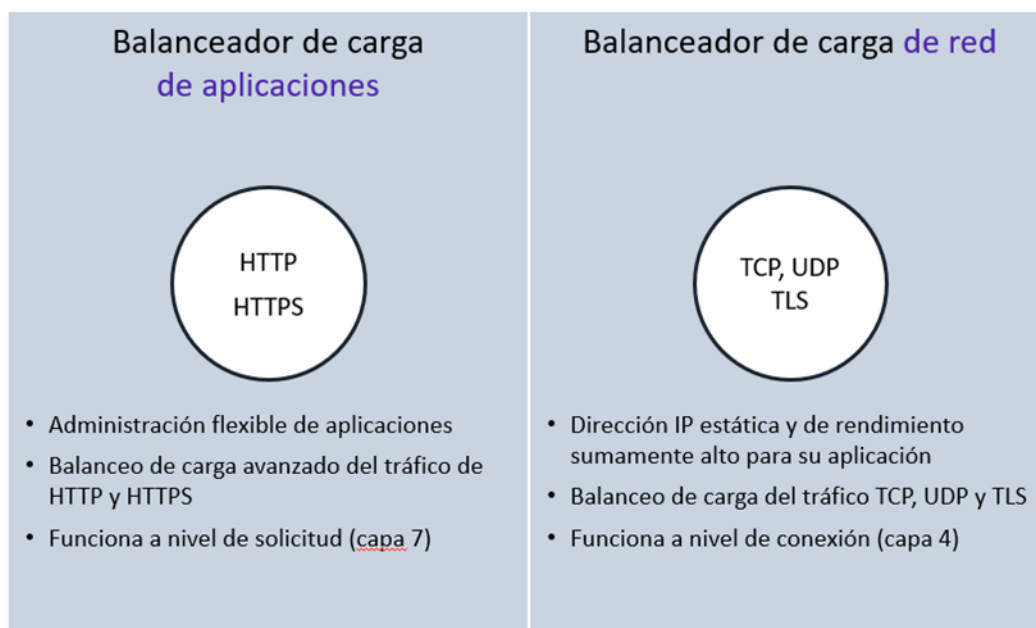


Figura 7. Balanceador de aplicaciones vs carga de red. Fuente: AWS, 2021.

Para **crear una aplicación de alta disponibilidad**, una práctica recomendada es lanzar recursos en varias zonas de disponibilidad y utilizar un balanceador de carga para

distribuir el tráfico entre esos recursos. Ejecutar las aplicaciones en varias zonas de disponibilidad da más disponibilidad si hay un error en un centro de datos.

Veamos la explicación con el siguiente esquema de red:

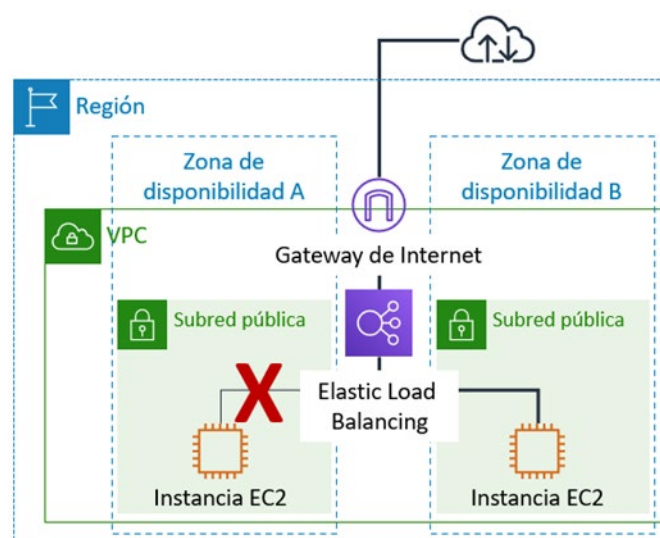


Figura 8. Esquema de red de un balanceador. Fuente: AWS, 2021.

En el patrón básico de aquí, dos servidores web se ejecutan en máquinas virtuales que están en zonas de disponibilidad diferentes. Las instancias están colocadas detrás de un balanceador de carga de **Elastic Load Balancing** que distribuye el tráfico entre ellas. Si un servidor deja de estar disponible, el balanceador de carga está configurado para detener la distribución del tráfico a la instancia que no esté en buen estado y comenzar el direccionamiento del tráfico a la instancia que esté en buen estado. De esta forma, la aplicación sigue estando disponible si hay un error en el centro de datos en una de las zonas de disponibilidad.

La mayoría de las aplicaciones se pueden diseñar para que admitan dos zonas de disponibilidad por región de proveedor de nube.

Si utiliza orígenes de datos que solo admiten conmutaciones por error principales o secundarias, es posible que a la aplicación no le resulte útil utilizar más zonas de

disponibilidad. Las zonas de disponibilidad están distribuidas de forma física, así que tampoco obtendrá demasiados beneficios si duplica los recursos en tres o más zonas de una región.

Al igual que la arquitectura que acaba de analizar, las **máquinas virtuales** están colocadas **detrás de un balanceador de carga**, que distribuye el tráfico público entrante entre ellas. Si uno de los servidores deja de estar disponible, el balanceador de carga está configurado para detener la **distribución** del tráfico a las instancias que no estén en buen estado y comenzar el **direccionamiento** del tráfico a las instancias que estén en buen estado.

Puede incorporar un segundo balanceador de carga en la arquitectura para direccionar el tráfico entrante desde las instancias de las subredes públicas hasta las instancias de las subredes privadas.

Si tiene recursos en varias zonas de disponibilidad que comparten una **gateway NAT** y la zona de disponibilidad de la gateway NAT no funciona, los recursos de las demás zonas de disponibilidad perderán el acceso a Internet. Es una práctica recomendada tener gateways NAT en ambas zonas de disponibilidad para velar por una disponibilidad alta.

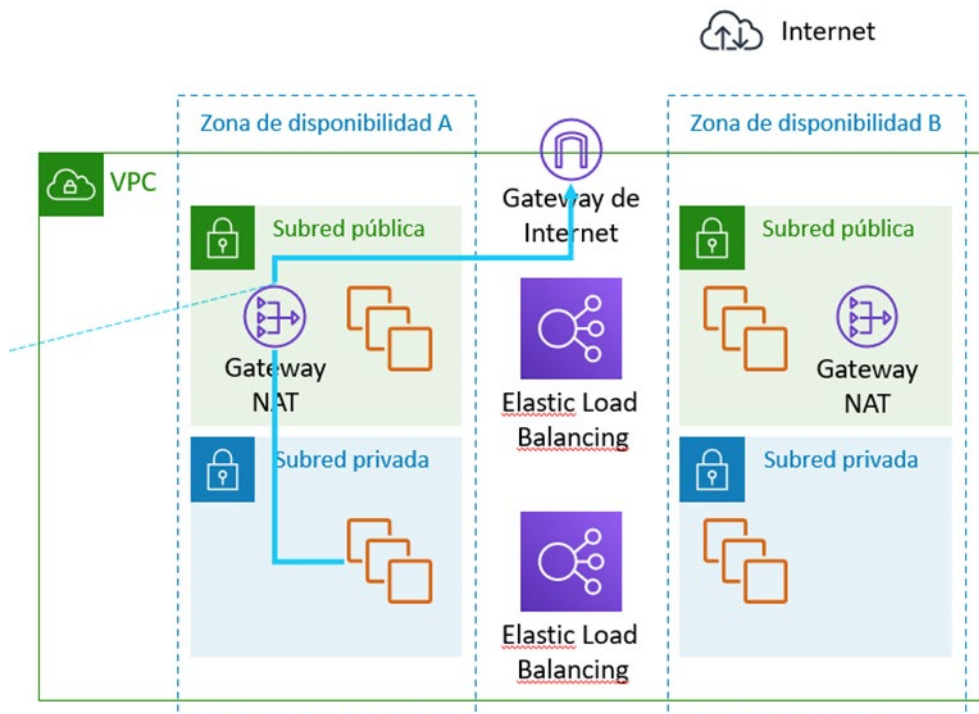


Figura 9. Esquema de red con ELB. Fuente: AWS, 2021.

Alta disponibilidad con DNS

También puede implementar la disponibilidad alta y la tolerancia a errores en varias regiones en la arquitectura de red mediante los servicios de DNS.

Los servicios de DNS son un servicio escalable y de disponibilidad alta en la nube. Este servicio está diseñado para ser un método económico y de confianza para redirigir a los usuarios a aplicaciones de Internet.

Traduce nombres (como ejemplo.com) a las direcciones IP numéricas (como 192.0.2.1) que los equipos utilizan para conectarse entre sí.

Conecta eficazmente las **solicitudes** de los usuarios a la infraestructura que se ejecuta en la nube, como máquinas virtuales, balanceadores de carga ELB o almacenamiento de objeto.

Puede utilizar DNS para **configurar comprobaciones de estado** de DNS a fin de direccionar el tráfico a puntos de enlace en buen estado o monitorear de manera independiente el estado de la aplicación y los puntos de enlace.

Servicios DNS también ofrece el **registro de nombres de dominio**. Puede adquirir y administrar nombres de dominio, como ejemplo.com, y DNS configurará automáticamente los valores de DNS para los dominios.

Los servicios de DNS admiten varios **tipos de políticas de direccionamiento**, las cuales determinan cómo se responde a las consultas:

- ▶ **Direccionamiento simple (turno rotativo):** distribuye la cantidad de solicitudes de la manera más uniforme posible entre todos los servidores participantes.
- ▶ **Direccionamiento de turno rotativo ponderado:** permite asignar ponderaciones a conjuntos de registros de recursos para especificar la frecuencia con la que se atienden las diferentes respuestas. Es posible que desee utilizar esta capacidad para hacer pruebas A/B, en las que envía una pequeña porción del tráfico a un servidor en el que hizo un cambio de software. Por ejemplo, supongamos que cuenta con dos conjuntos de registros asociados a un mismo nombre de DNS: uno con ponderación 3 y el otro con ponderación 1. En este caso, el 75 % del tiempo, DNS devolverá el conjunto de registros que tenga la ponderación 3, y el 25 % de las veces, lo hará con el conjunto de registros que tenga la ponderación 1. Las ponderaciones pueden oscilar entre 0 y 255.
- ▶ **Direccionamiento basado en la latencia (LBR):** se utiliza si tiene recursos en varias regiones y desea direccionar el tráfico a la región que ofrezca la mejor latencia. El direccionamiento basado en la latencia se encarga de direccionar a los clientes al punto de enlace de AWS (por ejemplo, máquinas virtuales, direcciones IP elásticas o balanceadores de carga) que ofrece la experiencia más rápida en función de las mediciones de rendimiento real de las distintas regiones donde se ejecuta la aplicación.

- ▶ **Direccionamiento de geolocalización:** permite elegir los recursos que atienden al tráfico en función de la ubicación geográfica de los usuarios (el origen de las consultas de DNS). Si utiliza el direccionamiento de geolocalización, puede localizar el contenido de su sitio web y presentarlo de forma parcial o total en el idioma de sus usuarios. También puede usar el direccionamiento basado en geolocalización para limitar la distribución de contenido a solo las ubicaciones para las que tenga derechos de distribución. Otro uso que se le puede dar es relativo al balanceo de carga entre puntos de enlace de una forma predecible y fácil de administrar para que la ubicación de cada usuario se dirija sistemáticamente al mismo punto de enlace.
- ▶ **Direccionamiento de geoproximidad:** le permite direccionar el tráfico en función de la distancia física que haya entre los usuarios y los recursos si está utilizando DNS Traffic Flow. También puede direccionar más o menos tráfico a cada recurso mediante la especificación de una tendencia positiva o negativa. Si crea una política de flujo de tráfico, puede especificar una región, si utiliza recursos de nube o la latitud y longitud de cada punto de enlace.
- ▶ **Direccionamiento tras conmutación por error (a nivel de DNS):** se utiliza si desea configurar la conmutación por error activa-pasiva. DNS lo ayuda a detectar interrupciones en su sitio web y a redirigir a los usuarios hacia ubicaciones alternativas donde la aplicación funcione de manera adecuada. Si habilita esta característica, los agentes de comprobación de estado de DNS monitorearán cada ubicación o punto de enlace de la aplicación para determinar su disponibilidad. Puede aprovechar esta característica para aumentar la disponibilidad de la aplicación orientada a los clientes.

- **Direccionamiento de respuesta con varios valores:** utilícelo si desea direccionar el tráfico de forma aproximadamente aleatoria a varios recursos, como servidores web. Puede crear un registro de respuesta con varios valores para cada recurso. También puede asociar opcionalmente una comprobación de estado de DNS con cada registro. Por ejemplo, imaginemos que administra un servicio web HTTP con doce servidores web, cada uno de los cuales tiene su propia dirección IP. Ningún servidor web podía manejar solo todo el tráfico. Sin embargo, si crea una docena de registros de respuesta con varios valores, DNS responde a consultas de DNS con hasta ocho registros en buen estado en respuesta a cada consulta de DNS. DNS da diferentes respuestas a distintos solucionadores de DNS. Si un servidor web deja de estar disponible después de que un solucionador almacena una respuesta en la memoria caché, el software de cliente puede intentar utilizar otra dirección IP en la respuesta.

Con la **conmutación por error a nivel de DNS**, puede ayudarlo a detectar interrupciones en su sitio web y a redireccionar a los usuarios a ubicaciones alternativas donde la aplicación funcione adecuadamente. Si habilita esta característica, los agentes de comprobación de estado de DNS **monitorearán cada ubicación** o punto de enlace de la aplicación para determinar su disponibilidad. Puede aprovechar esta característica para **aumentar la disponibilidad** de la aplicación orientada a los clientes.

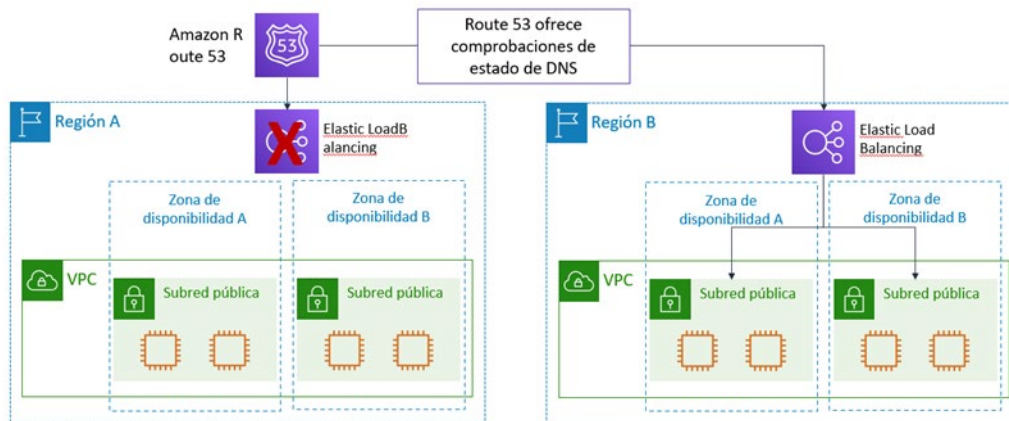


Figura 10. Disponibilidad alta y DNS en varias regiones. Fuente: AWS, 2021.

7.5. Monitorización y observabilidad

Según afirma Amazon Web Services (2021), el monitoreo y la observabilidad son unos componentes esenciales de una arquitectura reactiva.

El **monitoreo** puede ayudarlo a realizar lo siguiente:

- ▶ Hacer un seguimiento del funcionamiento y el rendimiento de sus recursos.
- ▶ Hacer un seguimiento de la utilización de recursos y del rendimiento de las aplicaciones para asegurarse de que la infraestructura esté satisfaciendo la demanda.
- ▶ Decidir qué permisos establecer para los recursos de AWS a fin de alcanzar los objetivos de seguridad que desee.

El monitoreo también puede ayudarlo a comprender y administrar el costo de la infraestructura de nube.

Los proveedores de nube ofrecen **herramientas de monitoreo y generación de informes**, entre las que se incluyen las siguientes:

- ▶ **Cost Explorer** lo ayuda a visualizar, comprender y administrar los **costos** y el **uso** que hace de nube con granularidad diaria o mensual. Puede utilizar esta solución para ver datos de hasta hace trece meses, lo que le permite ver patrones de cómo gasta los recursos de nube a lo largo del tiempo.
- ▶ **Budgets** le permite definir **presupuestos personalizados** que generarán una alerta si los costos o el uso rebasan, o si se prevé que rebasarán, el importe presupuestado.
- ▶ **El informe de uso y costo** obtiene el conjunto más completo de **datos** de uso y costo de nube disponibles e incluye **metadatos** adicionales sobre los servicios, los precios y las reservas.
- ▶ **Cost Optimization Monitor** es una arquitectura de soluciones con que se procesan automáticamente **informes de facturación** detallados para dar métricas granulares que puede buscar, analizar y visualizar en un panel que puede personalizar. La solución le da información sobre el uso y el costo del servicio, que puede desglosar por período, cuenta, recurso o etiquetas.

CloudWatch es un servicio de **monitoreo y observación** creado para ingenieros de DevOps, desarrolladores, ingenieros de fiabilidad de sitios y administradores de TI. CloudWatch ofrece **datos e información** procesable para monitorear las aplicaciones, responder a cambios de rendimiento que afectan a todo el sistema, optimizar el uso de recursos y lograr una vista unificada del estado de las operaciones.

Puede utilizar CloudWatch para recopilar y hacer un seguimiento de las **métricas**, que son variables con que puede analizar los recursos y las aplicaciones. Puede crear alarmas de CloudWatch para la observación de métricas y el envío de notificaciones.

Además, si se rebasa un límite, CloudWatch puede cambiar automáticamente los recursos que esté monitoreando.

Por ejemplo, usted puede monitorear el uso de la CPU y las lecturas y las escrituras en disco de las máquinas virtuales. A continuación, puede utilizar estos datos para determinar si debe lanzar instancias adicionales para manejar el aumento de la carga. También puede utilizar estos datos para detener instancias infrautilizadas con el objetivo de ahorrar dinero.

Además de monitorear las métricas integradas que se incluyen con el proveedor de nube, puede monitorear sus propias métricas personalizadas. Con CloudWatch, obtiene **visibilidad de todo el sistema** respecto de la utilización de los recursos, el rendimiento de las aplicaciones y el estado operativo.



Figura 10. Elementos de la monitorización en la nube. Fuente: AWS, 2021.

Las métricas son datos sobre el rendimiento de los sistemas.

Muchos servicios ofrecen métricas para los recursos de forma predeterminada, como máquinas virtuales, volúmenes de Elastic Block Store y máquinas virtuales, de base de datos de RDS. También puede habilitar el monitoreo detallado de algunos recursos, como las máquinas virtuales, o publicar sus propias métricas de aplicación. CloudWatch puede **cargar todas las métricas en su cuenta** (las métricas de recursos

de nube y las métricas de aplicación que proporcione) para búsquedas, representaciones gráficas y alarmas.

Puede utilizar **CloudWatch Logs** para monitorear y almacenar los **archivos de registro de orígenes** como máquinas virtuales, API de seguridad, DNS y otros servicios de nube, además de acceder a ellos.

Por ejemplo, puede utilizar CloudWatch Logs para monitorear aplicaciones y sistemas mediante **datos de registros**. Con CloudWatch Logs se puede hacer un **seguimiento** de la cantidad de errores que se producen en los registros de aplicaciones. Envía una notificación si la tasa de error rebasa un límite especificado.

Además, puede utilizar **CloudWatch Logs Insights** para **analizar los registros** en cuestión de segundos. Le ofrece consultas y visualizaciones rápidas e interactivas. Puede visualizar los resultados de las consultas mediante gráficos de líneas o de área apilada y agregarlas a un panel de CloudWatch.

Puede utilizar una **alarma** para iniciar automáticamente acciones en su nombre. Una alarma **vigila una métrica individual** durante un período que usted ha determinado. Realiza una o más acciones especificadas en función del valor de la métrica relativa a un límite a lo largo del tiempo. La acción es una notificación que se envía por ejemplo a una política de Auto Scaling. También puede agregar alarmas a paneles.

Las alarmas invocan acciones únicamente para los **cambios de estado prolongados**. Las alarmas de CloudWatch no invocan acciones simplemente porque estén en un cierto estado. El estado debe haber cambiado y debe mantenerse durante el número de períodos especificado.

Un **evento** indica un **cambio en el entorno**. Puede ser un entorno de nube, un servicio o aplicación de socios SaaS o una de sus propias aplicaciones o servicios personalizados. Por ejemplo, en las máquinas virtuales. Genera un evento si el estado de una máquina virtual cambia de pendiente a ejecución y Auto Scaling genera

eventos cuando lanza o termina instancias. Se publica eventos cuando usted hace llamadas a la API de los servicios de nube. También puede programar eventos que se generen de forma periódica.

Puede establecer **reglas de direccionamiento** para determinar el lugar al que se enviarán los datos con el objetivo de crear arquitecturas de aplicaciones que reaccionen en tiempo real a todos los orígenes de datos.

Una regla **empareja eventos** entrantes y los direcciona a destinos para su procesamiento. Con una única regla se puede efectuar el direccionamiento hacia varios destinos, los cuales se procesan en paralelo. Las reglas no se procesan en un orden concreto. Ello permite a las distintas partes de una organización buscar y procesar los eventos que les interesan. Con una regla se puede **personalizar** la notación de objetos JavaScript (JSON) que se envía al destino pasando solo ciertas partes o sobrescribiéndolo con una constante.

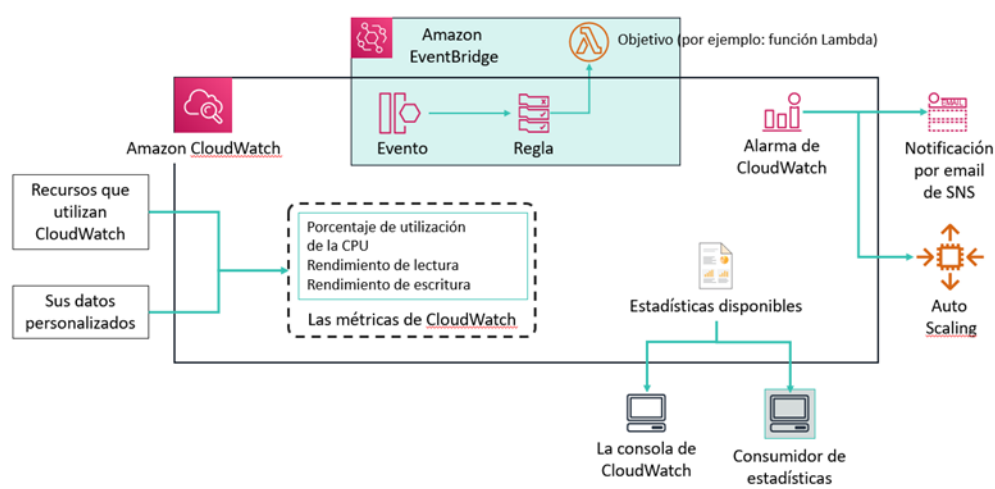


Figura 11. Arquitectura de monitorización y eventos. Fuente: AWS, 2021.

¿Qué es la observabilidad?

«¿Mi sistema funciona o no?» «¿Es rápido o lento según la experiencia de mis usuarios finales?» «¿Qué KPI y SLA debemos establecer y cómo sabemos si se cumplen?» Cuando se opera a la velocidad y escala de la nube, no se puede permitir volar a ciegas: hay que ser capaz de responder a una amplia gama de **preguntas operativas y empresariales** como éstas. Tiene que ser capaz de **detectar** los problemas a medida que surgen (idealmente antes de que interrumpen la experiencia del cliente), **responder** rápidamente y **resolverlos** lo antes posible. Para obtener esta información, necesita sistemas observables.

Monitoreo y observabilidad

La «observabilidad» describe lo bien que se puede entender lo que ocurre en un sistema, a menudo, mediante instrumentos para recopilar métricas, registros o rastreos. En la nube, la observabilidad puede volverse difícil de conseguir debido a la complejidad del sistema. Ya sea en los centros de datos o en la nube, para lograr excelencia operacional y cumplir con los objetivos del negocio, debe comprender cómo se desempeñan sus sistemas. Las soluciones de la observabilidad permiten **recopilar y analizar** los datos de las aplicaciones y la infraestructura para que pueda **comprender sus estados internos** y recibir alertas a fin de solucionar y resolver problemas con la disponibilidad y el rendimiento de la aplicación para mejorar la experiencia del usuario final.

¿Cuál es la diferencia entre la observabilidad y el monitoreo?

Aunque a veces el término «monitoreo» se define como algo diferente a la observabilidad, el monitoreo es una actividad que hace que el **sistema** sea **observable**, junto con otras actividades como el rastreo y el registro.

Generalmente, verá que se describe al monitoreo, el rastreo y el registro como los «tres pilares de la observabilidad». Sin embargo, existen otras herramientas para ayudarlo a lograr la observabilidad, como generadores de perfiles e IA/Operaciones, que se analizan a continuación.

En el siguiente vídeo *Monitorización de servicios en Cloud* aprenderemos a llevar a cabo la observabilidad de nuestras infraestructuras de red. Esto es algo más que la monitorización tradicional y aprenderemos a gestionar las alertas, métricas, *logs*, trazas, segmentos de nuestros servicios construidos.



Accede al vídeo

A modo de resumen, en este tema hemos aprendido:

- ▶ Utilizar Auto Scaling dentro de una arquitectura para fomentar la elasticidad.
- ▶ Explicar cómo escalar los recursos de las bases de datos.
- ▶ Implementar un balanceador de carga de aplicaciones para crear un entorno de alta disponibilidad.
- ▶ Utilizar DNS para la conmutación por error a nivel de DNS.
- ▶ Crear un entorno de alta disponibilidad.
- ▶ Diseñar arquitecturas que utilicen CloudWatch para monitorear los recursos y reaccionar en consecuencia.

7.6. Referencias bibliográficas

Amazon Web Services. (2022). *Use Elastic Load Balancing to distribute traffic across the instances in your Auto Scaling group.*
<https://docs.aws.amazon.com/autoscaling/ec2/userguide/autoscaling-load-balancer.html>

AWS. (2021, diciembre). *Training Track: Cloud Architecting*. Manuscrito no publicado.

Google Cloud. (2022). *Productos de redes de Google Cloud.*
<https://cloud.google.com/products/networking>

Microsoft. (2022). *Sistema de nombres de dominio (DNS).*
<https://docs.microsoft.com/es-es/windows-server/networking/dns/dns-top>

File Cloud Blog

FileCloud (2022). An Introduction to High Availability Architecture. *FileCloud Blog*. <https://www.filecloud.com/blog/an-introduction-to-high-availability-architecture/>

Centro de recursos para saber más sobre alta disponibilidad.

BlueCat

BlueCat. (2021, octubre 21). Comparing AWS, Azure, and GCP cloud DNS services. *Blue Cat Blog*. <https://bluecatnetworks.com/blog/comparing-aws-azure-and-gcp-cloud-dns-services/>

Son muchas las opciones y configuraciones que tienen los servidores de nombres de dominio (DNS). A través de este recurso, podemos ampliar la información de este servicio de los tres proveedores de nube principales.

CloudAcademy

Dzenan Dzevlan. (2019, noviembre 20). Application Load Balancer vs. Classic Load Balancer. *Cloud Academy Blog*. <https://cloudacademy.com/blog/application-load-balancer-vs-classic-load-balancer/>

Podemos consultar todos los datos sobre balanceadores y grupos de auto escalado en la web de temas técnicos para nube.

1. Una característica de una arquitectura reactiva es la elasticidad. La elasticidad significa:

A. Que la infraestructura puede ampliarse.

B. Que la infraestructura puede contraerse.

_ C. Que la infraestructura puede ampliarse o contraerse si cambian las necesidades de capacidad.

D. Ninguna de las anteriores.

Una característica de una arquitectura reactiva es la elasticidad. La elasticidad significa que la infraestructura puede ampliarse o contraerse si cambian las necesidades de capacidad.

2. El escalado es una técnica que se utiliza para lograr elasticidad. El escalado es la capacidad de:

A. Reducir la capacidad de cómputo de una aplicación.

_ B. Aumentar o reducir la capacidad de cómputo de una aplicación.

C. Aumentar la capacidad de cómputo de una aplicación.

D. Cambiar la oferta a la demanda siempre que quieras.

El escalado es una técnica que se utiliza para lograr elasticidad. El escalado es la capacidad de aumentar o reducir la capacidad de cómputo de una aplicación.

3. El Auto Scaling le permite crear arquitecturas de disponibilidad alta que abarcan:

_ A. Varias zonas de disponibilidad en una región.

B. Varias zonas de disponibilidad en varias regiones.

C. Varias regiones y una zona de disponibilidad

D. Varias zonas de disponibilidad en varios proveedores de nube.

El Auto Scaling le permite crear arquitecturas de disponibilidad alta que abarcan varias zonas de disponibilidad en una región. Si una zona de disponibilidad pasa a estar en mal estado o no está disponible.

4. El escalado dinámico significa ajustar la capacidad de su aplicación para afrontar cambios:

- ☐ A. En la demanda a fin de poder optimizar la disponibilidad, el rendimiento y los costos.
- ☐ B. En la demanda a fin de poder optimizar el rendimiento.
- ☐ C. En la demanda a fin de poder optimizar los costos.
- ☐ D. Ninguna de las anteriores.

El escalado dinámico significa ajustar la capacidad de su aplicación para afrontar cambios en la demanda a fin de poder optimizar la disponibilidad, el rendimiento y los costos.

5. Si hace un escalado ascendente o descendente de la instancia de base de datos, el tamaño de almacenamiento permanece:

- ☐ A. Igual en escalados asimétricos.
- ☐ B. Diferente si se hace un escalado descendente.
- ☐ C. Diferente si se hace un escalado ascendente.
- ☐ D. Igual y no se ve afectado por el cambio.

Si hace un escalado ascendente o descendente de la instancia de base de datos, el tamaño de almacenamiento permanece igual y no se ve afectado por el cambio.

6. Con ELB se distribuye automáticamente el tráfico de:

- ☐ A. Aplicaciones salientes entre varios destinos.
- ☐ B. Aplicaciones entrantes entre varios destinos.
- ☐ C. Máquinas virtuales entrantes entre varios destinos.
- ☐ D. Ninguna de las anteriores.

Con ELB se distribuye automáticamente el tráfico de aplicaciones entrante entre varios destinos, tales como máquinas virtuales, contenedores, direcciones IP y funciones serverless. Puede controlar la carga variable del tráfico de la aplicación en una o más zonas de disponibilidad.

7. Los servicios de DNS son un servicio escalable y:

- A. De baja latencia en la nube.
- B. De balanceo en la nube.
- _ C. De disponibilidad alta en la nube.
- D. Solo para empresas en data center locales.

Los servicios de DNS son un servicio escalable y de disponibilidad alta en la nube. Este servicio está diseñado para ser un método económico y de confianza para redirigir a los usuarios a aplicaciones de Internet.

8. Los servicios de DNS admiten varios tipos de políticas de direccionamiento. ¿Cuál es no correcta?

- A. Geoproximidad.
- B. Basado en latencia.
- C. Simple.
- _ D. Doble.

Direccionamiento simple, Direccionamiento de turno rotativo ponderado, Direccionamiento basado en la latencia (LBR), Direccionamiento de geolocalización, Direccionamiento de geoproximidad, Direccionamiento tras conmutación por error.

9. Los proveedores de nube ofrecen herramientas de monitoreo y generación de informes. ¿Cuál no una de ellas?

- A. Cost Explorer.
- _ B. Facturación múltiple.
- C. Budgets.
- D. Informes de uso y costo.

Cost explorer, Budgets, Informes de uso y costo y Cost optimization.

10. ¿Cuál no es un elemento de monitorización en nube?

- A. Métricas.
- B. Eventos.
- C. Alarmas.
- _ D. Funciones como servicio.

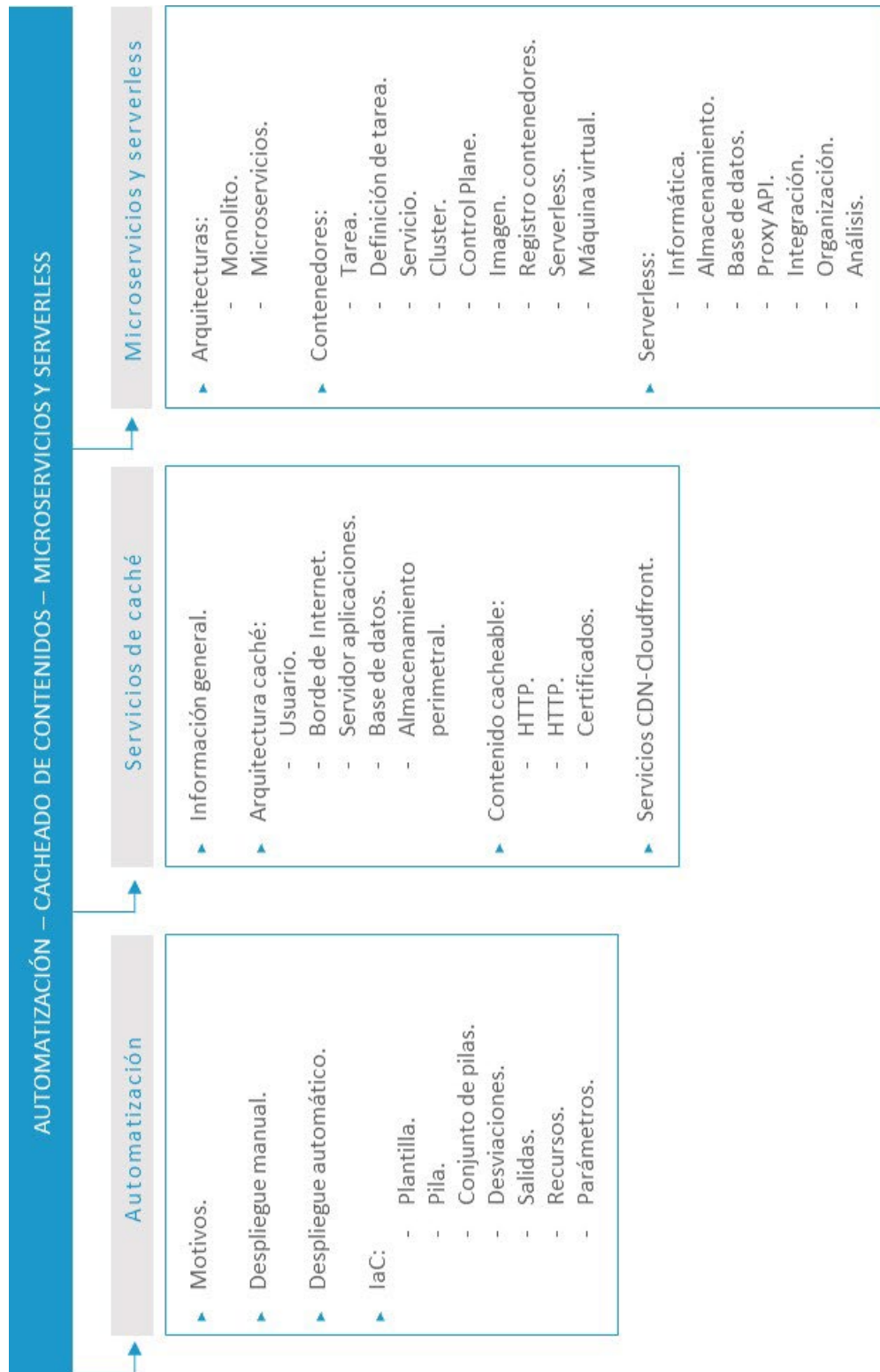
Eventos, Reglas, Objetivos, Métricas, Registros y alarmas.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Automatización, contenidos, microservicios y *serverless*

Índice

Esquema	3
Ideas clave	4
8.1. Introducción y objetivos	4
8.2. Motivos para automatizar	5
8.3. Automatización de la arquitectura	8
8.4. Servicios de caché	14
8.5. Microservicios y arquitecturas sin servidor	25
8.6. Referencias bibliográficas	40
A fondo	41
Test	42



8.1. Introducción y objetivos

Hoy en día, todo es código. Hemos llegado al punto que un «*data center*» está definido por código en los proveedores de nube pública.

Vamos a aprender a **definir recursos de nube** a través de manifiesto en **ficheros** de texto plano y ver como se generan **plantillas** para tener todas nuestras infraestructuras definidas por código.

Aprenderemos también cómo funciona el **caché de contenidos** en la nube y, finalmente, haremos una introducción a los **microservicios** en la nube y las arquitecturas sin servidor.

Con el estudio de este tema pretendemos alcanzar los siguientes **objetivos**:

Reconocer cuándo se debe implementar automatización y por qué.

Identificar cómo modelar, crear y administrar una recopilación de recursos de en nube.

Utilizar plantillas para configurar una arquitectura.

Reconocer cómo el almacenamiento de contenido en caché puede mejorar el rendimiento de las aplicaciones y reducir la latencia.

Indicar las características de los microservicios.

Refactorizar una aplicación monolítica en microservicios y utilizar contenedores para implementar los microservicios.

Explicar la arquitectura sin servidor.

En los vídeos de este tema podremos aprender los siguientes conceptos:

Vídeo 1. Entenderemos la infraestructura como código (IaC) y desplegaremos algunos ejemplos de infraestructuras definidas en un manifiesto en fichero YAML.

Vídeo 2. Publicaremos una página web estática y la distribuiremos en una CDN. También registraremos la web en un servicio DNS.

8.2. Motivos para automatizar

Según afirma Amazon Web Services (2022), crear un entorno informático a gran escala requiere mucho tiempo y energía.

Muchas organizaciones comienzan a utilizar la nube creando manualmente un almacenamiento o iniciando una máquina virtual y ejecutando un servidor web en ella. Luego, con el paso del tiempo, agregan manualmente más recursos a medida que descubren que ampliar su uso en la nube puede satisfacer **necesidades empresariales adicionales**. Sin embargo, pronto puede resultar difícil administrar y mantener manualmente estos recursos.

Estas son algunas de las **preguntas** que debe plantear:

¿Dónde quiere esforzarse? ¿En el diseño o en la implementación? ¿Cuáles son los riesgos de las implementaciones manuales?

¿Cuál sería la forma ideal de actualizar los servidores de producción? ¿Cómo desplegará implementaciones en varias regiones geográficas? Cuando algo falle (y algo siempre falla) ¿cómo planea administrar la restauración a la última versión válida conocida?

¿Cómo depurará las implementaciones? ¿Puede corregir los errores en su aplicación antes de implementar la implementación con los clientes? ¿Cómo descubrirá lo que está mal y cómo lo arreglará para que permanezca arreglado?

¿Cómo administrará las dependencias de los distintos sistemas y subsistemas de su organización?

Por último, ¿es realista que pueda realizar todas estas tareas a través de configuraciones manuales?



Figura 1. Despliegue manual de recursos en nube. Fuente: AWS, 2021.

Crear recursos y agregar nuevas características y funcionalidades manualmente a su entorno no es escalable. Si usted es responsable de una aplicación corporativa grande, podría no tener suficientes personas para administrar todo manualmente.

La creación de arquitectura y aplicaciones desde cero no tiene un control de versiones inherente.

En caso de emergencia, es útil poder restaurar la pila de producción a una versión anterior, pero eso no es posible cuando crea el entorno manualmente. Tener un **registro de seguimiento de auditoría** es muy importante para muchas situaciones de conformidad y seguridad. Es peligroso permitir que cualquier persona de su organización controle y edite manualmente sus entornos.

Por último, la **coherencia** es fundamental cuando se desea minimizar los riesgos. La automatización le permite mantener la coherencia.

También debe tener en cuenta qué tan bien los enfoques manuales cumplen con los principios de diseño de Well-Architected Framework.

Uno de los seis principios de diseño de excelencia operativa es realizar **operaciones como código**. En la nube, puede aplicar la misma disciplina de ingeniería que utiliza para el código de aplicaciones en todo el entorno. Puede definir toda su carga de trabajo (aplicaciones, infraestructura y otros recursos) como código y actualizarla con código. Puede utilizar *scripts* para sus procedimientos de operaciones y automatizar su ejecución mediante desencadenadores en respuesta a eventos. Cuando las operaciones se realizan como código, se **limita el error humano** y se producen **respuestas coherentes** a los eventos.

Otro principio de diseño de excelencia operativa es realizar cambios frecuentes, pequeños y reversibles. Es decir, diseñar **cargas de trabajo** para habilitar actualizaciones regulares de los componentes para poder aumentar el flujo de cambios beneficiosos hacia su carga de trabajo. Realice cambios en pequeños incrementos, que puedan revertirse si no lo ayudan a identificar y resolver los problemas que se introdujeron en su entorno. Cuando sea posible, intente no afectar a los clientes cuando realice estos cambios.

Por último, uno de los principios de diseño del pilar de **fiabilidad** de buena arquitectura es administrar los cambios en la **automatización**. Los cambios en su infraestructura deben realizarse a través de la automatización. Por lo tanto, los cambios que deben administrarse son cambios en la automatización.

Hacer cambios en los sistemas de producción es una de las áreas de riesgo más grandes para muchas organizaciones. En las operaciones, utilice la automatización donde sea práctico, como para probar e implementar cambios, agregar o eliminar capacidad y migrar datos.

8.3. Automatización de la arquitectura

Según afirma Amazon Web Services (2021), **CloudFormation** aprovisiona recursos de forma repetible. Le permite crear y reconstruir su infraestructura y aplicaciones sin necesidad de realizar acciones manuales o escribir *scripts* personalizados. Con CloudFormation, creará un documento que describe cuál debe ser su **infraestructura**, incluidos todos los **recursos de nube** que deberían formar parte de la implementación. Piense en este documento como un **modelo**.

Cuando utiliza CloudFormation para crear recursos, esto se conoce como una pila de CloudFormation. Puede crear una pila, actualizar o eliminar una pila. Por lo tanto, puede aprovisionar recursos de manera ordenada y predecible.

El uso de CloudFormation le permite tratar a **su infraestructura como código** (Infrastructure as Code, IaC). Puede utilizar cualquier **editor de código** para escribirla, insertarla en un sistema de control de versiones, como GitHub o CodeCommit, y revisar los archivos con los miembros del equipo antes de implementarlos en los entornos correspondientes.

Si el documento de CloudFormation que crea para modelar su implementación ya está registrado en un sistema de control de versiones, siempre puede eliminar una pila, retirar una versión anterior del documento y crear una pila a partir de dicha versión. Con el **control de versiones**, puede utilizar las capacidades esenciales de restauración.

En la Figura 2 se muestra cómo funciona CloudFormation.

Primero, define los **recursos de AWS que desea crear**. En el ejemplo que se muestra aquí, crea algunas máquinas virtuales, un balanceador de carga, un grupo de Auto Scaling y una zona alojada en DNS. Los recursos se definen en una plantilla de

CloudFormation. Puede crear la plantilla desde cero o utilizar una plantilla prediseñada.

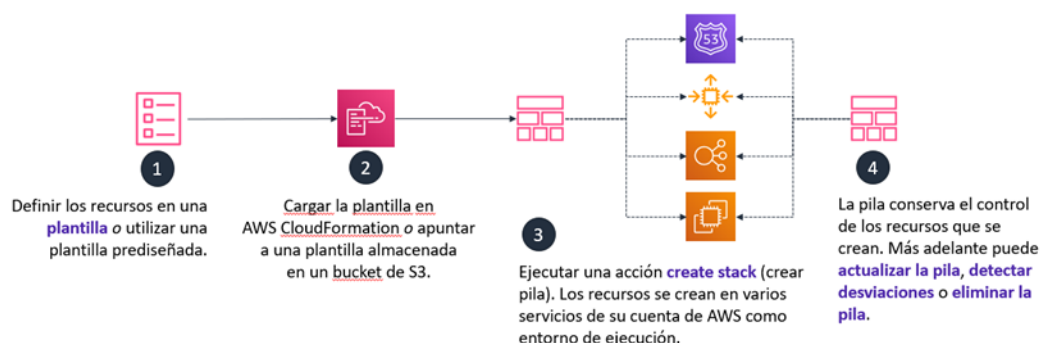


Figura 2. Funcionamiento de infraestructura como código. Fuente: AWS, 2021.

Luego, debe **cargar la plantilla** en CloudFormation. También puede almacenar la plantilla en un almacenamiento e indicarle a CloudFormation la ubicación donde está almacenada.

En tercer lugar, debe **ejecutar la acción create stack** (crear pila). Cuando realice esta acción, el servicio CloudFormation lee lo que se especifica en la plantilla y crea los recursos deseados en su cuenta. Una única pila puede crear y configurar recursos en una única región a través de varios servicios.

Por último, puede **observar el progreso del proceso** de creación de la pila. Una vez que la pila se haya creado correctamente, los recursos que creó existirán en su cuenta. El objeto de la pila permanece y actúa como un timón para todos los recursos que creó. Esto es útil cuando desea realizar acciones posteriores. Por ejemplo, es posible que desee actualizar la pila (para crear recursos adicionales o modificar recursos existentes) o eliminar la pila (lo que limpiará y eliminará los recursos creados por la pila).

Infraestructure as Code (IaC) es un término de la industria que se refiere al proceso de aprovisionar y administrar recursos en la nube definiéndolos en un

archivo de plantilla que es legible por humanos y consumible por las máquinas.

La popularidad de la **laC** continúa creciendo porque proporciona una **solución viable** para desafíos como cómo replicar, volver a implementar y reutilizar la infraestructura de manera fácil, confiable y coherente.

Ahora, considere algunos de los beneficios de la laC con más detalle. Si construye una infraestructura con código, obtendrá beneficios como la capacidad de **implementar entornos complejos rápidamente**. Con solo una plantilla (o una combinación de plantillas), puede replicar entornos complejos una y otra vez.

En la Figura 3, una sola plantilla se puede utilizar para crear tres pilas diferentes. Cada pila se puede crear con rapidez, generalmente en cuestión de minutos. Cada pila replica los detalles de configuración compleja de forma coherente.

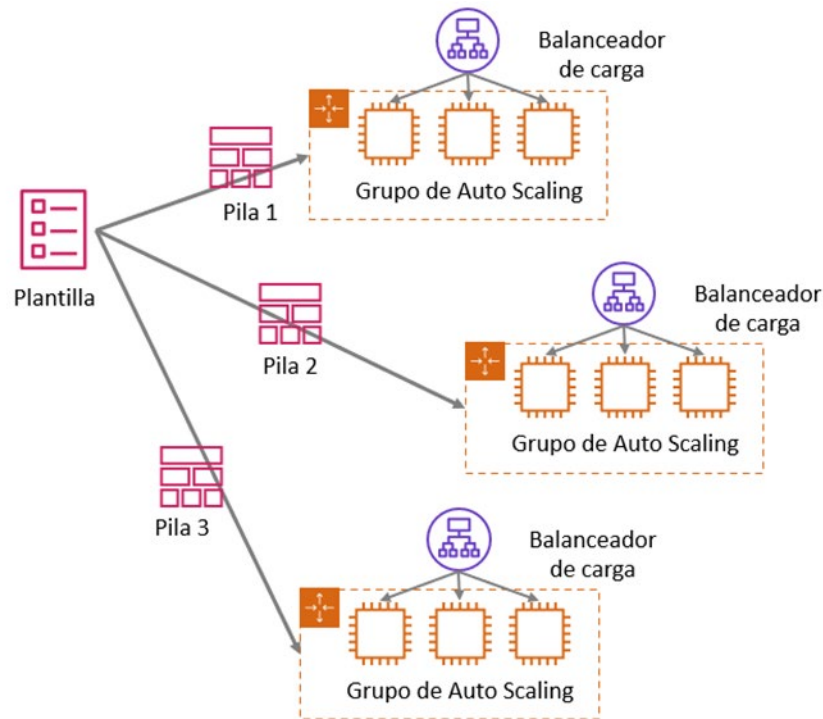


Figura 3. Beneficios de infraestructura como código. Fuente: AWS, 2021.

Si la pila 2 es su entorno de prueba y la pila 3 es su entorno de producción, puede confiar en que, si sus trabajos de prueba funcionan bien en el entorno de prueba, también funcionarán bien en el entorno de producción. La plantilla minimiza el riesgo de que el entorno de prueba esté configurado de manera diferente del entorno de producción.

Además, si tiene que realizar una actualización de configuración en el entorno de prueba, puede actualizar la plantilla con el cambio y actualizar todas las pilas. Este proceso ayuda a garantizar que las modificaciones realizadas en un solo entorno se propaguen de forma fiable a todos los entornos que deberían recibir la actualización.

Otro beneficio es que es más fácil **limpiar todos los recursos** que se crearon en su cuenta para admitir un entorno de prueba cuando ya no los necesite. Esto ayuda a **reducir los costos** que no están asociados a recursos que ya no necesita y ayuda a mantener su cuenta libre de códigos mal diseñados, complicados o no deseados.

Plantillas IaC

Las plantillas de **CloudFormation** se pueden escribir en archivos de notación de objetos de JavaScript (JSON) o YAML Ain't Markup Language.

YAML está optimizado para **facilitar la lectura**. Los mismos datos que se almacenan en JSON necesitarán menos líneas para almacenarse en YAML, ya que en YAML no se utilizan llaves ({}) y se utilizan menos comillas (""). Otra ventaja de YAML es que admite los **comentarios incrustados en forma nativa**. Por último, a muchas personas les resulta más fácil depurar documentos YAML que con JSON. Con JSON, puede ser difícil encontrar comas o llaves faltantes o mal colocados. YAML no tiene este problema.

A pesar de todas las ventajas de YAML, **JSON** ofrece algunas ventajas únicas. En primer lugar, se lo utiliza ampliamente en sistemas informáticos. Su **ubicuidad** es una ventaja, ya que los datos que se almacenan en JSON pueden usarse con confianza en muchos sistemas sin necesidad de transformarlos. Además, suele ser **más fácil generar y analizar** JSON que YAML.

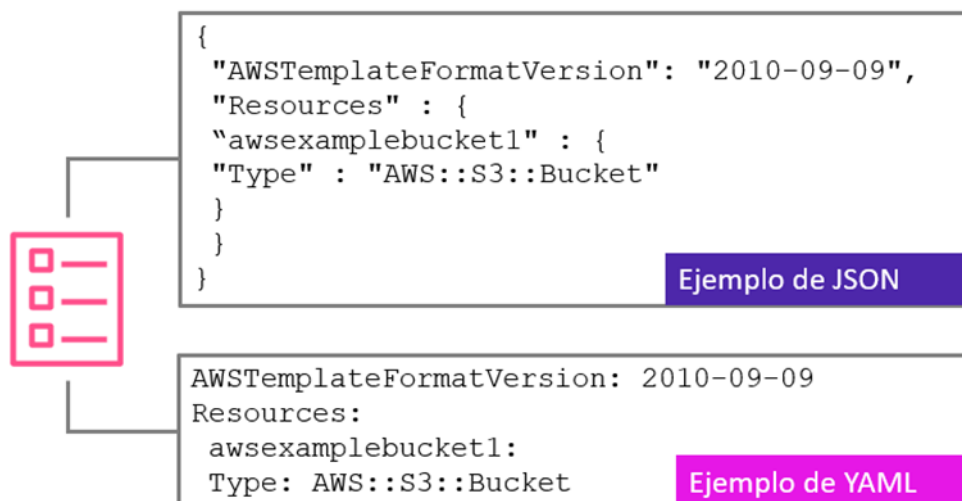


Figura 4. Plantilla json vs yaml. Fuente: AWS, 2021.

En la Figura 5, la plantilla de CloudFormation crea una máquina virtual. Aunque este ejemplo no ilustra todas las secciones posibles de una plantilla, sí resalta algunas de las secciones más utilizadas, incluidos parámetros, recursos y salidas.

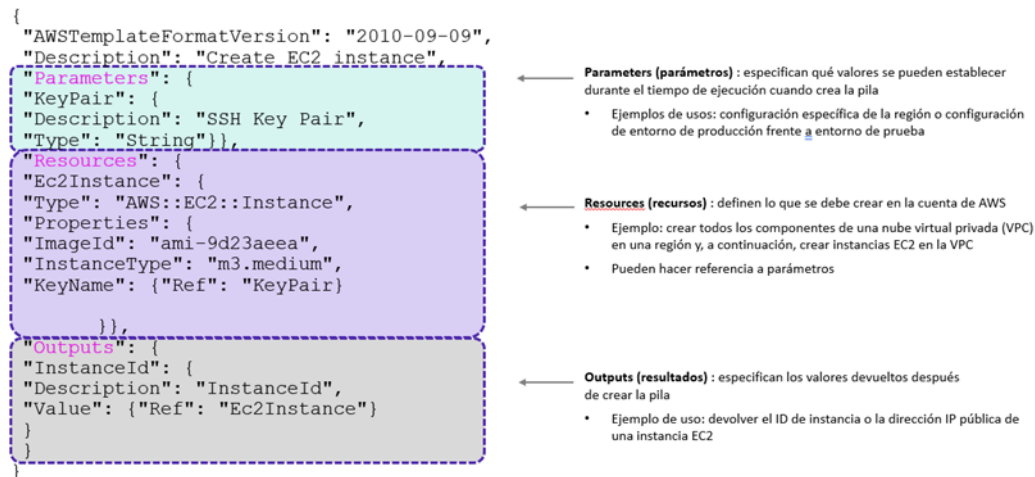


Figura 5. Secciones de una plantilla. Fuente: AWS, 2021.

Parameters (parámetros) es una sección opcional de la plantilla. Los parámetros son **valores** que se pasan a su plantilla en tiempo de ejecución (al crear o actualizar una pila). Puede hacer referencia a los parámetros de las secciones **Resources** (recursos) y **Outputs** (salidas) de la plantilla. El nombre del parámetro y su descripción aparecen en la página *Specify Parameters* (especificación de parámetros) cuando un usuario utiliza la plantilla en el asistente *Create Stack* (crear pila).

Resources (recursos) es una sección obligatoria de la plantilla. Se utilizan para especificar los recursos que se crearán, junto con sus propiedades. En este ejemplo, se especifica un recurso de tipo máquina virtual EC2.

Finalmente, el ejemplo muestra la sección **Outputs** (salidas). Esta sección describe los valores que se devuelven cuando ve las propiedades de su pila. En el ejemplo, se declara una salida InstanceId.

En el siguiente vídeo, *Infraestructura como código (IaC)*, vamos a ver en qué consiste la infraestructura como código (IaC). Para ello utilizaremos varias plantillas en formato YAML. También usaremos un diseñador de plantillas de IaC. Finalmente lanzaremos los ficheros y veremos la construcción de las infraestructuras definidas en los manifiestos de los ficheros.



Accede al vídeo

8.4. Servicios de caché

Información general sobre caché

Según afirma Amazon Web Services (2021), la **velocidad** es importante, ya sea que su aplicación ofrezca las últimas novedades, una clasificación de los diez mejores elementos, un catálogo de productos o la venta de entradas para un evento. La velocidad a la que entrega contenido **determina el éxito** de su aplicación. Si alguien necesita tener acceso a ciertos datos, ya sea para una página web o un informe del que dependen decisiones empresariales, esos datos se pueden entregar de forma mucho más rápida si se almacenan en **caché**.

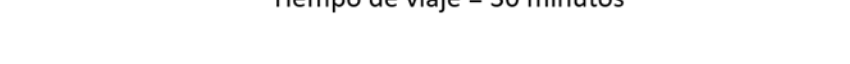
En informática, una caché es una capa de almacenamiento de datos de alta velocidad. A diferencia de una base de datos que, generalmente, almacena datos de una forma completa y duradera, una caché almacena un subconjunto de datos de manera transitoria.

El **objetivo principal** de una caché es potenciar el rendimiento de la recuperación de datos reduciendo la necesidad de acceder a la capa de almacenamiento subyacente que es más lenta. Las futuras solicitudes de datos almacenados en caché se atienden

© Universidad Internacional de La Rioja (UNIR)

Los datos de una caché se suelen almacenar en hardware de acceso rápido, como la memoria de acceso aleatorio (RAM).

Tiempo de viaje = 30 minutos



Su casa

Ferretería

En su lugar, puede almacenar los suministros que usa regularmente en una unidad de almacenamiento cerca de su casa. De esta manera, le lleva menos tiempo acceder a estos suministros que dirigirse a la ferretería.

Sin embargo, aun así, podría ir a la tienda para renovar sus suministros. En este ejemplo, la unidad de almacenamiento es **análoga** a una caché.

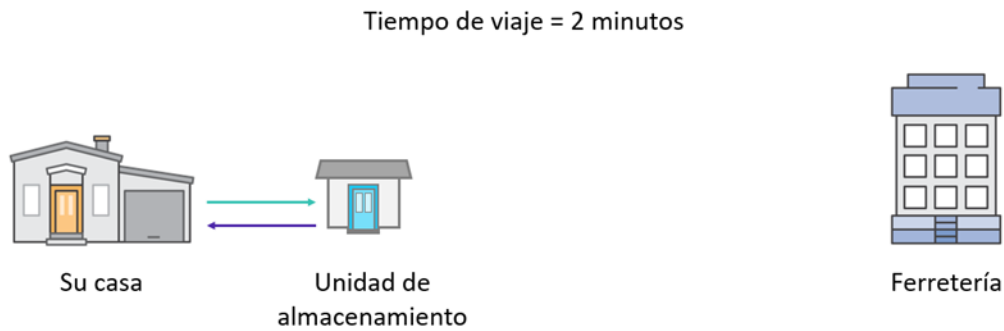


Figura 7. Tiempo con caché. Fuente: AWS, 2021.

A la hora de decidir qué datos desea almacenar en caché, tenga en cuenta los siguientes **factores**:

Velocidad y gasto: las consultas de bases de datos que consumen mucho tiempo y las consultas complejas de uso frecuente suelen crear cuellos de botella en las aplicaciones. Generalmente, si los datos requieren la adquisición de una consulta lenta y costosa, es posible que se almacenen en caché. Por ejemplo, las consultas que realizan combinaciones en varias tablas son más lentas y costosas que las consultas simples en tablas únicas. Sin embargo, incluso los datos que requieren una consulta relativamente rápida y sencilla, posiblemente se sigan considerando candidatos para almacenamiento en caché en función de otros factores.

Datos y patrones de acceso: determinar qué almacenar en caché también implica comprender los datos en sí mismos y los patrones de acceso a ellos. Por ejemplo, no tiene sentido almacenar en caché páginas web que devuelven resultados de búsqueda de naturaleza inusualmente dinámica. Para que el almacenamiento en caché suponga un beneficio significativo, los datos deberían ser relativamente estáticos y el acceso a ellos debería ser frecuente, como sucede con un perfil personal en una red social. Por el contrario, no deseará almacenar en caché datos si dicho almacenamiento no supone ninguna ventaja en términos de velocidad o costos. Por ejemplo, no tiene sentido almacenar en caché páginas web que

devuelven los resultados de una búsqueda, ya que estas consultas y sus resultados son casi siempre únicos.

Obsolescencia: según la definición, los datos almacenados en la caché son datos obsoletos. Incluso si no están obsoletos de acuerdo con ciertas circunstancias, los datos almacenados en caché deben considerarse obsoletos y tratarse como tal. A la hora de determinar si es posible que sus datos se almacenen en caché, debe evaluar también la tolerancia de su aplicación a los datos obsoletos. Es posible que la aplicación sea capaz de tolerar datos obsoletos en un contexto, pero no en otro. Por ejemplo, considere una aplicación que ofrece un precio de acciones cotizadas en la bolsa en un sitio web. Con esta aplicación, un breve retraso podría ser aceptable si presenta una exención de responsabilidad que mencione que los precios podrían retrasarse hasta en minutos. Cuando una aplicación debe ofrecer el precio de las acciones a un agente que está concretando una venta o una compra, se busca disponer de datos en tiempo real.

Una caché proporciona **acceso de alto rendimiento y baja latencia** a datos de aplicación a los que se accede comúnmente, ya que almacena los datos en la memoria.

Con el almacenamiento en caché, puede lograr lo siguiente:

Mejora la velocidad de su aplicación.

Reduce la latencia en la respuesta que experimentan los usuarios con su aplicación.

Reduce el tiempo de procesamiento de aplicaciones y el tiempo de acceso a la base de datos para cargas de trabajo de lectura intensiva, como redes sociales, videojuegos, uso compartido de contenido multimedia y portales de preguntas y respuestas. En las aplicaciones de escritura intensiva, el almacenamiento en caché no suele verse como un gran beneficio. Sin embargo, incluso las aplicaciones de escritura intensiva suelen tener una relación de lectura/escritura superior a uno, lo que implica que el almacenamiento en caché de lectura seguirá siendo beneficioso.

Esta sencilla arquitectura (Figura 8) de aplicación web muestra cómo fluyen los datos desde el usuario y hacia él. Puede utilizar el almacenamiento en caché en cada capa para **mejorar el rendimiento y la usabilidad** generales de su aplicación. Estas capas incluyen sistemas operativos, capas de red —como redes de entrega de contenido (CDN) y sistemas de nombres de dominio (DNS)— aplicaciones web y bases de datos.

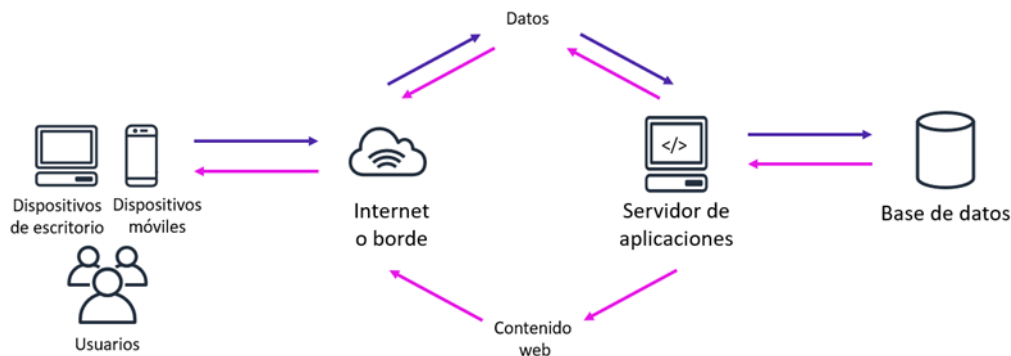


Figura 8. Arquitectura con caché. Fuente: AWS, 2021.

En esta arquitectura, puede usar el almacenamiento en caché para lo siguiente:

Acelerar la recuperación de información de los sitios web.

Almacenar un mapeo de nombres de dominio a direcciones IP.

Acelerar la recuperación de contenido web de servidores web o servidores de aplicaciones.

Acelerar el rendimiento de la aplicación y el acceso a los datos.

Reducir la latencia asociada a las solicitudes de consulta de bases de datos.

Almacenamiento en caché perimetral

Cuando alguien busca su sitio web o utiliza su aplicación, su solicitud viaja a través de muchas redes diferentes para llegar a su **servidor de origen**. El servidor de origen, que también se conoce como origen, almacena las versiones originales y definitivas de sus objetos (por ejemplo, objetos web, imágenes y archivos multimedia). El

número de **saltos de red** y la **distancia** que debe recorrer la solicitud afectan significativamente el rendimiento y la capacidad de respuesta del sitio web.

Además, la **latencia de red** puede depender de la **ubicación geográfica** del servidor de origen. Cuando el tráfico web está geográficamente disperso, no siempre es viable (ni rentable) replicar la infraestructura completa en todo el mundo. En este caso, una red de entrega de contenido (CDN) puede ser útil.

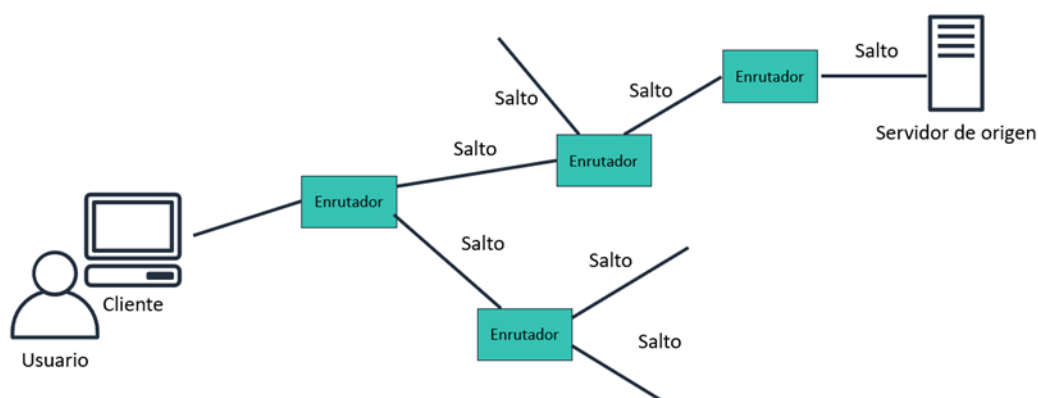


Figura 9. Latencia de red. Fuente: AWS, 2021.

Una red de entrega de contenido (CDN) es un sistema de servidores de almacenamiento en caché distribuido a nivel mundial.

Una **CDN** almacena en caché **copias de archivos solicitados** comúnmente que están alojados en el servidor de origen de la aplicación. Estos archivos pueden incluir contenido estático, como archivos HTML, CSS, JavaScript, de imagen y de vídeo. La CDN entrega una copia local del contenido solicitado desde un borde de caché o un punto de presencia que ofrece la entrega más rápida al solicitante.

Una CDN global acelera la entrega de contenido a los usuarios.

Dicho contenido puede ser **contenido estático y dinámico**, archivos multimedia que utilizan HTTP o HTTPS y vídeo en *streaming* (tanto vídeo bajo demanda como *streaming* en directo).

La CDN ofrece una caché de **varios niveles** de forma predeterminada. Las cachés perimetrales regionales mejoran la latencia y reducen la carga en los servidores de origen cuando el objeto aún no está almacenado en caché en el borde. Además, la CDN ofrece múltiples opciones para transmitir su contenido multimedia, tanto archivos pregrabados como eventos en directo. Los ofrece con el **rendimiento elevado y sostenido** que se requiere para la entrega 4K a los espectadores de todo el mundo.

La CDN brinda **protección** tanto a nivel de **red** como a nivel de **aplicación**. Las aplicaciones y el tráfico se ven beneficiados por distintas protecciones integradas, como *firewall* web perimetral o denegación de servicio.

La CDN admite la **comunicación bidireccional** en tiempo real a través del protocolo WebSocket. Esta conexión persistente permite a los clientes y los servidores enviar datos en tiempo real entre sí sin incurrir en el costo de establecer conexiones repetidas veces. Esto es especialmente útil para aplicaciones de comunicación, como chat, colaboración, juegos y negociaciones financieras.

También admite **métodos HTTP** (DELETE, GET, HEAD, OPTIONS, PATCH POST, PUT), que mejoran el rendimiento de los sitios web dinámicos. Estos sitios web tienen formularios web, casillas de comentarios e inicio de sesión, botones de agregar al carro y otras características que cargan datos de los usuarios. Por lo tanto, puede utilizar un **único nombre de dominio** para entregar todo el sitio web a través de la CDN, lo que acelera tanto la descarga como la carga de las partes de su sitio web.

En la Figura 10 se muestra cómo el contenido estático y dinámico puede componer una aplicación web dinámica. Esta aplicación web se entrega con el protocolo HTTPS para el cifrado de las solicitudes de página de usuario y las páginas que se devuelven

desde un servidor web. Puede usar una CDN o una caché perimetral para almacenar el contenido estático. Dicho contenido puede incluir objetos web (por ejemplo, documentos HTML, hojas de estilo CSS o archivos JavaScript), archivos de imagen y archivos de vídeo.

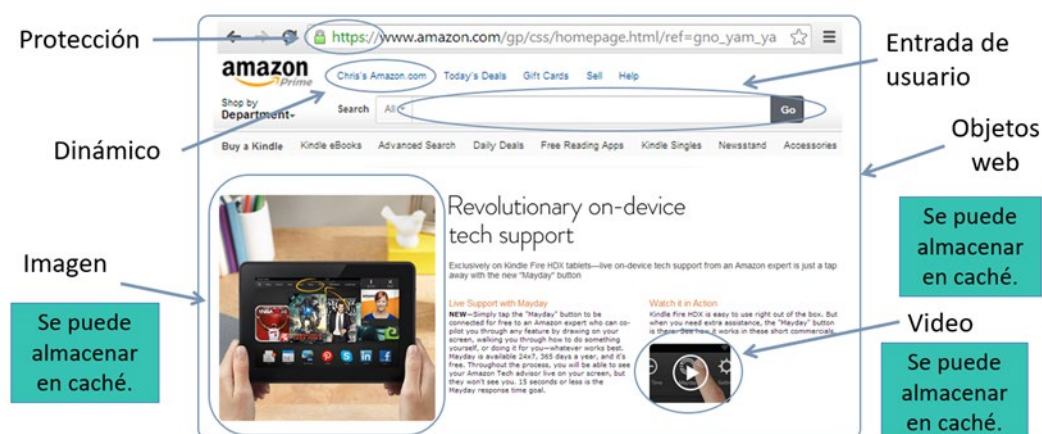


Figura 10. Tipo de contenido en CDN. Fuente: AWS, 2021.

No se puede almacenar en caché contenido generado de manera dinámica ni datos generados por el usuario.

Además, puede configurar la CDN para exigir a los espectadores que utilicen HTTPS al solicitar sus objetos, de modo que las conexiones se cifren cuando se comunique con los espectadores. También puede configurarla para usar HTTPS a la hora de obtener objetos del origen, de modo que las conexiones se cifren cuando la CDN se comunique con su origen.

La CDN entrega su **contenido** a usuarios a través de una red mundial de centros de datos que reciben el nombre de **ubicaciones perimetrales**.

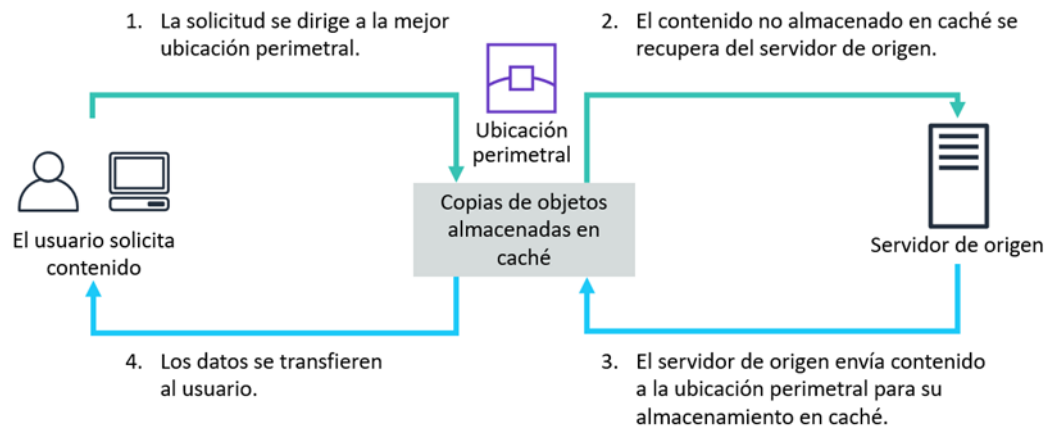


Figura 11. Funcionamiento almacenamiento en caché. Fuente: AWS, 2021.

Cuando un usuario solicita contenido que usted atiende con CDN el DNS dirige la solicitud a la ubicación periférica que mejor atienda la solicitud. Normalmente, es la **ubicación periférica más cercana** la que tiene la **latencia más reducida**. La CDN revisa la caché para comprobar si dispone del contenido solicitado (paso 1). Si el contenido está en la caché, la CDN lo entrega inmediatamente al usuario (paso 4). Es posible que el contenido no esté actualmente en la caché. Si no lo está, la CDN reenvía la solicitud al servidor de origen que usted identificó como origen de la versión definitiva de su contenido (paso 2). El servidor de origen envía el contenido de vuelta a la ubicación periférica (paso 3) y la CDN luego reenvía el contenido al usuario (paso 4). También agrega el contenido a la caché en la ubicación periférica para la próxima vez que alguien lo solicite.

A medida que los objetos pierden popularidad, las ubicaciones periféricas individuales podrán eliminarlos a fin de liberar espacio para el contenido más solicitado. Para el contenido menos popular, la CDN dispone de **cachés periféricas regionales**. Las cachés periféricas regionales son ubicaciones de la CDN que se implementan a nivel mundial y están cerca de sus espectadores. Están ubicadas entre el servidor de origen y las ubicaciones periféricas globales que entregan contenido directamente a los espectadores. Una caché periférica regional tiene una memoria caché más grande que una ubicación periférica individual, por lo que los objetos permanecen en la caché periférica regional durante más tiempo.

Si desea utilizar la CDN para distribuir su contenido, debe crear un **punto de distribución**.

Especifica el servidor de origen que aloja los archivos.

A continuación, especifica detalles sobre cómo realizar un seguimiento de la entrega de contenido y cómo administrarla.

La CDN asigna un nombre de dominio a la nueva distribución.

La CDN envía la configuración de su distribución, pero no el contenido, a todas las ubicaciones perimetrales.

Almacenamiento de sesiones web en caché

Cuando un usuario o un servicio interactúan con una aplicación web, envían una solicitud HTTP y la aplicación devuelve una respuesta. Una secuencia de tales transacciones se denomina **sesión**. Cada solicitud es independiente de las transacciones anteriores. Por lo tanto, las sesiones se utilizan para administrar la autenticación de los usuarios y almacenar sus datos mientras interactúan con la aplicación. Por ejemplo, al emplear sesiones, los usuarios no tienen que enviar sus credenciales por cada solicitud que realicen al servidor.

Puede **administrar las sesiones** de los usuarios de distintas maneras (de forma predeterminada, un balanceador de carga dirige cada solicitud independientemente de la instancia registrada con la carga más pequeña). Para utilizar las sesiones rápidas, los clientes deben admitir las *cookies*.

Las **sesiones rápidas** son rentables porque se almacenan en los servidores web que ejecutan sus aplicaciones. Por lo tanto, eliminan la latencia de la red y aceleran la recuperación de esas sesiones. Sin embargo, en caso de error en la instancia, es probable que pierda las sesiones almacenadas en esa instancia.

Otra **desventaja** de las sesiones rápidas es que pueden limitar la escalabilidad de su aplicación. Con las sesiones rápidas, el balanceador de carga no puede equilibrar

realmente la carga cada vez que recibe una solicitud de un cliente. Con las sesiones rápidas, el balanceador de carga se ve obligado a enviar todas las solicitudes al servidor original en el que se creó el estado de la sesión. Si ese servidor está muy cargado, la recepción de tantas solicitudes puede provocar una carga desigual entre los servidores y afectar el tiempo de respuesta del usuario.

En lugar de utilizar sesiones rápidas, puede designar una capa en su arquitectura que sea capaz de almacenar sesiones de una manera escalable y sólida fuera de la instancia. Una opción es mantener los datos de sesión en una **caché distribuida**, como muestra este diagrama de arquitectura de la Figura 12. Tiene sentido implementar esta arquitectura en un entorno dinámico cuando la cantidad de servidores web cambia para ajustarse a la carga y no desea arriesgarse a perder sesiones.

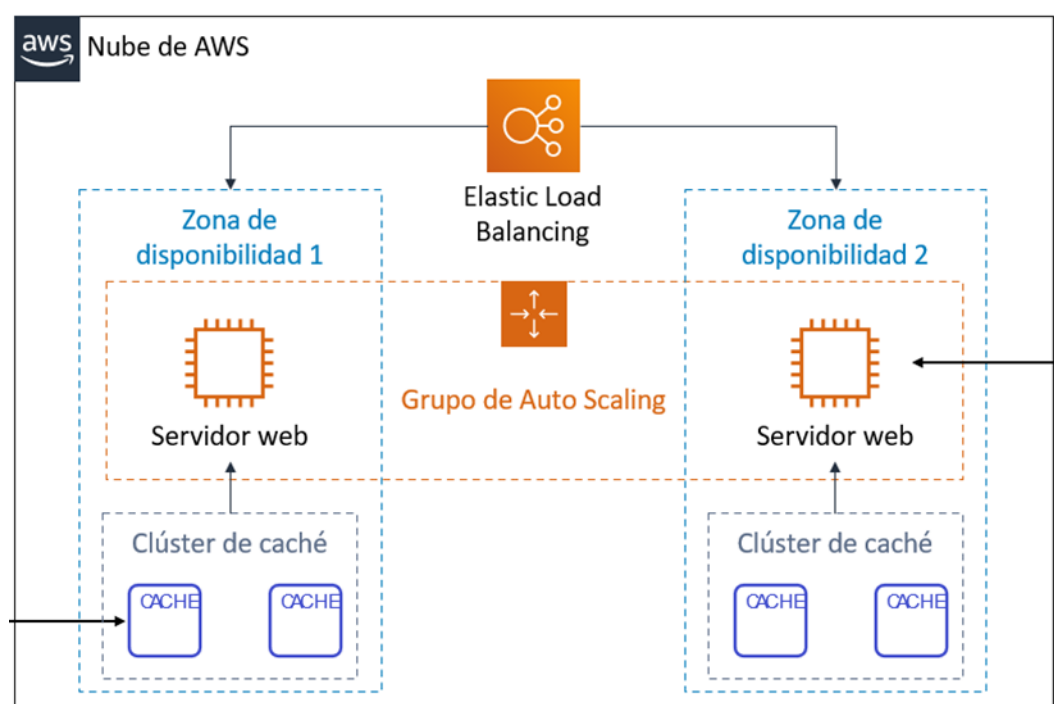


Figura 12. Sesiones de caché distribuidas. Fuente: AWS, 2021.

En el vídeo *Cacheado de una web estática en una CDN* aprenderemos a usar el servicio de caché de contenidos (*Content Delivery Network*). Con ello, veremos que podemos acercar los contenidos a los usuarios para evitar retardos y satisfacer en el servicio a nuestros clientes.



Accede al vídeo

8.5. Microservicios y arquitecturas sin servidor

Microservicios

Los microservicios son un enfoque de arquitectura y organización para el desarrollo de software en el que las aplicaciones están compuestas por servicios independientes que se comunican a través de interfaces de programación de aplicaciones (API) bien definidas.

Este enfoque está diseñado para acelerar los ciclos de implementación.

Según afirma Amazon Web Services (2021), el enfoque de microservicios fomenta la innovación y la propiedad, además de mejorar la capacidad de mantenimiento y escalabilidad de las aplicaciones de software.

Para comprender los beneficios de los microservicios, primero considere una **aplicación monolítica**.

En la Figura 13, los **tres procesos** (usuarios, temas y mensajes) de una **aplicación monolítica** de foro están estrechamente acoplados. Se ejecutan como un **único servicio**. Esto significa que, si un proceso de la aplicación experimenta un pico de demanda, se debe escalar toda la arquitectura. Agregar o mejorar características se

vuelve más complejo a medida que la base de código crece, lo que limita la experimentación y dificulta la implementación de nuevas ideas.

La **disponibilidad** de aplicaciones monolíticas también está en **riesgo** porque muchos procesos dependientes y estrechamente acoplados aumentan el impacto que puede producir un solo error en el proceso.

Supongamos que la misma aplicación se ejecuta en una **arquitectura de microservicios**. Cada proceso de la aplicación se crea como un **componente independiente** que se ejecuta como un servicio. Los servicios se comunican mediante operaciones de API ligeras. Cada servicio está a cargo de una única función que puede asistir a varias aplicaciones. Debido a que los servicios se ejecutan de forma independiente, se pueden actualizar, implementar y escalar para satisfacer la demanda de funciones específicas por parte de una aplicación.

Una arquitectura de microservicios proporciona **iteración, automatización y agilidad** general mucho más rápidas. Se comienza rápido, se encuentran errores rápido y se recupera rápido.

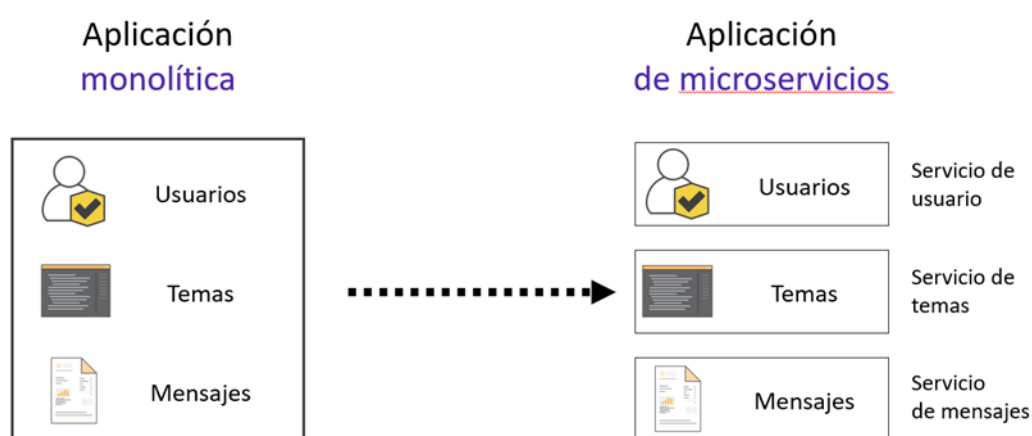


Figura 13. Aplicación monolítica vs. microservicios. Fuente: AWS, 2021.

Los microservicios comparten algunas características comunes:

Descentralizados: las arquitecturas de microservicios son sistemas distribuidos con administración descentralizada de datos. No dependen de un esquema unificado en una base de datos central. Cada microservicio tiene su propia vista sobre los modelos de datos. Los microservicios también son descentralizados en la forma en que se desarrollan, implementan, administran y operan.

Independientes: cada servicio componente de una arquitectura de microservicios se puede cambiar, actualizar o reemplazar de forma independiente sin afectar el funcionamiento de los otros servicios. Los servicios no necesitan compartir ninguno de sus códigos o implementaciones con otros servicios. Del mismo modo, los equipos responsables de los diferentes microservicios pueden actuar independientemente de los demás.

Especializado: cada servicio componente está diseñado para un conjunto de capacidades y se centra en un dominio específico. Si el código para un servicio componente determinado alcanza cierto nivel de complejidad, dicho servicio se puede dividir en dos o más servicios.

Políglota: los microservicios no siguen un único enfoque. Los equipos tienen la libertad de elegir la mejor herramienta para resolver sus problemas específicos. En consecuencia, las arquitecturas de microservicios adoptan un enfoque heterogéneo respecto de los sistemas operativos, los lenguajes de programación, los almacenes de datos y las herramientas. Este enfoque se denomina de persistencia y programación políglotas.

Cajas negras: los servicios componentes individuales están diseñados como cajas negras, lo que significa que los detalles de su complejidad están ocultos de los otros componentes. Las comunicaciones entre los servicios se producen a través de API bien definidas para evitar las dependencias implícitas y ocultas.

Se crean, se ejecutan: DevOps es un principio organizativo clave para los microservicios, donde el equipo responsable de crear un servicio también es responsable de operarlo y mantenerlo en producción.

Microservicios con contenedores

Cuando se crea una arquitectura de microservicios, se pueden utilizar contenedores para la **capacidad** de procesamiento.

Los contenedores son un **método de virtualización** del sistema operativo que le permite ejecutar una aplicación y sus dependencias en procesos de recursos aislados.

Un contenedor es un **paquete de software** ligero e independiente. Contiene todo lo que una aplicación de software necesita para funcionar, como ser, el código de la aplicación, el motor de tiempo de ejecución, las herramientas del sistema, las bibliotecas del sistema y las configuraciones.

Su contenedor

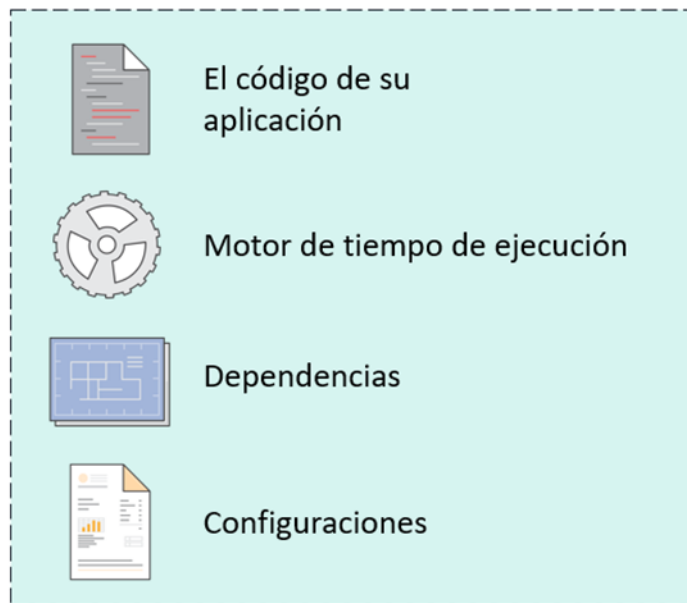


Figura 14. Partes de un contenedor. Fuente: AWS, 2021.

Los contenedores lo ayudan a garantizar una **implementación** rápida, uniforme y de confianza de las aplicaciones independientemente del entorno. Los contenedores también ofrecen un **control** más minucioso de los recursos, lo que mejora la eficiencia de la infraestructura.

Un contenedor se crea a partir de una plantilla de solo lectura que se denomina **imagen**. Por lo general, las imágenes se crean a partir de un **Dockerfile**, un archivo de texto sin formato que especifica todos los componentes incluidos en el contenedor. Puede crear imágenes desde cero o utilizar imágenes que otros hayan creado y publicado en un registro de contenedores público o privado.

La **imagen de contenedor** es la instantánea del sistema de archivos que está disponible para el contenedor. Por ejemplo, es posible que tenga el sistema operativo Debian como una imagen de contenedor. Cuando ejecute este contenedor, un sistema operativo Debian estará disponible para él. También puede empaquetar

todas sus dependencias de código en la imagen de contenedor y usarla como artefacto de código.

Las imágenes de contenedor se almacenan en un registro. Puede descargar las imágenes del registro y ejecutarlas en el clúster.

Puede ejecutar los contenedores si utilizan los servicios de contenedor en la nube. Es un servicio de administración de contenedores altamente escalable y de alto rendimiento. Es compatible con los contenedores de Docker y le permite ejecutar fácilmente aplicaciones en un clúster administrado de máquinas virtuales.

El servicio de contenedores en nube es un servicio de clúster escalable para alojar contenedores que presenta las siguientes características:

Puede escalar hasta miles de contenedores de Docker en segundos.

Monitorea la implementación de contenedores.

Administra el estado del clúster que ejecuta los contenedores.

Programa los contenedores por medio de un programador integrado o un programador de terceros (por ejemplo, Apache Mesos, Blox).

Es extensible mediante el uso de API.

Puede lanzarse con en formato *serverless* o con máquinas virtuales.

Es posible ejecutar los clústeres a escala mediante la combinación de instancias de spot con instancias reservadas e instancias bajo demanda.

Contenedores en la nube es un servicio regional que simplifica la ejecución de contenedores de aplicación con alta disponibilidad en diversas zonas de disponibilidad dentro de una región. Puede crear clústeres en una nube virtual privada (VPC) nueva o existente. Un **clúster** es una agrupación lógica de recursos.

Después de que el clúster esté activo y en ejecución, puede definir servicios y definiciones de tareas que especifiquen qué imágenes de contenedores de Docker se deben ejecutar en los clústeres.

Una **definición de tareas** es un **archivo de texto** en formato JavaScript Object Notation (JSON). Describe uno o más contenedores, hasta un máximo de diez, que forman su aplicación. Puede pensarlo como un proyecto para su aplicación. Las definiciones de tareas especifican parámetros para la aplicación, por ejemplo, qué tipo de contenedores y de lanzamientos se van a utilizar. Otros parámetros incluyen qué puertos deben abrirse para la aplicación y qué volúmenes de datos se deben utilizar con los contenedores en la tarea.

Un servicio le permite **especificar** cuántas **copias** de la definición de tareas se van a ejecutar y mantener en un clúster. De manera opcional, puede utilizar un **balanceador de carga** de Elastic Load Balancing para distribuir el tráfico entrante en los contenedores del servicio. El servicio mantiene ese número de tareas y coordina la programación de las tareas con el balanceador de carga.

Después de haber creado una definición de tareas para su aplicación, puede especificar el número de tareas que se ejecutarán en su clúster. Una tarea es la instancia creada de una definición de tareas dentro de un clúster.

El servicio descarga las imágenes de contenedor de un registro que usted especifica y ejecuta dichas imágenes en el clúster.

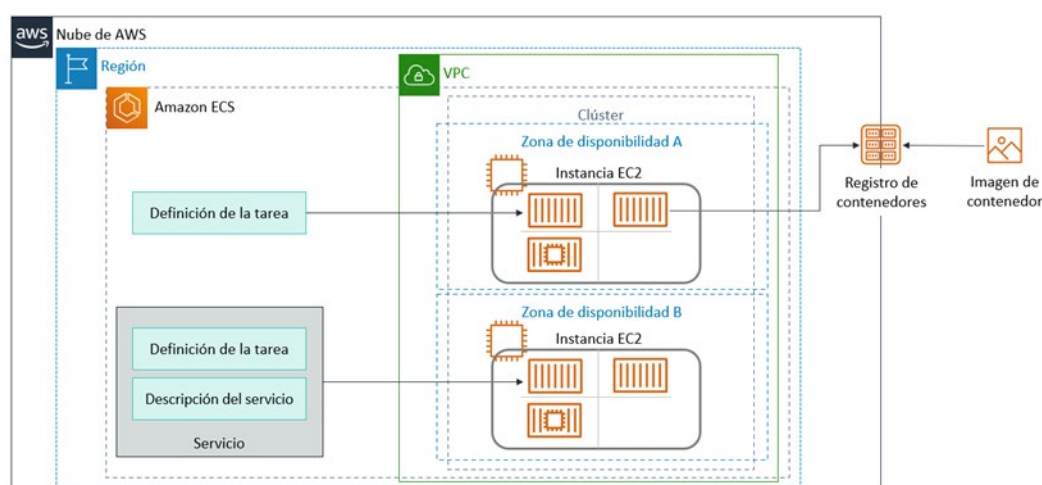


Figura 15. Servicio de contenedores en la nube de AWS. Fuente: AWS, 2021.

El servicio ofrece dos **tipos de lanzamiento** para alojar las aplicaciones en contenedores. Puede utilizar el tipo de lanzamiento de **sin servidor** para alojar el clúster en una infraestructura sin servidor administrada por la nube. Lo único que debe hacer es empaquetar la aplicación en contenedores, especificar los requisitos de CPU y memoria, definir las políticas de redes y de Identity and Access Management (IAM) y lanzar la aplicación.

Alternativamente, si desea tener más control, puede utilizar el tipo de lanzamiento en **máquinas virtuales** para alojar las tareas en un clúster de máquinas virtuales que usted administre.

Una instancia de contenedor es una máquina virtual que ejecuta el agente de contenedores.

Puede usar el servicio de contenedores para programar la colocación de los contenedores en su clúster en función de las necesidades de recursos, las políticas de aislamiento y los requisitos de disponibilidad.

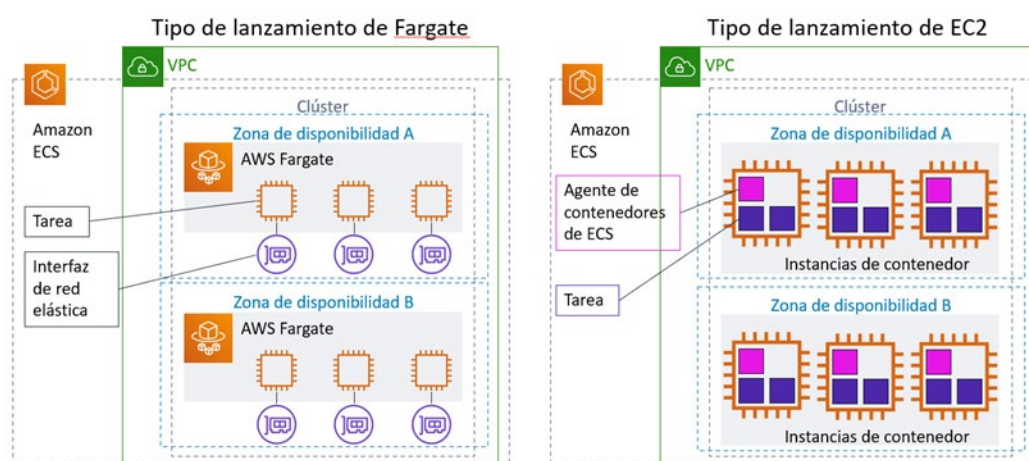


Figura 16. Modos de lanzamiento de contenedores en AWS. Fuente: AWS, 2021.

Puede crear un **grupo de Auto Scaling**. El grupo de Auto Scaling contiene instancias de contenedor cuya cantidad se puede escalar o reducir mediante el uso de alarmas de monitorización. Si configura el grupo de Auto Scaling para eliminar instancias de

contenedor, las tareas que se ejecuten en las instancias del contenedor eliminado se detendrán. Si las tareas se ejecutan como parte de un servicio, el servicio de contenedores reiniciará esas tareas en otra instancia si los recursos necesarios están disponibles. Son ejemplos de tales recursos necesarios la CPU, la memoria, los puertos. Sin embargo, las tareas que se iniciaron de forma manual no se reinician de forma automática.

También puede aprovechar el servicio de Auto Scaling que proporciona más **control** sobre la forma de escalar las tareas en un clúster. Aumenta la **velocidad** y la **fiabilidad** del escalado horizontal de clústeres. Le brinda control sobre la cantidad de capacidad de reserva que se mantiene en el clúster y administra automáticamente la terminación de la instancia en el caso de reducción de capacidad (reducción horizontal).

Con el escalado automático de clústeres, puede configurar el servicio de contenedores para que, de forma automática, **aumente o reduzca horizontalmente** la escala del grupo de Auto Scaling. El escalado automático de clústeres depende de los proveedores de capacidad, que vinculan el clúster de ECS con los grupos de Auto Scaling que desea utilizar. Cada grupo de Auto Scaling está asociado a un proveedor de capacidad y cada proveedor de capacidad solo tiene un grupo de Auto Scaling.

Descomposición de monolitos (despliegue en AWS)

Una vez más, considere una aplicación monolítica de foro en la cual se ejecuta toda la aplicación como un único servicio. Para rediseñar esta aplicación usando una arquitectura de microservicios, puede ejecutar cada proceso de la aplicación como un **servicio independiente** dentro de su propio contenedor. Con una arquitectura de microservicios, los servicios se pueden escalar y actualizar independientemente de los demás.

Para implementar la aplicación monolítica como una aplicación de microservicios, primero **Cree y etiquete una imagen** para cada servicio. Luego, registre las imágenes con **Amazon Elastic Container Registry (Amazon ECR)**.

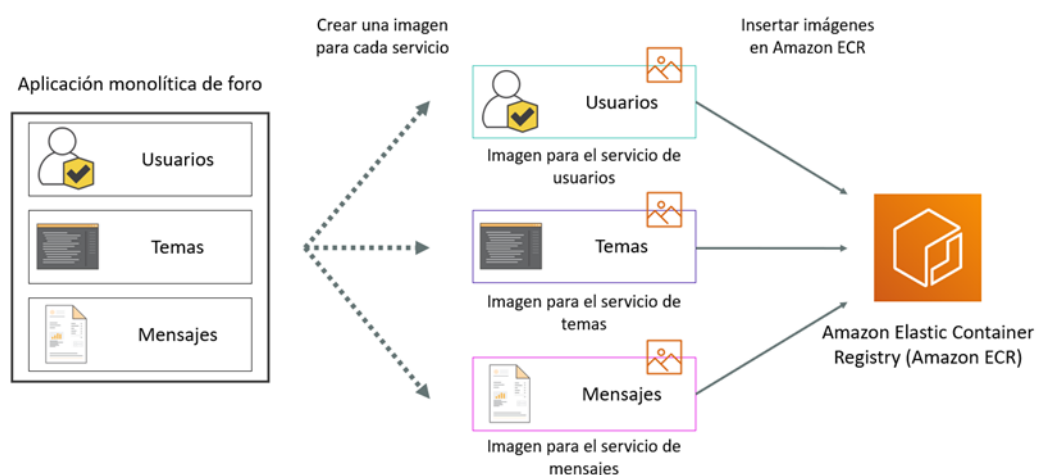


Figura 17. Descomposición de monolitos. Crear contenedores. Fuente: AWS, 2021.

A continuación, **elija un tipo de lanzamiento y cree un nuevo servicio** para cada pieza de la aplicación monolítica original. Amazon ECS implementa cada servicio en su propio contenedor en un clúster de ECS.

Luego, **cree un grupo de destino** para cada servicio. El grupo de destino realiza un seguimiento de las instancias y los puertos de cada contenedor que se ejecuta para ese servicio.

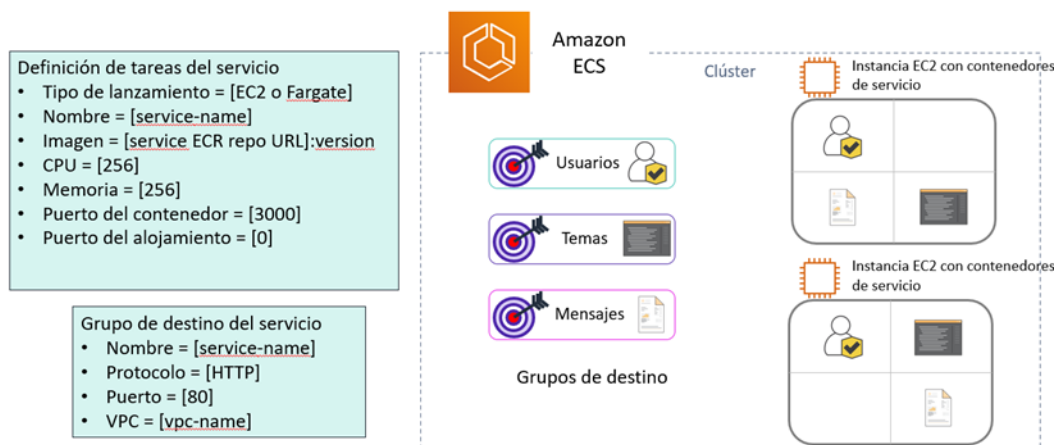


Figura 18. Descomposición de monolitos. Tareas y grupos de destino. Fuente: AWS, 2021.

Por último, **crea un balanceador de carga de aplicaciones y configura las reglas** del agente de escucha para conectarse a los servicios. El agente de escucha comprueba las solicitudes de conexión que ingresan al balanceador de carga y utiliza las reglas para dirigir el tráfico de forma adecuada. En el ejemplo, el agente de escucha del balanceador de carga de aplicaciones escucha las solicitudes de servicio HTTP en el puerto 80 y las dirige al servicio apropiado.

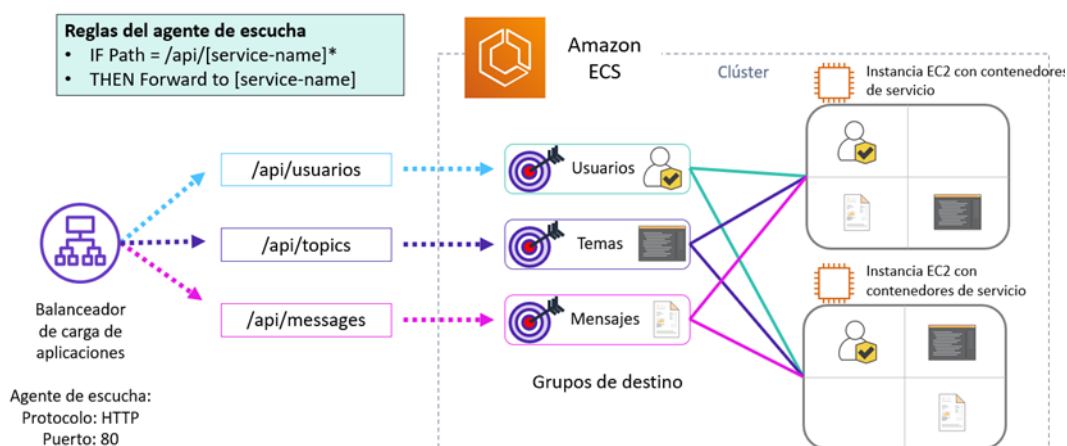


Figura 19. Descomposición de monolitos. Conectar balanceador. Fuente: AWS, 2021.

Arquitecturas sin servidor

Los principios que definen a la informática sin servidor como modelo operativo incluyen los siguientes:

No hay infraestructura que aprovisionar o administrar (no hay ningún servidor que aprovisionar, operar ni al cual aplicar parches).

Escala de forma automática por unidad de consumo (escala por unidad de trabajo o consumo más que por unidad de servidor).

Modelo de precios de pago por valor (solo se paga por la duración que se ejecuta un recurso, en lugar de por unidad de servidor).

Disponibilidad y tolerancia a errores integrada (no es necesario diseñar la disponibilidad porque está integrada en el servicio).

Mediante la informática sin servidor, puede crear aplicaciones modernas con **mayor agilidad** y **menor costo** total de propiedad (TCO). Si utiliza una arquitectura sin servidor, puede centrarse en su producto principal. No tiene que preocuparse por administrar ni operar servidores ni entornos de tiempo de ejecución, ya sea en la nube o en las instalaciones. Esta reducción de tareas le permite **recuperar tiempo y energía** que puede dedicar a desarrollar productos que escalen y sean de confianza. Por último, las arquitecturas sin servidor le permiten crear aplicaciones de microservicios.

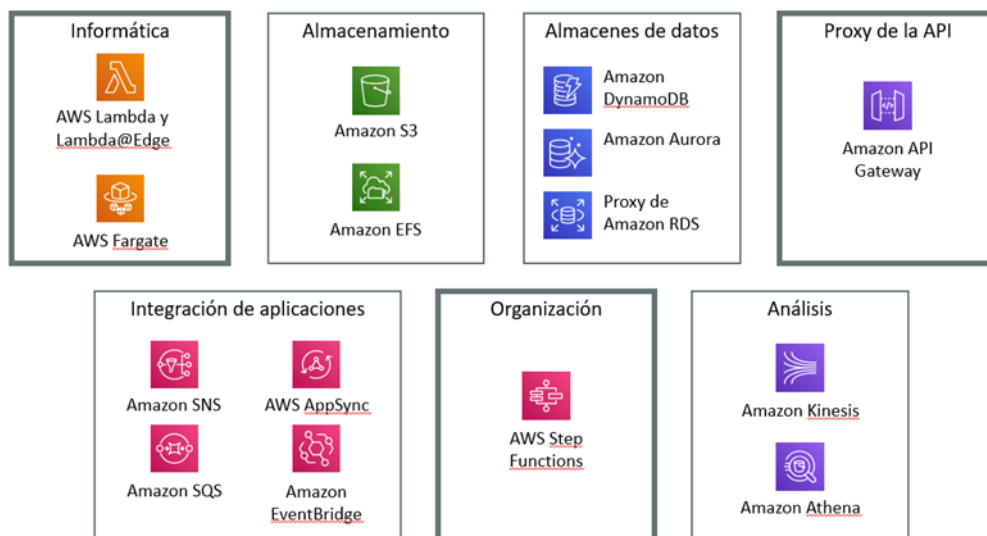


Figura 20. Oferta servicios serverless en AWS. Fuente: AWS, 2021.

Como ejemplo de servicio sin servidor, vemos el **AWS Lambda**. Es un servicio de informática completamente **administrado** que ejecuta código en respuesta a eventos y administra automáticamente los recursos informáticos subyacentes. Lambda **ejecuta código** en una infraestructura informática de gran disponibilidad y se encarga de toda la **administración** de los recursos informáticos, incluidos el mantenimiento del servidor y del sistema operativo, el aprovisionamiento de capacidad, el escalado automático, el monitoreo del código y el registro.

AWS Lambda es compatible de forma nativa con Java, Go, PowerShell, Node.js, C#, Python y el código Ruby. Además, proporciona una API de tiempo de ejecución que le permite utilizar cualquier lenguaje de programación adicional para crear sus funciones.

AWS Lambda se integra con otros servicios de AWS para invocar las funciones de Lambda. Una función de Lambda es un **código personalizado** que se escribe en uno de los lenguajes que admite Lambda. Puede configurar desencadenadores para invocar una función en respuesta a los eventos del ciclo de vida de los recursos, responder a las solicitudes HTTP entrantes, consumir los eventos de una cola o realizar ejecuciones programadas.

Un origen de eventos es la entidad que publica el evento en Lambda. La función de Lambda procesa el evento y Lambda ejecuta la función en su nombre.

Las funciones de Lambda son **sin estado**, lo que significa que no tienen afinidad con la infraestructura subyacente. Lambda puede lanzar rápidamente tantas copias de la función como sea necesario para escalar a la velocidad de los eventos entrantes.

Cuando crea una función de Lambda, define los **permisos** para la función y especifica qué eventos la activan. Además, crea un **paquete de implementación** que incluye el código de la aplicación, las dependencias y las bibliotecas que se necesitan para ejecutar el código. Por último, configura los **parámetros de tiempo** de ejecución, como la memoria, el tiempo de espera y la concurrencia. Cuando se invoca su función, Lambda ejecuta un entorno basado en el tiempo de ejecución y las opciones de configuración seleccionadas.

Cuando se invoca una función de Lambda, el código comienza a ejecutarse en el **controlador**. El controlador es un método o una función de código específico que se crea e incluye en el paquete. Cuando se crea una función de Lambda, se debe especificar el controlador. Cada lenguaje compatible tiene sus propios requisitos respecto de cómo se puede definir un controlador de funciones y hacer referencia a él dentro del paquete. Después de que el controlador se invoca de forma correcta dentro de su función de Lambda, el entorno de tiempo de ejecución pertenece al código que escribió.

El controlador siempre toma **dos objetos**: el objeto de evento y el objeto de contexto. El **objeto de evento** proporciona información acerca del evento que desencadenó la función de Lambda. Este evento puede ser un objeto predefinido que genera un servicio de AWS o un objeto personalizado definido por el usuario en forma de cadena serializable. Un ejemplo de tal cadena podría ser un objeto Java estándar (POJO) o una secuencia JSON.

Handler()

Función que se ejecutará
luego de la invocación

Objeto de evento

Datos enviados durante la
invocación de la función de
Lambda

Objeto de contexto

Métodos disponibles
para interactuar con la
información de tiempo de
ejecución (ID de solicitud, grupo
de registro, etc.)

```
import json

def lambda_handler(event, context):
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Hello world')
    }
```

Figura 21. Anatomía de una función Lambda. Fuente: AWS, 2021.

En el **contenido** del objeto de evento, se incluyen todos los **datos** y los **metadatos** que su función de Lambda necesita para impulsar la lógica. El contenido y la estructura del objeto de evento **varían** en función del origen de eventos que lo creó. Por ejemplo, un evento creado por API Gateway contiene detalles relacionados con la solicitud HTTPS realizada por el cliente de API, como la ruta de acceso, la cadena de consulta y el cuerpo de la solicitud. Sin embargo, un evento creado por Amazon incluye detalles sobre el *bucket* y el nuevo objeto.

AWS genera el objeto de contexto y proporciona metadatos sobre el entorno de tiempo de ejecución. El objeto de contexto permite que el código de su función interactúe con el entorno de tiempo de ejecución de Lambda. El contenido y la estructura del objeto de contexto varían en función del tiempo de ejecución del lenguaje que utiliza su función de Lambda.

8.6. Referencias bibliográficas

Amazon Web Service. (2022). *Content Delivery Network (CDN) Caching*.
<https://aws.amazon.com/caching/cdn/>

AWS. (2021, diciembre). *Training Track: Cloud Architecting*. Manuscrito no publicado.

Serverless Stack (2022). *¿Qué es Serverless?* <https://serverless-stack.com/chapters/es/what-is-serverless.html>.

CloudFlare

Cloudflare. (2022). *¿Qué es el almacenamiento caché?* <https://www.cloudflare.com/es-es/learning/cdn/what-is-caching/>

Uno de los proveedores de nube importantes con respecto a caché. Podemos averiguar todo sobre esta tecnología en su web.

Serverless.com

Serverless. (2022). *Serverless Google Cloud Functions Provider Documentation.* <https://www.serverless.com/framework/docs/providers/google>

La importancia en las nuevas arquitecturas del mundo serverless hacen de esta tecnología algo indispensable en el conocimiento tecnológico actual. Desde esta web podemos tener un vistazo global del mundo sin servidor de todos los proveedores y servicios.

Atlassian

Atlassian. (2022). *Comparación entre la arquitectura monolítica y la arquitectura de microservicios.* <https://www.atlassian.com/es/microservices/microservices-architecture/microservices-vs-monolith>

Desde el proveedor Atlassian líder en el sector tecnológico en muchos productos nos dan una visión más completa de la arquitectura a través de microservicios.

Tener un registro de seguimiento de auditoría es muy importante para muchas situaciones de:

- A. Resiliencia y control.
- B. Control de gasto y seguridad.
- _ C. Conformidad y seguridad.
- D. Seguridad y resiliencia.

Tener un registro de seguimiento de auditoría es muy importante para muchas situaciones de conformidad y seguridad. Es peligroso permitir que cualquier persona de su organización controle y edite manualmente sus entornos.

La popularidad de la IaC continúa creciendo porque proporciona una solución viable para:

- A. Desafíos multicuenta.
- _ B. Desafíos como cómo replicar.
- C. *Backup y restore* de infraestructuras críticas.
- D. Desafíos de las nuevas tecnologías.

La popularidad de la IaC continúa creciendo porque proporciona una solución viable para desafíos como cómo replicar, volver a implementar y reutilizar la infraestructura de manera fácil, confiable y coherente.

Las plantillas de CloudFormation se pueden escribir en archivos de notación de objetos de:

- _ A. JavaScript o Yaml.
- B. Javascript y XML.
- C. XML y Java.
- D. En cualquier formato de texto plano.

Las plantillas de CloudFormation se pueden escribir en archivos de notación de objetos de JavaScript (JSON) o YAML Ain't Markup Language (YAML).

En informática, una caché es una capa de almacenamiento de:

- ☐ A. Datos de alta velocidad.
- B. Microprocesadores de última generación.
- C. Datos de tiempo de vida pequeños.
- D. Ninguna de las anteriores.

En informática, una caché es una capa de almacenamiento de datos de alta velocidad. A diferencia de una base de datos, que generalmente almacena datos de una forma completa y duradera, una caché almacena un subconjunto de datos de manera transitoria.

A la hora de decidir qué datos desea almacenar en caché, que factor no se tiene en cuenta:

- A. Datos y patrones de acceso.
- B. Velocidad y gasto.
- C. Obsolescencia.
- ☐ D. Distancias de las métricas de los *routers*.

Datos y patrones de acceso, velocidad y gasto, obsolescencia.

Una red de entrega de contenido (CDN) es un sistema de servidores de almacenamiento en:

- A. En caché en formato denominado *mesh*.
- ☐ B. Caché distribuido a nivel mundial.
- C. Caché compartido en cada una de las regiones del proveedor de nube.
- D. Con sistema operativo de High Performance Computing.

Una red de entrega de contenido (CDN) es un sistema de servidores de almacenamiento en caché distribuido a nivel mundial. Una CDN almacena en caché copias de archivos solicitados comúnmente que están alojados en el servidor de origen de la aplicación.

Los microservicios son un enfoque de arquitectura y organización para el desarrollo de:

A. Software en el que las aplicaciones están compuestas por pasarelas de comunicaciones independientes entre sí.

B. Software en el que las aplicaciones están compuestas por servidores Linux.

_ C. Software en el que las aplicaciones están compuestas por servicios independientes que se comunican a través de interfaces de programación de aplicaciones (API) bien definidas.

D. Hardware y software en el que las aplicaciones están compuestas por servicios independientes.

Los microservicios son un enfoque de arquitectura y organización para el desarrollo de software en el que las aplicaciones están compuestas por servicios independientes que se comunican a través de interfaces de programación de aplicaciones (API) bien definidas.

¿Cuál no es una característica compartida de los microservicios?

A. Descentralizados.

B. Independientes.

C. Políglota.

_ D. Efímeros.

Descentralizados, independientes, políglotas, cajas negras, especializados.

Los contenedores son un método de virtualización del sistema operativo que le permite:

- A. Ejecutar una aplicación en sistemas Linux.
- _ B. Ejecutar una aplicación y sus dependencias en procesos de recursos aislados.
- C. Ejecutar aplicaciones Java y sus dependencias en procesos de recursos aislados.
- D. Ejecutar una aplicación y sus dependencias en procesos de recursos compartidos.

Los contenedores son un método de virtualización del sistema operativo que le permite ejecutar una aplicación y sus dependencias en procesos de recursos aislados.

Mediante la informática sin servidor, puede crear aplicaciones modernas con mayor agilidad:

- A. Y mayor resiliencia al cambio.
- B. Y menos sistemas de seguridad.
- C. Y menos recursos físicos.
- _ D. Y menor costo total de propiedad (TCO).

Mediante la informática sin servidor, puede crear aplicaciones modernas con mayor agilidad y menor costo total de propiedad (TCO). Si utiliza una arquitectura sin servidor, puede centrarse en su producto principal. No tiene que preocuparse por administrar ni operar servidores ni entornos de tiempo de ejecución, ya sea en la nube o en las instalaciones.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

DevOps, DevOps Culture y DevSecOps

Índice

Esquema	3
Ideas clave	4
9.1. Introducción y objetivos	4
9.2. ¿A qué llamamos DevOps?	6
9.3. Principios y metodologías DevOps	11
9.4. Introducción a la colaboración	17
9.5. Factores que aceleran la adopción de DevOps	22
9.6. Seguridad en DevOps (DevSecOps)	28
9.7. Referencias bibliográficas	39
A fondo	41
Test	42

DEVOPS - CULTURA DEVOPS - DEVSECOPS

DevOps

▲ Conceptos DevOps:

- Sistemas ágiles.
- Herramienta de cambio.
- Aceleración de la innovación.
- Colaboración.
- Despliegue automático.
- Incremento de la eficiencia.
- Reducción de fallos.
- Aceleración del tiempo de recuperación.

▲ Principios:

- Fomentar el ambiente de trabajo.
- Establecer responsabilidades.
- Mejora continua.
- Automatización.
- Priorizar al cliente.
- Aceptar el fracaso.
- Equipos multidisciplinares.

Cultura DevOps

▲ Prácticas:

- Identificar objetivos de negocio.
- Medición de la cultura.
- Creación ambiente intercambio.
- Equipo DevOps.
- Procesos DevOps.

▲ Problemas:

- Procesos para controlar.
- Falta visibilidad.
- Poca capacidad innovación.
- Dificultades técnicas evitables.
- Miedo y resistencia al cambio.
- Shadow IT.

DevSecOps

▲ Soportes:

- Personas.
- Procesos.
- Tecnología.

▲ Perfiles:

- DevOps Engineer.
- Traditional Security.
- Agile Project Manager.

▲ Prácticas:

- Formación.
- Seguridad integrada.
- Seguridad en todo el Ciclo de vida.
- Automatización.
- Monitorizar cumplimiento seguridad.
- Herramientas de seguridad.
- Manifiesto DevOps.

Esquema

9.1. Introducción y objetivos

DevOps no puede entenderse únicamente como un conjunto de tecnologías y mejores prácticas, sino que debe entenderse como una **revolución cultural**. Es una corriente de cambio dentro de las organizaciones en la que se prima la **colaboración** y la **interacción abierta** entre los equipos de tecnología de la información (en siglas, TI) y aquellos de desarrollo. El **objetivo primordial** de esta sinergia es la de alcanzar la máxima eficiencia y agilidad en el desarrollo, despliegue y mantenimiento de las aplicaciones.

Numerosas organizaciones han incluido la virtualización como la mejor forma de optimizar sus recursos computacionales, ya que da nueva vida a infraestructuras obsoletas y obtienen una flexibilidad mucho mayor a la hora de utilizar sus recursos computacionales. La revolución de la virtualización ya no es tal y el nuevo ciclo de cambio ya está en marcha.

Ya no es suficiente hablar de Cloud Computing como una forma de obtener recursos computacionales de manera elástica y en un modelo de pago por uso, sino que, además, aparece el concepto de **datacenter virtual**, donde podemos tener sus ventajas con los beneficios de la nube.

Por lo general, solemos utilizar el término **cambio cultural** para implantar procesos de mejora continua, emplear metodologías ágiles y adoptar el enfoque DevOps. Esto obedece a la imposibilidad de adoptar las transformaciones de forma superficial. Tal y como hemos visto con anterioridad, esta metodología exige un cambio en la gestión de los recursos, la organización de los equipos y también en la forma de pensar. Es necesario pasar del **control al autoservicio**, del **miedo a la visibilidad**, de los **silos a**

la colaboración abierta y ninguno de estos cambios se puede imponer, de un cambio organizativo muy beneficioso a la vez que radical.

A través del estudio de este tema, se pretende conseguir los siguientes **objetivos**:

Comprender el concepto de DevOps y cómo se aplican las tecnologías y mejores prácticas que se conocen como DevOps.

Conocer los factores que intervienen en la implementación del cambio cultural necesario para la adopción de la metodología DevOps en una organización.

Comprender qué implica el cambio cultural de una organización para la adopción de la metodología DevOps.

Conocer las herramientas de colaboración que ayudan a los equipos DevOps a realizar su trabajo de forma exitosa.

Introducirnos en la seguridad en el mundo DevOps.

En el vídeo de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Repasaremos las buenas prácticas y principios de DevOps y veremos las herramientas y servicios que posemos utilizar en nube pública.

9.2. ¿A qué llamamos DevOps?

DevOps es un término relativamente nuevo para describir lo que también ha sido llamado como **administración de sistemas ágiles**. DevOps se centra en el trabajo y la colaboración de los equipos de desarrollo y operaciones.

En la actualidad, las empresas se mueven a la velocidad de la nube y es por ello por lo que DevOps se ha convertido en un enfoque cada vez más extendido y popular para la entrega de software. Los equipos de desarrollo y operaciones utilizan esta metodología para construir, probar, implementar y monitorizar las aplicaciones debido a que les permite: actuar con **velocidad**, mantener altos **índices de calidad** y **controlar los cambios** con suma rapidez.

DevOps es esencial para cualquier empresa que aspire a ser ágil y que pretenda ser capaz de responder rápidamente a las demandas del mercado. Por lo tanto, DevOps es un **enfoque**:

Hacia la entrega de software ágil.

Que promueve una colaboración estrecha entre las líneas de negocio, desarrollo y operaciones de TI.

Que elimina las barreras entre las partes interesadas y los clientes.

Es necesario que todas las partes involucradas en el proceso de entrega de software estén dispuestas a colaborar entre sí. Los **equipos de desarrollo** necesitan diseñar, desarrollar, entregar y ejecutar el software de la manera más rápida y confiable posible. Los **equipos de operaciones** deben identificar y resolver los problemas de forma temprana mediante la monitorización, la predicción de fallos, la administración de los entornos y la solución de problemas. La combinación de este enfoque común permite monitorizar y analizar los cuellos de botella y optimizar las capacidades de DevOps que busca, ante todo, fomentar la colaboración entre el área de negocio y

los departamentos de desarrollo y operaciones para entregar y ejecutar el software lo antes posible.

DevOps como herramienta de cambio para las organizaciones

Para ir adentrándonos en la materia, necesitamos comprender la **necesidad** y el **valor** de DevOps a nivel de negocio, así como los **principios** de DevOps. Como sabrán, realizar un cambio en el «*business as usual*» siempre es difícil y, por lo general, requiere de una inversión. Así que, cada vez que una organización adopta cualquier nueva tecnología, metodología o enfoque, la adopción de esta tiene que ser impulsada por una **necesidad de negocio**. Es decir que, al desarrollar un modelo de negocio para la adopción de DevOps, se debe entender la **necesidad de la empresa**, incluyendo los retos a los que se enfrenta.

En un entorno DevOps, los equipos son responsables de ofrecer nuevas características, pero también estabilidad, escalabilidad, fiabilidad y otro sinfín de propiedades comunes a un software de calidad. Es por ello por lo que las responsabilidades se equilibran para garantizar que ambos equipos (desarrollo y operaciones) tengan **visibilidad del rendimiento** de la aplicación a través de todas las etapas del ciclo de vida.

Debido a los beneficios que reporta a todo el conjunto de la organización, DevOps continúa ganando **adeptos** año tras año desde su aparición. Según los datos de la popular web DevOps.com, la tasa de adopción aumentó sustancialmente, de 2015 a 2016, del 66 % al 74 %. ¿Qué impulsa a las empresas hacia la adopción de DevOps? La respuesta es muy sencilla: los largos plazos de entrega del software hasta su paso a producción, derivado de los sistemas de trabajo tradicionales, impiden a las empresas brindar **servicios de vanguardia** y mejorar la **experiencia** del cliente. Para mantener el ritmo de las demandas del mercado, los equipos de TI deben construir, implementar, probar y lanzar software en ciclos cada vez más rápidos.

Debido a que DevOps mejora la forma en que una empresa entrega valor a sus clientes, proveedores y socios, podemos afirmar que es un **proceso esencial de negocio** y no solo una capacidad de TI. No debemos olvidar que esta metodología ofrece un retorno significativo de la inversión, en inglés Return of Investment (en adelante, ROI), por diversas razones:

Aceleración de la innovación

Con un sólido equipo de operaciones y desarrollo que trabaje acompasadamente, las aplicaciones se pueden desarrollar y desplegar con mayor rapidez. Esto es vital, ya que el **éxito empresarial** actual depende, en gran medida, de la **capacidad de una organización** para innovar más rápido que la competencia. Los ingenieros DevOps pueden aprovechar las herramientas tecnológicas de las que disponen para comprobar los datos de rendimiento de sus aplicaciones en tiempo real, y así, comprender el impacto de los cambios introducidos en el código. A su vez, las soluciones a los problemas derivados del software se proveen en plazos de tiempo más cortos porque los miembros del equipo solo necesitan verificar los últimos cambios de código para detectar el origen de los errores.

Mejora de la colaboración

En vez de intentar desdibujar las diferencias entre ambos departamentos (desarrollo y operaciones), un entorno DevOps exitoso procura construir un puente basado en la **comunicación** y la **colaboración** para crear una sinergia. La cultura de desarrollo de software, entonces, se enfoca en **objetivos, responsabilidades y logros compartidos**, en lugar de objetivos individuales. Cuando estos equipos adquieren un buen nivel de confianza y colaboración, pueden experimentar, investigar e innovar de manera más efectiva. El entorno de desarrollo se vuelve, progresivamente, más uniforme a medida que todos los miembros del equipo trabajan para alcanzar objetivos compartidos.

Incremento de la eficiencia

Las herramientas automatizadas y las plataformas de producción estandarizadas son elementos clave de las mejores prácticas de DevOps que ayudan a que las implementaciones sean más predecibles y liberen al personal de TI de tediosas tareas repetitivas. Con un entorno de pruebas e integración automatizadas, los desarrolladores no necesitan desperdiciar su tiempo confiando en que se completen los procesos de integración de código. Estas herramientas ofrecen **oportunidades adicionales** para mejorar la eficiencia:

La **infraestructura escalable**, como las soluciones basadas en la nube, ayudan a acelerar las pruebas y los procesos de implementación al aumentar el acceso a los recursos de hardware.

Las **herramientas de compilación y desarrollo** ayudan a acortar los ciclos de desarrollo y acelerar la entrega del producto.

Los **flujos de trabajo de entrega continua** ayudan a entregar software de manera más rápida y frecuente.

Reducción del número de fallos

Los ciclos de desarrollo más cortos, derivados de DevOps, promueven **entregas de código más frecuentes**. Con estas implementaciones modulares, los equipos pueden descubrir e identificar, de forma temprana, problemas asociados a la configuración, el código o la infraestructura.

DevOps eleva el grado de compromiso de los miembros del equipo durante todo el ciclo de vida de una aplicación, lo que da como resultado un código de mayor calidad.

Por lo general, se requieren menos cambios y modificaciones porque los desarrolladores identifican y eliminan posibles problemas a medida que escriben el código. Según un informe reciente elaborado por Puppet Labs, llamado State of

DevOps, las organizaciones que adoptan una cultura DevOps tienen sesenta veces menos fallos que aquellas que no implementan DevOps.

Aceleración del tiempo de recuperación

Como hemos dicho anteriormente, debido a que las implementaciones de DevOps son más frecuentes y «pequeñas», los **errores** son **fácilmente detectables** y es por ello por lo que las **soluciones** se encuentran con **mayor rapidez**. El equipo necesitará verificar los últimos cambios en el código para poder resolver una incidencia en vez de revisar enormes cantidades de código. Los tiempos de resolución son intrínsecamente más rápidos porque, además, la responsabilidad en la solución de los problemas y las correcciones pertinentes corresponden a **un solo equipo**: el equipo DevOps. En el informe [State of DevOps](#) de Puppet Labs, se establece que los equipos DevOps de alto rendimiento se recuperan de los fallos 168 veces más rápido que aquellos con menor rendimiento.

Aumento de la satisfacción laboral

DevOps promueve un ambiente de trabajo basado en el **rendimiento**, en contraposición con una cultura basada en reglas, estatus o poder. Esto reduce los obstáculos burocráticos y fomenta la **responsabilidad compartida** entre todos los miembros. El resultado es un equipo de trabajo más satisfecho y productivo, que ayuda a impulsar el rendimiento del negocio. Los desarrolladores y los ingenieros de operaciones, generalmente, prefieren un entorno DevOps porque pueden trabajar de manera más eficiente y cambiar de roles cuando sea necesario. A su vez, tienen una mejor comprensión sobre cómo impacta su función dentro de TI y dentro de la organización en su conjunto.

La entrega rápida de software es crucial en la era digital actual y la cultura de DevOps es el motor de este proceso. En otras palabras, permite a las empresas acelerar la comercialización de sus servicios y productos al mercado y desplegar nuevas funciones, de manera rápida y eficiente. La adopción de DevOps no es un proceso

simple, pero cuando se realiza correctamente, la inversión genera dividendos que duplican el esfuerzo inicial.

9.3. Principios y metodologías DevOps

En el corazón de los principios DevOps se encuentra el **aprendizaje colaborativo** y las **relaciones de colaboración** entre desarrollo y operaciones. Estos se centran en aumentar el ritmo de trabajo de forma planificada para obtener despliegues e implementaciones más frecuentes, al tiempo que mejora la estabilidad, la resistencia y la seguridad del entorno de producción. Para que una organización pueda abrazar los principios de DevOps, debe reforzar y enfatizar este enfoque holístico en todas sus áreas y no únicamente en los departamentos de desarrollo y operaciones.

Estos **principios fundamentales** pueden resumirse en la siguiente lista:

Fomentar un ambiente de trabajo colaborativo: como ya hemos comentado, la esencia de la metodología DevOps es el trabajo conjunto entre los equipos de desarrollo y operaciones, a fin de crear una **sinergia** que se centre en la consecución de objetivos comunes. Para lograr este objetivo, las organizaciones necesitan fomentar una **comunicación eficiente** y permitir que ambos equipos compartan ideas y resuelvan problemas de forma conjunta. Esta «rotura de los silos» permite a las empresas alinear a sus trabajadores, procesos y herramientas hacia un enfoque centrado en el cliente.

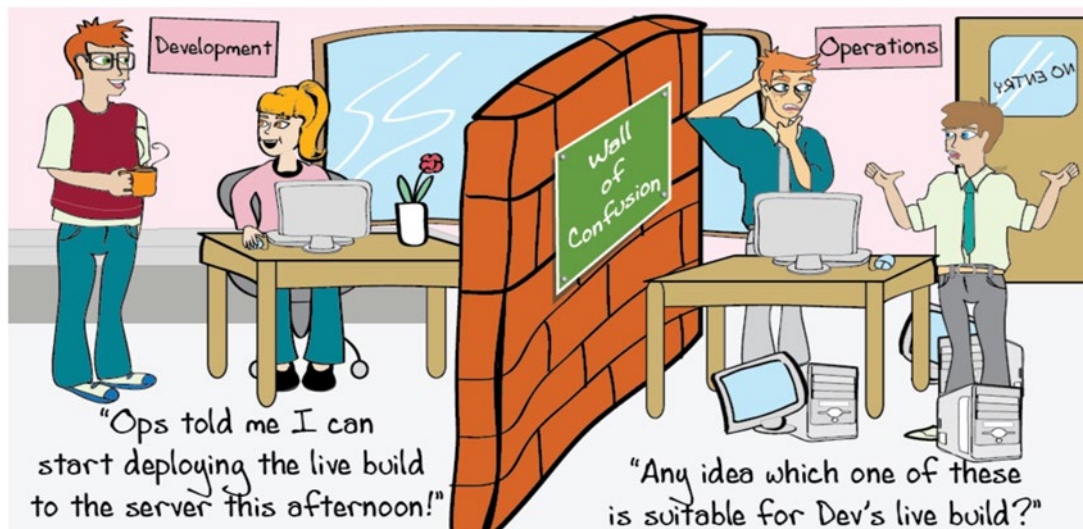


Figura 1. Qué es DevOps. Fuente: Buehring, 2021.

Establecer una responsabilidad de extremo a extremo: en el modelo tradicional de desarrollo de software, desarrollo y operaciones tenían roles separados. Pero en un entorno DevOps, ambos grupos trabajan como un **equipo** que es totalmente **responsable** de la aplicación, de principio a fin. Tradicionalmente, los desarrolladores escribían código y operaciones desplegaba ese código, pero eso producía todo tipo de ineficiencias, problemas de rendimiento y entornos impredecibles. Todos los miembros del equipo DevOps son responsables de velar por la **calidad y fiabilidad** de las aplicaciones que desarrollan.

Fomentar la mejora continua: la responsabilidad de extremo a extremo involucra a **todas las áreas** de la organización, ya que estas deben adaptarse continuamente a las circunstancias cambiantes (ya sea el surgimiento de nuevas tecnologías, las nuevas y variadas necesidades de los clientes o los cambios en la legislación). DevOps pone énfasis en la mejora continua para **optimizar** el rendimiento, los costes y la velocidad de entrega.

Fomentar la automatización: para hacer frente a los numerosos ciclos de desarrollo y responder de inmediato a los comentarios de los clientes, las organizaciones deben implementar procesos automatizados. Afortunadamente, en los últimos

años han surgido variadas y numerosas herramientas de automatización que agilizan la gestión de procesos en gran medida.

Dar prioridad a las necesidades del cliente: DevOps requiere que las organizaciones tengan la misma **flexibilidad** que las *start-up lean*, ya que estas pueden innovar continuamente, cambiar de estrategia cuando sea necesario e invertir en la creación de nuevas características (*features*) para aumentar la **satisfacción** de sus clientes. Los equipos de DevOps deben estar al día de todo lo que ocurre en el mercado para satisfacer constantemente las necesidades y demandas cambiantes de los consumidores. Los datos recopilados de los procesos automatizados deben revisarse con frecuencia para garantizar que se cumplen los **objetivos** de rendimiento establecidos por el área de negocio.

Aceptar el fracaso y aprender de él: la aceptación del fracaso fomenta un clima de aprendizaje que tendrá un impacto positivo en la cultura organizacional. Cuando los equipos se sienten psicológicamente seguros para actuar y tienen la capacidad de transformar radicalmente su trabajo, pueden incurrir en fallos. Resulta vital convertir esos fallos en **oportunidades** de aprendizaje.

Formar equipos multifuncionales: es necesario que los equipos de DevOps se involucren en cada etapa del ciclo de vida del desarrollo de software, desde la planificación, la construcción, la implementación, y la retroalimentación hasta la mejora continua. Para ello, es necesario que el equipo sea **multifuncional y equilibrado** y que cada miembro posea un conjunto habilidades idóneas para su rol en TI.

Además de estos principios, existen otros que se centran, más específicamente, en el **área de TI** y que abogan por una mejora integral de sus procesos. Sin embargo, tienen un enfoque holístico para DevOps y organizaciones de todos los tamaños pueden adoptarlos. Estos **principios** son:

Trabajar en entornos similares a producción durante el desarrollo del software y la ejecución de pruebas. Hace alusión al concepto DevOps de «desviación a la izquierda» o *shift left*. El objetivo que persigue este principio es el de permitir que los equipos de desarrollo y calidad trabajen en entornos similares a producción a fin de que puedan verificar el comportamiento y desempeño de la aplicación mucho antes de que esté lista para su despliegue. Desde una perspectiva de operaciones, este principio tiene un enorme valor porque permite que el equipo pueda ver, desde una fase temprana del ciclo, cómo se comportará su entorno cuando soporte a la aplicación.

Realizar despliegues mediante procesos repetibles y fiables. Permite que el área de operaciones dé soporte a un proceso de desarrollo de software ágil e iterativo, durante todas las fases del ciclo de vida que atraviesa el código. La automatización es esencial para crear procesos que sean iterativos, frecuentes, repetibles y fiables, por lo que la organización debe crear una fuente o suministro de entregas que permita que el despliegue sea continuo, automatizado y validado a través de pruebas. Los despliegues frecuentes también permiten a los equipos poner a prueba sus propios procesos y, a largo plazo, hace que se reduzca el riesgo de fallos.

Supervisar y validar la calidad operativa. Pone énfasis en la monitorización del código desde una fase temprana al exigir que el código sea validado a través de pruebas automatizadas al comienzo del desarrollo y con una alta frecuencia, a fin de controlar las características funcionales y no funcionales de la aplicación. Esta supervisión frecuente proporciona una alerta temprana sobre cuestiones operativas o de calidad que puedan ocurrir más tardíamente en producción.

Amplificar los bucles de retroalimentación. Exige a las organizaciones la creación de canales de comunicación eficaces que permitan a todas las partes interesadas acceder y participar en un ciclo comunicativo. De ello se desprende que el desarrollo de código tendrá el foco puesto en las prioridades del proyecto y podrá ajustarse a los requisitos del proyecto si estos merman. También implica que los

pases a producción se harán mediante la mejora de los entornos de producción y que los planes de entrega se modificarán en pos del negocio.

Metodologías DevOps

DevOps tuvo sus orígenes en **Agile**. Para comprender mejor esta afirmación, es importante recordar que Agile promueve el desarrollo de software a través de iteraciones cortas, la entrega continua de nuevas características, así como correcciones de errores en ciclos rápidos, de entre dos a cuatro semanas. Por su parte, DevOps elimina los silos que separan a los equipos de desarrollo y operaciones con el objetivo de disminuir el tiempo de respuesta a sus clientes y garantizar la entrega continua de software.

DevOps no solo involucra a todas las áreas de una organización en el proceso de desarrollo (líneas de negocio, proveedores involucrados en la entrega de software y propios consumidores), sino que además lo hace de una manera en la que **acelera el desarrollo y mejora la calidad** de todos sus procesos. Esto conduce a la creación de una **cultura de la innovación** que permite a las organizaciones colaborar y reaccionar con agilidad ante los cambios del mercado.

Las **metodologías** de DevOps incluyen:

Integración continua que incluye a la creación de código, la construcción, la integración y las pruebas.

Entrega continua que incluye a la integración continua, pero se centra en las entregas.

Despliegue continuo que se centra en automatizar las entregas de la forma más rápida posible.

Gestión de la configuración y monitorización continua.

Estrategia DevOps

Como hemos visto, DevOps se basa en el concepto de **continuidad**. No es una casualidad que hayamos mencionado el concepto de integración, entrega y despliegue continuos. La estrategia de DevOps se centra en la **capacidad empresarial** para la entrega continua de software que permite a los clientes aprovechar las oportunidades del mercado y, a la vez, reducir el tiempo de retroalimentación de estos.

Permite a los equipos trabajar de forma conjunta y esforzarse en solucionar problemas que puedan aparecer en cualquier parte del ciclo de vida del desarrollo de software. El cliente representa la prioridad máxima de estos equipos y su bienestar es una responsabilidad compartida entre todos.

La estrategia DevOps acelera la entrega de software, ayuda a equilibrar la velocidad, los costes, la calidad y el riesgo y aporta una mayor capacidad para innovar. Además, reduce el tiempo de retroalimentación de los clientes, lo que ayuda a mejorar su experiencia.

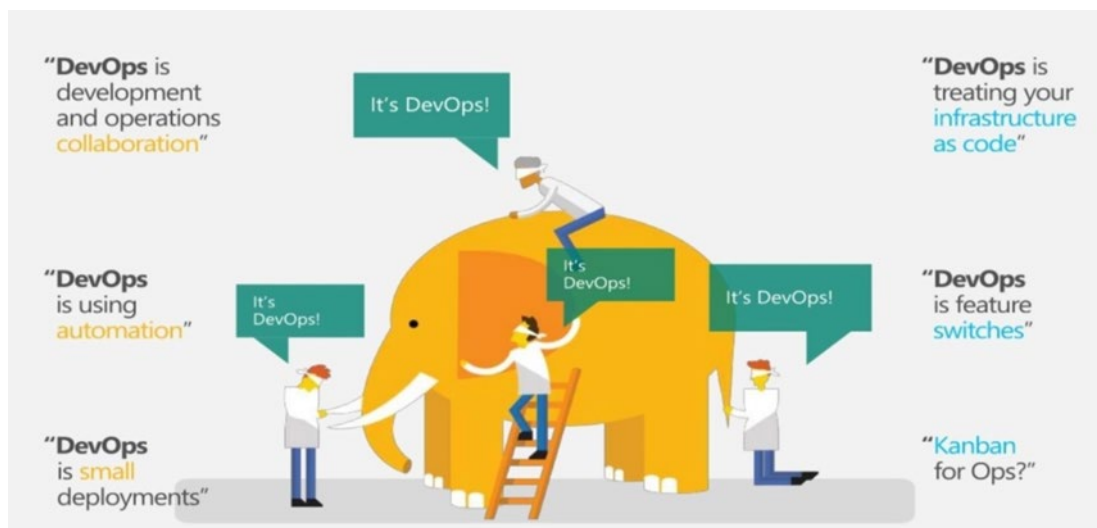


Figura 2. Cómo aplicar la cultura DevOps a tu empresa. Fuente: Bravent IT Consulting, 2019.

9.4. Introducción a la colaboración

Si quisiéramos escribir un decálogo para DevOps, probablemente uno de los elementos más importantes sería la **colaboración**. Recordemos que DevOps integra dos mundos: el del desarrollo y el de las operaciones. Estos individuos adquieren conocimientos y capacidades de ambos entornos y se abren, por tanto, **canales de comunicación** dentro de las organizaciones que permiten alcanzar una mayor agilidad, capacidad requerida para la Integración continua, la entrega continua y los frecuentes *releases* (lanzamientos).

Desde sus cimientos, DevOps es un **movimiento cultural** que involucra a las personas. Una organización puede adoptar los procesos más eficientes o tantas herramientas automatizadas como le sea posible, pero todas ellas serán inútiles si no están integradas con las personas que deben ejecutar esos procesos y utilizar esas herramientas.

En la construcción de una cultura DevOps, por lo tanto, está la clave de la
adopción DevOps.

Una cultura DevOps se caracteriza por un alto grado de colaboración a través de los **roles**, centrado en el bienestar y los objetivos del negocio (en lugar de los objetivos exclusivos de cada departamento), la confianza y el aprendizaje a través de la experiencia. La construcción de una cultura no es como la adopción de un proceso o una herramienta, ya que requiere de cierta la **ingeniería social** de los equipos y cada uno está compuesto por personas que tienen predisposición, experiencia y prejuicios propios. Esta diversidad puede hacer que la construcción de la cultura se convierta en un proceso extremadamente desafiante y difícil.

Las prácticas **Lean** y **Agile** son el núcleo de DevOps y, si la organización ya ha aplicado alguno de estos métodos, podrán apalancar la transición hacia una cultura DevOps.

Identificación de los objetivos de negocio

La primera tarea en la creación de una cultura es conseguir que todos vayan en la **misma dirección** y trabajen por el **mismo objetivo**, es decir, la identificación de los objetivos de negocio comunes para el equipo y la organización en su conjunto. Cuando el equipo sabe cuál es el objetivo común por el que está trabajando y cómo se medirá el progreso hacia ese objetivo, disminuyen los riesgos y el miedo individual y colectivo.

DevOps no es el objetivo final de una organización, sino que es una herramienta que permite alcanzar los objetivos propuestos.

Actualmente, DevOps debe abordar nuevos **retos** empresariales, por lo que la organización puede usar esos desafíos como **punto de partida** para identificar los objetivos que se desean alcanzar y, entonces, desarrollar un conjunto de hitos o peldaños que permitan alcanzar esos objetivos y que los equipos utilicen como guía.

Medición de la cultura

La medición de la cultura es extremadamente difícil. ¿Cómo se mide con precisión la mejora de la colaboración o de la moral de los equipos? Se podría plantear una medición directa de las actitudes y valores de un equipo mediante encuestas, pero las encuestas pueden tener un alto índice de error estadístico, ya que suelen ser, por lo general, pequeñas. A la inversa, se puede tomar una **medida indirecta** mediante el seguimiento de la **frecuencia** con la que un miembro del equipo de desarrollo contacta a un miembro del equipo de operaciones o de *Quality Assurance* (en siglas, QA) para colaborar en la resolución de un problema sin tener que pasar por los canales oficiales o sin que se lo ordene un superior. La **colaboración** y la **comunicación** entre las personas y los equipos es la **esencia de la cultura** de DevOps.

Creación de un ambiente de intercambio

Después de identificar los objetivos comunes de negocio, el siguiente paso en la construcción de una cultura DevOps es que los **líderes** de la organización trabajen con sus equipos para crear un entorno de colaboración e intercambio.

Los líderes deben velar por la eliminación de todas las barreras autoimpuestas para que haya cooperación.

Por lo general, se suele **valorar el desempeño** de los equipos de operaciones a través del *uptime* (tiempo de actividad sin caída) y la estabilidad, y a los equipos de desarrollo por las nuevas características (*features*) entregadas.

La realidad es que esta práctica enfrenta a estos dos grupos: **operaciones** (sabe que la mejor manera de protegerse es no aceptar ningún cambio) y **desarrollo** (tiene poco o ningún incentivo para centrarse en QA). DevOps elimina esta fricción entre ambas áreas, valorando el éxito desde la concepción de una responsabilidad compartida en la entrega de nuevas capacidades y *features*, de forma rápida y segura.

Los líderes de la organización deben fomentar la colaboración mediante la **mejora de la visibilidad**.

El establecimiento de un conjunto común de herramientas de colaboración es esencial, especialmente cuando los equipos están localizados en diferentes puntos del globo y no tienen la posibilidad de trabajar juntos en el mismo espacio físico.

Resulta crucial dar **visibilidad** a todas las partes involucradas en un proyecto sobre cuáles son los **objetivos y el estado** de un proyecto. Esto facilitará la construcción de una cultura DevOps basada en la confianza y la colaboración. Este cambio en la dinámica de grupos y el traspaso de la información tendrán un impacto a nivel

humano y es probable que algunas personas deban ser reasignadas a otros proyectos mientras se esté adoptando esta nueva cultura.

Antes de continuar, te invito a que veas este enlace, Modelos de licenciamiento

Creative Commons:

<https://creativecommons.org/licenses/?lang=es> ES

Creación de un equipo DevOps

Algunas organizaciones (como Netflix) no tienen equipos independientes de desarrollo y operaciones, en cambio, tienen un único equipo llamado **No-Ops**, que comparte ambos roles y responsabilidades. Otras organizaciones han tenido éxito con los **equipos DevOps de enlace**, que resuelven cualquier conflicto y fomentan la colaboración. Dicho equipo está conformado por representantes de todos los equipos y tienen una participación en la aplicación que está siendo desarrollada. Cualquiera sea la organización de equipos, si decides tener un equipo DevOps, el objetivo más importante es asegurarte de que funciona como un **centro de excelencia**, que facilita la colaboración y no añade un nuevo nivel de burocracia.

Procesos DevOps

Anteriormente, hemos hablado sobre el **papel** que tienen las **personas** y la **cultura** en la adopción de DevOps.

Los procesos definen lo que esas personas deben hacer y cuándo deben hacerlo.

Por tanto, la organización puede tener una gran cultura de colaboración, pero si la gente está realizando las tareas de forma errónea o, por el contrario, están trabajando eficazmente, pero en el camino equivocado, el fracaso es todavía más probable.

Como ya hemos mencionado, DevOps es una capacidad que afecta a toda la empresa, haciendo que el negocio sea más ágil y que mejore la entrega de características a los clientes. También podemos ir más allá y entenderlo como un **proceso de negocio**: un conjunto de actividades o tareas que produce un resultado específico (producto o servicio) para los clientes.

El proceso de negocio de DevOps consiste en llevar las capacidades fijadas normalmente por los directivos, dueños o inversores del negocio, a través del desarrollo y las pruebas, hasta que llegue a producción.

Aunque este proceso de negocio no es lo suficientemente maduro para ser capturado en un flujo de procesos simples, es necesario poder capturar los procesos que la organización utiliza actualmente para producir y entregar las características. Esto servirá como **punto de partida** para realizar las modificaciones, cambios y vueltas de tuerca necesarias para **impulsar el cambio** hacia una cultura DevOps.

En la Figura 3 podemos visualizar cómo estos objetivos de negocio se integran con las tareas y capacidades del equipo de tecnología de la información (en adelante, TI). En un ciclo de vida DevOps, personas y procesos deben integrarse bajo el lema de la integración y entrega continua e incorporar las herramientas de software adecuadas para un resultado exitoso.

El resultado de dicha integración permitirá crear una **sinergia** que beneficiará a los **clientes y usuarios finales**, pero también a la **organización**, que ganará agilidad, flexibilidad y eficiencia.

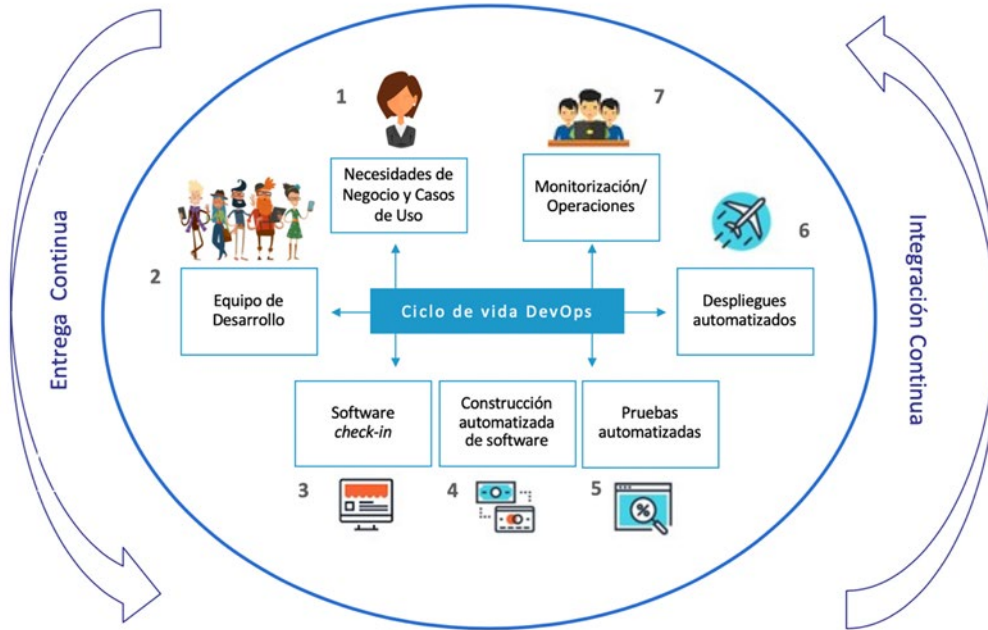


Figura 3. Ciclo de vida DevOps. Fuente: elaboración propia.

9.5. Factores que aceleran la adopción de DevOps

Para que la adopción de DevOps sea exitosa, es necesario que la organización tenga el **objetivo de innovar**. Sin embargo, también es frecuente que la decisión de implantar este enfoque se produzca por la aparición y repetición de problemas recurrentes:

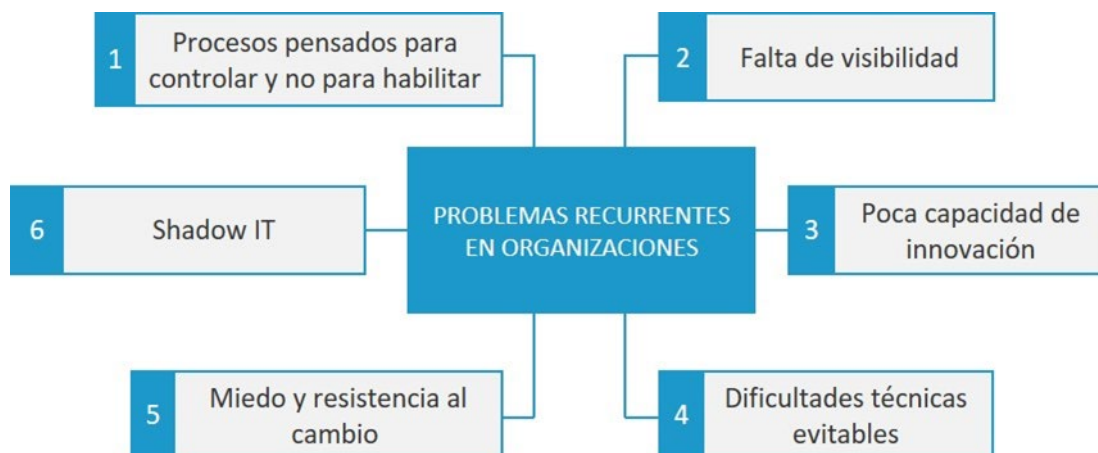


Figura 4. Problemas recurrentes en las organizaciones. Fuente: elaboración propia.

Procesos pensados para controlar y no para habilitar

Imaginemos el siguiente escenario: un equipo de cien desarrolladores necesita tener acceso a los recursos de pruebas para realizar sus tareas habituales y cotidianas (desarrollo, demos, etc.) y la empresa desea que estos recursos se consuman en la nube con el objetivo de que sean más ágiles y evitar así una inversión en hardware.

En una organización tradicional, podrían surgir muchos **inconvenientes** si se diese a los desarrolladores acceso a esos recursos (gasto descontrolado, poca visibilidad, imposibilidad de prever, etc.). A continuación, probablemente se definiría un proceso para **recontar y contabilizar los recursos** que se están consumiendo y, de alguna forma, aprobar este consumo. Por último, es posible que se generase un *workflow* (ciclo) de aprobaciones que, si bien funcionaría, dejaría mucho que desear en términos de eficiencia y agilidad.

Desde el enfoque DevOps, debemos responder al interrogante de cómo se puede implementar un sistema para que los desarrolladores puedan acceder de forma eficiente y rápida a los recursos, con control y visibilidad.

La respuesta, como es de esperar, será diferente a la de una empresa tradicional. Se definirán las **políticas** que permitan a los desarrolladores autoabastecerse de los **recursos** necesarios, mediante unas limitaciones definidas *a priori*. Un ejemplo real puede ser el de una empresa que cuente con un portal autoservicio (*self-service*) que tenga un presupuesto ya aprobado para permitir al equipo que invierta en herramientas y recursos, de acuerdo con las políticas que ha dictado la empresa. De esta manera, se elimina la burocracia de los ciclos de aprobación.

Recordemos que este enfoque favorece la independencia, la responsabilidad, la eficiencia y, por supuesto, la colaboración.

Falta de visibilidad

El hecho de establecer **múltiples puntos de control** es una forma de asegurar que las tareas, las personas, los recursos y demás elementos que componen la organización, sean gestionados de forma correcta. Sin embargo, también es sabido que todos estos puntos de control y **procesos burocráticos** pueden generar cuellos de botella o incluso generar una pérdida de tiempo y dinero por la monitorización de aspectos que no generan ningún beneficio a la compañía.

En línea con lo que hemos comentado en el apartado anterior, el hecho de tener una buena visibilidad reporta muchas más **ventajas** que ejercer un control excesivo. Por decirlo de otra forma, una buena visibilidad en conjunto, con altos niveles de responsabilidad y compromiso, permitirá al equipo **reaccionar** ante cualquier situación **y cambiar de rumbo**, mientras que un control férreo ralentizará a todo el equipo y no garantiza un resultado exitoso.

Otro caso de **falta de visibilidad**, lo podemos encontrar en las organizaciones que utilizan cuentas Amazon Web Services (en siglas, AWS) no asociadas, lo que eleva el gasto a nivel corporativo porque, al no haber una asociación de cuentas, no se pueden aprovechar de los descuentos corporativos por volumen que ofrece el proveedor de nube.

Poca capacidad de innovación

La innovación en sí misma implica **agilidad**. Resulta imposible realizar cambios realmente significativos e innovadores en una organización encorsetada por una maraña de reglas. Debe haber un cambio de filosofía en las empresas que aporte más **flexibilidad** a sus procesos.

Algunas veces es fácil identificar los puntos de mejora, sin embargo, resulta difícil encontrar la mejor solución posible a un determinado *issue*, así que aquí es donde la

innovación entra en escena. Hablando desde un punto de vista meramente técnico, además de un buen análisis de la situación y de las posibles soluciones, es importante que los equipos tengan la libertad para probar, cometer errores y aprender de ellos.

En esto se basa, principalmente, la innovación, así que trataremos a este proceso como un proceso de definición, prueba, error, éxito y /o aprendizaje; un ciclo con tantas iteraciones como sean necesarias.

Dificultades técnicas evitables

DevOps aboga por la **eliminación de la complejidad**, ya que esta va en detrimento de la productividad y la eficiencia. Seguramente conozcas aquel dicho que reza: «Lo bueno, si simple, dos veces bueno». Pues bien, en muchos entornos las dificultades técnicas que se perciben como «técnicas» son en realidad **dificultades humanas** y, a menudo, están causadas porque se trabaja con procesos extremadamente complejos, que no hacen más que incrementar los puntos de fallo.

En las organizaciones tradicionales, existen ciclos de aprobación de cambios de código, comités de aprobación de pase a producción y, por supuesto, esto no hace más que añadir complicaciones a la hora de revertir el pase y corregir el código. En una organización DevOps, la **gobernanza** está en manos del **equipo DevOps**, y ellos pueden acceder al código y subsanar cualquier problema que surja en un breve plazo de tiempo, sin la necesidad de que un comité o departamento los autorice (esta es la verdadera agilidad a la que hacemos referencia cuando hablamos de DevOps).

Miedo y resistencia al cambio

El miedo es endémico a la vida y al ser humano, de la misma forma que es el peor enemigo de la innovación y la colaboración. Nos encontraremos con el simple miedo **al cambio**, que incluso es un motivador positivo en las proporciones adecuadas, o el miedo a la **pérdida de control**, que en el caso de DevOps debería entenderse como

un paso hacia la independencia y la visibilidad. Pero el peor, sin dudas, es el miedo a la **pérdida del statu quo** por parte de aquellas personas que quieren ser imprescindibles.

En un equipo DevOps, las personas pueden jugar diferentes roles en el transcurso de un proyecto, pero también **comparten información** sobre los avances, estados y alcance, entre otras cosas. Aquellos miembros que se guardan información, conocimiento y herramientas, creyendo que así serán imprescindibles, tienen cada vez menos cabida en las organizaciones que desean crecer.

Como indica Paz Cariñena (2016), las personas imprescindibles no atesoran los conocimientos como propios, sino que los **comparten**, están siempre predispuestas a **ayudar** a los demás y a emprender nuevos retos. Estas personas son imprescindibles porque sin ellas las cosas se podrían hacer (no vamos a decir que no), pero con más esfuerzo o incluso con menos calidad. Pero lo más importante de todo, es que el mundo gira cuando no están, porque consiguen «enseñar a pescar» y no solo dan pescado cuando se necesita.

Esa es la fórmula que describe el espíritu de un DevOps: colaborar + compartir
+ ayudar + producir.

Shadow IT

El Shadow IT (o TI oculta), engloba a todas las tecnologías y elementos dentro de TI que están fuera de control o simplemente son desconocidas. Siguiendo nuestro enfoque de colaboración, innovación y agilidad, la falta de control no es el problema, sino que lo es la **no estandarización** y la **falta de visibilidad** derivada de ello.

Es importante comprender que, en muchos casos, la necesidad imperiosa de ejercer control es lo que ha causado el Shadow IT, es decir, la imposibilidad de acceder a los recursos en tiempo y forma. Numerosas organizaciones, grandes o medianas, se han

visto en la tesitura de estar usando recursos en la nube antes de poder siquiera diseñar una estrategia de nube.

Entre los **ejemplos** más frecuentes de Shadow IT, podemos destacar el **uso** de:

Aplicaciones de almacenamiento en la nube, como Google Docs o Dropbox.

Redes no permitidas (como la red pública de Internet).

Bring your own device (en siglas, BYOD) no autorizado.

Aplicaciones de terceros como las de sistema en la nube tipo *Software as a Service* (SaaS).

Una de las principales razones que motivan a los empleados a usar aplicaciones no autorizadas, además de su bajo coste, es la facilidad que estas ofrecen en cuanto a acceso, mantenimiento y despliegue.

Entre las principales **consecuencias** que acarrea el Shadow IT podemos encontrar (Rivas, 2018):

Fuga de datos sensibles: a través de aplicaciones de terceros que no estén debidamente supervisadas por el equipo de seguridad de TI.

Procesos lentos: el despliegue de ciertas aplicaciones puede ralentizar u obstaculizar el desempeño de los procesos de la organización.

Ineficiencia en la resolución de problemas: en caso de que algún dispositivo o aplicación de Shadow IT genere un incidente, puede resultar complicado para los profesionales de TI encontrar la fuente del problema.

En el vídeo *Herramientas DevOps en Cloud* vamos a realizar un repaso sobre la cultura DevOps. Veremos en qué consiste, las herramientas necesarias de hoy en día para un ingeniero de desarrollo y de operaciones, así como las diferentes tecnologías y buenas prácticas que debe conocer.



Accede al vídeo

9.6. Seguridad en DevOps (DevSecOps)

La falacia de la sofisticación

Los fallos de seguridad en los programas no tienen por qué ser sofisticados.

¿Alguno conoce StuxNet? Se aprovecha de una mala configuración por defecto de los sistemas Windows en los puertos USB.

¿Alguien conoce el error de desbordamiento de búfer? Se aplica no solo en IT, sino en los coches para modificar los cuentakilómetros. Se basa en que los números necesitan representarse en tipos de datos y es necesario elegir el tipo adecuado y proteger la operación matemática de modo adecuado para evitar dicho desbordamiento.

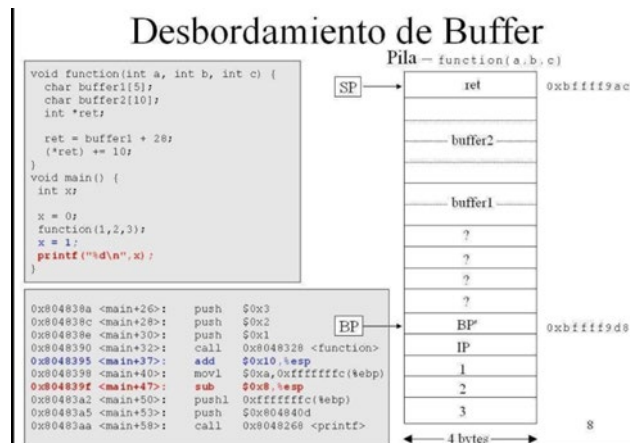


Figura 5. Ejemplo de desbordamiento de búfer visualizado en un cuentakilómetros. Fuente: EcuRed, s.f.

DevOps se convierte en DevSecOps

La seguridad del software debe estar presente e incrustada en todas las fases
del ciclo DevOps para convertirlo en DevSecOps.

El gráfico siguiente muestra cómo la **seguridad** es un **proceso continuo**, cómo los elementos de detección temprana en el software y de detección durante la operación son vitales para reducir el tiempo de exposición a las amenazas y evitar un mal mayor.

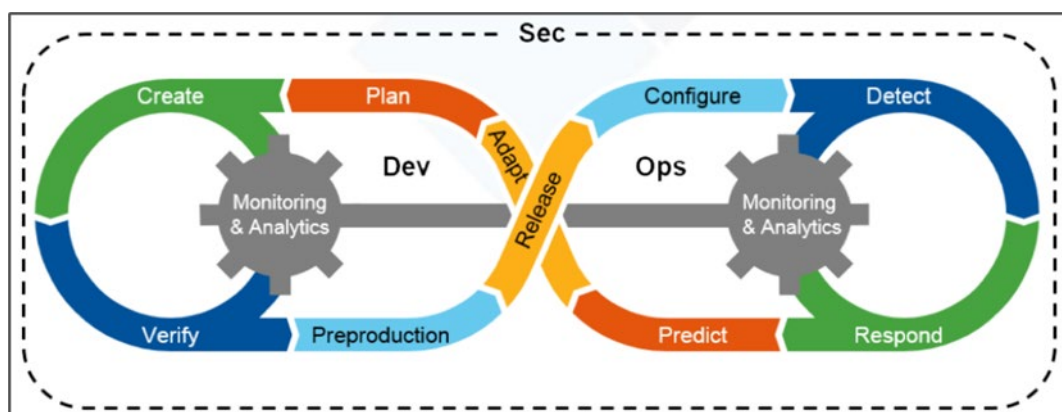


Figura 6. Ciclo DevSecOps. Fuente: Claranet, s.f.

La aplicación de DevSecOps en la organización debe tener tres soportes que están interrelacionados:

Personas (los equipos que aplican e interiorización la seguridad desde el diseño, que sienten que la seguridad es labor de todos).

Procesos (ayudan con las automatizaciones a que la seguridad se verifique).

Tecnología (determina el tipo de errores de seguridad inherentes al tipo de lenguaje y al tipo de entornos donde se ejecuta).

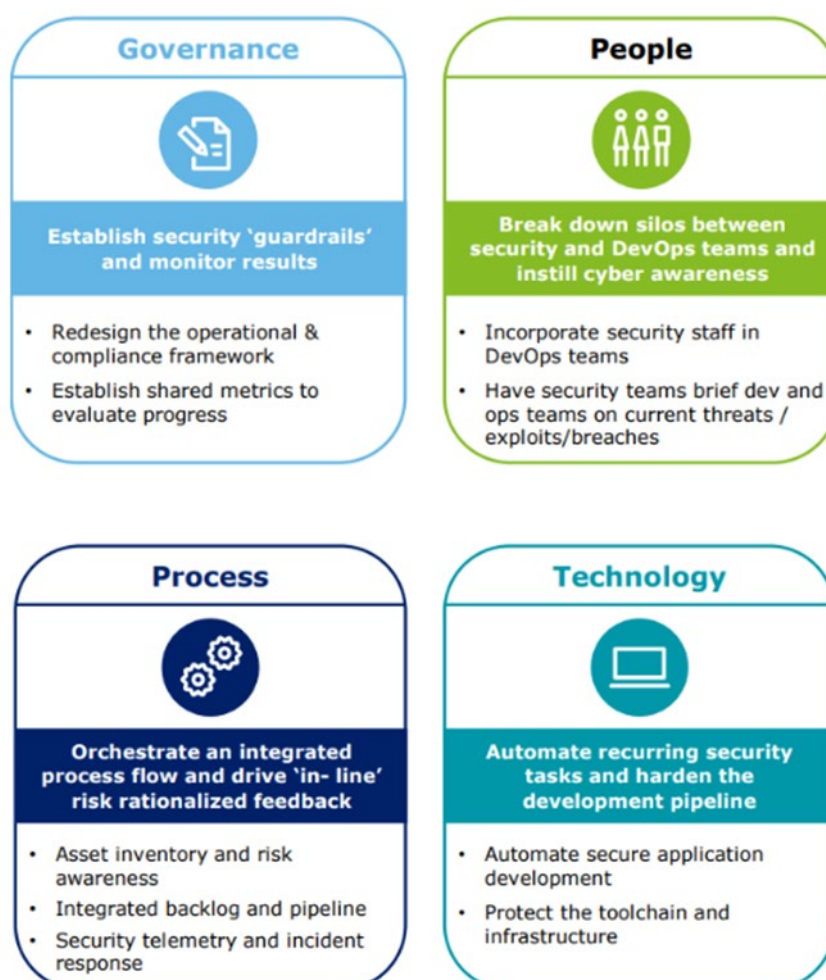


Figura 7. Soportes de DevSecOps. Fuente: Cuesta, 2022.

La seguridad del software no es una lastra en el tiempo de desarrollo.

Cuanto antes se identifiquen los problemas, menos coste se tiene en la resolución. Este **principio *shift-left*** que es conocido para las incidencias funcionales, rara vez se aplica a las de seguridad que deben tomarse como un **requisito funcional** más en el desarrollo.

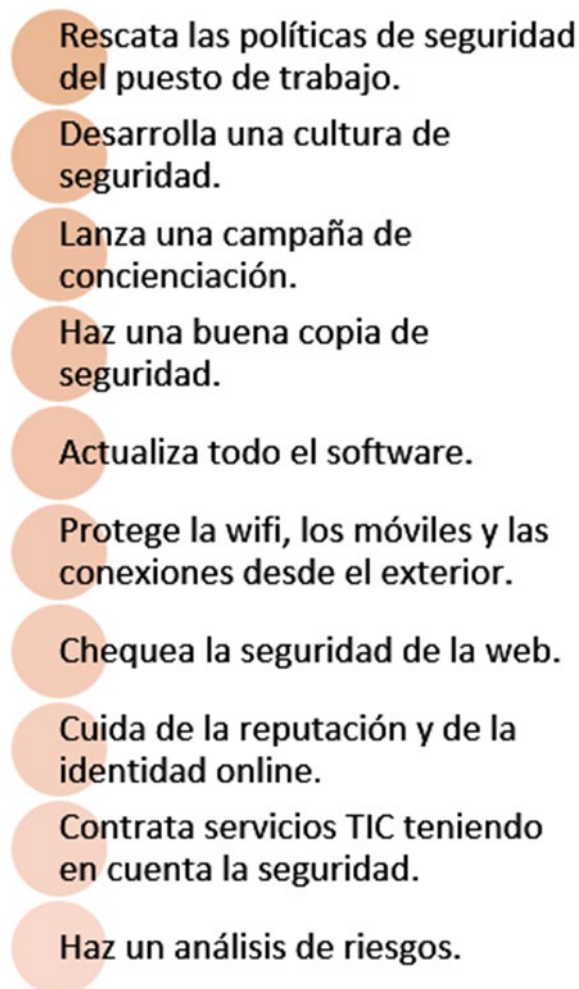


Figura 8. Pilares de la ciberseguridad. Fuente: Instituto Nacional de Ciberseguridad, 2017.

Hay que asegurar que los equipos entiendan que **la seguridad es cuestión de todos**, que es vital para integrarla dentro del ciclo y eficientar costes y tiempo para arreglar los problemas.

Interiorizar el cambio

DevSecOps es una **filosofía**, no es un producto que se compra. De hecho, es una **necesidad**. Existen herramientas que ayudan a integrar la seguridad dentro del ciclo, pero no hay una herramienta única que se pueda comprar e integrar para cubrir todas las etapas. Es importante que las personas entiendan que la **seguridad** es algo que debe hacerse en **cada etapa** y que no es algo que las herramientas *per se* puedan solucionar.

¿Quién es el ingeniero DevSecOps?

El rol de ingeniero DevSecOps no es un superhombre. Puede ser tanto un **perfil de seguridad de software tradicional** que evoluciona a un mundo agile, o un perfil de **ingeniero DevOps** que se enriquece con conocimientos de seguridad. No es una tarea difícil, pero exige que los desarrolladores asuman el principio de que **la seguridad** debe estar en el **diseño del código** desde el inicio.

Traditional Security Vs. DevSecOps (Rugged DevOps)

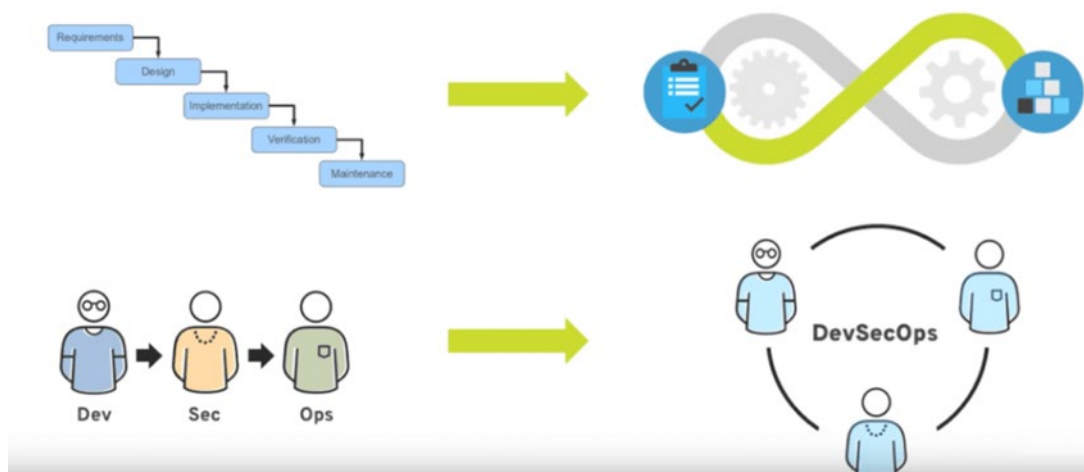


Figura 9. El perfil del ingeniero DevSecOp. Fuente: Singh, 2022.

¿Quién es el Security Champion?

El rol del *Security Champion* en el equipo es importante para divulgar los principios DevSecOps y asegurar que se están aplicando.

Su función debe empezar creando buenas prácticas y dando formación a los equipos sobre la importancia de iniciar las pruebas de seguridad en las etapas más iniciales del desarrollo. Y puede ser cualquier miembro del equipo o puede estar en una tribu, según el modelo organizativo.

En el apartado A fondo tienes a tu disposición un recurso con el que podrás ampliar información.

Formación en seguridad

Practicar la **codificación segura** es **deber** de todos los miembros del equipo DevOps, asegurando que se cumplen las prácticas recomendadas. Aquí, la formación en los **lenguajes de programación** es importante para evitar un mal uso de funciones y errores. Puede parecer un coste, pero un buen conocimiento del lenguaje evita problemas de seguridad.

Debe completarse con **formación** en temas de seguridad a los miembros del equipo, porque el factor humano es uno de los más vulnerables en la cadena de seguridad (como el *phishing* y los olvidos de mantener actualizado a los últimos niveles de parches el software).

Seguridad como parte integrante del modelo

Desde el inicio de la fase de planificación incorporando los requisitos no funcionales de seguridad y durante todo el ciclo.

Dar a los requisitos de seguridad el mismo peso que los requisitos funcionales.

Esto implica agregar controles y procesos de seguridad, así como la automatización de las tareas principales de seguridad en todo el ciclo de vida. Así, se permite a los desarrolladores abordar **vulnerabilidades** conocidas por adelantado y, por lo tanto, proporcionar **software seguro y resistente**.

Probar la seguridad durante todo el ciclo

Para asegurar que en todo momento hemos incorporado seguridad en el ciclo, vamos a realizar **pruebas de seguridad** dentro del proceso continuo y como una parte integral del ciclo de desarrollo. Es decir, tendremos que realizarlas en:

Aplicaciones.

Api.

Contenedores.

Datos.

Procesos.

Microservicios desplegados.

El objetivo es que cuanto antes localicemos los errores, más fácil será eliminarlos. Y así, minimizar las vulnerabilidades sin importar dónde aparezcan.

Automatizar todo lo posible

La automatización **reduce errores de seguridad** porque los programadores pueden ser muy buenos, pero son humanos. La labor del DevSecOps es ayudarles con la introducción de automatismos y herramientas que validen el código en busca de problemas de seguridad.

Estas herramientas, principalmente de análisis estático de código en los entornos de desarrollo, pueden **ahorrar tiempo del ciclo** mientras que una mala elección puede complicarlo. Y si es posible, el análisis se debe repetir en entorno de prueba antes de empaquetar las soluciones para comprobar errores de seguridad en la integración.

Aparte de reducir los errores humanos, la automatización de los chequeos de la seguridad junto con la gestión de la configuración, las pruebas integradas y otras tareas relativas a los procesos reducen la carga de trabajo de los equipos DevOps.

Hay que automatizar todas las pruebas de funcionalidad y de seguridad no funcional tanto a nivel de aplicación como de **infraestructura y configuración**, así como realizar pruebas de seguridad de la **lógica** de la aplicación. Esto es lo que denominaremos *Security as Code*.

Monitorizar siempre el cumplimiento de la seguridad del software

Como hemos visto en el gráfico de Gartner, resolver los incidentes lo antes posible en el ciclo es mejor. Por ello, es necesario obtener información en **tiempo real**, tanto del desarrollo como de la operación, para permitir tomar decisiones más acertadas y rápidas que reduzcan el impacto de las vulnerabilidades. Además, la recopilación de estos datos puede ayudar a mejorar el ciclo de vida con la detección de fallas que puedan ser resueltas con mejores automatizaciones.

Deberá habilitarse *logs* antes y después de una detección de vulnerabilidad e integrar en los cuadros de seguimiento.

Herramientas para integrar seguridad en el ciclo

Hay varias herramientas DevOps que podemos usar para asegurar, con muy pequeñas adaptaciones, algunas de estas funciones de seguridad:

Docker: integrando seguridad en la gestión de los contenedores.

Chef: para automatizar pruebas de seguridad en los despliegues.

Puppete: para probar el cumplimiento y aplicar políticas de seguridad en la integración.

Ansible: para definir y automatizar las mejores prácticas de seguridad como pueden ser la aplicación de políticas estándares de seguridad, configuración de reglas de *firewall*, bloqueo de ciertos usuarios, etc.

SaltStack: para finalmente automatizar reglas de seguridad.

Pero no se debe olvidar que los propios programas y procesos de gestión de seguridad pueden tener **errores de codificación** y, por tanto, es recomendable realizar **chequeos manuales** dentro de las pruebas de seguridad.

En la sección A fondo tienes disponible un recurso con el que podrás completar información.

Usando proveedores en Cloud, como AWS o Azure, es importante señalar que existe el concepto de «responsabilidad compartida» de infraestructura en el que el proveedor se encarga de la seguridad en unos niveles dejando al usuario el resto de los niveles.

En este caso, el margen de maniobra de reducir vulnerabilidades en esa parte se reduce a la notificación a los proveedores de soporte, aunque su eficacia suele ser bastante alta.

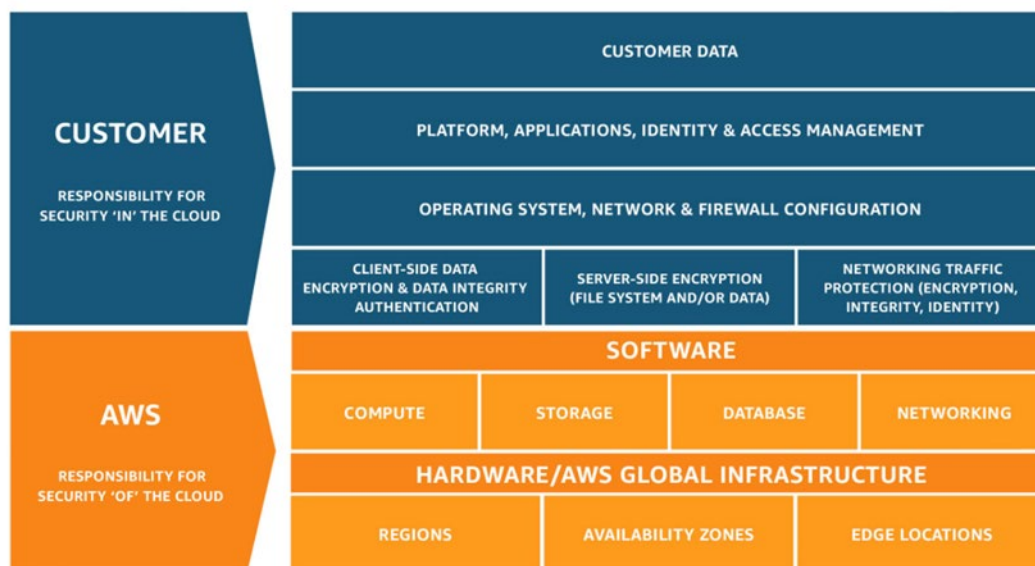


Figura 10. Responsabilidad Compartida entre proveedor Cloud y usuario. Fuente: Amazon Web Service, 2022.

Todos los proveedores de Cloud ofrecen **herramientas de modo gratuito** para realizar un chequeo básico de la infraestructura (AWS Trusted Advisor, Azure Advisor, etc.) y, también de modo avanzado por un pago de servicio, por ejemplo, el AWS Inspector.

En caso de querer usar herramientas gratuitas para revisar la infraestructura y configuración de nuestra instalación AWS, existen **herramientas de código abierto** como la herramienta Scout2.

Puedes descargar esta herramienta a través de la siguiente dirección web:

<https://nccgroup.github.io/Scout2/> .

De este modo vamos a ir incorporando herramientas en todo el ciclo DevOps, siendo un ejemplo no extensivo, el siguiente:



Figura 11. Herramientas de un ciclo DevOps. Fuente: Bhanda y Badam, 2022.

Manifiesto DevSecOps

Para acabar el tema, como no podía ser de otra manera, y como resumen hay que recordar el **Manifiesto DevSecOps** (DevSecOps.org, 2020) para tener siempre presente que la función de la **seguridad** no es entorpecer al desarrollo y a la operación, sino ayudar que **el producto** tenga **mayor calidad y menos posibilidades de fallo**, en este caso por temas de seguridad.

Los **principios** son los siguientes:

«Leaning in over Always Saying «No».

Data & Security Science over Fear, Uncertainty and Doubt.

Open Contribution & Collaboration over Security-Only Requirements.

Consumable Security Services with APIs over Mandated Security Controls & Paperwork.

Business Driven Security Scores over Rubber Stamp Security.

Red & Blue Team Exploit Testing over Relying on Scans & Theoretical Vulnerabilities.

24x7 Proactive Security Monitoring over Reacting after being Informed of an Incident.

Shared Threat Intelligence over Keeping Info to Ourselves.

Compliance Operations over Clipboards & Checklists.

9.7. Referencias bibliográficas

Amazon Web Service. (2022). *Modelo de responsabilidad compartida.*

<https://aws.amazon.com/es/compliance/shared-responsibility-model/>

Bhanda, S. y Badam, R. (2022). *Comprehensive List of DevOps Tools 2022.* Qentelli

<https://www.qentelli.com/thought-leadership/insights/devops-tools>

Bravent. (2019, junio 3). *Cómo aplicar la cultura DevOps a tu empresa.*

<https://www.bravent.net/como-aplicar-la-cultura-devops-a-tu-empresa/>

Buehring, S. (2020, diciembre 2021). *What is DevOps: Free ebook.* Knowledge Train.

<https://www.knowledgetrain.co.uk/it/devops/what-is-devops>

Claranet. (s.f.). *La mentalidad DevSecOps: de la metodología lineal al «shifting left y*

shifting right». <https://www.claranet.es/blog/la-mentalidad-devsecops>

Cuesta, J. (2022, marzo 22). *Evolucionando hacia DevSecOps*. OpenExpo. <https://openexpoeurope.com/es/evolucionando-hacia-devsecops/>

Devsecops. (2020). *Manifiesto*. <https://devsecops.org/>

EcuRed. (s.f.). *Desbordamiento de búfer*. [https://www.ecured.cu/Desbordamiento de b%C3%BAfer](https://www.ecured.cu/Desbordamiento_de_b%C3%BAfer)

Instituto Nacional de Ciberseguridad. (2017, abril 17). 10 pilares básicos para construir la ciberseguridad de tu negocio. *Blog de INCIBE*. <https://www.incibe.es/protege-tu-empresa/blog/10-pilares-basicos-construir-ciberseguridad-tu-negocio>

Paz Cariñena Amigo. (2016, enero 28). *Personas imprescindibles... Las que se lo creen y las que de verdad lo son* [Publicación]. LinkedIn. Recuperado el 9 de noviembre de 2022. <https://www.linkedin.com/pulse/personas-imprescindibles-las-que-se-lo-creen-y-de-son-paz/>

Rivas, G. (2018, agosto 17). Shadow IT: ¿Cómo hacer de la TI invisible una oportunidad para tu organización? *Tech-Blog*. <https://www.gb-advisors.com/es/shadow-it/>

Singh, M. (2022, febrero 23). *DevSecOps Engineers Roles and Responsibilities*. DevOpsSchool. <https://www.devopsschool.com/blog/devsecops-engineers-roles-and-responsibilities/>

What is DevOps?

IBM Technology. (2019, septiembre 9). *What is DevOps?* [Vídeo]. YouTube. <https://youtu.be/UbtB4sMaaNM>

Andrea Crawford, ingeniera y CTO (*Chief Technology Officer*) de DevOps en IBM, explica con sus propias palabras qué es DevOps, cuál es valor que aporta a la organización y cómo sus prácticas y herramientas intervienen en el ciclo de vida de una aplicación.

La cultura DevOps

Exceltic. (2018, octubre 26). *La cultura DevOps* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=Te47jTCDfVM>

Este recurso te ayudará a comprender un poco más sobre la cultura DevOps dentro de las empresas y sus beneficios.

DevSecOps Manifiesto

Página de DevSecOps. <https://www.devsecops.org/>

A semejanza del *Agile Manifesto*, los autores plasman su filosofía de trabajo: cómo acercar las operaciones de seguridad al producto y al usuario final y no al revés.

DevOps es un término relativamente nuevo para describir lo que también ha sido llamado como:

- A. Gestión de desarrollo y operaciones.
- B. Administración de desarrolladores.
- ☐ C. Administración de sistemas ágiles.
- D. Ninguna de las anteriores.

DevOps es un término relativamente nuevo para describir lo que también ha sido llamado como administración de sistemas ágiles. DevOps se centra en el trabajo y la colaboración de los equipos de desarrollo y operaciones.

En un entorno DevOps, los equipos son responsables de:

- A. La gestión TI de la empresa.
- ☐ B. Ofrecer nuevas características, pero también estabilidad, escalabilidad y fiabilidad.
- C. Ofrecer nuevos productos hardware y software.
- D. Ofrecer integración entre desarrolladores y administradores.

En un entorno DevOps, los equipos son responsables de ofrecer nuevas características, pero también estabilidad, escalabilidad, fiabilidad y otro sinfín de propiedades comunes a un software de calidad. El almacenamiento de bloque es una forma de almacenamiento persistente y aplicable que puede montarse como un dispositivo en una máquina virtual.

Un Entorno DevOps exitoso procura:

- ☐ A. Construir un puente basado en la comunicación y la colaboración.
- B. Construir una sólida relación entre dirección y departamento TI.
- C. Construir entornos de pre y producción.
- D. Ninguna de las anteriores.

En vez de intentar desdibujar las diferencias entre ambos departamentos (desarrollo y operaciones), un entorno DevOps exitoso procura construir un puente basado en la comunicación y la colaboración para crear una sinergia.

¿Cuál no es una oportunidad adicional en una herramienta de automatización en el mundo DevOps?

- ☐ A. Integración de incidentes.
- B. Infraestructura escalable.
- C. Herramientas de compilación y desarrollo.
- D. Flujos de trabajo de entrega continua.

Infraestructura escalable, herramientas de compilación y desarrollo y flujos de trabajo de entrega continua.

En el corazón de los principios DevOps se encuentra el:

- A. Aprendizaje ágil.
- B. Aprendizaje múltiple.
- C. Aprendizaje empresarial.
- ☐ D. Aprendizaje colaborativo.

En el corazón de los principios DevOps se encuentra el aprendizaje colaborativo y las relaciones de colaboración entre desarrollo y operaciones. Estos se centran en aumentar el ritmo de trabajo de forma planificada para obtener despliegues e implementaciones más frecuentes, al tiempo que mejora la estabilidad, la resistencia y la seguridad del entorno de producción.

La estrategia de DevOps se centra en la capacidad empresarial para:

- A. La entrega del producto log.
- ☐ B. La entrega continua de software.
- C. La entrega de características funcionales.
- D. Adaptarse a la tecnología.

La estrategia de DevOps se centra en la capacidad empresarial para la entrega continua de software, que permite a los clientes aprovechar las oportunidades del mercado y, a la vez, reducir el tiempo de retroalimentación de estos.

Para que la adopción de DevOps sea exitosa, es necesario que la organización tenga el objetivo de:

- A. Gestionar el cambio.
- B. Adaptarse.
- ☐ C. Innovar.
- D. Mejorar.

Para que la adopción de DevOps sea exitosa, es necesario que la organización tenga el objetivo de innovar. Sin embargo, también es frecuente que la decisión de implantar este enfoque se produzca por la aparición y repetición de problemas recurrentes.

El Shadow IT (o TI oculta), engloba a todas:

- A. Las personas que no son del departamento de TI.
- B. Las personas que se mueven al departamento de TI desde otros departamentos.
- C. Las directrices generadas desde el departamento de TI que no se cumplen.
- ☐ D. Las tecnologías y elementos dentro de TI que están fuera de control.

El Shadow IT (o TI oculta), engloba a todas las tecnologías y elementos dentro de TI que están fuera de control o simplemente son desconocidas. Siguiendo nuestro enfoque de colaboración, innovación y agilidad, la falta de control no es el problema, sino que lo es la no estandarización y la falta de visibilidad derivada de ello.

Las métricas de evaluación tradicionales pasan a un segundo plano y la evaluación de los objetivos de la empresa debe centrarse en:

- A. La satisfacción de los requisitos del *product backlog*.
- _ B. La satisfacción del cliente.
- C. Los beneficios empresariales trimestrales.
- D. Con cultura DevOps no se necesita realizar métricas de evaluación.

Las métricas de evaluación tradicionales pasan a un segundo plano y la evaluación de los objetivos de la empresa debe centrarse en la satisfacción del cliente.

La aplicación de DevSecOps en la organización debe tener tres soportes interrelacionados

- A. Personas, procesos y normas.
- B. Principios, procesos y personas.
- C. Procesos, tecnología y negocio.
- _ D. Personas, procesos y tecnología.

Los equipos que aplican e interiorización la seguridad desde el diseño son personas que sienten que la seguridad es labor de todos. Los procesos ayudan con las automatizaciones a que la seguridad se verifique y la tecnología determina el tipo de errores de seguridad inherentes al tipo de lenguaje y al tipo de entornos donde se ejecuta.

Administración de Sistemas, Virtualización, Cloud
Computing y DevOps

Entornos de integración y entrega continua

Índice

Esquema	3
Ideas clave	4
10.1. Introducción y objetivos	4
10.2. ¿Qué es integración continua?	6
10.3. Repositorio de código	9
10.4. Plataforma de desarrollo	19
10.5. Gestión de versiones	25
10.6. Herramientas orquestadoras de CI	31
10.7. Estrategias de despliegue	34
10.8. Herramientas de gestión de entornos	40
10.9. Promoción entre entornos de la plataforma	
DevOps	43
10.4. Referencias bibliográficas	46
A fondo	47
Test	49

ENTORNOS DE INTEGRACIÓN Y ENTREGA CONTINUA

Integración Continua

▶ Etapas de los Scripts:

- Descargar.
- Compilar.
- Desplegar.
- Enviar Informes.

▶ Repositorio de código:

- Definición.
- Funciones: seguridad, control de cambios y gestión de ramas.
- Git: servidor y cliente.

▶ Plataformas de desarrollo:

- Definición.
- GitHub.
- GitLab.
- BitBucket.

▶ Gestion de versiones:

- Major.
- Minor.
- Patch.

▶ Herramientas orquestadoras de CI:

- Orquestadores.
- *Build*.
- Análisis estático de código.
- Repositorios de artefactos.
- *Testing* automático.
- Despliegues.
- Monitorización.

Despliegue continuo

▶ Estrategias de despliegue:

- Big bang.
- Rolling upgrade.
- Blue-green.
- Canary releases.
- Dark launches.

▶ Herramientas de gestión de entornos:

- Spinnaker.
- Argo CD.
- AWS CodePipeline.

▶ Promoción entre entornos:

- Docker.
- Kubernetes.

10.1. Introducción y objetivos

Dentro de todo el proceso de gestión del software (SCM), una de las etapas que más tiempo suele consumir es el **desarrollo del software**, por lo que es una de las fases donde más **esfuerzo** hay que realizar definiendo procedimientos eficientes que reduzcan su duración.

Dentro de la fase de desarrollo, las **tareas de testing** consumen buena parte del tiempo. Uno de esos procedimientos que se han instaurado en los equipos de desarrollo de software es el de la integración continua (CI).

La **integración continua** es un procedimiento automatizado que se ejecuta periódicamente. Detecta errores de software con el objetivo de reducir la duración de la fase de *testing and bug-fixing* del desarrollo de software.

Profundizaremos en las **herramientas** de las que disponemos para poner en marcha un sistema de integración continua, así como las **buenas prácticas** en el desarrollo y en la organización del equipo de trabajo que impulsan un desarrollo ordenado permitiendo automatizar la integración y alcanzar un mayor control del proyecto.

Las siglas CD puede hacer referencia a dos **conceptos similares**, aunque estrechamente ligados o complementarios:

Continuous Delivery: ejecuta todos los pasos necesarios para ir a producción, excepto la actualización del entorno de producción, paso que se realizará manualmente.

Continuous Deployment: ejecuta todos los pasos, incluyendo el propio despliegue/actualización en el entorno de producción, de manera automática o semiautomática.

Nos centraremos, específicamente, en el **Continuous Deployment** revisando las diferentes estrategias que existen para la puesta en producción, así como las herramientas de las que se dispone para facilitar estas tareas y algunos aspectos para tener en cuenta en el propio desarrollo.

Con el estudio de este tema se conseguirán los siguientes **objetivos**:

Conocer por qué y para qué surgen las técnicas de integración continua.

Repasar los principios de los repositorios de código y los fundamentos de Git.

Asimilar qué es SCM y cuáles son sus principales beneficios y principios.

Comprender las ventajas de las plataformas de desarrollo dentro del ciclo de SCM.

Saber diferenciar entre las múltiples plataformas de desarrollo con sus ventajas e inconvenientes.

Entender la necesidad del versionado semántico para el proceso de CI.

Repasar las principales herramientas que usamos en un proceso de CI.

Conocer las distintas estrategias de despliegue en producción.

Estudiar las ventajas e inconvenientes de cada una.

Entender cuándo son convenientes.

Conocer aspectos para tener en cuenta en el desarrollo de aplicaciones en escenarios con CD.

Descubrir herramientas DevOps para la gestión de entornos y despliegues.

En el vídeo de este tema podremos aprender los siguientes **conceptos**:

Vídeo 1. Realizaremos una integración y despliegue continuo con un servicio de nube pública para poner en servicio una página web estática.

10.2. ¿Qué es integración continua?

Construir y desplegar (*build and deploy*) una aplicación es un **proceso complejo** que suele llevar bastante tiempo si se ejecuta de forma manual, especialmente en sistemas grandes. Actualmente, es habitual que todo proyecto tenga incorporados los **scripts** necesarios para compilar toda una aplicación, resolviendo las dependencias, actualizando los repositorios locales y desplegando la aplicación en el entorno de desarrollo local, de manera que estas tareas puedan ejecutarse lo más **automáticamente posible**.

¿Cuál es el problema?

Cada desarrollador prueba la aplicación en su **entorno local** y dentro del contexto en el que está trabajando, ya sea un módulo concreto, una pantalla, una modificación en el esquema de BD, un servicio web, etc., pero sin verificar que el resto de la aplicación sigue funcionando correctamente o, incluso, sin comprobar si una librería desarrollada dentro del proyecto sigue funcionando y compilándose sin problemas.

Esa librería se encuentra ya precompilada y almacenada en su entorno local, aunque es posible que otro equipo de trabajo haya realizado cambios en ella y el desarrollador no sea consciente y, por tanto, no la actualice y ejecute pruebas que verifiquen que es compatible con la aplicación.

Los procesos de **CI/CD** se encargan de realizar una **validación completa y automatizada** de todo el proyecto, más allá de las pruebas locales que cada desarrollador pueda realizar.

Mediante la ejecución de unos scripts que se basan en comandos sobre unas **herramientas de apoyo** (*ant, maven, git, etc.*) se consigue automatizar completamente todo este proceso de construcción, verificación y despliegue de la aplicación.

Estos scripts se ejecutan cada cierto tiempo (minutos, horas o cada noche) y siguen un proceso que se puede resumir en cuatro etapas:

Etapas de los scripts	
1	Descargar todo el código fuente del proyecto desde el repositorio de código, accediendo a la/s rama/s donde se encuentra el último código validado por los desarrolladores.
2	Compilar todo el código, comprobando previamente que los artefactos establecidos como dependencias se encuentran en el entorno de desarrollo descargados de sus correspondientes repositorios y actualizados.
3	Desplegar la aplicación en un entorno de pruebas y ejecutar test sobre la aplicación, ya sean test unitarios, pruebas de usuario automatizadas, test de servicios web, etc.
4	Enviar informes sobre el resultado de todo el proceso, indicando, entre otros detalles: ▶ Que el código se pudo recuperar de los repositorios. ▶ Que cumple con los estándares de calidad en cuanto a análisis de código estático. ▶ Que se pudo compilar sin errores o dependencias rotas. ▶ Que se pudo desplegar en el entorno de pruebas. ▶ Que las pruebas automáticas se ejecutaron correctamente o, si no, cuáles fallaron y con qué datos de entradas falló para así reproducir el error. ▶ Etc.

Tabla 1. Etapas de los scripts. Fuente: elaboración propia.

De esta manera, se consigue implantar un procedimiento automatizado en el que de forma continua se **verifica** toda la aplicación, **integrando** todos sus componentes y validando que todos siguen funcionando como se estableció.

En cuanto se produce un error, los desarrolladores son informados rápidamente para que este pueda ser corregido. Si no se hiciera de esta manera, el error quizás no se detectaría hasta mucho tiempo después lo que podría provocar más errores en el código, así como regresiones que reducirían la calidad global del software.

Cuanto antes se detecte un error y se corrija más se reduce la cantidad total de errores en el código. Menos errores conllevan a menos tiempo en la fase de *testing and bug-fixing* que es una de las **fases más complejas** dentro de la etapa de desarrollo de software.

Cuando se habla de integración continua (CI) normalmente se acompaña también otro término: **CD** y puede hacer referencia a dos procesos similares:

Continuous Delivery.

Continuous Deployment.

Se trata de dos procesos muy parecidos, aunque con un matiz en cuanto a la automatización del proceso de despliegue.

Algunos proyectos usan CI/CD hasta la entrega de la *release* (que se desplegará luego manualmente a producción). Se trataría de **CI/CDelivery**. Cuando el proceso de despliegue en producción también es automatizable, se habla de **CI/CDeployment**, ya que el sistema es capaz de automatizar todo el proceso y actualizar el sistema de producción de forma automática, lo cual es una tarea más compleja en algunos proyectos, especialmente en cambios de grandes de versiones o en sistema muy grandes con *~0% downtime*.

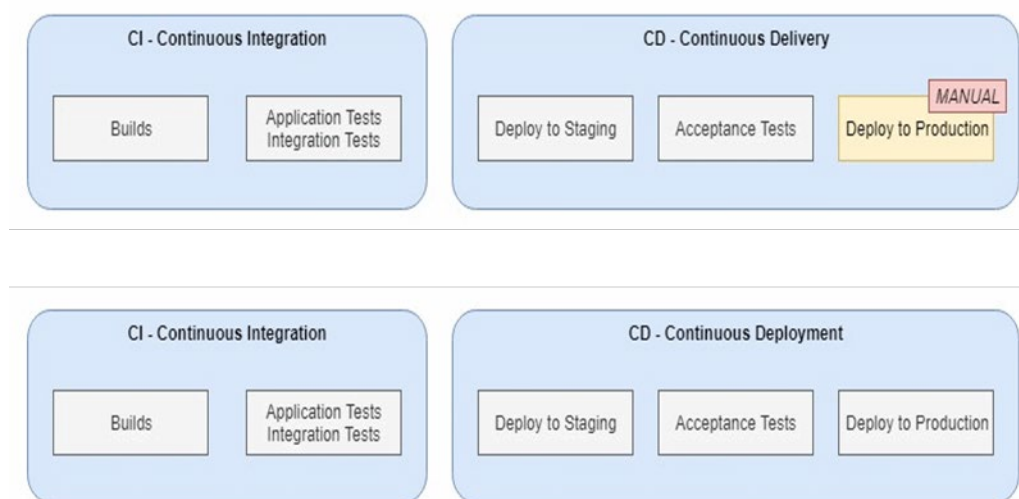


Figura 1. Integración, entrega y despliegue continuo. Fuente: Serrano, 2019,

10.3. Repositorio de código

¿Qué es un repositorio de código?

Un repositorio de código es el **almacén de los ficheros** que conforman el código fuente de un sistema. Aunque pueda dar la impresión de que no existían repositorios de código antes de la aparición de Git/SVN lo cierto es que los repositorios son tan antiguos como el mismo código.

En los orígenes, cada versión de una aplicación se guardaba en copias de directorios de un disco duro y la gestión que se hacía sobre el repositorio era completamente **manual**, por lo que resulta sencillo imaginar los **conflictos** que surgían cuando se trataba de gestionar el código fuente de sistemas con una considerable envergadura como un sistema operativo, por ejemplo, en los que pueden trabajar cientos de personas en diferentes módulos, compartiendo librerías y accesos a código del núcleo común del sistema.

Aunque en el pasado se trataba como un «todo» tanto al propio almacén de código como a las herramientas que permitían trabajar sobre ese almacén de código, lo habitual es tener de **forma diferenciada** el almacén y las herramientas, incluso hay herramientas que acceden a diferentes tipos de almacenes o almacenes que implementan los protocolos de diferentes herramientas.

Por ejemplo:

BitBucket sería un almacén y Git una herramienta que puede trabajar sobre ese almacén remoto, puesto que ese almacén ofrece servicios que implementan el protocolo definido por Git.

No obstante, Git se puede considerar como un almacén por sí mismo, ya que no requiere de un almacén remoto para funcionar, pero sería una excepción; además, no es frecuente trabajar con Git sin un almacén remoto de respaldo e integración.

Los sistemas más tradicionales funcionaban en modo cliente (herramienta) más servidor (almacén). SVN sería un ejemplo de esta arquitectura.

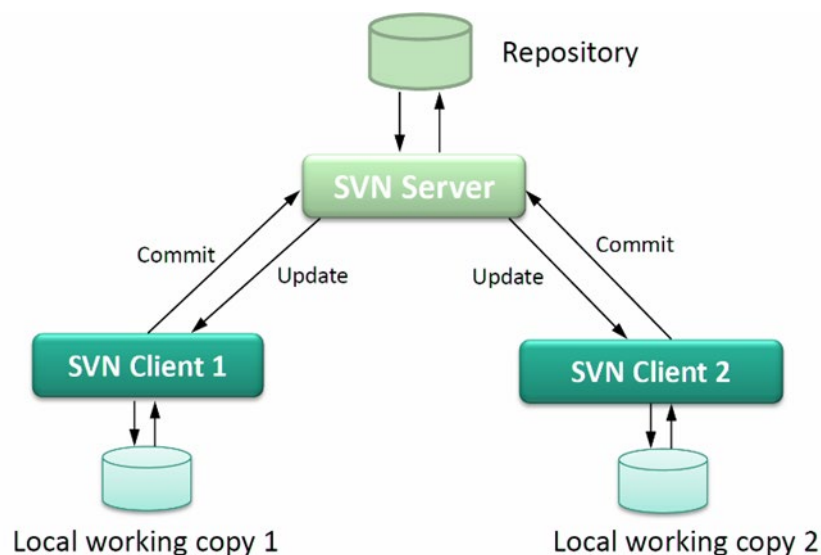


Figura 2. Arq. Cliente-Servidor en Repositorio de código. Fuente: L.palm, 2011.

En cualquier equipo de desarrollo se comentan (por fortuna, cada vez menos) anécdotas en cuanto a **código perdido**, sistemas que han dejado de funcionar al hacer una *release*, *bugs* detectados en producción en módulos que siempre han funcionado bien, etc.

Por ejemplo:

Dos desarrolladores pueden estar trabajando sobre el mismo fichero de código fuente. El primero en acabar guarda sus cambios. El segundo desarrollador guarda también sus cambios sobrescribiendo el mismo fichero y eliminando, por tanto, los cambios realizados por el primer desarrollador.

Un sistema de control de versiones actual avisaría al segundo desarrollador de que está intentando guardar una versión ya modificada y le propondría una forma de combinar ambos cambios (*merge*) de forma que no se pierda el trabajo de ninguno de los dos desarrolladores.

Es por todos estos motivos, y muchos otros, que los repositorios de código tuvieron que **necesariamente evolucionar** con el paso del tiempo para conseguir una mayor eficiencia en el trabajo en equipo en sistemas informáticos grandes.

Tras la extraordinaria mejora de las telecomunicaciones y los equipos de trabajo a lo largo de los últimos cuarenta años, los **repositorios de código** han incorporado **funciones** y **herramientas** que eran impensables en los orígenes y que, con la experiencia de millones de usuarios por todo el mundo, durante muchos años han definido un **modelo de trabajo** basado en las necesidades y *best-practices* para la gestión de código de grandes proyectos y la coordinación de equipos de trabajo remotos y desacoplados.

Las principales **funciones** de un repositorio de código se podrían resumir en las siguientes:

Principales funciones de un repositorio de código	
Seguridad	Mantener todo el código centralizado de manera que no puedan perderse datos, permitiendo que forme parte de las políticas de <i>backup</i> , así como evitando los accesos no autorizados.
Control de cambios	Cualquier cambio que se realice en algún fichero del repositorio queda almacenado y diferenciado de otras versiones, por lo que siempre se puede recuperar el código que había en cualquier momento (versión/commit). Inicialmente, estos sistemas se llamaban «controles de versiones» puesto que esta era precisamente la principal funcionalidad que ofrecían.
Gestión de ramas	Esta función está muy relacionada con las versiones, pero a diferencia de una versión (<i>commit</i>) que es cualquier cambio que se sube al repositorio, una rama engloba un conjunto de <i>commits</i> que dan un sentido conceptual al conjunto, tales como «la rama para la versión 2.1», «la rama para corregir el bug1254», un <i>Proof of Concept</i> (PoC) para la mejora de rendimiento, etc.

Tabla 2. Principales funciones de un repositorio de código. Fuente: elaboración propia.

En la historia de la gestión de software han existido **grandes sistemas de gestión** de repositorios de código usados por las mayores compañías de software, tanto clásicas como recientes. Desde los sistemas más clásicos como SourceSafe, CVS, Perforce o Mercurial hasta los más modernos como Subversion (SVN) o Git (con BitBucket, Github, Gitlab, etc.), cada sistema ha ido añadiendo o mejorando alguna **funcionalidad o arquitectura** para adaptarse a las **necesidades y capacidades** del momento.

Este capítulo no podría completarse sin profundizar en la herramienta actual por excelencia y estándar de facto para gestionar **repositorios de código (Git)**, ni tampoco sin explicar el proceso de **gestión de software o SCM**, por lo que se abarcarán a continuación.

GIT

Git es la **herramienta de control de versiones** más usada en la actualidad. Desde sus comienzos en 2005, con una arquitectura innovadora, ha ido ganando adeptos a este sistema revolucionario dejando atrás a los antiguos estándares como SVN o CVS.

Cabe recordar que en un sistema de gestión de código tenemos dos **piezas** fundamentales:

El servidor: almacena de forma centralizada (y segura) todos los cambios que se realizan en el código.

El cliente: proporciona al usuario las herramientas (comandos, utilidades) para trabajar de forma local con el código y comunicarse con el servidor.

Git es un **cliente** que es capaz de comunicarse con muchos tipos de servidores distintos (todos deben ofrecer servicios que implementen el protocolo de Git) y que proporciona al usuario las herramientas para descargar actualizaciones de un proyecto, subir sus cambios, crear nuevas ramas, etc. Precisamente, la **gestión de ramas** es la principal **fortaleza** de Git.

Git es un **sistema de control de versiones distribuido** (DCVS). A diferencia de otros sistemas centralizados, Git (al igual que otros DCVS del mercado) envía toda la información del repositorio a los clientes, de manera que, en caso de un fallo grave del servidor, sería posible reconstruir todo el modelo de datos en el servidor a partir de la información de los clientes.

Esto permite una gran **autonomía a los clientes**, puesto que son capaces de ejecutar comandos para gestionar ramas, *commits*, etc. de manera totalmente independiente del servidor, sin conexión a Internet y, únicamente cuando estamos preparados para compartir los cambios sincronizamos contra el servidor. Es en este momento cuando pueden surgir **conflictos** en el caso de que otro usuario haya modificado el mismo archivo desde su puesto de trabajo, sobre la misma rama y al mismo tiempo.

Conceptos de los sistemas de control de versiones y plataformas

Antes de entrar en estos detalles, es conveniente realizar un rápido repaso de los principales conceptos de los sistemas de control de versiones y plataformas de desarrollo que también se analizarán más adelante:

Fichero. Es la **unidad básica de almacenamiento** en un repositorio. Todos los contenidos del repositorio se basan en ficheros, al igual que el fichero es la unidad básica en un sistema de ficheros del sistema operativo.

Versión de un fichero. Identifica el **contenido** de un fichero en un instante determinado. Un fichero puede registrar cambios a lo largo de su existencia y cada uno de esos cambios se almacena sin sobrescribir al anterior de manera que se puede trazar toda la evolución de ese fichero.

Commit. Es un **punto de control** donde el usuario decide qué quiere informar al repositorio de la creación de una nueva versión sobre uno o varios ficheros. El fichero puede modificarse diez veces haciendo correcciones locales, pero solo cuando tenemos el fichero preparado y corregido es cuando hacemos un *commit*

para indicar que tenemos una nueva versión que contiene los cambios necesarios para un determinado propósito (un cambio solicitado, un *bug* corregido o simplemente para salvar los cambios antes de irse de vacaciones el desarrollador para que el resto del equipo pueda continuar con el desarrollo).

Es importante que los desarrolladores realicen *commits* frecuentemente para evitar que el código resida únicamente en sus PCs. Un *commit* puede incluir varios ficheros a la vez, es decir, se notifica al mismo tiempo de la modificación de varios ficheros todos con un mismo propósito. Los *commits* tienen una intención de agrupación conceptual. Todo *commit* tiene un código de identificación único en el repositorio.

Rama. Es una **secuencia de *commits*** con una intención también conceptual.

Por ejemplo, la versión 1.3 de una aplicación se compone de veintisiete *commits*: cinco de ellos implementan los cambios funcionales y técnicos dentro del alcance de esa versión, quince *commits* corrigen errores de versiones previas y siete corrigen errores de la versión en desarrollo. En total, tenemos una secuencia lineal de veintisiete *commits* que juntos engloban todas las modificaciones que se han realizado en cualquier fichero del proyecto para cumplir con los objetivos de esa versión.

Una rama también puede crearse simplemente para la corrección de un error, para realizar una prueba de concepto o, en general, para cualquier motivo que requiera poder modificar el código y almacenarlo en el servidor sin que otros usuarios sean conscientes siquiera de esos cambios para que no les afecte a sus desarrollos.

Dentro de una rama se pueden crear subramas y niveles de ramas. En realidad, el concepto de «rama» comienza a perder un poco su significado original, puesto que una rama puede partir de una rama y concluir en otra diferente. Se debe hablar más bien de líneas base de código que pueden recombinarse entre ellas

libremente. No obstante, en los desarrollos habituales las ramas sí suelen mantener una jerarquía y comenzar y terminar en la misma rama.

Merge. Cuando se realiza un **cambio en un fichero** (como parte de un *commit*), pueden darse dos situaciones:

El fichero **no ha sido modificado** por nadie en esa rama mientras se hacían los cambios; en este caso, el fichero puede ser sobrescrito completamente con la nueva versión.

El fichero **sí ha sido modificado** en paralelo por otro desarrollador; en este otro caso, no se puede sobrescribir puesto que perderíamos los cambios del otro desarrollador.

Es en este punto cuando tenemos que realizar un *merge*, es decir, **combinar** tanto la versión del primer desarrollador como la versión del segundo desarrollador, de manera que generemos una versión del fichero que incluya ambos cambios y sea válida para ambos desarrolladores.

En ocasiones, los dos desarrolladores estarán tocando partes diferentes del mismo fichero y es posible que esta combinación «*mergeo*» sea muy sencilla e incluso automática. Sin embargo, hay momentos en los que los cambios afectan a la **misma porción de código**, y es cuando se produce el *merge conflict*.

Merge conflict. Cuando dos desarrolladores han modificado a la vez un mismo fichero el sistema de control de versiones avisa al segundo desarrollador (el que ha hecho el *commit* más tarde) del conflicto de código. En caso de no haber modificado el mismo código, el sistema le propondrá **aplicar la combinación automáticamente**; sin embargo, cuando las mismas líneas de código están afectadas, le mostrará el conflicto al desarrollador para que lo resuelva **manualmente**: dando prioridad al código del primer desarrollador, dándosela al

segundo, o bien escribiendo manualmente un código que contemple ambos cambios.

Los conflictos son uno de los momentos más complejos de la gestión de código y debe realizarse con sumo cuidado para evitar daños graves a la calidad del código.

Deben hacerse con un buen **conocimiento del propósito** por el que ambos desarrolladores estaban haciendo ese cambio en el fichero y buscando que ambos cambios queden recogidos en la versión final combinada.

Merge request (o pull-request): se trata de **combinar dos ramas** y, de manera habitual, concretamente, combinar una rama hija (*feature*) en una rama padre (máster).

Cuando se crea una rama para cumplir un determinado objetivo y se finaliza, esos cambios deben aplicarse en la línea base de código. La línea base contiene el **código definitivo** que incluye todos los cambios y que está **libre de errores** conocidos. Sobre esta línea base (o línea maestra) se van abriendo diversas ramas para objetivos de mayor o menor envergadura.

Desde la rama «máster» se puede crear una rama para la versión 2.3 que se planea publicar en cinco meses. Dentro de la rama «v2.3» se crean tantas ramas como requisitos funcionales se van a implementar (por ejemplo). Cada **equipo** de trabajo trabaja en una **rama aislada** sin verse afectado por los cambios que van realizando el resto de los equipos. Cuando el equipo completa su rama, tiene que llevar sus cambios a la rama padre «v2.3». La rama hija puede contener cambios de cuarenta ficheros, de los cuales diez pueden haber sido modificados en paralelo por otros equipos (*merge conflict*). Es en este punto cuando entra en juego el **merge-request**.

El *merge-request* (o *pull-request*) es una **solicitud de un equipo a otro** equipo (o a un supervisor) para que **revise el código** que está a punto de subir a la rama

padre (por ejemplo). Ese código ya está libre de conflictos (se han resuelto por el equipo que solicita el *merge-request*) y solo queda a la espera de que otro equipo lo revise para verificar que es correcto y que el código puede sobrescribir la línea base. El equipo supervisor validará la resolución de los conflictos, la calidad del código, etc., y dará su visto bueno para ser incorporado a la línea base.

Estos son los conceptos y acciones principales que se realizan continuamente al trabajar con sistemas de control de versiones, además, lógicamente, de las acciones para descargar el código del servidor, crear ramas, actualizar la copia local con los últimos cambios subidos al repositorio, etc.

Git es un sistema muy potente para gestionar el código de equipos que trabajan en paralelo sobre el mismo código.

Gracias a los avances en los sistemas de control de versiones, ahora es mucho **más eficiente** trabajar en proyectos grandes sin que el esfuerzo en la coordinación de equipos suponga un gran porcentaje del esfuerzo global del proyecto, como lo era antes de la existencia de estas herramientas.

Los conceptos más básicos de Git se asimilan rápidamente, pero Git es un sistema **muy complejo**, cuyo manejo a nivel experto requiere muchos años de **experiencia** y mucha **formación**.

10.4. Plataforma de desarrollo

¿Qué es una plataforma de desarrollo?

Antes de entrar en detalle en las opciones principales de mercado que tenemos disponibles como plataformas de desarrollo, es necesario situar el **contexto de estos sistemas**.

Una plataforma de desarrollo es un **sistema**, generalmente basado en web, desde el que un desarrollador, un equipo de desarrollo o toda una compañía **gestiona el desarrollo de un proyecto y su evolución**.

La piedra angular de estos sistemas es un **gestor de repositorios** (generalmente basado en Git) sobre el que los desarrolladores almacenan todo el código de sus proyectos. Alrededor de este repositorio se añaden de manera integrada otras funcionalidades que implementan las técnicas de SCM para facilitar el desarrollo, el trabajo en equipo y la gestión de los proyectos.

Ejemplo de proceso de desarrollo

Si un usuario detecta un error, puede entrar en la plataforma y abrir un *ticket* con los detalles (o una solicitud de mejora o nueva funcionalidad) y queda asignado a un responsable de atender ese *ticket*.

El equipo de gestión del proyecto prioriza las nuevas tareas y decide atender ese *ticket* en la siguiente versión, por lo que genera una rama de desarrollo con la nueva versión e indica en un documento (en la misma plataforma) vinculado a esa rama cuál es el alcance de la rama, y qué *tickets* se van a atender.

Cuando se va a trabajar en un determinado *ticket*, se abre una rama «*feature*» y se vincula la rama al *ticket*, de forma que se pueda trazar la evolución del código para saber qué modificaciones se hicieron para atenderlo, qué desarrolladores trabajaron en él o qué pruebas se hicieron; también pueden documentar el cambio, etc.

Todo esto, mejora el proceso de desarrollo al quedar todo mejor documentado y, especialmente, habilitando una trazabilidad completa.

Estos sistemas no se quedan únicamente en añadir una wiki y un sistema de *ticketing* a un repositorio Git (aunque sí es lo que más se usa en la práctica), sino que ofrecen otras **características** como:

Gestión de *merge/pull request*.

Gestión de permisos e integración con sistema corporativos de autenticación.

Herramientas de CI/CD para permitir que el código siga el flujo de integración/test/despliegue automático.

Herramientas de comparación de código entre ramas y *commits* para detectar fácilmente las diferencias entre versiones.

Integración con sistemas de terceros para poder extender y adaptar el entorno de desarrollo a las necesidades particulares de cada organización hasta el punto de establecer sistemas de *web-hooks*.

Amplia selección de servicios web para maximizar las capacidades de adaptación e integración con otros sistemas.

Cuando se habla de **plataforma de desarrollo**, el primer sistema que habitualmente viene a la mente es **Github** que se puede considerar el primer gran precursor de plataformas de desarrollo. No fue la originaria, pero sí la primera en explotar este mercado.

Otras grandes plataformas de uso muy frecuente son **GitLab** y **BitBucket**, aunque a una distancia considerable en cuanto a número de usuarios y de proyectos almacenados en sus repositorios, lo cual no quiere decir que sean peores, ya que, como veremos más adelante, no se trata de ser mejor o peor, sino de adaptarse a las necesidades particulares y concretas de cada proyecto o equipo. A nivel de costes, suelen ofrecer versiones gratuitas, alojadas y versiones *enterprise* de pago, estas últimas con funciones adicionales propias de grandes sistemas corporativos, además del soporte técnico, mantenimiento, etc.

El **funcionamiento** de estos tres gigantes es **muy similar**, con algunas diferencias a la hora de implementar las funciones que ofrecen o la forma de integrarse con otros sistemas, por lo que la transición de un sistema a otro no suele ser complejo, siempre que no se haya adaptado mucho el sistema, lo que sí supondría una barrera de salida a la hora de cambiar de plataforma.

En el momento de decidir qué plataforma usar hay otros **factores externos** a la propia funcionalidad del sistema que inclinan la balanza hacia un lado u otro. Por ejemplo, la versión gratuita de GitHub no está pensada para grandes desarrollos privados, puesto que GitHub «fuerza» a sus usuarios a que el código que se almacene en sus repositorios sea **código abierto**, disponible a cualquier otro desarrollador; no obstante, hay excepciones para permitir almacenar parte del **código de forma privada**. Veremos estos detalles y una comparativa general entre estos sistemas en los siguientes apartados.

GitHub

Fue la primera gran plataforma de desarrollo, nacida en 2007-2008. Durante sus años de existencia ha acumulado unas **cifras** muy superiores al resto de plataformas:

Número de **usuarios**: alrededor de 30 millones.

Número de **organizaciones** que trabajan con GitHub: 1.5 millones.

Número de **repositorios** alojados: alrededor de 100 millones.

En cuanto a **características más relevantes** para definir GitHub serían las siguientes:

Es una herramienta de uso gratuito que también cuenta con una versión Enterprise con funcionalidades específicas para trabajar en entornos privados.

No es un producto *open-source*.

El código que se almacena en la versión gratuita de GitHub es *open-source*, aunque es posible almacenar una pequeña cantidad de código de forma privada.

En 2018, la compañía fue comprada por Microsoft. Muchos usuarios vieron un movimiento peligroso en cuanto a continuidad de la versión gratuita y la relación con el mundo *open-source*, lo que provocó una migración de usuarios y proyectos a otros sistemas, especialmente GitLab, a pesar de que GitHub no ha sufrido cambios relevantes desde la adquisición.

GitLab

GitLab surgió en 2011 y mantiene a 1000 desarrolladores aproximadamente en plantilla, además de 1400 colaboradores *open-source*. El **volumen de negocio** está a otro nivel respecto a GitHub:

Número de **usuarios**: es muy difícil de conocer, pero se estima que la cifra está alrededor de 1 millón de usuarios (30 veces menos que GitHub).

Número de **organizaciones** que trabajan con GitLab: 100.000 (10-15 veces menos).

Número de **repositorios** alojados: desconocido, al permitir instalaciones privadas del producto.

Las **principales características** de GitLab son:

Es un producto *open-source* que permite la instalación privada *on-premise*. Por este motivo, se desconoce el volumen real total de usuarios o de repositorios. Una única consultora de IT puede tener miles de usuarios y repositorios albergados en GitLab de forma privada.

Al igual que GitHub, tiene una versión pública de *hosting* donde los usuarios trabajan de forma colaborativa entre todos los usuarios y puede compartir código; así mismo, tiene una versión Enterprise.

La principal diferencia de GitLab respecto a GitHub es que permite instalar la versión Community Edition *on-premise*, lo que abre las posibilidades de almacenar código privado; esto supone una gran ventaja respecto a las opciones gratuitas de GitHub.

Otra diferencia importante es la incorporación de sistemas de CI/CD de manera totalmente integrada en la plataforma, por lo que no es necesario usar sistemas de terceros.

BitBucket

BitBucket es la solución de Atlassian para la **gestión de repositorios** y constituye el tercer gigante en plataformas de desarrollo. Como todos los productos de la compañía, el **nivel de integración** entre sus productos es **muy alto** por lo que resulta una opción idónea cuando la compañía usa buena parte de sus productos: Jira, Confluence, Bamboo, BitBucket, etc.

A nivel de **volumen de negocio**, las **cifras** que proporciona Atlassian son:

Número de **usuarios**: 6 millones.

Número de **organizaciones** que trabajan con BitBucket: 1 millón.

Número de **repositorios** alojados: alrededor de 30 millones.

En cuanto a sus principales **características**:

Fuerte integración con el resto de los productos de Atlassian, especialmente con Jira y Bamboo lo que permite una muy buena trazabilidad entre tareas/incidentes y código fuente.

Tiene una versión gratuita privada (*on-premise*) limitada a cinco usuarios, pero ilimitada en cuanto a número de repositorios privados.

La versión comercial está muy orientada a grandes compañías, permitiendo varios tipos de instalaciones según el volumen.

Bitbucket está también disponible de forma gratuita siempre que los proyectos que se alberguen sean *open-source*; similar a como funciona GitHub.

Comunidad:

GitHub: gran comunidad de desarrollo y posibilidad de compartir y colaborar.

Mucha información en las redes para realizar integraciones del producto con casi cualquier sistema de terceros.

GitLab: comunidad más pequeña y el uso suele ser privado. Bastante información disponible para realizar integraciones.

BitBucket: comunidad más pequeña y el uso suele ser privado.

10.5. Gestión de versiones

Desde un punto de vista literal del concepto, el versionado semántico consiste en **dotar de semántica** (significado) a una versión y, por lo tanto, indicar qué **tipo de cambio** está ofreciendo una nueva versión respecto a su predecesora. El ejemplo más claro para entender las versiones está en las librerías de uso común que se usan en el desarrollo: jQuery, Spring, Hibernate, etc.

El origen de estos conceptos es muy antiguo, pero la aplicación práctica masiva de esta convención es más reciente y cobra especial sentido para la gestión de dependencias una de las fases de la CI. Empecemos exponiendo un caso práctico para detectar el problema y, a partir de ahí, comprender la solución.

Ejemplo

Si tenemos una aplicación que está fragmentada en diez módulos independientes y aislados, cada uno con una serie de dependencias a unas librerías concretas, puede darse el caso de que cada módulo use una versión diferente de la misma librería y no sea posible hacer la *build* de la aplicación completa.

El módulo de Ventas se ha desarrollado sobre jQuery 3.4.0, mientras que el panel principal del sistema usa jQuery 2.2. Esto impide que se pueda construir la *build* puesto que las versiones son incompatibles y se detectará esta incompatibilidad a la hora de hacer la *build* en la comprobación de dependencias.

Para solucionar este problema, se debe llegar a un acuerdo para que todos los módulos especifiquen qué versiones requieren como mínimo y como máximo.

El módulo de ventas, en realidad, necesita cualquier versión superior o igual a la 2.1, porque es cuando se publicó una determinada funcionalidad. Por su lado, el panel principal del sistema necesita jQuery 2.2 o superior porque, en esa versión, se corrigió un *bug* que afectaba a ese módulo. Además, no puede ser una versión superior o igual a la 3.0 porque cambia la API completamente y ya no es compatible.

Para solucionar estas dependencias, una vez que cada módulo ha indicado cuáles son sus requisitos para cada librería, la herramienta de gestión de dependencias buscará qué versiones son **compatibles** para todos los módulos que, en el ejemplo anterior, podría ser la versión 2.1 o 2.2 (sin entrar en detalles de *patch version*), por lo que elegirá una de esas dos versiones para generar la *build*.

Esto sería una introducción al problema que queremos resolver. Para que esto funcione, se debe mantener una política estricta de versionado en la una versión 2.2 es completamente compatible con una 2.1, es decir, que si una aplicación funcionaba con la versión 2.1, también seguirá funcionando con una 2.2 (salvo que haya algún **bug** concreto, pero el objetivo es que funcione).

Sin embargo, si generamos una versión 3.0, ya estamos diciendo que no tiene por qué ser compatible con una aplicación que funcionaba con una 2.X. Puede ser que funcione o puede que no; no hay ningún compromiso.

Es ahí donde estamos dando información semántica con el número de las versiones. No existe una convención 100 % exacta para la nomenclatura completa de las versiones de un componente, pero sí que lo hay para, al menos, los primeros números de la versión.

Versión 2.0.9-prealpha1-SNAPSHOT.

Versión 2.1.1-beta1-build34562.

Versión 2.1.1-RC3.2.

Versión 2.2.0.

Versión 3.0.

Podemos encontrar versiones de este estilo, aunque cada vez es más difícil encontrar una versión que no especifique, como mínimo, los tres primeros números de la versión.

4.2.1
MAJOR Minor patch

Figura 3. Software *versioning*. Fuente: Wikipedia, 2022.

El **primer dígito** indica la versión del componente. Se conoce como **major version** (también se puede encontrar como *breaking*).

El **segundo dígito** indica la versión de una mejora realizada sobre ese componente, pero respetando la interfaz del componente. Se conoce como **minor version** (también se puede encontrar como *feature*).

El **tercer dígito** indica la versión correctiva dentro de la versión *feature*, es decir, una versión para solucionar bugs. Se conoce como una **patch version** (también se puede encontrar como *fix*).

A continuación del *patch version*, se puede escribir un guion y continuar añadiendo más detalles a la versión, pero como mínimo se deberían especificar estos tres números de versión.

¿Por qué son tan importantes estos números?

Entendamos un **componente** como una **pieza de software** que expone una interfaz pública que será usada por otros componentes. Esta interfaz define, entre otros detalles:

Unas acciones.

Unas entradas.

Una salida.

Un comportamiento.

Unos errores/excepciones.

Para poder realizar un desarrollo usando un componente, debemos tener la seguridad de que ese componente se va a comportar según **indica su interfaz**. Si no, no será posible la comunicación con ese componente o el resultado no será el esperado.

Si hemos validado que nuestra aplicación funciona correctamente con jQuery 3.4.0, ¿podemos garantizar que va a funcionar también con jQuery 2.1.0? No, no podemos garantizar que funcionará, aunque no podemos tampoco asegurar que no lo hará; tendremos que iniciar un **ciclo exhaustivo de pruebas** para validar que la versión antigua del componente también es 100 % compatible con nuestro desarrollo (con las funciones que necesita nuestro módulo de ese componente).

Puesto que la ejecución de un ciclo exhaustivo de pruebas es algo que queremos **evitar** por el sobrecoste que conlleva, la primera solución debería realizarse a nivel de **arquitectura**, especificando en las reuniones de diseño técnico qué versiones se pueden usar de cada componente de forma global.

Por ejemplo, vamos a realizar un desarrollo sobre **Angular 5**, nos solicitan una modificación sobre una aplicación que usa jQuery 2.1.0. Son restricciones que se imponen a **nivel de arquitectura** y el equipo de desarrollo debe cumplirlas para evitar problemas graves de incompatibilidad durante el desarrollo.

Una vez se han detallado los fundamentos y las bases del versionado, pasamos a explicar en qué consiste concretamente el **versionado semántico**. Como se ha comentado anteriormente, el **objetivo** es garantizar que el componente se comportará como el desarrollador espera y también queremos tener cierta flexibilidad a la hora de poder hacer pequeños cambios de versiones, como pasar de una 2.1.9 a una 2.2.4 sin tener que revisar de nuevo todo el código o ejecutar un ciclo completo de pruebas. ¿Cómo podemos (casi) asegurar esto? Gracias al versionado semántico.

Siempre que nos movamos entre versiones dentro de una **misma *major version***, tendremos la garantía (salvo que existan *bugs*) de que tendremos disponible la **misma interfaz pública** que estaba disponible en la **versión inicial** (X.0.0) de ese componente.

Por lo tanto, si nuestra aplicación funciona con una versión 2.0.0, tenemos buena seguridad de que también funcionará con una 2.1.1, una 2.2.5, etc.

Si pasamos a una *major version* inferior o superior tenemos una probabilidad alta de que la interfaz haya cambiado en el todo o en una parte (puede que exista la misma función, pero su comportamiento sea diferente, o la salida sea ligeramente diferente en algunos casos, etc.) o puede que no haya cambiado en absoluto.

Si necesitamos hacer uso de una **característica** (*feature*) concreta que salió en una determinada **minor version** (por ejemplo, 2.1.0), tenemos la garantía de que la aplicación también funcionará con **cualquier versión** 2.1.X y, además, tenemos la garantía de que funcionará correctamente en una **minor version superior** (por ejemplo 2.2.0).

Si necesitamos hacer uso de una versión concreta donde se corrige un *bug*, es decir, una *patch version*, tenemos la garantía (de nuevo, salvo errores en el componente) de que nuestra aplicación funcionará igualmente en una *patch-version* superior.

Ahora bien, no siempre funcionará en cualquier *minor version* superior, pero sí que saldrá una *patch version* dentro de las *minor versions* superiores donde también se arreglará ese mismo *bug*.

Por ejemplo, supongamos el siguiente caso:

Se publica una versión 2.1.9 que resuelve un error crítico de *Null Pointer Exception*.

- Previamente ya existía una versión 2.2.2 con *features* nuevas.
- También existía una versión 2.3.1 con *features* nuevas.
- Existía una versión 1, con *minors* 1.1, 1.2, 1.3 y cuya última versión fue la 1.3.8.

En este caso, es muy probable (las *best-practices* indican qué se debe hacer):

- Que se publique una 2.2.3 que arregle ese mismo *bug*, dentro de la rama v2.2.
- Que se publique una 2.3.2 arreglando ese mismo *bug*.
- Que se publique una 1.3.9 arreglando ese mismo *bug* en una versión antigua dentro del ciclo de soporte de versiones antiguas.
- Es probable que no se publiquen *patch versions* para las ramas 1.0, 1.1 y 1.2, pero esto ya dependerá del acuerdo de soporte de cada fabricante.

Resumiendo, toda esta casuística, el **versionado semántico** nos *indica* que:

Si cambiamos la *major version*, estamos informando de que no hay garantía de que la nueva versión sea compatible con versiones anteriores.

Si cambiamos la *minor version*, estamos informando de que las aplicaciones que funcionaban con *minor versions* anteriores (dentro de la misma *major version*), seguirán funcionando sin problemas, o incluso mejor.

Si cambiamos la *patch-version*, estamos informando de que las aplicaciones que funcionaban con *patch versions* y *minor versions* anteriores, lo seguirán haciendo con esta nueva versión, ya que solo hemos corregido *bugs*.

Gracias a toda esta convención, es posible establecer **reglas de definición** de dependencias que indican las **versiones** de cada componente que se requiere para **construir un módulo**. Con esto, el sistema de gestión de dependencias buscará aquellas versiones que sean **compatibles** con todos los módulos (o dará un error si

no es posible), se descargará la última versión compatible con todos (si la tiene ya descargada) y dará el visto bueno para iniciar el proceso de **generación de la *build***.

Para definir estas reglas se usan una serie de **operadores** (^, >, =, <, etc.) que indican la versión mínima o máxima de un componente que es compatible con ese módulo.

10.6. Herramientas orquestadoras de CI

Para automatizar tareas disponemos de muchas herramientas que se encargan o bien de ejecutar una tarea concreta, o bien de orquestar todos los procesos en general.

Existen una gran cantidad de herramientas, entre las que destacan:

Orquestadores:

- Jenkins.
- Bamboo.
- CircleCI.
- Travis.

Build:

- Ant.
- Maven.
- Gradle.
- NuGet.

Análisis estático de código:

- Sonar.
- FindBugs.

Repositorio de artefactos:

Nexus.
Artifactory.

Testing automático:

jUnit.
jMeter.
Cucumber.
Selenium/Selenide.

Despliegues y monitorización:

Ansible.
Kubernetes.
Docker.
Argo CD.
Graphana.
Etc.

Disponemos de una gran cantidad de **herramientas** que permiten **automatizar** todos los procesos con el fin de que el tiempo de despliegue sea el más corto posible, sin errores y se pueda ejecutar con la máxima frecuencia.

Jenkins

Jenkins es un producto de software libre cuyo **propósito principal** es orquestar las tareas de CI/CD.

Sus **componentes principales** serían el sistema de conexión con un repositorio de código (soporta prácticamente todos los sistemas de repositorio actuales) y la ejecución de scripts basados en Ant, Maven, Bash, etc.

Dentro de un repositorio concreto, se definen una serie de **pipelines** (procesos completos que se componen de tareas individuales organizadas en etapas) que se pueden ejecutar sobre ramas concretas de ese repositorio.

Por ejemplo, un *pipeline* puede ser generar una *build* sobre una rama concreta del repositorio. También, puede ser todo el proceso de despliegue de una *build* en un entorno de producción, preproducción, etc.

Para ejecutar un *pipeline* se pueden solicitar una serie de **parámetros** que, junto con la rama desde la que queremos ejecutar el *pipeline*, reúnen todos los datos necesarios para pasárselos al código del *pipeline* y ejecutarlo.

Además de los *pipelines* y los diferentes tipos de *scripts*, Jenkins se puede **extender mediante plugins y**, actualmente, existen miles para añadir acciones a los procesos que se ejecutan en Jenkins. Por ejemplo, hay *plugins* para notificar por correo cuando inicia o finaliza un proceso, enviando también el resumen del estado del proceso (si ha acabado correctamente o con errores, qué errores, qué nivel de calidad de código de sonar ha obtenido, cuántos test automáticos han fallado, etc.).

También puede forzar la ejecución de un *pipeline* cuando se ha realizado un *commit* sobre una rama del repositorio, etc. Hay una gran comunidad de desarrollo detrás de Jenkins y sus *plugins*, mejorando el producto y añadiendo nuevas funciones.

Como **beneficio principal** al uso de Jenkins, destaca la capacidad de acercar el complejo mundo de las tareas de *build and deploy* a usuarios que únicamente deben pulsar un botón o, incluso, ni siquiera eso. Hay entornos en los que construir y desplegar una aplicación puede suponer la ejecución de 150-200 *scripts* que, únicamente, un reducido grupo de DevOps conoce. Jenkins proporciona la simplificación necesaria para acercar estos procesos a cualquier miembro del equipo de desarrollo, viendo las etapas por las que va atravesando el proceso y consultando los logs que se emiten en cada uno.

Como en la mayoría de los productos *open-source* de calidad, la mejor forma de comenzar con Jenkins o resolver dudas es revisar la documentación en la página del producto que puedes acceder desde [aquí](#), además de algunos vídeos introductorios.

10.7. Estrategias de despliegue

Cuando vamos a llevar una aplicación a producción, hay muchos **factores** que afectan a la decisión de **cómo realizar esta actualización**. Por ejemplo, hay sistemas que permiten una caída completa del sistema, otros que utilizan máquinas físicas muy limitadas y que no permiten tener varios entornos en paralelo, BDs compartidas, etc.

Al final, según las restricciones de nuestro sistema y los objetivos del negocio, se necesita aplicar un sistema de despliegue u otro. Tampoco en este caso se puede hablar de sistemas mejores o peores, sino que un sistema de despliegue es bueno cuando cumple con las **expectativas del negocio** y es **viable con la infraestructura** disponible.

Existen cinco grandes grupos de estrategias de despliegue.

Hablamos de grupos porque cada sistema informático requiere un **despliegue ligeramente diferente a otro**, por lo que es complicado hablar de estrategias idénticas en todos los casos, sino que es habitual que aparezcan algunas variantes en cada estrategia para cada sistema particular. No obstante, hablaremos de estrategias de despliegue refiriéndonos a estos cinco grupos:

Grupos de estrategias de despliegue	
1	<i>Big bang deployment.</i>
2	<i>Rolling upgrade.</i>
3	<i>Blue-green</i> (también conocido como <i>Red-black</i>).
4	<i>Canary releases</i> (también conocido como <i>Red-black</i> por algunos autores).
5	<i>Dark launches</i> o <i>feature toggles</i> .

Tabla 3. Grupos de estrategias de despliegue. Fuente: elaboración propia.

Es importante destacar que no hay una que sea mejor que otra, sino que la mejor estrategia es la que mejor se adapta al negocio y a los recursos.

Ejemplo de estrategias

No parece lógico realizar una inversión económica (DevOps e infraestructura) en un sistema automatizado de tipo *canary deployment* (que veremos más adelante) cuando el sistema de información tiene una ventana de mantenimiento de 12 h todos los días, 68 h los fines de semana en los que se permite un *downtime* completo para cualquier tarea de mantenimiento.

Con ambas estrategias se consigue el objetivo de negocio deseado, por lo que la mejor estrategia sería la más económica (una variante de *big bang deployment*, seguramente), en este caso.

No obstante, en sistemas modernos relacionados con el mundo *online* (por ejemplo, web, móvil, televisión, etc.), las estrategias son diametralmente opuestas, ya que el negocio fuerza a que el sistema tenga un *cut-over* rápido, fiable y sin pérdidas en el posible *rollback*.

Big bang deployment

Se trata del sistema de despliegue **más sencillo** y, además, el **más empleado** en la mayoría de los sistemas de información de gestión empresarial.

Como su nombre indica, este tipo de despliegue pone en marcha la nueva aplicación de golpe y en todos los servidores a la vez. Previamente, se han parado todos los servicios, por lo que **el downtime es alto o muy alto** (en relación con sistemas *online*). Una vez que se han desplegado en todos los servidores, se ejecutan pruebas de verificación y, en caso de conseguir la aceptación, se habilita el acceso al nuevo sistema. No se trata de un sistema manual (aunque también podría serlo), es una **estrategia** más para tener en cuenta en sistemas CI/CD 100 % automatizados.

Este sistema de despliegue es **muy seguro, sencillo de mantener y desarrollar**, puesto que el sistema no necesita ser capaz de trabajar con dos versiones a la vez, ya que la versión antigua se desactiva completamente. Este matiz es muy importante en casos en los que, por ejemplo, la nueva versión requiere realizar cambios sensibles en el modelo de datos y la infraestructura únicamente dispone de una base de datos. Al haber cortado el tráfico a la versión antigua, se pueden realizar los cambios en frío en el modelo de datos para que la nueva versión funcione sin problemas con el único modelo de datos que se va a encontrar en BD.

Rolling upgrade

Avanzamos un poco más en la complejidad del proceso de despliegue y, una evolución natural del Big bang deployment sería el Rolling upgrade, conocido en español como **despliegue incremental** según diversos autores; aunque una traducción más adecuada sería **despliegue escalonado o despliegue secuencial**.

Big bang se caracterizaba por no permitir coexistir dos versiones a la vez y por tener un *downtime* alto. Con Rolling upgrade esas dos características ya no están presentes,

aunque sí se incorpora una nueva característica en cuanto a las posibilidades de **crecimiento** de la infraestructura.

Rolling upgrade consiste en ir apagando instancias del servidor (desconectándolas del *pool* de producción del balanceador de carga) y ejecutar los procesos de **actualización en cada instancia** una a una. Cada vez que una instancia está actualizada, se ejecutan las pruebas de aceptación y se devuelve de nuevo al *pool* de producción, **coexistiendo** en el *pool* tanto instancias **antiguas** como **nuevas**.



Figura 4. Rolling upgrade deployment. Fuente: Skowronski, 2018.

En el gráfico anterior, las instancias azules representan la versión antigua y las verdes las instancias que se actualizan, prueban y devuelven actualizadas con la versión nueva.

Blue-green deployment

Estas estrategias de despliegue tienen **diferentes denominaciones** según autores y organizaciones. También se denomina Red-black (con el mismo propósito) o simplemente A/B. Algunos autores también denominan Red-black a Canary releases que veremos en el siguiente capítulo. Para evitar confusiones con el término Red-black, en este tema hablaremos de Blue-green y Canary releases.

El **objetivo principal** es mantener dos entornos paralelos prácticamente idénticos entre sí de manera que un entorno esté en producción y el otro quede a la espera para entrar en producción en cualquier momento (**rollback inmediato**).

A uno de los entornos se le denomina *Blue* (o Red, o A) y contiene la versión del sistema que está en producción. Todo el tráfico se redirige al entorno Blue. El otro entorno (Green, Black o B) se usa para desplegar la versión nueva; todas las pruebas finales se ejecutan sobre el nuevo entorno.

Canary releases

Esta estrategia de despliegue es una **evolución de Blue-green** y de **Rolling upgrade**. Netflix acuñó el término Red-black a esta estrategia y es la usada en sus sistemas de despliegue. No obstante, Red-black podría considerarse como una variante de Canary releases.

El concepto **Canary testing** consiste en ofrecer una **nueva versión** a un **grupo reducido de usuarios**, de tal forma que sean cobayas (o canarios) del nuevo sistema y, en caso de que algo funcione de forma incorrecta, solo afecte a ese conjunto reducido de usuarios.

Una vez que la aplicación en producción tiene el visto bueno (mediante *testing* sobre producción o por métricas sobre las acciones de los usuarios reales), se amplía el acceso a la nueva versión a más usuarios, para lo cual se habilitan más instancias de la nueva versión y se van desactivando (no eliminando) las instancias de la versión antigua, de manera que la capacidad de computación no se vea comprometida para el dimensionamiento de carga prevista.

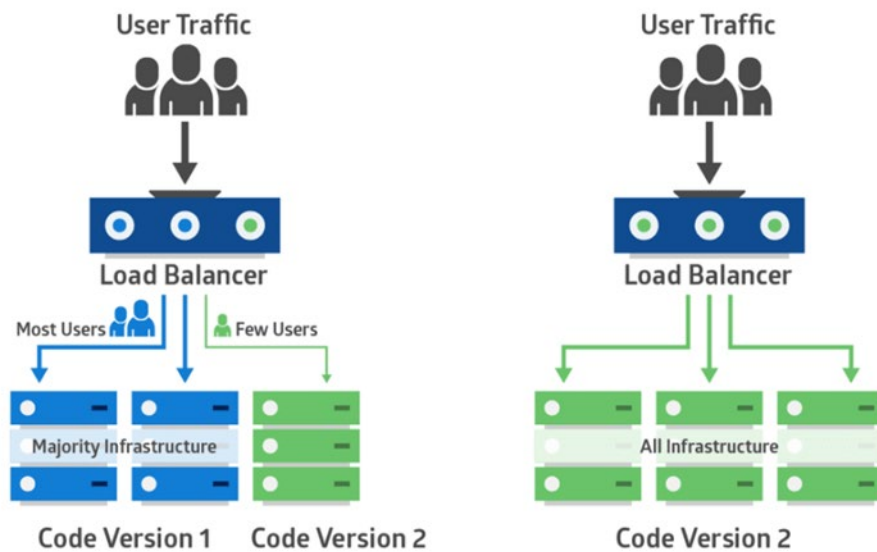


Figura 5. Canary release. Fuente: Skowronski, 2018.

Este salto, desde la versión antigua a la nueva, puede realizarse de forma **incremental** (como en Rolling upgrade) o de forma **brusca** (Blue-green).

Dark deployment / Features toggles

Estas estrategias de despliegue son menos conocidas en el ámbito divulgativo, aunque sí son de uso **frecuente en muchos sistemas**, incluso en algunos tan conocidos como el S.O. Android, el navegador Chrome, Gmail, etc.

Se trata de una estrategia en el que el equipo de pruebas, generalmente, son **usuarios internos** que son los únicos que conocen cómo acceder a los nuevos cambios, o bien usuarios **externos** con perfil más técnico que aceptan activar algunas funcionalidades que no están disponibles por defecto para el público general.

10.8. Herramientas de gestión de entornos

Hasta ahora, hemos estudiado estrategias de despliegue y analizado las ventajas e inconvenientes de todas ellas para elegir la que más conviene en cada sistema y cada situación concreta. Sin embargo, todas estas estrategias pueden llevarse a cabo de forma completamente manual o automatizada.

El **objetivo** del despliegue continuo es realizar estos despliegues de forma **automatizada o casi automatizada** (autorizaciones manuales, por ejemplo) de manera que se reduzca el tiempo necesario para poner en producción unos cambios realizados en la aplicación. Esto es especialmente **útil** en sistemas que requieren un *time to market* lo más reducido posible, así como en sistemas con cientos/miles de instancias en producción en los que no sería posible llevar a cabo un proceso de actualización manual en un tiempo prudencial.

Para lograr estos procesos automatizados necesitamos una serie de **herramientas** que faciliten la automatización de algunas tareas, desde la generación de la *build*, como ya vimos en el tema anterior, como el despliegue de las imágenes en las instancias, reinicio de servicios, auto *trigger* del despliegue, etc. Veremos alguna de estas herramientas.

La primera idea que puede venirnos a la cabeza es que por qué hace falta algo más que Jenkins (o similares), cuando estos son capaces de conectar con cualquier sistema, son entornos distribuidos, extensibles, integrables, personalizables, etc. Sí, es cierto que todo se podría orquestar con Jenkins, de la misma manera que toda una aplicación web se podría desarrollar escribiendo código HTML desde cero o bien usando librerías de componentes gráficos y *frameworks* JavaScript, pero aquí entramos en el debate entre **generalización versus especialización**.

Cuando nos centramos en **tareas de despliegue**, hay herramientas orientadas a estas tareas que facilitan mucho estos procesos ofreciendo funcionalidades complejas a golpe de *click*. Quizás no sean tan abiertas o personalizables (que también lo pueden ser), pero están orientadas a unas tareas concretas que se pueden parametrizar de forma rápida y sin grandes desarrollos.

Repasaremos algunas de las principales **herramientas de mercado**, aunque hay muchas otras:

Spinnaker.

Argo CD.

AWS CodePipeline.

Spinnaker

Imaginemos los entornos de producción de una compañía tan grande como Netflix, con un volumen de tráfico de los mayores del mundo donde constantemente están actualizando los sistemas, tanto para corregir *bugs* como para ofrecer una mejor experiencia de usuario y nuevas características que posicionen su producto delante de la competencia.

Esta compañía claramente necesita el mejor sistema posible para **reducir** el *time to market* de su producto, **garantizando** un 0 % *downtime* (ninguno nos imaginamos Netflix caído) y con todas las garantías de que el *upgrade* se realizará con éxito, y permitiendo Canary tests en algunos casos concretos, segmentando el *target* de cada versión, etc.

Argo CD

Argo es una *suite* de productos vinculados al mundo DevOps. Argo CD es la herramienta que se encarga de las **tareas de despliegue**. Es una aplicación que facilita el despliegue únicamente en Kubernetes. A diferencia de Spinnaker, no es tan potente ni ofrece tantas acciones rápidas de forma nativa.

Está implementado como un **controlador de Kubernetes** y, siguiendo la metodología de **GitOps**, es decir, definir de forma declarativa en Git (en un fichero tipo YAML) cuál es el estado deseado para un sistema, y el producto se encargará de llegar a ese estado. Mediante este lenguaje declarativo, se definen las instancias que queremos en cada entorno, qué imagen debería haber en cada entorno, qué parámetros tiene cada uno, qué pruebas se deben ejecutar, etc.

AWS CodePipeline

Dentro del universo AWS Cloud Computing, tenemos el producto AWS CodePipeline que ofrece también una herramienta de gestión de flujos basada en el **modelado de pipelines**, pero más orientada a cubrir todo el proceso de CI/CD, de forma similar a Jenkins.

La principal **fortaleza** de CodePipeline es su fuerte integración con los **productos AWS**, lo que facilita y optimiza las tareas de *build & deploy* cuando usamos los productos AWS.

En el vídeo *DevOps-CI/CD Pipeline* vamos a realizar un ejemplo del despliegue web de una página web con un «*pipeline*» de integración y despliegue continuo. Para ello vamos a utilizar un servicio de nube pública que nos gestiona todos los pasos de una forma fácil y rápida.



Accede al vídeo

10.9. Promoción entre entornos de la plataforma DevOps

Hoy en día, el procedimiento de **distribución y despliegue de las aplicaciones** ha cambiado radicalmente respecto a hace veinte o treinta años. Ya no tiene sentido, casi en ningún caso, usar un DVD o un *pendrive* para instalar una aplicación, sino que, generalmente, las aplicaciones residen en **servidores** y se descarga un cliente que accede al servidor.

No obstante, el proceso de entrega debe abarcar todas las posibilidades y puede concluir con una publicación en un **repositorio**, sin instalar nada en ningún entorno de producción. Este sería el caso del equipo de trabajo de un *framework* como Spring, React, etc., cuyo proceso finaliza cuando la librería se ha subido al repositorio de artefactos.

Si nos centramos en las aplicaciones de servidor, es decir, aquellas que requieren la instalación/actualización de unos artefactos en un grupo de servidores, hoy en día también ha cambiado mucho el procedimiento de despliegue, desde enviar un binario (.exe,.jar, etc.) a un servidor para que se despliegue en un servidor de aplicaciones, como se realizaba antiguamente, a empaquetar la aplicación en

imágenes (contenedores o máquinas virtuales) que se envían a los servidores para que se instalen.

Analizaremos las **herramientas** de las que disponemos para llevar a cabo los **despliegues** basados en contenedores que son las que, hoy en día, más se usan en el mercado debido a su alto rendimiento, capacidad de encapsulado y aislamiento.

En primer lugar, hay que realizar una **diferenciación entre contenedores y máquinas virtuales**. Ambos son *sandboxes* que se pueden ejecutar conjunta y aisladamente sobre un mismo sistema operativo y un mismo hardware, con la principal diferencia de que la máquina virtual incluye todo un sistema operativo y las aplicaciones, y el contenedor únicamente incluye las aplicaciones, por lo que es mucho más ligero y el tiempo de arranque es sensiblemente más rápido, proporcionando el aislamiento suficiente para las tareas que se encomiendan.

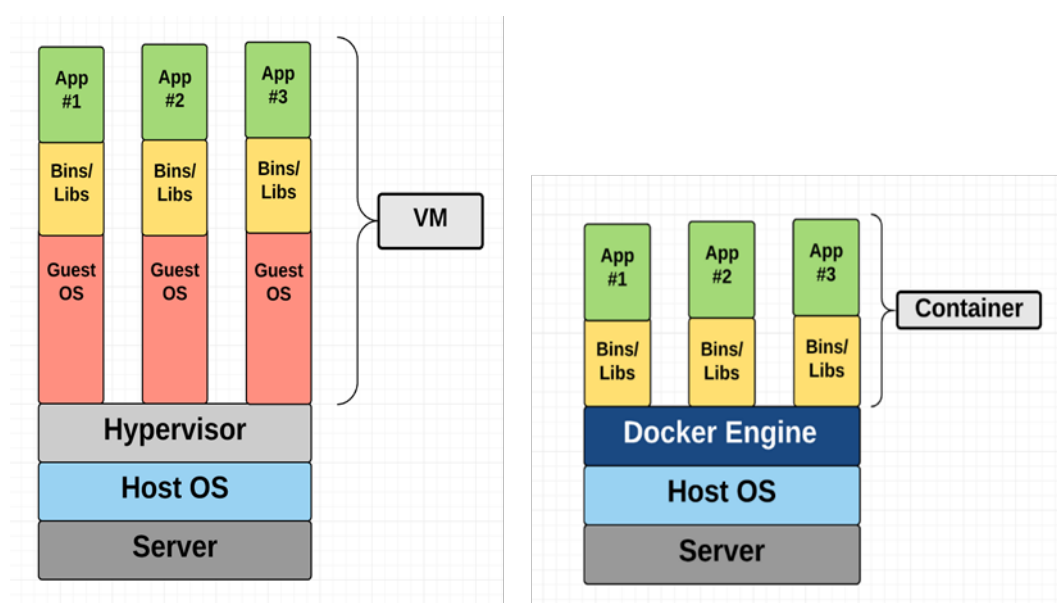


Figura 6. Diferencias entre VMs y Contenedores. Fuente: Kasireddy, 2016.

De esta manera, es posible tener una imagen Docker (un contenedor) con una instalación completa de MySQL, o un servidor de aplicaciones, o un servidor web, etc. Y es posible tener todos esos servicios corriendo en la misma máquina física y mismo sistema operativo, respetando los espacios de usuario entre ellos y sin interferencias.

El concepto de contenedor, aunque es relativamente nuevo, ya existía desde hace tiempo de forma similar y se encuentra implementado de forma nativa en sistemas operativos como Solaris, con las «Zones» y en otros sistemas. Sin embargo, hace no muchos años, se detectó la potencia de esta tecnología para llevar a cabo despliegues de paquetes completos estancos en servidores, facilitando los procedimientos de despliegue.

Para ello, hay dos sistemas que son los que principalmente se usan:

Docker.

Kubernetes.

10.10. Referencias bibliográficas

Serrano, J. (2019, febrero 25). *Integración, entrega y despliegue continuo. Diferencias y similitudes*. Blog Geeks. <https://geeks.ms/jorge/author/jorge/>

Lpalm. (2011, junio 26). *File:SVN Server Client Structure.png*. Wikibooks. https://en.wikibooks.org/wiki/File:SVN_Server_Client_Structure.png

Software versioning (2022, octubre 18). *En Wikipedia*. https://en.wikipedia.org/w/index.php?title=Software_versioning&oldid=1116804459

Skowronski, J. (2018, noviembre 21). *Intro to deployment strategies: blue-green, canary, and more*. DEVCommunity. <https://dev.to/mostlyjason/intro-to-deployment-strategies-blue-green-canary-and-more-3a3>

Kasireddy, P. (2016, marzo 4). *A Beginner-Friendly Introduction to Containers, VMs and Docker*. FreeCodeCamp. <https://www.freecodecamp.org/news/a-beginner-friendly-introduction-to-containers-vms-and-docker-79a9e3e119b/>

Integración Continua

Fowler, M. (2006, mayo 1). *Continuous Integration*.
<https://www.martinfowler.com/articles/continuousIntegration.html>

Es un artículo bastante extenso, pero muy bien explicado y con un análisis en profundidad sobre la integración continua poniendo el foco, especialmente, en las mejores prácticas dentro de la metodología de *Continuous Integration*. Todo esto, de la mano de un gran autor de artículos sobre Ingeniería del Software que complementará los conocimientos sobre la integración continua explicados en este tema.

Control de versiones

Control de versiones. (2022, septiembre 28). En *Wikipedia*.
https://es.wikipedia.org/wiki/Control_de_versiones

Un buen artículo que resume los principales conceptos, funciones y clasificaciones de los sistemas de control de versiones. En caso de no conocer el funcionamiento y los principales comandos de un sistema actual de control de versiones, se trata de una lectura altamente recomendada para seguir el hilo de la integración continua, así como otros temas posteriores.

Tutorial sobre Jenkins

Simplilearn. (2019, noviembre 18). *Jenkins Full Course | Jenkins Tutorial For Beginners | Jenkins Tutorial* / *Simplilearn*. [Vídeo]. YouTube.
<https://www.youtube.com/watch?v=FX322RVNGj4>

Existe un amplio debate acerca de la mejor forma de aprender a manejar un producto. Unos apuestan por los tutoriales y otros por manuales de usuario. El manual lo tenemos disponible [aquí](#), pero la recomendación de este autor es iniciar el *hands-on roadmap* de la mano de estos recursos de YouTube que explican los conceptos y procesos con una gran calidad didáctica.

1. ¿Cuáles no son etapas de los scripts de código?

- A. Descargar y compilar.
- B. Descargar y desplegar.
- ☐ C. Desplegar y enviar test de errores.
- D. Enviar informes y descargar.

Descargar, compilar, desplegar, enviar informes.

Un repositorio de código es el:

- A. Almacén de secretos de la aplicación.
- ☐ B. Almacén de los ficheros que conforman en el código fuente de un sistema.
- C. Almacén de los ficheros que conforman el código testeado de un sistema.
- D. Almacén de los ejecutables verificados.

Un repositorio de código es el almacén de los ficheros que conforman el código fuente de un sistema. Aunque pueda dar la impresión de que no existían repositorios de código antes de la aparición de Git/SVN lo cierto es que los repositorios son tan antiguos como el mismo código.

Git es un cliente que es capaz de:

- ☐ A. Comunicarse con muchos tipos de servidores distinto.
- B. Gestionar cualquier código.
- C. Comunicarse con servidores Linux.
- D. Ninguna de las anteriores.

Git es un cliente que es capaz de comunicarse con muchos tipos de servidores distintos (todos deben ofrecer servicios que implementen el protocolo de Git) y que proporciona al usuario las herramientas para descargar actualizaciones de un proyecto, subir sus cambios, crear nuevas ramas, etc. Precisamente, la gestión de ramas es la principal fortaleza de Git.

¿Qué es considerado un *Merge Conflict*?

- _ A. Cuando dos desarrolladores han modificado a la vez un mismo fichero del sistema.
- B. Cuando dos desarrolladores abren ramas en un Git con el mismo nombre.
- C. Cuando dos desarrolladores usan el mismo usuario en el mismo repositorio Git.
- D. Cuando dos líneas de código en un repositorio Git coinciden y se fusionan automáticamente.

Cuando dos desarrolladores han modificado a la vez un mismo fichero el sistema de control de versiones avisa al segundo desarrollador (el que ha hecho el *commit* más tarde) del conflicto de código.

Una plataforma de desarrollo es un sistema:

- A. Se gestionan los desarrolladores y sus tareas.
- B. Se gestionan los test de código.
- C. Se gestiona las versiones de código.
- _ D. Se gestiona el desarrollo de un proyecto y su evolución.

Una plataforma de desarrollo es un sistema, generalmente basado en web, desde el que un desarrollador, un equipo de desarrollo o toda una compañía gestiona el desarrollo de un proyecto y su evolución.

En una versión denominada 5.4.3 ¿Qué indica el número 4?

- A. Indica la versión del componente.
- _ B. Indica la versión de una mejora realizada sobre ese componente.
- C. Indica la versión correctiva dentro de la versión *feature*.
- D. Indica la versión *feature* dentro de la versión correctiva.

El segundo dígito indica la versión de una mejora realizada sobre ese componente, pero respetando la interfaz del componente. Se conoce como *minor version* y también se puede encontrar como *feature*.

¿Qué herramienta de CI no es un orquestador?

- A. Jenkins.
- B. Bamboo.
- _ C. Gradle.
- D. Travis.

Jenkins, Bamboo, CircleCI, Travis.

¿Cuál no es una estrategia de despliegue?

- A. Big bang deployment.
- B. Rolling upgrade.
- C. Blue-green.
- _ D. Lower TTL.

Big bang deployment, Rolling upgrade, Blue-green, Red-black, Canary releases, Dark launches o feature toggles.

Rolling upgrade consiste en ir:

- A. Apagando instancias del servidor y ejecutar los procesos de actualización en todas máquinas a la vez.
- _ B. Apagando instancias del servidor y ejecutar los procesos de actualización en cada instancia una a una.
- C. Apagando instancias del servidor todas menos una y ejecutar los procesos de actualización en esa instancia.
- D. Ninguna de las anteriores.

Rolling upgrade consiste en ir apagando instancias del servidor (desconectándolas del *pool* de producción del balanceador de carga) y ejecutar los procesos de actualización en cada instancia una a una.

El objetivo del despliegue continuo es:

- A. Realizar estos despliegues de forma autónoma.
- B. Realizar estos despliegues con scripts de ficheros json.
- C. Realizar estos despliegues de forma gradual.
- _ D. Realizar estos despliegues de forma automatizada o casi automatizada.

El objetivo del despliegue continuo es realizar estos despliegues de forma automatizada o casi automatizada (autorizaciones manuales, p.ej.) de manera que se reduzca el tiempo necesario para poner en producción unos cambios realizados en la aplicación.